SERVER & CLOUD

BAND 1

DOCKER
UND CONTAINER

Von den Grundlagen bis zum
produktiven Containerbetrieb



SERVER & CLOUD

Docker und Container

Von den Grundlagen bis zum produktiven Containerbetrieb

BAND 1

Vorwort

Server und Cloud tragen heute fast jede IT-Umgebung. Wer Dienste bereitstellt, braucht Wissen darüber, wie moderne Serverinfrastrukturen aufgebaut, verwaltet, abgesichert und dauerhaft betrieben werden. Die Buchreihe „Server & Cloud“ vermittelt dieses Wissen. Sie erklärt zuerst die grundlegenden Konzepte und führt dann zu fortgeschrittenen Technologien. Im Mittelpunkt stehen neben den theoretischen Erklärungen das praktische Verständnis und die eigenständige Umsetzung in realistischen IT-Umgebungen.

Die Buchreihe

Die Reihe umfasst vier eigenständige Bände. Jeder Band lässt sich für sich lesen und enthält ausführliche Erklärungen, technische Diagramme, vollständige Befehle, Schritt-für-Schritt-Anleitungen, Laborübungen, Praxisprojekte, Fehleranalysen und Lernkontrollen.

Band 1, Docker und Container, ist das Buch, das Sie in den Händen halten. Es führt in die Containertechnologie und die Architektur von Docker ein und behandelt die Installation, die Verwaltung von Containern, Docker Images, Docker Compose, Netzwerke, Volumes, die Sicherheit und den produktiven Betrieb. Außerdem zeigt es Docker unter Linux und unter Windows, mit WSL 2, PowerShell und Windows-Containern.

Band 2, Kubernetes-Grundlagen, widmet sich der Container-Orchestrierung. Er erklärt die Architektur von Kubernetes mit Nodes, Pods, Deployments, Services und Ingress, die Verwaltung von Konfigurationen und Geheimnissen mit ConfigMaps und Secrets, Persistent Volumes sowie Skalierung und Hochverfügbarkeit.

Ziel ist, Betrieb und Verwaltung verteilter Containeranwendungen zu verstehen und praktisch umzusetzen.

Band 3, Webserver und Reverse Proxy, behandelt den Aufbau und die Administration von Webservern mit nginx und Apache. Er erklärt HTTP und HTTPS, DNS, den Reverse Proxy, die TLS-Verschlüsselung und Zertifikate, Let's Encrypt und Certbot, die Konfiguration der Firewall und die Absicherung öffentlich erreichbarer Webdienste.

Band 4, Cloud-Grundlagen, führt in das Cloud Computing ein. Er behandelt IaaS, PaaS und SaaS, Public, Private und Hybrid Cloud, virtuelle Server, Netzwerke, Speicherlösungen, Sicherheit, Kostenmodelle und praktische Cloud-Infrastrukturen. Ein Schwerpunkt liegt auf dem Vergleich von AWS, Microsoft Azure und Hetzner.

Der Grundgedanke

Die Bücher sollen technisches Wissen verständlich und praxisnah vermitteln. Sie zeigen, welche Technologien es gibt, und erklären vor allem, wie diese funktionieren, wie sie zusammenarbeiten und wie man sie selbstständig einsetzt. Am Ende der Reihe sollen Sie moderne Server- und Cloud-Infrastrukturen selbstständig verstehen, aufbauen, verwalten und sicher betreiben können.

Die Reihe richtet sich an Einsteiger, an angehende Fachinformatiker für Systemintegration, an Linux- und Windows-Administratoren, an IT-Techniker und an technisch interessierte Menschen, die ihre Kenntnisse im Bereich Server und Cloud erweitern möchten.

Zu diesem Band

Dieser Band beginnt bei den Grundlagen und setzt keine Docker-Kenntnisse voraus. Sie sollten wissen, was ein Server und ein Betriebssystem sind, und bereit sein, mit der Kommandozeile zu arbeiten, die Teil II einführt. Das Buch gliedert sich in acht Teile, und jeder Teil endet mit einer Lernkontrolle. Die sieben Praxisprojekte bauen aufeinander auf und führen in Projekt 7 zu einer vollständigen, abgesicherten Serverinfrastruktur. Am Ende des Buches finden Sie die Lösungen,

eine Befehlsreferenz, Prüfungsfragen, zusätzliche Praxisaufgaben und ein Glossar.

Alle Beispiele laufen in derselben Laborumgebung: einem Debian-13-Server mit dem Namen `docker01`, dem Benutzer `admin` und dem Netz `192.168.100.0/24`. Ein Dollarzeichen vor einem Befehl bedeutet, dass Sie ihn als normaler Benutzer eingeben, eine Raute steht für Root. Diese Zeichen tippen Sie nicht mit. Die gezeigten Ausgaben sind typische Beispiele. Kennungen, Zeiten, Größen und Versionsnummern sehen bei Ihnen anders aus.

Docker und Windows ändern sich schneller, als ein Buch erscheinen kann. Prüfen Sie versionsabhängige Angaben deshalb vor dem Einsatz in der aktuellen Dokumentation der Hersteller. Üben Sie auf Testsystemen und nicht auf Produktivsystemen, denn einige Übungen verändern Firewall, Benutzerrechte und Netzwerkeinstellungen.

Inhaltsverzeichnis

Vorwort ii

I Grundlagen der Containertechnologie 1

1 Einführung in die Servervirtualisierung 2
- 1.1 Was ist Virtualisierung? . . . 2
- 1.2 Virtuelle Maschinen . . . 4
- 1.3 Hypervisor Typ 1 und Typ 2 . . . 4
- 1.4 Containertechnologie . . . 6
- 1.5 Unterschiede zwischen VMs und Containern . . . 7
- 1.6 Vorteile und Nachteile . . . 8
- 1.7 Typische Einsatzgebiete . . . 9

2 Container verstehen 10
- 2.1 Was ist ein Container? . . . 10
- 2.2 Images und Container . . . 11
- 2.3 Container-Lifecycle . . . 12
- 2.4 Isolation und Ressourcenverwaltung . . . 13
- 2.5 Linux Namespaces . . . 14
- 2.6 Control Groups (cgroups) . . . 16
- 2.7 Union Filesystems . . . 17
- 2.8 Container und Linux-Kernel . . . 19

3 Einführung in Docker 20
- 3.1 Entwicklung und Hintergrund . . . 20
- 3.2 Docker-Architektur . . . 22
- 3.3 Docker Engine . . . 22
- 3.4 Docker-Daemon . . . 23

3.5 Docker-Client ... 24
3.6 Docker Registry und Docker Hub ... 25
3.7 Kommunikation zwischen Client und Daemon ... 25

Lernkontrolle zu Teil I 27

II Docker installieren und konfigurieren 31

4 Vorbereitung der Linux-Umgebung 32
4.1 Das Übungssystem ... 32
4.2 Debian 13 vorbereiten ... 33
4.3 Ubuntu Server vorbereiten ... 35
4.4 System aktualisieren ... 36
4.5 Benutzer und Gruppen ... 37
4.6 Netzwerk überprüfen ... 38
4.7 Speicherplatz kontrollieren ... 39

5 Docker installieren 41
5.1 Installationswege im Überblick ... 41
5.2 Alte Pakete entfernen ... 42
5.3 Das Docker-Repository einrichten ... 42
5.4 Docker Engine, CLI und Compose installieren ... 44
5.5 Dienstverwaltung mit systemd ... 45
5.6 Docker automatisch starten ... 47
5.7 Installation überprüfen ... 47
5.8 Sicherheitsrelevante Einstellungen ... 49

6 Erste Schritte mit Docker 52
6.1 docker version und docker info ... 52
6.2 docker run ... 53
6.3 docker ps ... 56
6.4 docker images und docker pull ... 57
6.5 docker stop, start und restart ... 58
6.6 docker rm und docker rmi ... 59
6.7 docker logs ... 60
6.8 docker inspect ... 61
6.9 docker exec ... 62
6.10 Aufräumen nach den Übungen ... 63

Praxisprojekt 1: Docker-Grundinstallation 65

Lernkontrolle zu Teil II 72

III Images und Container verwalten 75

7 Docker Images 76

7.1 Aufbau eines Images ... 76
7.2 Der Image-Speicher ... 78
7.3 Image-Tags und Prüfsummen ... 79
7.4 Images laden und hochladen ... 80
7.5 Images bauen ... 81
7.6 Der Image-Cache ... 83
7.7 Image-Größen optimieren ... 84
7.8 Image-Versionierung ... 85

8 Dockerfile 86

8.1 Aufbau und Syntax ... 86
8.2 FROM ... 87
8.3 RUN ... 88
8.4 COPY und ADD ... 88
8.5 WORKDIR ... 89
8.6 ENV und ARG ... 89
8.7 EXPOSE ... 90
8.8 USER ... 90
8.9 CMD und ENTRYPOINT ... 91
8.10 HEALTHCHECK ... 92
8.11 Ein vollständiges Beispiel ... 93
8.12 Mehrstufige Builds ... 95
8.13 Best Practices ... 96

9 Containerverwaltung 98

9.1 Container erstellen, starten und stoppen ... 98
9.2 Interaktive Container und Detached Mode ... 99
9.3 Umgebungsvariablen ... 99
9.4 Ressourcenlimits ... 101
9.5 Container untersuchen ... 102
9.6 Container bereinigen ... 104

9.7 Automatische Neustartregeln 105
Lernkontrolle zu Teil III 107

IV Speicher und Netzwerke 110

10 Docker Storage 111
10.1 Das Dateisystem des Containers 111
10.2 Drei Wege, Daten zu speichern 112
10.3 Volumes 113
10.4 Bind Mounts 115
10.5 tmpfs 116
10.6 Persistente Datenspeicherung in der Praxis 117
10.7 Sicherung und Wiederherstellung 117
10.8 Rechte und Berechtigungen 119
10.9 Speicherverwaltung 120

11 Docker Networking 121
11.1 Netzwerkarchitektur 122
11.2 Netzwerkarten im Überblick 124
11.3 Bridge Networks 124
11.4 Benutzerdefinierte Netzwerke 125
11.5 Containerkommunikation 126
11.6 DNS innerhalb von Docker 127
11.7 Host Network und None 128
11.8 Macvlan 129
11.9 Portfreigaben 130
11.10 Firewall und Netzwerkisolation 131
Praxisprojekt 2: Eigener Webserver 135
Lernkontrolle zu Teil IV 143

V Docker Compose und Datenbanken 147

12 Einführung in Docker Compose 148
12.1 Zweck und Funktionsweise 148
12.2 YAML-Grundlagen 150
12.3 Die Datei compose.yaml 150
12.4 Services 153

12.5 Networks 154
12.6 Volumes 155
12.7 Environment 156
12.8 depends_on und Healthchecks 157
12.9 Neustartregeln 159

13 Mehrere Container gemeinsam betreiben 160
13.1 Docker Compose installieren und prüfen 160
13.2 Eine Anwendung definieren 161
13.3 Container starten 163
13.4 Container stoppen 165
13.5 Protokolle auswerten 166
13.6 Konfiguration ändern 166
13.7 Updates durchführen 167
13.8 Projekte verwalten 168

14 Datenbankumgebungen 171
14.1 MariaDB mit Docker 172
14.2 PostgreSQL mit Docker 174
14.3 Persistente Datenspeicherung 176
14.4 Benutzer und Passwörter 177
14.5 Datenbank-Backups 178
14.6 Wiederherstellung 179
14.7 Verbindungen zwischen Containern 180

Praxisprojekt 3: Datenbankserver 182

Praxisprojekt 4: Multi-Container-Anwendung 191

Lernkontrolle zu Teil V 203

VI Webdienste und Serveranwendungen 207

15 Webserver mit Docker 208
15.1 nginx installieren 208
15.2 Webserver-Container betreiben 209
15.3 Eigene Webseiten bereitstellen 211
15.4 Bind Mounts für Webinhalte 213
15.5 Portfreigaben 214
15.6 Logs und Fehleranalyse 215

16 Reverse Proxy mit Docker 217
16.1 Grundlagen eines Reverse Proxy . . . 218
16.2 nginx als Reverse Proxy . . . 219
16.3 Mehrere Webanwendungen . . . 220
16.4 Interne Docker-Netzwerke . . . 223
16.5 Namensauflösung im Proxy . . . 224
16.6 Grundlagen von TLS . . . 225
16.7 Sicherheitsaspekte . . . 228
17 Portainer 230
17.1 Einführung . . . 230
17.2 Installation . . . 231
17.3 Die Weboberfläche . . . 234
17.4 Container verwalten . . . 235
17.5 Images verwalten . . . 236
17.6 Volumes verwalten . . . 236
17.7 Netzwerke verwalten . . . 237
17.8 Stacks . . . 237
17.9 Die Benutzeroberfläche absichern . . . 237
Praxisprojekt 5: Reverse Proxy 240
Lernkontrolle zu Teil VI 252

VII Sicherheit und professioneller Betrieb 256

18 Docker Security 257
18.1 Angriffsflächen . . . 258
18.2 Root-Rechte . . . 259
18.3 Der Docker-Socket . . . 261
18.4 Das Least-Privilege-Prinzip . . . 262
18.5 Non-Root-Container . . . 263
18.6 Read-only-Dateisysteme . . . 264
18.7 Linux Capabilities . . . 265
18.8 Seccomp . . . 267
18.9 AppArmor . . . 267
18.10 Secrets . . . 268
18.11 Image-Sicherheit . . . 270

18.12 Schwachstellenanalyse ... 270
18.13 Supply-Chain-Sicherheit ... 271

19 Wartung und Monitoring 274
19.1 Containerüberwachung ... 274
19.2 Ressourcenverbrauch: CPU und RAM ... 277
19.3 docker stats ... 278
19.4 Logs ... 279
19.5 Speicherplatz ... 280
19.6 Updates und Patches ... 281
19.7 Backup-Strategien ... 284
19.8 Wiederherstellungsplanung ... 287

20 Fehlerdiagnose 289
20.1 Der Container startet nicht ... 289
20.2 Das Image kann nicht geladen werden ... 291
20.3 Der Port ist bereits belegt ... 292
20.4 Netzwerkprobleme ... 294
20.5 Berechtigungsfehler ... 295
20.6 Volume-Probleme ... 296
20.7 Datenbankverbindungsfehler ... 297
20.8 Der Docker-Daemon startet nicht ... 298
20.9 Systematische Diagnosemethoden ... 299

Praxisprojekt 6: Sicherer Docker-Server 302

Praxisprojekt 7: Vollständige Serverinfrastruktur 316

Lernkontrolle zu Teil VII 341

VIII Docker unter Windows 344

21 Docker unter Windows: Desktop, WSL 2 und Windows Server 345
21.1 Grundlagen von Docker unter Windows ... 346
21.2 Docker Desktop unter Windows 11 ... 348
21.3 Windows Subsystem for Linux (WSL 2) ... 351
21.4 Docker mit Windows PowerShell ... 354
21.5 Linux-Container unter Windows ... 358
21.6 Windows-Container ... 359
21.7 Docker und Container auf Windows Server ... 366

21.8 Docker Compose unter Windows 369
21.9 Docker Desktop und Sicherheit 371
21.10 Vergleich: Docker unter Linux und Windows 374
21.11 Praxisprojekte 376
21.12 Lernkontrolle und Prüfungsvorbereitung 406
21.13 Zusammenfassung 411
Lösungen zu den Lernkontrollen 413
Befehlsreferenz 435
Prüfungsfragen 520
Zusätzliche Praxisaufgaben 537
Glossar 544

TEIL I

Grundlagen der Containertechnologie

KAPITEL 1

Einführung in die Servervirtualisierung

Bevor Docker ins Spiel kommt, lohnt sich ein Blick auf das Problem, das Container lösen. Dieses Kapitel erklärt, warum Administratoren Server überhaupt aufteilen, welche Techniken es dafür gibt und an welcher Stelle Container in diese Geschichte passen. Vorkenntnisse in Virtualisierung sind nicht nötig. Sie sollten lediglich wissen, was ein Server und ein Betriebssystem sind.

1.1 Was ist Virtualisierung?

Ein Server ist zunächst ein ganz gewöhnlicher Rechner: Prozessor, Arbeitsspeicher, Festplatten und eine Netzwerkkarte, dazu ein Betriebssystem, das alles verwaltet. Früher bekam jede Anwendung ihren eigenen Rechner. Der Mailserver stand auf einer Maschine, die Datenbank auf einer zweiten, die Intranetseite auf einer dritten. Das hatte gute Gründe. Wenn eine Anwendung abstürzte oder ein Update brauchte, blieben die anderen unberührt. Außerdem benötigten verschiedene Programme oft verschiedene Versionen derselben Bibliothek, und zwei Programme, die sich nicht vertragen, lassen sich am einfachsten auf getrennten Rechnern betreiben.

Der Preis dafür war hoch. Die meisten dieser Maschinen langweilten sich. Ein Mailserver, der tagsüber ein paar Prozent der Rechenleistung braucht, belegt trotzdem ein komplettes Gehäuse samt Netzteil, Stellplatz im Rack und Strom-

rechnung. Wer zehn Anwendungen betrieb, kaufte zehn Server und nutzte jeden zu einem kleinen Teil.

Virtualisierung löst dieses Problem, indem sie die Verbindung zwischen Hardware und Anwendung lockert. Statt jede Anwendung direkt auf einem eigenen Rechner laufen zu lassen, schiebt man eine Softwareschicht dazwischen. Diese Schicht teilt die vorhandenen Ressourcen auf und gibt jeder Anwendung den Eindruck, sie habe ein eigenes System für sich allein. Ein einzelner kräftiger Server kann so die Arbeit von vielen kleinen übernehmen.

Fachbegriff: Virtualisierung

Virtualisierung bedeutet, Ressourcen wie Rechenleistung, Speicher oder Netzwerk durch eine Softwareschicht zu abstrahieren. Anwender sehen dann ein logisches System, das nicht mehr eins zu eins einem bestimmten Stück Hardware entspricht. Abstrahieren heißt hier: Die Details der darunterliegenden Technik werden verborgen.

Ein Vergleich aus dem Alltag hilft beim Einordnen. Ein Einfamilienhaus hat eigene Wände, einen eigenen Hausanschluss und eine eigene Heizung. Ein Mehrfamilienhaus teilt sich Fundament, Dach und Anschlüsse, doch jede Wohnung hat eine Tür mit Schloss. Die Bewohner sehen einander nicht in die Küche. Für Server bedeutet das: Man baut ein Gebäude und vermietet darin mehrere abgeschlossene Wohnungen. Wie dick die Wände zwischen den Wohnungen sind, unterscheidet die beiden Verfahren, um die es in diesem Buch geht. Virtuelle Maschinen bauen gewissermaßen vollständige Häuser in das Gebäude hinein. Container begnügen sich mit abgeschlossenen Wohnungen.

1.2 Virtuelle Maschinen

Eine virtuelle Maschine, kurz VM, ist ein Computer, der vollständig in Software existiert. Sie hat einen virtuellen Prozessor, virtuellen Arbeitsspeicher, eine virtuelle Festplatte und eine virtuelle Netzwerkkarte. Auf diesem simulierten Rechner läuft ein ganz normales Betriebssystem, das sogenannte Gastsystem. Es weiß nicht, dass die Hardware, auf der es zu laufen glaubt, in Wirklichkeit nur vorgespielt wird. Der Rechner, der die VMs beherbergt, heißt Host.

Jede VM bringt ihr eigenes Betriebssystem mit, und zwar einschließlich eines eigenen Kernels. Der Kernel ist der Kern eines Betriebssystems. Er verwaltet Prozesse, Speicher, Geräte und Dateien und vermittelt zwischen den Programmen und der Hardware. Weil jede VM ihren eigenen Kernel besitzt, kann auf demselben Host eine Debian-VM neben einer Windows-VM laufen, und beide stören einander nicht. Ein Fehler im Kernel der einen VM bleibt in dieser VM.

Die virtuelle Festplatte einer VM ist auf dem Host eine gewöhnliche, oft mehrere Gigabyte große Datei. Das macht VMs gut handhabbar: Man kann sie kopieren, sichern, auf einen anderen Host verschieben oder auf einen früheren Zustand zurücksetzen, indem man einen sogenannten Snapshot einspielt. Dem stehen die Kosten gegenüber. Jede VM startet ein vollständiges Betriebssystem, belegt dafür dauerhaft Arbeitsspeicher und braucht beim Start meist einige Sekunden bis Minuten.

1.3 Hypervisor Typ 1 und Typ 2

Die Softwareschicht, die virtuelle Maschinen erzeugt und verwaltet, heißt Hypervisor. Sie teilt dem Gast Prozessorzeit und Speicher zu, leitet Zugriffe auf Geräte weiter und sorgt dafür, dass keine VM in den Bereich einer anderen schaut. Moderne Prozessoren unterstützen das mit eigenen Erweiterungen (bei Intel heißt die Technik VT-x, bei AMD AMD-V). Ohne diese Unterstützung wären VMs spürbar langsamer.

Fachbegriff: Hypervisor

Ein Hypervisor ist Software, die virtuelle Maschinen erzeugt, ausführt und gegeneinander abschottet. Man unterscheidet zwei Bauarten, je nachdem, ob der Hypervisor direkt auf der Hardware oder auf einem normalen Betriebssystem läuft.

Ein *Hypervisor vom Typ 1* läuft direkt auf der Hardware, ohne ein gewöhnliches Betriebssystem darunter. Man spricht deshalb auch von „bare metal", also vom blanken Metall. Er ist die unterste Schicht und dafür gebaut, VMs zu betreiben. Typ-1-Hypervisor setzt man auf Servern in Rechenzentren ein. Bekannte Vertreter sind VMware ESXi, Microsoft Hyper-V, Xen und KVM. KVM (Kernel-based Virtual Machine) ist ein Teil des Linux-Kernels. Ein Linux-Server mit KVM ist damit selbst der Hypervisor.

Ein *Hypervisor vom Typ 2* läuft als gewöhnliches Programm auf einem vollwertigen Betriebssystem, etwa unter Windows, macOS oder einem Linux-Desktop. Das Betriebssystem des Hosts sitzt zwischen Hardware und Hypervisor und verursacht zusätzlichen Aufwand. Typ-2-Hypervisor eignen sich gut für Entwickler und zum Lernen, weil man sie wie jedes andere Programm installiert. Beispiele sind Oracle VirtualBox und VMware Workstation.

	Typ 1	Typ 2
Läuft auf	direkt auf der Hardware	auf einem Host-Betriebssystem
Typischer Einsatz	Rechenzentrum, Produktivbetrieb	Entwicklung, Tests, Lernumgebung
Leistung	hoch, wenig Zusatzaufwand	etwas geringer, wegen der Zwischenschicht
Beispiele	ESXi, Hyper-V, Xen, KVM	VirtualBox, VMware Workstation

Hinweis

Wenn Sie dieses Buch begleitend am eigenen Rechner durcharbeiten, genügt eine lokale Virtualisierungslösung. Auf einem Windows-Rechner können Sie beispielsweise Hyper-V mit einer Debian-VM verwenden, unter Li-

nux eignet sich KVM mit dem Programm `virt-manager`. Was Sie für das Übungssystem brauchen, beschreibt Kapitel 4.

1.4 Containertechnologie

Container gehen einen anderen Weg. Sie simulieren keinen Rechner. Stattdessen läuft eine Anwendung als ganz gewöhnlicher Prozess direkt auf dem Betriebssystem des Hosts, bekommt aber vom Kernel eine eingeschränkte Sicht auf das System. Der Prozess sieht nur seine eigenen Unterprozesse, ein eigenes Dateisystem und ein eigenes Netzwerk. Alles andere bleibt ihm verborgen. Von innen fühlt sich das an wie ein eigener kleiner Rechner. Von außen betrachtet ist es ein Prozess unter vielen.

Fachbegriff: Container

Ein Container ist ein Prozess (oder eine Gruppe von Prozessen), der vom Betriebssystemkern so abgeschottet wird, dass er ein eigenes Dateisystem, eigene Prozesse und ein eigenes Netzwerk zu haben scheint. Alle Container eines Hosts teilen sich denselben Kernel.

Das Dateisystem des Containers enthält alles, was die Anwendung zum Laufen braucht: das Programm selbst, seine Bibliotheken und die nötigen Hilfsdateien. Ein Betriebssystemkern gehört nicht dazu, den leiht sich der Container vom Host. Deshalb sind Container klein. Das Grundsystem eines schlanken Containers umfasst oft nur wenige Megabyte, während eine VM mehrere Gigabyte belegt. Ein Container startet, ohne ein Betriebssystem hochzufahren. Er startet einen Prozess, und das dauert meist nur Sekundenbruchteile.

Die Idee ist älter, als viele denken. Schon 1979 kam mit dem Befehl `chroot` ein Mittel in Unix, das einem Prozess ein anderes Wurzelverzeichnis vorspielt. Später folgten FreeBSD Jails, Solaris Zones und LXC unter Linux. Docker hat diese Technik nicht erfunden. Der Beitrag von Docker liegt darin, sie so bedienbar zu machen, dass heute auch Einsteiger nach wenigen Minuten einen Webserver im Container betreiben. Kapitel 3 erzählt diese Entwicklung im Zusammenhang.

1.5 Unterschiede zwischen VMs und Containern

Die folgende Abbildung stellt beide Ansätze gegenüber. Es kommt darauf an, was jeweils zwischen der Hardware und der Anwendung liegt.

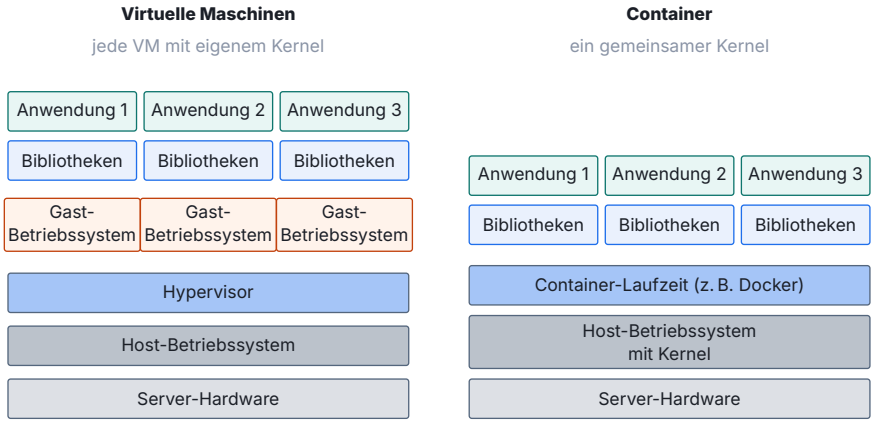


Abbildung 1.1: Virtuelle Maschinen und Container im Vergleich

Auf der linken Seite trägt jede VM ein komplettes Gastsystem mit sich herum. Rechts teilen sich alle Container den Kernel des Hosts und bringen nur die Dateien mit, die ihre Anwendung braucht. Daraus ergeben sich die praktischen Unterschiede, die die Tabelle zusammenstellt.

Merkmal	Virtuelle Maschine	Container
Kernel	eigener Kernel pro VM	Kernel des Hosts wird geteilt
Startzeit	Sekunden bis Minuten	meist Sekundenbruchteile
Größe	mehrere Gigabyte	oft nur wenige Megabyte bis einige hundert
Speicherbedarf	fest für das Gastsystem reserviert	nur, was die Anwendung nutzt
Gastsysteme	beliebig (Linux, Windows, BSD)	nur Systeme mit dem Kernel des Hosts
Isolation	sehr stark, eigene Hardware-Illusion	gut, aber durch gemeinsamen Kernel schwächer
Dichte pro Host	dutzende	oft hunderte

1.6 Vorteile und Nachteile

Die Stärken von Containern ergeben sich unmittelbar aus ihrer Bauweise. Weil sie so wenig Ballast mitschleppen, laufen auf einem Host weit mehr Container als VMs. Sie starten schnell, was beim Entwickeln und beim Automatisieren viel Wartezeit spart. Außerdem lassen sich Container reproduzierbar beschreiben: Die Anleitung, wie ein Container aufgebaut ist, steht in einer kleinen Textdatei. Wer diese Datei hat, bekommt auf jedem Linux-Rechner dieselbe Umgebung. Der Satz „Bei mir läuft es aber“ verliert damit viel von seinem Schrecken.

Dem stehen Einschränkungen gegenüber. Der gemeinsame Kernel ist zugleich die Schwachstelle. Gelingt es einem Angreifer, aus einem Container auszubrechen und den Kernel anzugreifen, betrifft das den ganzen Host und damit alle anderen Container. Bei VMs müsste er zusätzlich den Hypervisor überwinden, was deutlich schwerer ist. Zudem kann ein Linux-Container nur dort laufen, wo ein Linux-Kernel vorhanden ist. Ein Windows-Programm lässt sich nicht in einem Linux-Container betreiben, und ein Container mit einem anderen Kernel als dem des Hosts ist nicht möglich.

Docker Desktop und der unsichtbare Linux-Kernel

Wer Docker unter Windows oder macOS nutzt, hat scheinbar Linux-Container auf einem fremden Betriebssystem laufen. Das funktioniert nur, weil Docker Desktop im Hintergrund eine kleine Linux-VM startet und die Container darin betreibt. Der Kernel-Grundsatz gilt also weiter. In diesem Buch arbeiten wir ausschließlich mit echten Linux-Servern, auf denen Docker direkt aufsetzt.

Virtuelle Maschinen behalten deshalb ihren festen Platz. Sie sind die richtige Wahl, wenn Sie ein anderes Betriebssystem brauchen als das des Hosts, wenn zwei Mandanten strikt getrennt bleiben müssen oder wenn eine Anwendung einen ganzen Rechner mit allen Eigenheiten erwartet. Ihr Nachteil sind Größe, Startzeit und der dauerhaft reservierte Arbeitsspeicher.

1.7 Typische Einsatzgebiete

In der Praxis schließen sich beide Techniken nicht aus. Viele Unternehmen betreiben ihre Docker-Hosts selbst als virtuelle Maschinen. Die VM sorgt für die starke Trennung nach außen, die Container darin für die schlanke Verwaltung der Anwendungen. Das ist auch die Umgebung, die dieses Buch voraussetzt: eine Linux-VM oder ein Linux-Server, auf dem Docker läuft.

Container finden sich vor allem dort, wo viele kleine, gleichartige Dienste zu betreiben sind. Dazu gehören Webserver und Reverse-Proxys, Datenbanken für Entwicklung und Tests, Anwendungen, die aus vielen kleinen Diensten bestehen (sogenannte Microservices), und die automatischen Abläufe, mit denen Entwickler Software bauen und prüfen. Auch im Homelab sind sie beliebt, weil sich dort Programme wie Medienserver, Passwortverwaltungen oder Wikis in Minuten aufsetzen lassen, ohne das Hostsystem mit Abhängigkeiten zu füllen. Räumt man später auf, genügt es, den Container zu löschen. Auf dem Host bleibt nichts zurück.

KAPITEL 2

Container verstehen

Docker gilt vielen als Magie: Ein Befehl, und ein fertiger Webserver läuft. Dahinter steckt eine Handvoll Funktionen des Linux-Kernels, die Docker geschickt zusammenschaltet. Dieses Kapitel öffnet die Kiste. Sie lernen, was ein Container im Betriebssystem tatsächlich ist, und führen dazu kleine Versuche durch, für die Sie Docker noch gar nicht brauchen. Ein Debian-System mit Root-Rechten per `sudo` genügt.

2.1 Was ist ein Container?

Das Wort Container weckt die Vorstellung einer kleinen, geschlossenen Kiste, in der etwas steckt. Technisch trifft das nur zum Teil zu. Im Kernel gibt es den Begriff „Container“ gar nicht. Es gibt Prozesse, und es gibt Funktionen, mit denen sich Prozesse voneinander abschirmen lassen. Ein Container ist das Ergebnis, wenn man einen Prozess mit diesen Funktionen einhüllt.

Ein Beispiel macht das greifbar. Startet man einen nginx-Webserver in einem Container, dann läuft auf dem Host ein Prozess namens `nginx`. Er taucht in der Prozessliste des Hosts auf wie jedes andere Programm. Der Unterschied liegt in dem, was dieser Prozess sieht. Er bekommt ein eigenes Dateisystem, in dem nur nginx und seine Dateien liegen. Er sieht nur sich selbst und seine Kindprozesse, und er hat ein eigenes Netzwerk mit einer eigenen IP-Adresse. Außerdem lässt sich festlegen, wie viel Rechenzeit und Speicher er höchstens verbrauchen darf.

Container sind keine kleinen VMs

Ein häufiger Denkfehler besteht darin, einen Container wie eine winzige virtuelle Maschine zu behandeln, in die man sich einloggt, in der man Software installiert und die man monatelang laufen lässt. Ein Container ist besser als Hülle um genau einen Prozess zu verstehen. Endet der Prozess, endet der Container. Diese Sichtweise erklärt viel von dem, was Docker später tut und lässt.

2.2 Images und Container

Zwei Begriffe begleiten Sie durch das ganze Buch: Image und Container. Sie werden oft vermischt, bezeichnen aber verschiedene Dinge.

Ein *Image* ist eine schreibgeschützte Vorlage. Es enthält ein Dateisystem mit allem, was eine Anwendung braucht, dazu ein paar Angaben darüber, welcher Befehl beim Start ausgeführt werden soll. Ein Image läuft nicht. Es liegt auf der Platte und wartet, ähnlich wie ein Installationsprogramm, das noch nicht gestartet wurde.

Ein *Container* entsteht, wenn man aus einem Image eine Instanz erzeugt. Docker legt dann über das schreibgeschützte Image eine dünne beschreibbare Schicht und startet den Prozess. Aus einem einzigen Image lassen sich beliebig viele Container erzeugen, und jeder bekommt seine eigene beschreibbare Schicht. Änderungen in einem Container berühren weder das Image noch die anderen Container.

Fachbegriff: Image und Container

Ein Image ist die unveränderliche Vorlage, ein Container die ausgeführte (oder gestoppte) Instanz davon. Wer Programmiersprachen kennt, kann an Klasse und Objekt denken: Aus einer Klasse entstehen beliebig viele Objekte.

Daraus folgt eine Eigenschaft, die Einsteiger zunächst überrascht: Was im Container geschrieben wird, gehört zu diesem Container und verschwindet mit ihm.

Löscht man den Container, ist auch seine beschreibbare Schicht weg. Daten, die bleiben sollen, gehören in Volumes, ein Thema von Kapitel 10.

2.3 Container-Lifecycle

Ein Container durchläuft im Betrieb mehrere Zustände. Wer diese Zustände kennt, versteht auch, was Befehle wie `docker start` und `docker stop` bewirken.

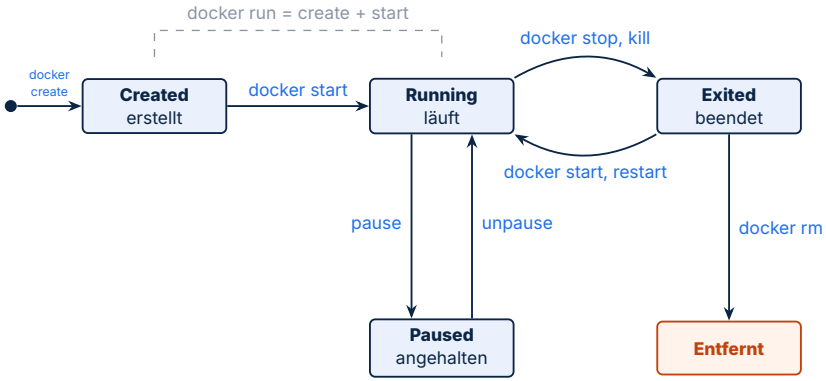


Abbildung 2.1: Zustände eines Containers und die Befehle für die Übergänge

Ein neu erzeugter Container steht zunächst im Zustand *created*. Alle Ressourcen sind vorbereitet, aber noch läuft kein Prozess. Mit `docker start` wechselt er nach *running*. Dort arbeitet der Hauptprozess, den das Image vorgibt. Im Alltag überspringt man den ersten Schritt meistens: Der Befehl `docker run` erzeugt den Container und startet ihn in einem Zug.

Endet der Hauptprozess, sei es, weil er fertig ist, abgestürzt ist oder von Docker beendet wurde, geht der Container nach *exited*. Er läuft dann nicht mehr, existiert aber weiter, samt seiner beschreibbaren Schicht. Das ist wichtig: Ein gestoppter Container belegt noch Platz und lässt sich mit `docker start` wieder aufwecken. Erst `docker rm` entfernt ihn endgültig.

Dazu kommen weitere Zustände. Mit `docker pause` lässt sich ein laufender Container einfrieren (*paused*), alle seine Prozesse stehen dann still, bis `docker`

`unpause` sie wieder freigibt. Der Zustand *restarting* tritt auf, wenn eine Neustartregel den Container gerade wieder hochfährt. Kapitel 9 zeigt, wie man solche Regeln festlegt.

> **Vorschau**
>
> Alle Befehle aus diesem Abschnitt führen Sie in Kapitel 6 selbst aus. Wenn Sie dort angekommen sind, lohnt es sich, nach jedem Befehl `docker ps -a` aufzurufen und den Zustand mit der Abbildung zu vergleichen.

2.4 Isolation und Ressourcenverwaltung

Zwei Fragen bestimmen, ob ein Prozess in einer Wohnung lebt oder im offenen Großraumbüro. Die erste lautet: Was darf der Prozess sehen? Die zweite: Wie viel darf er verbrauchen? Der Linux-Kernel beantwortet sie mit zwei getrennten Mechanismen. *Namespaces* schränken die Sicht ein, *Control Groups* (kurz cgroups) begrenzen den Verbrauch. Ein Container ist im Kern die Kombination aus beidem, ergänzt um ein eigenes Dateisystem, das aus dem Image stammt.

2.5 Linux Namespaces

Ein Namespace verpackt eine Systemressource so, dass die Prozesse darin eine eigene Version davon sehen. Der Kernel kennt mehrere Arten, die jeweils eine andere Ressource trennen.

Namespace	Schalter bei `unshare`	Was er trennt
mnt	`--mount`	Einhängepunkte, also die Sicht auf das Dateisystem
pid	`--pid`	Prozessnummern; der erste Prozess bekommt die Nummer 1
net	`--net`	Netzwerkkarten, IP-Adressen, Routen und Ports
uts	`--uts`	Hostname und Domainname
ipc	`--ipc`	gemeinsamer Speicher und Nachrichtenwarteschlangen
user	`--user`	Benutzer- und Gruppennummern
cgroup	`--cgroup`	die Sicht auf die cgroup-Hierarchie
time	`--time`	Zeitversatz der Systemuhren

Docker erzeugt für jeden Container standardmäßig eigene mnt-, pid-, net-, uts- und ipc-Namespaces. Auf Hosts mit cgroup v2 kommt ein eigener cgroup-Namespace hinzu. Der user-Namespace ist in der Grundeinstellung nicht aktiv. Das hat Folgen für die Sicherheit, auf die Kapitel 18 zurückkommt.

Sie können jeden dieser Namespaces ohne Docker ausprobieren. Das Programm `unshare` startet einen Befehl in neuen Namespaces. Der erste Versuch trennt den Hostnamen.

```
$ hostname
docker01
$ sudo unshare --uts bash
# hostname testhost
# hostname
testhost
# exit
$ hostname
```

```
docker01
```

Zeilen mit `$` gibt ein normaler Benutzer ein, Zeilen mit `#` stehen für eine Root-Shell. Die Zeichen selbst tippen Sie nicht mit. Innerhalb der neuen Shell hat sich der Hostname geändert. Nach dem Verlassen ist der Host unverändert, denn die Änderung betraf nur den uts-Namespace der Shell. Genau so verhält sich ein Container: Er darf seinen Hostnamen setzen, ohne den Host zu berühren.

Der zweite Versuch trennt die Prozessliste.

```
$ sudo unshare --pid --fork --mount-proc bash
# ps aux
USER   PID %CPU %MEM    VSZ   RSS TTY   STAT START   TIME COMMAND
root     1  0.0  0.1   8264  5120 pts/0 S    10:15   0:00 bash
root     2  0.0  0.1  10072  1536 pts/0 R+   10:15   0:00 ps aux
# exit
```

Im neuen pid-Namespace trägt die Shell die Prozessnummer 1, und von den Hunderten Prozessen des Hosts ist nichts zu sehen. Die Schalter `--fork` und `--mount-proc` sind nötig, damit die Shell tatsächlich im neuen Namespace startet und das Verzeichnis `/proc`, aus dem `ps` seine Daten liest, zum neuen Namespace passt. Die genauen Zahlen in Ihrer Ausgabe weichen ab.

Der dritte Versuch zeigt das Netzwerk.

```
$ sudo unshare --net ip link
1: lo: <LOOPBACK> mtu 65536 qdisc noop state DOWN mode DEFAULT group
    ↪ default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
```

Im frischen net-Namespace existiert nur eine einzige Schnittstelle, `lo`, und die ist noch nicht einmal eingeschaltet. Die Netzwerkkarte des Hosts ist unsichtbar. Docker verbindet jeden Container anschließend über ein virtuelles Kabel mit einem virtuellen Switch auf dem Host. Wie das aussieht, steht in Kapitel 11.

Mit `lsns` listet man alle Namespaces auf, die gerade auf dem System existieren. Sobald später Container laufen, tauchen dort für jeden Container neue Einträge auf.

2.6 Control Groups (cgroups)

Namespaces sorgen dafür, dass ein Prozess nicht sieht, was ihn nichts angeht. Sie verhindern aber nicht, dass er den ganzen Arbeitsspeicher auffrisst. Dafür gibt es Control Groups. Eine cgroup ist eine Gruppe von Prozessen, für die der Kernel Grenzen festlegt und Verbrauchswerte mitzählt. Mögliche Grenzen betreffen Prozessorzeit, Arbeitsspeicher, die Zahl der Prozesse und die Geschwindigkeit von Plattenzugriffen.

Fachbegriff: Control Group (cgroup)

Eine cgroup fasst Prozesse zusammen und ordnet ihnen Ressourcenlimits zu. Der Kernel verwaltet cgroups in einer baumartigen Struktur, die als Dateisystem unter `/sys/fs/cgroup` sichtbar ist.

Es gibt zwei Generationen dieser Technik, cgroup v1 und cgroup v2. Moderne Distributionen wie Debian 13 und aktuelle Ubuntu-LTS-Versionen verwenden standardmäßig v2. Welche Version Ihr System nutzt, verrät ein einzelner Befehl.

```
$ stat -fc %T /sys/fs/cgroup
cgroup2fs
```

Steht dort `cgroup2fs`, arbeitet das System mit v2. Eine Ausgabe wie `tmpfs` deutet auf v1 hin. Welche Ressourcen sich begrenzen lassen, steht in einer Datei, die den Namen `cgroup.controllers` trägt.

```
$ cat /sys/fs/cgroup/cgroup.controllers
cpuset cpu io memory hugetlb pids rdma misc
```

Die Liste kann je nach Kernel kürzer oder länger sein. Wichtig sind für uns `cpu`, `memory`, `io` und `pids`. Jeder Prozess gehört zu genau einer cgroup. Wo die eigene Shell eingeordnet ist, zeigt `/proc/self/cgroup`.

```
$ cat /proc/self/cgroup
0::/user.slice/user-1000.slice/session-3.scope
```

Die Zahlen in Ihrer Ausgabe unterscheiden sich. Der Pfad zeigt, dass `systemd` die Sitzung in eine cgroup gesteckt hat. Nach demselben Prinzip legt Docker für jeden Container eine eigene cgroup an. Auf einem System mit `systemd` und cgroup v2 verwendet Docker dazu in der Grundeinstellung den `systemd`-Treiber, sodass die Container als Einheiten mit Namen wie `docker-<ID>.scope` erscheinen. Wenn Sie in Kapitel 9 einen Container mit `--memory` begrenzen, schreibt Docker den Wert in genau diese cgroup. Der Kernel setzt ihn durch.

2.7 Union Filesystems

Das dritte Bauteil betrifft das Dateisystem. Ein Image besteht aus mehreren übereinanderliegenden Schichten, den *Layern*. Die unterste enthält ein minimales Grundsystem, darüber liegen Schichten mit installierten Programmen und der Anwendung. Alle diese Schichten sind schreibgeschützt. Der Container soll sie trotzdem so sehen, als bildeten sie ein einziges Dateisystem, in dem sich Dateien ändern lassen.

Diese Aufgabe übernimmt ein Union-Dateisystem. Unter Linux ist *OverlayFS* das Mittel der Wahl, das Docker verwendet. OverlayFS legt mehrere Verzeichnisse übereinander und zeigt das Ergebnis als eines. Die schreibgeschützten Schichten heißen *lower*, die beschreibbare *upper*, und die zusammengeführte Sicht heißt *merged*. Ein viertes Verzeichnis, *work*, braucht OverlayFS für interne Zwecke.

Auch das lässt sich von Hand nachbauen. Der Versuch legt vier Verzeichnisse an, hängt sie als OverlayFS ein und verändert danach Dateien.

```
$ mkdir -p ~/overlay-demo/lower ~/overlay-demo/upper ~/overlay-demo/
    ↪ work ~/overlay-demo/merged
$ cd ~/overlay-demo
$ echo "Datei aus dem Image" > lower/info.txt
$ echo "zweite Datei" > lower/rest.txt
$ sudo mount -t overlay overlay -o lowerdir=$PWD/lower,upperdir=$PWD/
    ↪ upper,workdir=$PWD/work $PWD/merged
$ ls merged
info.txt  rest.txt
```

Die zusammengeführte Sicht zeigt beide Dateien aus `lower`. Nun wird eine Datei geändert.

```
$ echo "geändert im Container" | sudo tee merged/info.txt
$ cat lower/info.txt
Datei aus dem Image
$ ls upper
info.txt
```

Die Änderung ist in `merged` sichtbar, in `lower` steht weiterhin der alte Text. Die neue Fassung landete in `upper`. Dieses Verfahren heißt *Copy-on-Write*: Eine Datei der unteren Schicht wird erst dann in die obere Schicht kopiert, wenn jemand sie ändern will. Bis dahin teilen sich alle Container dieselbe Originaldatei, was viel Speicherplatz spart.

Als letztes lösen wir eine Datei.

```
$ sudo rm merged/rest.txt
$ ls lower
info.txt  rest.txt
$ ls -l upper
total 4
-rw-r--r-- 1 root root   23 Nov 11 13:06 info.txt
c--------- 2 root root 0, 0 Nov 11 13:06 rest.txt
```

In `merged` ist `rest.txt` verschwunden, in `lower` aber nicht. Die Schicht `lower` ist ja schreibgeschützt. Stattdessen legt OverlayFS in `upper` eine Markierung an, die als Zeichengerät mit der Nummer 0,0 erscheint. Sie wird *Whiteout* genannt und teilt dem Dateisystem mit: Diese Datei gilt als gelöscht. Zum Aufräumen genügen zwei Befehle.

```
$ cd ~
$ sudo umount ~/overlay-demo/merged
$ sudo rm -r ~/overlay-demo
```

Docker arbeitet nach diesem Muster. Die Schichten eines Images spielen die Rolle von `lower`, die dünne beschreibbare Schicht eines Containers die von `upper`. Wo

Docker diese Verzeichnisse genau ablegt, hängt von der Docker-Version und vom eingestellten Image-Speicher ab. Kapitel 7 geht darauf ein.

2.8 Container und Linux-Kernel

Aus alldem ergibt sich ein Satz, den man nicht oft genug wiederholen kann: Alle Container eines Hosts teilen sich den Kernel des Hosts. Das lässt sich leicht überprüfen. Sobald Docker installiert ist, zeigt `uname -r` im Container dieselbe Kernel-Version an wie auf dem Host.

Der gemeinsame Kernel hat drei Konsequenzen. Erstens kann ein Linux-Container nur auf einem Linux-Kernel laufen, wie schon in Kapitel 1 beschrieben. Zweitens müssen Image und Host zur selben Prozessorarchitektur passen. Ein Image für `amd64` läuft nicht auf einem Rechner mit ARM-Prozessor, es sei denn, das Image wird für beide Architekturen angeboten. Drittens hängt die Sicherheit eines Containers an der Sicherheit des Kernels. Ein Update des Host-Kernels betrifft alle Container, und eine Lücke im Kernel kann alle gefährden.

> **Kernel-Updates gehören zum Containerbetrieb**
>
> Wer Docker produktiv betreibt, hält den Host genauso aktuell wie jeden anderen Server. Updates des Containers allein schützen nicht vor Fehlern im Kernel. In Kapitel 19 richten wir dafür einen Wartungsplan ein.

KAPITEL 3

Einführung in Docker

Die Technik, die Container ermöglicht, steckt im Linux-Kernel. Docker ist das Werkzeug darüber, das aus dieser Technik etwas macht, das sich bedienen lässt. Dieses Kapitel erklärt, woher Docker kommt, aus welchen Teilen es besteht und wie diese Teile miteinander reden. Mit dem Wissen daraus finden Sie sich später bei Fehlermeldungen leichter zurecht, denn die meisten Probleme entstehen an einer der Nahtstellen, die hier vorkommen.

3.1 Entwicklung und Hintergrund

Docker entstand bei einem Unternehmen namens dotCloud, das eine Plattform für Webanwendungen betrieb. Die Entwickler suchten nach einem Weg, Anwendungen samt ihren Abhängigkeiten so zu verpacken, dass sie überall gleich laufen. Im März 2013 stellten sie das Ergebnis als Open-Source-Projekt vor. Das Unternehmen benannte sich später nach dem Produkt um.

Die ersten Versionen setzten auf LXC auf, eine bereits vorhandene Sammlung von Werkzeugen für Linux-Container. 2014 ersetzte Docker LXC durch eine eigene Bibliothek namens libcontainer. Wichtiger als die Technik selbst war das Drumherum. Docker lieferte ein einfaches Kommando, ein verständliches Dateiformat zum Beschreiben von Images und eine öffentliche Sammelstelle, über die man fertige Images beziehen konnte. Plötzlich genügten wenige Zeilen, um eine Datenbank oder einen Webserver zu starten.

Mit der Verbreitung wuchs der Wunsch nach offenen Standards. 2015 gründeten mehrere Firmen unter dem Dach der Linux Foundation die Open Container Initiative, kurz OCI. Sie beschreibt, wie ein Container-Image aussehen muss und wie eine Laufzeitumgebung es ausführt. Docker steuerte seine Laufzeit `runc` bei, die bis heute von vielen Werkzeugen verwendet wird. Die Komponente `containerd` wurde 2017 an die Cloud Native Computing Foundation übergeben. Ein Docker-Image ist deshalb kein proprietäres Format. Es lässt sich auch mit anderen Programmen wie Podman ausführen.

Fachbegriff: Docker, Docker Engine und Docker Desktop

Der Name Docker bezeichnet mehrere Dinge. *Docker Engine* ist die Software, die auf einem Linux-Server Container baut und ausführt, und sie ist das Thema dieses Buches. *Docker Desktop* ist ein Programm für Windows, macOS und Linux-Arbeitsplätze mit grafischer Oberfläche, das die Engine in einer VM mitbringt. Die Engine steht unter der Apache-Lizenz 2.0. Für Docker Desktop gelten eigene Lizenzbedingungen, und in größeren Unternehmen ist es kostenpflichtig.

3.2 Docker-Architektur

Docker arbeitet nach dem Client-Server-Prinzip. Zwei Programme sind beteiligt. Der *Docker-Client* nimmt Ihre Befehle entgegen. Der *Docker-Daemon* führt sie aus. Beide können auf demselben Rechner laufen, müssen es aber nicht. Hinzu kommt eine dritte Instanz, die *Registry*, ein Server, auf dem fertige Images lagern.

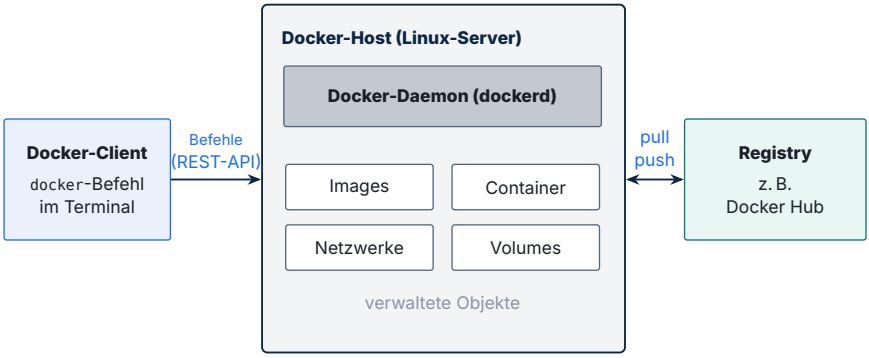


Abbildung 3.1: Docker-Client, Docker-Daemon und Registry

Der Ablauf ist stets derselbe. Sie geben im Terminal einen Befehl wie `docker run nginx` ein. Der Client macht daraus eine Anfrage und schickt sie an den Daemon. Der Daemon prüft, ob das Image `nginx` bereits lokal vorliegt. Wenn nicht, holt er es von der Registry. Anschließend erzeugt er daraus einen Container und startet ihn. Der Client zeigt nur an, was der Daemon zurückmeldet.

3.3 Docker Engine

Der Begriff Docker Engine fasst die Programme zusammen, die auf dem Server die eigentliche Arbeit leisten. Er umfasst den Daemon `dockerd`, die Komponente `containerd`, die den Lebenszyklus der Container verwaltet, und die Laufzeit `runc`, die den Container tatsächlich im Kernel anlegt. Die Aufgaben sind klar verteilt: `dockerd` versteht die Wünsche der Benutzer und verwaltet Images, Netzwerke und Volumes, `containerd` kümmert sich um Start, Stopp und Überwachung der

Container, und `runc` ruft die Kernel-Funktionen für Namespaces und cgroups auf.

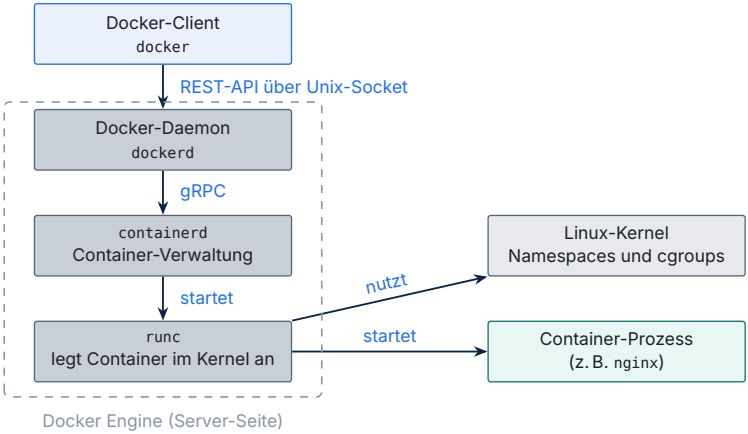


Abbildung 3.2: Zusammenspiel der Engine-Bausteine, vom Befehl bis zum laufenden Container

Der gestrichelte Rahmen in der Abbildung grenzt ab, was zur Engine zählt: Daemon, `containerd` und `runc`. Der Client gehört nicht dazu, er ist nur die Schnittstelle. Das hat einen praktischen Nutzen. Wenn der Daemon neu gestartet wird, können die laufenden Container bei richtiger Konfiguration weiterarbeiten, weil `containerd` sie hält. Und `runc` ist nur kurz im Spiel: Es legt den Container an, startet den Prozess und beendet sich dann. Ein kleines Hilfsprogramm namens `containerd-shim` bleibt als Elternprozess des Containers zurück und leitet die Ausgabe weiter.

3.4 Docker-Daemon

Der Daemon heißt `dockerd` und läuft als Hintergrunddienst mit Root-Rechten. Das Wort Daemon bezeichnet unter Linux ein Programm, das ohne Terminal im Hintergrund arbeitet. Auf Debian und Ubuntu startet `systemd` den Dienst beim Hochfahren. Zum Docker-Dienst gehören zwei Einheiten, die Sie später mit `systemctl` ansprechen: `docker.service` für den Daemon selbst und `docker.socket` für die Schnittstelle, über die der Client ihn erreicht.

Der Daemon verwaltet alle Objekte, aus denen eine Docker-Umgebung besteht: Images, Container, Netzwerke und Volumes. Container-Daten, Volumes und Netzwerkeinstellungen legt er unter `/var/lib/docker` ab. Wo die Images liegen, hängt vom eingestellten Image-Speicher ab: Bei einer frischen Installation von Docker 29 verwaltet `containerd` sie unter `/var/lib/containerd`, ältere Installationen halten sie ebenfalls unter `/var/lib/docker`. Kapitel 7 erklärt den Unterschied. In beiden Verzeichnissen wächst mit der Zeit einiges an, deshalb sollten die Partitionen genug Platz bieten. Einstellungen des Daemons stehen in der Datei `/etc/docker/daemon.json`. Diese Datei existiert nach der Installation zunächst nicht. Man legt sie bei Bedarf selbst an, zum Beispiel um die Größe der Protokolle zu begrenzen.

> **Root-Rechte im Hintergrund**
>
> Der Daemon läuft als Root und erfüllt Aufträge von jedem, der ihn erreichen darf. Wer Docker bedienen kann, kann damit auch Dateien des Hosts lesen und verändern, zum Beispiel durch einen Container, der das Hauptverzeichnis einbindet. Abschalten lässt sich das nicht, es ergibt sich aus der Funktionsweise. Kapitel 18 zeigt, wie man damit umgeht.

3.5 Docker-Client

Das Programm `docker`, das Sie im Terminal aufrufen, ist der Client. Er enthält keine Containerlogik. Er übersetzt Befehle in Anfragen an den Daemon und gibt die Antworten aus. Deshalb kann ein Client auch einen Daemon auf einem anderen Rechner steuern. Docker spricht dann von verschiedenen *Kontexten*, die sich mit dem Befehl `docker context` verwalten lassen. Für dieses Buch genügt der Standardkontext, bei dem Client und Daemon auf demselben Server laufen.

Wer Docker benutzen will, braucht Zugriff auf den Daemon. In der Grundeinstellung ist das nur Root erlaubt, und der Client muss deshalb mit `sudo` aufgerufen werden. Lässt man einen Benutzer in die Gruppe `docker` aufnehmen, entfällt das `sudo`. Das ist bequem, doch es hat denselben Rang wie eine Root-Berechtigung, wie die Warnung im vorigen Abschnitt andeutet. Kapitel 5 behandelt diesen Punkt ausführlich.

3.6 Docker Registry und Docker Hub

Eine Registry ist ein Server, der Images speichert und ausliefert. Er funktioniert ähnlich wie ein Paketarchiv bei Linux-Distributionen, nur dass er ganze Images statt einzelner Programmpakete verwaltet. Die bekannteste Registry ist *Docker Hub*. Sie ist als Standardquelle eingetragen: Nennt man bei `docker pull` keinen anderen Server, sucht Docker dort.

Der Name eines Images folgt einem festen Schema, das Sie bald auswendig kennen werden.

```
registry.beispiel.de/team/webapp:1.4
```

Der erste Teil nennt die Registry. Er darf fehlen, dann gilt Docker Hub. Danach kommt der Namensraum, also meist ein Benutzer oder eine Firma, und dann der Name des Images. Hinter dem Doppelpunkt steht der *Tag*, der meist eine Version bezeichnet. Fehlt der Tag, ergänzt Docker automatisch `latest`. Das Image `nginx` ist daher eine Kurzform für `docker.io/library/nginx:latest`.

Auf Docker Hub gibt es besonders gepflegte Images, die *Docker Official Images*. Dazu zählen unter anderem nginx, MariaDB und PostgreSQL, die wir in diesem Buch verwenden. Daneben bieten Hersteller und Privatpersonen eigene Images an. Solche Images können fehlerhaft oder sogar schädlich sein, und deshalb sollte man bei fremden Quellen vorsichtig sein. Kapitel 18 zeigt, wie man Images prüft.

Neben Docker Hub gibt es weitere öffentliche Registries, etwa die GitHub Container Registry oder Quay. Firmen betreiben außerdem eigene, private Registries, um Images im internen Netz zu verteilen.

3.7 Kommunikation zwischen Client und Daemon

Zwischen Client und Daemon läuft eine ganz gewöhnliche Webschnittstelle, die REST-API. Der Client schickt HTTP-Anfragen, der Daemon antwortet mit Daten im JSON-Format. In der Grundeinstellung bleiben diese Anfragen auf dem Rechner und laufen über einen *Unix-Socket*, eine Datei im Dateisystem, die als Kommunikationskanal dient. Der Socket des Docker-Daemons heißt `/var/run/docker`.

`sock`. Wer diese Datei beschreiben darf, darf den Daemon steuern. Deshalb gehört sie standardmäßig Root und der Gruppe `docker`.

Sobald Docker installiert ist, können Sie die Schnittstelle selbst ansprechen, ohne den Client zu benutzen. Das Programm `curl` erledigt das mit einem Schalter für den Socket.

```
$ sudo curl --unix-socket /var/run/docker.sock http://localhost/
    ↪ version
```

Die Antwort ist ein JSON-Text mit Angaben zu Version und Betriebssystem. Der Client `docker version` fragt genau diese Schnittstelle ab und bereitet die Antwort lesbar auf. Das Beispiel zeigt, dass der Client nichts Besonderes kann. Jedes Programm, das HTTP über einen Unix-Socket spricht, kann Docker steuern.

Docker nicht ungeschützt ins Netz stellen

Der Daemon lässt sich auch über eine TCP-Verbindung erreichbar machen. Ohne Verschlüsselung und Anmeldung kann dann jeder im Netz, der den Port erreicht, Container starten und damit den Host übernehmen. Eine solche Einstellung gehört nicht in ein produktives Netz. Wer Docker aus der Ferne steuern will, nutzt am besten einen Kontext über SSH:

```
docker context create server1 --docker "host=ssh://admin@docker01"
```

Lernkontrolle zu Teil I

Die Aufgaben prüfen, ob Sie die Grundlagen aus den Kapiteln 1 bis 3 sicher beherrschen. Die Lösungen finden Sie im Anhang. Versuchen Sie zuerst, jede Aufgabe ohne Blick in die Lösung zu bearbeiten.

Verständnisfragen

V1. Erklären Sie in zwei Sätzen, warum Virtualisierung die Auslastung eines Servers verbessert.

V2. Worin unterscheidet sich ein Hypervisor vom Typ 1 von einem Hypervisor vom Typ 2? Nennen Sie für jeden Typ ein Beispiel.

V3. Warum startet ein Container in der Regel schneller als eine virtuelle Maschine?

V4. Was ist der Unterschied zwischen einem Image und einem Container?

V5. Welche Aufgabe haben Namespaces, und welche Aufgabe haben cgroups?

V6. Was geschieht mit den Daten in der beschreibbaren Schicht eines Containers, wenn man den Container löscht?

V7. Nennen Sie die drei Bausteine der Docker Engine und beschreiben Sie jeweils in einem Satz, was sie tun.

V8. Wie lautet der vollständige Name des Images, das man mit `nginx` meint?

Multiple-Choice-Aufgaben

Jeweils genau eine Antwort ist richtig.

M1. Welche Aussage über Container trifft zu?
(a) Jeder Container bringt einen eigenen Kernel mit.
(b) Alle Container eines Hosts teilen sich den Kernel des Hosts.
(c) Container benötigen immer einen Hypervisor.
(d) Linux-Container laufen nur auf Windows-Servern.

M2. Welcher Mechanismus des Kernels begrenzt den Arbeitsspeicher eines Containers?
(a) Ein Namespace
(b) OverlayFS
(c) Eine Control Group
(d) Eine Registry

M3. In welchem Zustand befindet sich ein Container, der erzeugt, aber noch nie gestartet wurde?
(a) created
(b) exited
(c) paused
(d) running

M4. Worüber spricht der Docker-Client in der Grundeinstellung mit dem Daemon?
(a) Über den Port 80
(b) Über die Datei `/etc/docker/daemon.json`
(c) Über Docker Hub
(d) Über den Unix-Socket `/var/run/docker.sock`

M5. Welchen Tag ergänzt Docker, wenn beim Image-Namen keiner angegeben ist?
(a) stable
(b) latest
(c) default
(d) current

M6. Welche Ausgabe zeigt `stat -fc %T /sys/fs/cgroup` auf einem System mit cgroup v2?
(a) tmpfs
(b) overlay
(c) cgroup2fs
(d) ext4

Praktische Aufgaben

P1. *uts-Namespace.* Starten Sie mit `unshare` eine Shell in einem neuen uts-Namespace, ändern Sie dort den Hostnamen auf `labor`, kontrollieren Sie ihn, verlassen Sie die Shell und prüfen Sie, ob der Hostname des Hosts unverändert ist.

P2. *pid-Namespace.* Zählen Sie auf dem Host die Zeilen von `ps aux` mit `ps aux | wc -l`. Starten Sie dann eine Shell in einem neuen pid-Namespace (mit `--fork` und `--mount-proc`) und zählen Sie dort erneut. Notieren Sie beide Zahlen und erklären Sie den Unterschied.

P3. *OverlayFS.* Bauen Sie die Umgebung aus Kapitel 2 nach. Legen Sie in `merged` eine neue Datei `neu.txt` an und stellen Sie fest, in welchem der Verzeichnisse `lower` und `upper` sie erscheint. Räumen Sie danach vollständig auf.

Fehlersuche

F1. Ein Teilnehmer startet `sudo unshare --pid --fork bash` und gibt `ps aux` ein. Er sieht Hunderte Prozesse und glaubt, der Namespace funktioniere nicht. Was hat er übersehen?

F2. Beim Einhängen von OverlayFS erscheint die Fehlermeldung `wrong fs type, bad option, bad superblock on overlay`. Der Befehl lautete:

```
sudo mount -t overlay overlay -o lowerdir=$PWD/lower,upperdir=$PWD/
    ↪ upper $PWD/merged
```

Was fehlt?

Szenario aus dem Betrieb

Ein kleines Büro betreibt drei interne Anwendungen auf drei zehn Jahre alten Servern. Anwendung A ist eine Wiki-Software für Linux, Anwendung B eine Aufgabenverwaltung für Linux, Anwendung C eine Branchensoftware, die nur unter Windows Server läuft. Die Geschäftsführung genehmigt einen einzigen neuen Server. Beschreiben Sie, wie Sie die drei Anwendungen darauf unterbringen würden, und begründen Sie Ihre Wahl zwischen virtuellen Maschinen und Containern.

TEIL II

Docker installieren und konfigurieren

KAPITEL 4

Vorbereitung der Linux-Umgebung

Docker läuft auf einem sauber eingerichteten Linux-System am zuverlässigsten. Wer bei Hostname, Benutzerrechten, Uhrzeit oder Speicherplatz schlampt, sucht später bei jeder Fehlermeldung an der falschen Stelle. Dieses Kapitel richtet das Übungssystem so ein, dass die folgenden Kapitel ohne Umwege funktionieren. Es beschreibt Debian 13 ausführlich und nennt für Ubuntu Server die Abweichungen.

4.1 Das Übungssystem

Als Übungssystem dient eine virtuelle Maschine mit Debian 13. Sie können dafür einen Typ-2-Hypervisor wie VirtualBox oder Hyper-V verwenden, die in Kapitel 1 vorgestellt wurden. Ein echter Server funktioniert genauso. Die folgenden Werte reichen für alle Projekte dieses Buches.

Eigenschaft	Empfehlung
Prozessor	2 virtuelle Kerne
Arbeitsspeicher	4 GB
Festplatte	40 GB, dynamisch wachsend
Netzwerk	Bridge-Modus, damit der Server eine Adresse im Heimnetz bekommt
Software bei der Installation	SSH-Server und Standard-Systemwerkzeuge, keine Desktopumgebung

Eine Desktopumgebung brauchen Sie nicht. Ein Server wird in der Praxis über die Kommandozeile und per SSH bedient, und genau das üben wir. Wenn Sie den SSH-Dienst bei der Installation mitinstalliert haben, können Sie sich vom eigenen Rechner aus anmelden und bequem Befehle aus dem Buch kopieren.

Zwei Namen legen wir für das ganze Buch fest, damit alle Beispiele zusammenpassen. Der Server heißt `docker01`, und der Administrator, mit dem Sie arbeiten, heißt `admin`. Wenn Sie bei der Installation andere Namen gewählt haben, ersetzen Sie sie in den Beispielen.

Prozessorarchitektur prüfen

Docker unterstützt auf Debian und Ubuntu die Architekturen amd64, armhf, arm64 und ppc64el (Ubuntu zusätzlich s390x). Welche Ihr System hat, verrät der Befehl `dpkg --print-architecture`. Auf einem normalen PC oder in einer VM auf einem PC steht dort `amd64`. Die Beispiele im Buch sind mit amd64 entstanden.

4.2 Debian 13 vorbereiten

Zuerst melden Sie sich als `admin` an, entweder an der Konsole der VM oder per SSH. Mit dem folgenden Befehl prüfen Sie, welches System läuft.

```
$ cat /etc/os-release
PRETTY_NAME="Debian GNU/Linux 13 (trixie)"
NAME="Debian GNU/Linux"
VERSION_ID="13"
VERSION="13 (trixie)"
VERSION_CODENAME=trixie
ID=debian
```

Die Zeile `VERSION_CODENAME=trixie` brauchen wir in Kapitel 5 wieder, denn das Docker-Repository wird nach dem Codenamen der Distribution ausgewählt.

4.2.1 sudo einrichten

Bei der Installation fragt Debian nach einem Passwort für den Benutzer `root`. Wenn Sie eines vergeben haben, ist das Programm `sudo` in der Grundeinstellung nicht installiert. `sudo` erlaubt es einem normalen Benutzer, einzelne Befehle mit Root-Rechten auszuführen, und bildet die Grundlage für alle Anleitungen in diesem Buch. Prüfen Sie zuerst, ob es vorhanden ist.

```
$ sudo --version
-bash: sudo: command not found
```

Erscheint diese Meldung, wechseln Sie mit `su` zum Benutzer Root, installieren `sudo` und nehmen `admin` in die gleichnamige Gruppe auf.

```
$ su -
Passwort:
# apt update
# apt install sudo
# usermod -aG sudo admin
# exit
```

Die Gruppenmitgliedschaft wirkt erst bei der nächsten Anmeldung. Melden Sie sich deshalb ab und wieder an. Danach testen Sie, ob `sudo` funktioniert.

```
$ sudo whoami
[sudo] Passwort für admin:
root
```

Dass der Befehl `whoami` unter `sudo` den Namen `root` ausgibt, bestätigt, dass `admin` Befehle mit Root-Rechten ausführen darf.

4.2.2 Hostname festlegen

Den Namen des Rechners setzen Sie mit `hostnamectl`. Das Programm gehört zu `systemd` und funktioniert auf Debian und Ubuntu gleich.

```
$ sudo hostnamectl set-hostname docker01
$ hostnamectl
 Static hostname: docker01
       Icon name: computer-vm
         Chassis: vm
Operating System: Debian GNU/Linux 13 (trixie)
          Kernel: Linux 6.12.x-amd64
    Architecture: x86-64
```

Die Ausgabe enthält weitere Zeilen, etwa zur Maschinen-ID. Die Kernel-Version hängt von Ihrer Installation ab. Damit der Rechner seinen eigenen Namen auflösen kann, sollte in der Datei `/etc/hosts` ein passender Eintrag stehen. Debian trägt dort bei der Installation den damals gewählten Namen ein. Öffnen Sie die Datei und ersetzen Sie den alten Namen durch `docker01`.

```
$ sudo nano /etc/hosts
127.0.0.1   localhost
127.0.1.1   docker01
```

Speichern Sie in `nano` mit [Strg]+[O] und verlassen Sie den Editor mit [Strg]+[X]. Die Zeilen darunter, die für IPv6 gedacht sind, lassen Sie unverändert.

4.3 Ubuntu Server vorbereiten

Ubuntu Server unterscheidet sich an wenigen Stellen von Debian. Der Installer legt den ersten Benutzer automatisch als Administrator an und nimmt ihn in die Gruppe `sudo` auf. Das Programm `sudo` ist daher schon vorhanden, und der Abschnitt zur Einrichtung entfällt. Docker unterstützt laut der offiziellen Installationsanleitung derzeit die Ubuntu-Versionen 22.04, 24.04, 25.10 und 26.04. Für Server nehmen Sie am besten eine Version mit Langzeitunterstützung (LTS), also 22.04, 24.04 oder 26.04.

```
$ cat /etc/os-release | head -n 5
PRETTY_NAME="Ubuntu 24.04.2 LTS"
NAME="Ubuntu"
```

```
VERSION_ID="24.04"
VERSION="24.04.2 LTS (Noble Numbat)"
VERSION_CODENAME=noble
```

Der Hostname lässt sich wie unter Debian mit `hostnamectl` setzen. Zwei weitere Unterschiede betreffen später die Firewall. Ubuntu bringt das Programm `ufw` mit, das aber ausgeschaltet ist. Und die Netzwerkkonfiguration liegt unter Ubuntu in Dateien im Verzeichnis `/etc/netplan`, bei Debian in der Regel in `/etc/network/interfaces` oder bei NetworkManager. Für das Übungssystem genügt in beiden Fällen die Adresse, die per DHCP vergeben wird.

Debian und Ubuntu in diesem Buch

Wo sich beide Systeme unterscheiden, steht der Unterschied im Text. Alles andere gilt für beide. Das betrifft insbesondere alle Docker-Befehle: Sobald Docker installiert ist, verhalten sich Debian und Ubuntu gleich.

4.4 System aktualisieren

Bevor Sie Docker installieren, bringen Sie das System auf den aktuellen Stand. Das schließt bekannte Sicherheitslücken und verhindert, dass die Docker-Pakete wegen veralteter Abhängigkeiten Probleme machen. Der Paketmanager heißt `apt`. Er arbeitet in zwei Schritten: Zuerst lädt er die Liste der verfügbaren Pakete, dann installiert er die Updates.

```
$ sudo apt update
$ sudo apt upgrade
```

Bei `apt upgrade` listet das Programm die Pakete auf und fragt nach. Bestätigen Sie mit [J] oder [Enter]. Wurde dabei der Kernel aktualisiert, läuft das System weiterhin mit dem alten Kernel, bis es neu startet. Das ist bei Containern besonders wichtig, denn alle Container nutzen den Kernel des Hosts. Starten Sie den Rechner deshalb nach einem Kernel-Update neu.

```
$ sudo reboot
```

Unter Ubuntu zeigt die Datei `/var/run/reboot-required` an, dass ein Neustart nötig ist. Unter Debian gibt es diese Datei in der Grundeinstellung nicht. Dort vergleichen Sie bei Bedarf die Ausgabe von `uname -r` mit dem zuletzt installierten Kernel-Paket.

4.5 Benutzer und Gruppen

Unter Linux gehört jeder Prozess einem Benutzer, und Dateien und Geräte schützen Gruppen. Den eigenen Benutzer und seine Gruppen zeigt der Befehl `id`.

```
$ id
uid=1000(admin) gid=1000(admin) groups=1000(admin),27(sudo),100(users
    ↪ )
```

Die Gruppen in Ihrer Ausgabe können abweichen. Wichtig ist, dass `sudo` dabei ist. Beim Installieren von Docker entsteht eine weitere Gruppe namens `docker`. Wer ihr angehört, darf den Docker-Daemon ansprechen, und das ist ein besonders weitreichendes Recht, wie Kapitel 3 ankündigte. Kapitel 5 geht darauf ein. Für den Moment genügt es zu wissen, wo man die eigenen Gruppen nachsieht.

Für alle Übungen legen wir ein gemeinsames Arbeitsverzeichnis an, in dem später die Projekte liegen. Es heißt `/srv/docker`. Das Verzeichnis `/srv` ist nach dem Dateisystemstandard für Daten gedacht, die der Server bereitstellt.

```
$ sudo mkdir -p /srv/docker
$ sudo chown admin:admin /srv/docker
$ ls -ld /srv/docker
drwxr-xr-x 2 admin admin 4096 Nov 11 10:02 /srv/docker
```

Die erste Zeile legt das Verzeichnis an, die zweite übergibt es an `admin`, damit Sie dort ohne `sudo` arbeiten können. Die Option `-p` sorgt dafür, dass fehlende Zwischenverzeichnisse mit angelegt werden und dass kein Fehler erscheint, wenn das Verzeichnis schon existiert.

4.6 Netzwerk überprüfen

Docker lädt bei fast jedem Schritt etwas aus dem Internet: Pakete bei der Installation, Images beim Start eines Containers. Ein stabiles Netzwerk mit funktionierender Namensauflösung ist deshalb Voraussetzung. Drei Prüfungen reichen aus. Die erste zeigt die Netzwerkschnittstellen und ihre Adressen.

```
$ ip -br address
lo               UNKNOWN        127.0.0.1/8 ::1/128
enp0s3           UP             192.168.100.10/24 fe80::a00:27ff:fe4e
    ↪ :66a1/64
```

Die Schnittstelle `lo` ist die Loopback-Schnittstelle, über die der Rechner mit sich selbst spricht. Die zweite Zeile zeigt die echte Netzwerkkarte. Ihr Name kann bei Ihnen anders lauten, etwa `eth0` oder `ens18`. Die Adresse in diesem Buch lautet `192.168.100.10`. Bei Ihnen hängt sie von Ihrem Netz ab. Die zweite Prüfung zeigt, über welchen Router der Rechner ins Internet kommt.

```
$ ip route
default via 192.168.100.1 dev enp0s3
192.168.100.0/24 dev enp0s3 proto kernel scope link src
    ↪ 192.168.100.10
```

Die Zeile mit `default` nennt den Standard-Gateway, hier `192.168.100.1`. Fehlt sie, kennt der Rechner keinen Weg ins Internet. Die dritte Prüfung testet gleich die Namensauflösung und eine echte Verbindung zum Docker-Server.

```
$ getent hosts download.docker.com
2600:9000:...  d2h6rq2oobu8v2.cloudfront.net download.docker.com
$ curl -sI https://download.docker.com/linux/debian/gpg | head -n 1
HTTP/2 200
```

Die genauen Adressen ändern sich ständig und sehen bei Ihnen anders aus. Entscheidend ist, dass `getent` überhaupt eine Adresse liefert und dass `curl` die Antwort `200` anzeigt. Ein `ping` eignet sich weniger für den Test, weil manche Netze ihn sperren, obwohl normale Verbindungen funktionieren.

Eine Voraussetzung wird oft übersehen: die Uhrzeit. Pakete werden über verschlüsselte Verbindungen geladen, und die Prüfung der Zertifikate schlägt fehl, wenn die Systemuhr stark falsch geht.

```
$ timedatectl
               Local time: Wed 2026-11-11 10:07:31 CET
           Universal time: Wed 2026-11-11 09:07:31 UTC
                 Time zone: Europe/Berlin (CET, +0100)
System clock synchronized: yes
              NTP service: active
```

Wichtig sind die letzten beiden Zeilen. Steht dort `System clock synchronized: yes`, gleicht das System die Uhr mit einem Zeitserver ab. Die Zeitzone stellen Sie bei Bedarf mit `sudo timedatectl set-timezone Europe/Berlin` ein.

4.7 Speicherplatz kontrollieren

Docker legt Container, Volumes und je nach Version auch die Images unter `/var/lib/docker` ab, bei frischen Installationen von Docker 29 liegen die Images unter `/var/lib/containerd`. In beiden Verzeichnissen wächst mit der Zeit einiges an. Ein einzelnes Image belegt je nach Inhalt von wenigen Megabyte bis zu mehreren Gigabyte, und die Protokolle der Container kommen hinzu. Prüfen Sie deshalb, wie viel Platz das System hat.

```
$ df -h /
Filesystem      Size  Used Avail Use% Mounted on
/dev/sda1        39G  2.6G   35G   7% /
$ free -h
               total        used        free      shared  buff/cache
    ↪   available
Mem:           3.8Gi       312Mi       3.1Gi       1.0Mi       589Mi
    ↪       3.5Gi
Swap:          1.0Gi          0B       1.0Gi
$ lsblk
NAME   MAJ:MIN RM  SIZE RO TYPE MOUNTPOINTS
sda      8:0    0   40G  0 disk├─
```

```
sda1   8:1    0   39G  0 part /├─
sda2   8:2    0    1K  0 part└─
sda5   8:5    0  975M  0 part [SWAP]
```

`df -h` zeigt den freien Platz in lesbaren Einheiten, `free -h` den Arbeitsspeicher, und `lsblk` den Aufbau der Datenträger. Ihre Zahlen unterscheiden sich. Als Faustregel sollten für die Übungen mindestens 15 GB frei sein.

Das Dateisystem darf nicht volllaufen

Läuft die Partition mit `/var/lib/docker` oder `/var/lib/containerd` voll, bleiben Container stehen, Datenbanken melden Fehler, und im schlimmsten Fall lässt sich der Docker-Daemon nicht mehr starten. Auf echten Servern liegen diese Verzeichnisse deshalb oft auf einer eigenen Partition oder einem eigenen Datenträger, damit ein volles Docker-Verzeichnis nicht das Betriebssystem blockiert. Kapitel 19 zeigt, wie Sie den Verbrauch überwachen.

KAPITEL 5

Docker installieren

Dieses Kapitel installiert Docker Engine aus dem offiziellen Docker-Repository. Das ist der Weg, den die Docker-Dokumentation für Debian und Ubuntu empfiehlt, und er hat den Vorteil, dass spätere Updates über den gewohnten Paketmanager kommen. Alle Schritte gelten für Debian 13 und Ubuntu Server. Wo sich beide unterscheiden, stehen zwei getrennte Blöcke.

5.1 Installationswege im Überblick

Es gibt vier Möglichkeiten, Docker auf einen Linux-Server zu bringen.

Die erste und empfohlene Möglichkeit ist das *offizielle Docker-Repository*. Man trägt es einmal in die Paketverwaltung ein, danach installiert und aktualisiert `apt` die Docker-Pakete wie jedes andere Programm. Die zweite Möglichkeit sind die *einzelnen Paketdateien*, die man von Hand herunterlädt und mit `dpkg` einspielt. Sie ist für Rechner ohne Internetzugang gedacht, bringt aber den Nachteil mit, dass man jedes Update ebenfalls von Hand holen muss. Die dritte ist ein *Installationsskript* von Docker, das die Distribution erkennt und alles automatisch einrichtet. Die Dokumentation empfiehlt es ausdrücklich nur für Test- und Entwicklungsumgebungen, weil man dabei kaum einstellen kann, was passiert. Die vierte Möglichkeit sind die Docker-Pakete der Distribution selbst, etwa das Paket `docker.io` unter Debian und Ubuntu. Sie werden von den Paketbetreuern der Distribution gebaut und können sich in Konfiguration und Versionsstand vom offiziellen Docker unterscheiden.

Wir verwenden die erste Möglichkeit. Alle Anleitungen und Ausgaben dieses Buches gehen davon aus.

5.2 Alte Pakete entfernen

Wenn auf dem System schon einmal Docker aus den Paketquellen der Distribution installiert wurde, kollidieren diese Pakete mit den offiziellen. Auch einzeln installierte Pakete für `containerd` und `runc` stören, denn Docker bringt beide in einem eigenen Paket mit. Der folgende Befehl entfernt alle bekannten Konfliktpakete, die vorhanden sind.

```
$ sudo apt remove $(dpkg --get-selections docker.io docker-compose
    ↪ docker-compose-v2 docker-doc docker-buildx podman-docker
    ↪ containerd runc | cut -f1)
```

`dpkg --get-selections` listet nur die Pakete aus dieser Namensliste auf, die tatsächlich installiert sind, und `cut -f1` schneidet davon die Paketnamen aus. Auf einem frisch installierten System meldet `apt`, dass es keines der Pakete gibt. Das ist in Ordnung. Vorhandene Images, Container und Volumes unter `/var/lib/docker` bleiben beim Entfernen der alten Pakete erhalten.

5.3 Das Docker-Repository einrichten

Die Einrichtung des Repositorys besteht aus zwei Teilen. Zuerst holt man den Schlüssel, mit dem Docker seine Pakete signiert. `apt` prüft damit bei jedem Download, ob ein Paket tatsächlich von Docker stammt und unterwegs nicht verändert wurde. Danach trägt man die Adresse des Repositorys in die Paketquellen ein.

5.3.1 Debian 13

```
$ sudo apt update
$ sudo apt install ca-certificates curl
$ sudo install -m 0755 -d /etc/apt/keyrings
```

```
$ sudo curl -fsSL https://download.docker.com/linux/debian/gpg -o /
    ↪ etc/apt/keyrings/docker.asc
$ sudo chmod a+r /etc/apt/keyrings/docker.asc
```

Die erste Zeile aktualisiert die Paketlisten, die zweite installiert zwei Hilfsprogramme: `ca-certificates` liefert die Zertifikate für verschlüsselte Verbindungen, `curl` lädt Dateien aus dem Netz. Die dritte Zeile legt das Verzeichnis `/etc/apt/keyrings` an, in dem `apt` Schlüssel erwartet. Die vierte lädt den Docker-Schlüssel dorthin. Bei `curl` bedeutet `-f`, dass Fehler als Fehler gemeldet werden, `-s` unterdrückt die Fortschrittsanzeige, `-S` zeigt trotzdem Fehlermeldungen, und `-L` folgt Weiterleitungen. Die letzte Zeile macht die Schlüsseldatei für alle lesbar, damit `apt` sie benutzen kann.

Nun folgt der Eintrag in die Paketquellen.

```
$ sudo tee /etc/apt/sources.list.d/docker.sources <<EOF
Types: deb
URIs: https://download.docker.com/linux/debian
Suites: $(. /etc/os-release && echo "$VERSION_CODENAME")
Components: stable
Architectures: $(dpkg --print-architecture)
Signed-By: /etc/apt/keyrings/docker.asc
EOF
$ sudo apt update
```

Der Befehl schreibt mit `tee` eine neue Datei, deren Inhalt zwischen den beiden `EOF`-Zeilen steht. Das Format heißt deb822 und ist die moderne Schreibweise für Paketquellen. Jede Zeile hat eine feste Bedeutung. `Types` nennt die Art der Quelle, `URIs` die Adresse, `Suites` die Distributionsversion, `Components` den Zweig und `Architectures` die Prozessorarchitektur. `Signed-By` verweist auf den eben geladenen Schlüssel. Die Teile in `$(…)` führt die Shell vorher aus und setzt das Ergebnis ein. Unter Debian 13 steht dort `trixie` und unter amd64 `amd64`.

Kontrolle

Mit `cat /etc/apt/sources.list.d/docker.sources` sehen Sie die fertige Datei. Dort sollten die konkreten Werte stehen (`Suites: trixie`), nicht

die Befehle. Stehen dort noch Dollarzeichen, hat die Shell die Ausdrücke nicht ersetzt, meist weil die Anführungszeichen um `EOF` fehlen oder dazugeschrieben wurden.

5.3.2 Ubuntu Server

Unter Ubuntu ändern sich die Adresse des Repositorys und die Art, den Codenamen zu bestimmen. Der Schlüssel kommt ebenfalls von einer anderen Adresse. Ubuntu-Derivate verwenden gelegentlich einen eigenen Codenamen, deshalb sucht die Anleitung zuerst nach `UBUNTU_CODENAME` und nimmt sonst `VERSION_CODENAME`.

```
$ sudo apt update
$ sudo apt install ca-certificates curl
$ sudo install -m 0755 -d /etc/apt/keyrings
$ sudo curl -fsSL https://download.docker.com/linux/ubuntu/gpg -o /
    ↪ etc/apt/keyrings/docker.asc
$ sudo chmod a+r /etc/apt/keyrings/docker.asc
$ sudo tee /etc/apt/sources.list.d/docker.sources <<EOF
Types: deb
URIs: https://download.docker.com/linux/ubuntu
Suites: $(. /etc/os-release && echo "${UBUNTU_CODENAME:-
    ↪ $VERSION_CODENAME}")
Components: stable
Architectures: $(dpkg --print-architecture)
Signed-By: /etc/apt/keyrings/docker.asc
EOF
$ sudo apt update
```

Auf einem normalen Ubuntu 24.04 steht danach `noble` in der Datei.

5.4 Docker Engine, CLI und Compose installieren

Jetzt folgt die eigentliche Installation. Ein einziger Befehl holt alle fünf Pakete.

```
$ sudo apt install docker-ce docker-ce-cli containerd.io docker-
    ↪ buildx-plugin docker-compose-plugin
```

Die fünf Pakete haben jeweils eine eigene Aufgabe.

Paket	Inhalt
`docker-ce`	der Docker-Daemon `dockerd`, die eigentliche Engine (CE steht für Community Edition)
`docker-ce-cli`	der Client, also das Programm `docker`
`containerd.io`	`containerd` und `runc`, die beiden Bausteine aus Kapitel 3
`docker-buildx-plugin`	die erweiterte Variante zum Bauen von Images (`docker buildx`)
`docker-compose-plugin`	Docker Compose als Erweiterung des Clients (`docker compose`)

`apt` installiert dazu die nötigen Abhängigkeiten, unter anderem die Werkzeuge für `iptables`. Auf einem Standardsystem laufen danach der Daemon und `containerd` bereits.

Compose als Plugin

Docker Compose heißt in diesem Buch durchgehend `docker compose` mit einem Leerzeichen. Das ist das Plugin, das mit dem Client mitkommt. Das alte Einzelprogramm `docker-compose` mit Bindestrich wird nicht mehr verwendet.

5.5 Dienstverwaltung mit systemd

Der Docker-Daemon läuft als Dienst unter `systemd`, dem Programm, das auf Debian und Ubuntu die Hintergrunddienste startet und überwacht. Den Zustand eines Dienstes zeigt `systemctl status`.

```
$ sudo systemctl status docker●
 docker.service - Docker Application Container Engine
```

```
     Loaded: loaded (/usr/lib/systemd/system/docker.service; enabled;
     ↪ preset: enabled)
     Active: active (running) since Wed 2026-11-11 10:21:04 CET; 2min
     ↪ 5s ago
TriggeredBy: ● docker.socket
       Docs: https://docs.docker.com
   Main PID: 2217 (dockerd)
      Tasks: 9
     Memory: 28.4M
```

Zwei Zeilen sind wichtig. In `Loaded` bedeutet `enabled`, dass der Dienst beim Systemstart automatisch hochfährt. In `Active` bedeutet `active (running)`, dass er gerade läuft. Mit `[Q]` verlassen Sie die Anzeige. Die weiteren Zeilen und Zahlen sehen bei Ihnen anders aus. Zum Docker-Dienst gehören drei Einheiten, die Sie mit `systemctl` ansprechen.

Einheit	Aufgabe
`docker.service`	der Docker-Daemon
`docker.socket`	die Socket-Datei `/var/run/docker.sock`, über die der Client den Daemon erreicht
`containerd.service`	der Dienst `containerd`, den Docker im Hintergrund nutzt

Die gebräuchlichen Befehle für den Dienst lauten:

```
$ sudo systemctl stop docker
$ sudo systemctl start docker
$ sudo systemctl restart docker
$ journalctl -u docker --since "10 minutes ago"
```

Die ersten drei halten den Daemon an, starten ihn und starten ihn neu. Der vierte zeigt seine Protokollmeldungen der letzten zehn Minuten. Das Protokoll des Daemons ist bei Startproblemen die erste Anlaufstelle, wie Kapitel 20 zeigt. Weil `systemd` den Daemon über den Socket aktiviert, startet er auf Wunsch auch von selbst, sobald ein Client anklopft. Wenn Sie den Dienst dauerhaft anhalten wollen, müssen Sie deshalb zusätzlich `docker.socket` stoppen.

5.6 Docker automatisch starten

Auf Debian und Ubuntu richten die Docker-Pakete den automatischen Start beim Systemstart selbst ein. Ob das bei Ihnen der Fall ist, prüfen Sie so:

```
$ systemctl is-enabled docker containerd
enabled
enabled
```

Steht dort etwas anderes, schaltet der folgende Befehl den automatischen Start für beide Dienste ein.

```
$ sudo systemctl enable docker.service containerd.service
```

Soll ein Dienst nicht mehr mit dem System starten, tauscht man `enable` gegen `disable` aus. Die Schreibweise mit `.service` ist optional, macht aber deutlich, welche Art von Einheit gemeint ist.

5.7 Installation überprüfen

Der klassische Test für eine Docker-Installation ist ein Container namens `hello-world`. Er tut nichts anderes, als eine Meldung auszugeben und sich zu beenden. Dabei durchläuft Docker aber den ganzen Weg vom Befehl bis zum laufenden Container.

```
$ sudo docker run hello-world
Unable to find image 'hello-world:latest' locally
latest: Pulling from library/hello-world
...: Pull complete
Digest: sha256:...
Status: Downloaded newer image for hello-world:latest

Hello from Docker!
This message shows that your installation appears to be working
    ↪ correctly.
```

Dahinter steht die Ausgabe des Containers, die den Ablauf in vier Schritten beschreibt. Der Client meldet sich beim Daemon. Der Daemon lädt das Image `hello-world` von Docker Hub, weil er es lokal nicht findet. Danach erzeugt er daraus einen Container, der den Text ausgibt, und schließlich reicht er den Text an Ihr Terminal weiter. Das ist der Ablauf aus Kapitel 3, jetzt in der Praxis. Die Zeilen mit `...` stehen für lange Prüfsummen, die bei Ihnen anders aussehen.

Zwei weitere Befehle zeigen die Versionen.

```
$ sudo docker version
Client: Docker Engine - Community
 Version:           29.x.y
 API version:       1.xx
 OS/Arch:           linux/amd64
 Context:           default

Server: Docker Engine - Community
 Engine:
  Version:          29.x.y
  API version:      1.xx (minimum version 1.xx)
  OS/Arch:          linux/amd64
 containerd:
  Version:          X.Y.Z
 runc:
  Version:          X.Y.Z
```

Die Ausgabe ist gekürzt, und die Versionsnummern lauten bei Ihnen anders. Sie sehen hier die Trennung zwischen Client und Server aus dem letzten Kapitel. Der Block `Client` beschreibt das Programm `docker`, der Block `Server` den Daemon samt `containerd` und `runc`. Erscheint der Server-Block nicht, sondern eine Fehlermeldung, kann der Client den Daemon nicht erreichen. Das ist einer der häufigsten Anfängerfehler, und Kapitel 20 beschreibt ihn genauer.

```
$ docker compose version
Docker Compose version vX.Y.Z
```

Der Befehl bestätigt, dass das Compose-Plugin da ist. Ihre Versionsnummer weicht ab.

5.8 Sicherheitsrelevante Einstellungen

Nach der Installation fehlen noch drei Entscheidungen, die man bewusst treffen sollte und nicht dem Zufall überlässt.

5.8.1 Der Zugriff ohne sudo

Bis jetzt stand vor jedem Docker-Befehl ein `sudo`. Das ist umständlich. Mit der Gruppe `docker` lässt es sich vermeiden. Die Docker-Pakete legen die Gruppe an, und wer ihr angehört, darf den Socket benutzen.

```
$ sudo usermod -aG docker admin
```

Die Option `-a` hängt die Gruppe an, ohne bestehende zu entfernen, und `-G` nennt die Gruppe. Lassen Sie `-a` weg, entfernt `usermod` den Benutzer aus allen anderen Gruppen, auch aus `sudo`. Damit die Änderung wirkt, melden Sie sich ab und wieder an. Wer es sofort braucht, kann in der aktuellen Shell mit `newgrp docker` die neue Gruppe aktivieren. Ein Test zeigt, ob es klappt:

```
$ docker run --rm hello-world
```

Die Option `--rm` sorgt dafür, dass Docker den Container nach dem Ende wieder löscht. Läuft der Befehl ohne `sudo` durch, funktioniert der Zugriff.

> **Die Gruppe docker entspricht Root**
>
> Die Docker-Dokumentation sagt es unmissverständlich: Die Gruppe `docker` gewährt dem Benutzer Root-Rechte. Wer Docker bedienen darf, kann einen Container starten, der das ganze Dateisystem des Hosts einbindet, und damit alles lesen und ändern. Nehmen Sie deshalb nur Benutzer auf, denen Sie ohnehin Root-Zugriff geben würden. Auf diesem Übungsserver ist das für `admin` in Ordnung. Auf einem echten Server entscheiden Sie das bewusst, und Kapitel 18 zeigt Alternativen wie den rootless-Modus. Wer lieber bei `sudo` bleiben möchte, stellt es in den folgenden Kapiteln jedem Docker-Befehl voran.

In allen weiteren Kapiteln stehen die Befehle ohne `sudo`. Das setzt voraus, dass `admin` in der Gruppe `docker` ist.

5.8.2 Firewall und veröffentlichte Ports

Hier lauert eine Überraschung, die viele Administratoren kalt erwischt. Wenn Sie einen Container-Port nach außen veröffentlichen, trägt Docker eigene Regeln in die Firewall des Kernels ein. Diese Regeln greifen vor den Regeln, die Programme wie `ufw` oder `firewalld` verwalten. Ein Port, den Sie in `ufw` gesperrt haben, kann deshalb trotzdem von außen erreichbar sein, sobald ein Container ihn veröffentlicht.

Außerdem ist Docker nach der offiziellen Dokumentation nur mit `iptables-nft` und `iptables-legacy` verträglich. Firewallregeln, die man direkt mit dem Programm `nft` anlegt, werden auf einem System mit Docker nicht unterstützt. Eigene Regeln gehören beim Standardverfahren in die Kette `DOCKER-USER`. Kapitel 11 und Kapitel 18 zeigen den richtigen Umgang damit. Bis dahin gilt: Veröffentlichen Sie nur Ports, die ein Container wirklich nach außen anbieten soll.

5.8.3 Begrenzte Protokolle

Docker schreibt die Ausgabe jedes Containers standardmäßig in eine Datei, ohne sie zu begrenzen. Bei einem gesprächigen Container wächst diese Datei, bis die Platte voll ist. Die Docker-Dokumentation empfiehlt deshalb, die Rotation der Protokolle einzuschalten. Das geschieht in der Konfigurationsdatei des Daemons, die es nach der Installation noch nicht gibt.

/etc/docker/daemon.json

```
{
  "log-driver": "json-file",
  "log-opts": {
    "max-size": "10m",
    "max-file": "3"
  }
}
```

Die Datei legen Sie zum Beispiel mit `sudo nano /etc/docker/daemon.json` an. Sie steht im JSON-Format, bei dem jede Klammer und jedes Anführungszeichen

stimmen muss. Die Einstellung begrenzt jede Protokolldatei auf 10 Megabyte und hebt höchstens drei Dateien auf. Pro Container sind das also etwa 30 Megabyte. Damit sie wirkt, starten Sie den Daemon neu.

```
$ sudo systemctl restart docker
$ docker info --format '{{.LoggingDriver}}'
json-file
```

Die neue Einstellung gilt nur für Container, die nach dem Neustart erzeugt werden. Bereits vorhandene Container behalten ihre alte Einstellung.

Wenn der Daemon nach der Änderung nicht startet

Ein Tippfehler in der `daemon.json` verhindert den Start des Daemons. `systemctl status docker` meldet dann `failed`, und `journalctl -u docker` nennt die Stelle des Fehlers. Meist fehlt ein Komma oder eine Klammer. Korrigieren Sie die Datei und starten Sie den Dienst erneut.

KAPITEL 6

Erste Schritte mit Docker

Mit diesem Kapitel beginnt die Arbeit am Terminal. Sie lernen die vierzehn Befehle kennen, die man täglich braucht, und führen sie an kleinen Beispielen selbst aus. Jeder Befehl wird erklärt: Wozu er dient, welche Optionen wichtig sind, was Docker ausgibt und welche Stolpersteine es gibt. Die Beispiele bauen aufeinander auf, deshalb sollten Sie sie in der gegebenen Reihenfolge nachvollziehen.

Alle Befehle stehen ohne `sudo`. Das setzt voraus, dass der Benutzer `admin` in der Gruppe `docker` ist, wie in Kapitel 5 beschrieben. Die Ausgaben in diesem Kapitel stammen von Docker Engine 29. Bei Ihnen weichen Container-Kennungen, Zeitangaben und Versionsnummern ab.

6.1 docker version und docker info

Die beiden ersten Befehle fragen den Zustand von Docker ab, ohne etwas zu verändern. Sie sind die erste Anlaufstelle, wenn etwas nicht wie erwartet funktioniert.

```
docker version
docker info
```

`docker version` kennen Sie bereits aus Kapitel 5. Es zeigt die Versionen von Client und Server getrennt an. `docker info` liefert dagegen eine Bestandsaufnahme des Daemons: wie viele Container und Images vorhanden sind, welcher Kernel läuft und wo Docker seine Daten ablegt. Ein Ausschnitt:

```
$ docker info
Client:
 Version:    29.x.y
 Context:    default

Server:
 Containers: 1
  Running: 0
  Paused: 0
  Stopped: 1
 Images: 1
 Server Version: 29.x.y
 Cgroup Driver: systemd
 Cgroup Version: 2
 Kernel Version: 6.12.x-amd64
 Operating System: Debian GNU/Linux 13 (trixie)
 Total Memory: 3.8GiB
 Docker Root Dir: /var/lib/docker
```

Die Ausgabe ist erheblich länger, hier steht nur ein Auszug. Einige Zeilen kennen Sie aus Kapitel 2. `Cgroup Version: 2` und `Cgroup Driver: systemd` bestätigen die Angaben über cgroups. `Docker Root Dir` nennt das Verzeichnis, in dem Docker Images, Container und Volumes ablegt. Möchten Sie nur einen einzelnen Wert, übergeben Sie eine Vorlage mit `--format`.

```
$ docker info --format '{{.ServerVersion}}'
29.x.y
```

Die Schreibweise mit den doppelten geschweiften Klammern stammt aus der Programmiersprache Go. Sie begegnet Ihnen bei `docker inspect` noch einmal.

6.2 docker run

`docker run` ist der Befehl, mit dem man am häufigsten arbeitet. Er erzeugt aus einem Image einen neuen Container und startet ihn, wie in Kapitel 2 beschrieben.

```
docker run [OPTIONEN] IMAGE [BEFEHL] [ARGUMENTE]
```

Hinter dem Namen des Images kann ein Befehl stehen, der den Standardbefehl des Images ersetzt. Ohne Angabe startet Docker das, was der Ersteller des Images vorgesehen hat. Ein erstes Beispiel nimmt das kleine Image `alpine`, ein minimales Linux-System, und lässt es einen Satz ausgeben.

```
$ docker run alpine echo "Hallo aus dem Container"
Unable to find image 'alpine:latest' locally
latest: Pulling from library/alpine
...: Pull complete
Status: Downloaded newer image for alpine:latest
Hallo aus dem Container
```

Beim ersten Mal lädt Docker das Image herunter, danach liegt es lokal vor, und der Start dauert Sekundenbruchteile. Der Container hat `echo` ausgeführt, die Ausgabe zurückgegeben und sich beendet, weil sein einziger Prozess fertig war. Er existiert aber noch. Das sehen wir gleich bei `docker ps`.

Die wichtigsten Optionen von `docker run` fasst die Tabelle zusammen.

Option	Wirkung
`-d`	startet den Container im Hintergrund (*detached*) und gibt sofort die Eingabe zurück
`-it`	verbindet das Terminal mit dem Container, damit man darin tippen kann (interaktiv)
`--name NAME`	vergibt einen festen Namen statt eines zufälligen
`-p HOST:CONTAINER`	veröffentlicht einen Container-Port auf einem Port des Hosts
`--rm`	löscht den Container automatisch, sobald er sich beendet
`-e NAME=WERT`	setzt eine Umgebungsvariable im Container

Die Optionen `-i` (interaktiv) und `-t` (Terminal) werden fast immer zusammen verwendet und deshalb als `-it` geschrieben. Das nächste Beispiel öffnet eine Shell im Container.

```
$ docker run -it --rm alpine sh
/ # hostname
3f2a9c1b7d44
/ # ps
PID   USER     TIME  COMMAND
    1 root      0:00 sh
    7 root      0:00 ps
/ # uname -r
6.12.x-amd64
/ # exit
```

Die Eingabeaufforderung `/ #` zeigt, dass Sie sich jetzt im Container befinden. Alle drei Ausgaben bestätigen, was Kapitel 2 theoretisch erklärt hat. Der Hostname ist die Kennung des Containers, und das ist der uts-Namespace. Die Shell trägt die Prozessnummer 1, und von den übrigen Prozessen des Hosts ist nichts zu sehen, das ist der pid-Namespace. Und `uname -r` nennt dieselbe Kernel-Version wie der Host, weil Container keinen eigenen Kernel haben. Mit `exit` verlassen Sie die Shell. Die Shell war der einzige Prozess des Containers, deshalb endet damit auch der Container, und dank `--rm` verschwindet er gleich ganz.

Der dritte Aufruf startet einen Webserver. Hier kommt die Option `-d` ins Spiel, denn ein Webserver soll im Hintergrund weiterlaufen.

```
$ docker run -d --name web -p 8080:80 nginx
Unable to find image 'nginx:latest' locally
latest: Pulling from library/nginx
...: Pull complete
Status: Downloaded newer image for nginx:latest
c41e8d0b6f2a8b1c0d7e5a9f3b2c4d6e8f0a1b3c5d7e9f1a2b4c6d8e0f2a4b6c
```

Die lange Zeichenfolge ist die vollständige Kennung des neuen Containers. Mit `--name web` haben wir ihm einen festen Namen gegeben, auf den wir uns in den folgenden Befehlen beziehen. Die Option `-p 8080:80` verbindet Port 8080 des Hosts mit Port 80 im Container, dem Port, auf dem nginx lauscht. Die Reihenfolge ist wichtig: Links steht der Port des Hosts, rechts der des Containers. Ob der Webserver antwortet, prüfen Sie mit `curl`.

```
$ curl -s http://localhost:8080 | head -n 5
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
```

Wenn Sie in einem Browser auf dem eigenen Rechner die Adresse `http://192.168.100.10:8080` aufrufen (mit der IP-Adresse Ihres Servers), sehen Sie dort die Startseite von nginx.

Port bereits belegt

Wenn auf dem Host schon ein Programm den Port 8080 benutzt, bricht `docker run` mit einer Meldung wie `port is already allocated` ab. Wählen Sie dann einen anderen Host-Port, etwa `-p 8081:80`.

6.3 docker ps

`docker ps` zeigt die Container an. Ohne Optionen erscheinen nur die laufenden. Die Ausgabe hat sieben Spalten: die verkürzte Kennung (`CONTAINER ID`), das verwendete Image (`IMAGE`), den Startbefehl (`COMMAND`), das Alter (`CREATED`), den Zustand (`STATUS`), die veröffentlichten Ports (`PORTS`) und den Namen (`NAMES`). Das ist breiter, als eine Buchseite zeigen kann. Die Beispiele in diesem Buch beschränken sich deshalb mit `--format` auf die Spalten, auf die es ankommt. Diese Option ist auch im Alltag nützlich, wenn das Terminalfenster schmal ist.

```
$ docker ps --format "table {{.Names}}\t{{.Status}}\t{{.Ports}}"
NAMES   STATUS         PORTS
web     Up 2 minutes   0.0.0.0:8080->80/tcp, [::]:8080->80/tcp
```

In der Spalte `PORTS` sehen Sie die Verbindung zwischen Host und Container. `0.0.0.0:8080` bedeutet, dass der Port auf allen IPv4-Adressen des Hosts erreichbar ist, `[::]` steht für die IPv6-Adressen. Der Container mit `echo` aus dem ersten Beispiel fehlt, denn er läuft nicht mehr. Mit der Option `-a` (*all*) erscheinen auch beendete Container.

```
$ docker ps -a --format "table {{.Names}}\t{{.Image}}\t{{.Status}}"
NAMES           IMAGE         STATUS
web             nginx         Up 3 minutes
sweet_ritchie   alpine        Exited (0) 6 minutes ago
happy_banach    hello-world   Exited (0) 20 minutes ago
```

Hier sind zwei beendete Container zu sehen: der Alpine-Container aus dem ersten Beispiel und der Testcontainer `hello-world` aus Kapitel 5. Beide haben einen zufälligen Namen bekommen (zum Beispiel `sweet_ritchie`), weil wir keinen angegeben haben. Der Status `Exited (0)` bedeutet, dass der Container sich mit dem Rückgabewert 0 beendet hat, also ohne Fehler. Ein anderer Wert wäre ein Hinweis auf ein Problem. Das ist der Zustand *exited* aus Abbildung 2.1. Es gibt zwei weitere nützliche Varianten.

```
$ docker ps -q
c41e8d0b6f2a
$ docker ps -a --filter "status=exited" --format "{{.Names}}"
sweet_ritchie
happy_banach
```

Die Option `-q` (*quiet*) gibt nur die Kennungen aus, praktisch für Skripte. `--filter` schränkt die Anzeige ein, hier auf beendete Container.

6.4 docker images und docker pull

Ein Container braucht ein Image. `docker pull` lädt ein Image aus einer Registry auf den Server, ohne einen Container zu starten.

```
$ docker pull mariadb:11
11: Pulling from library/mariadb
...: Pull complete
Status: Downloaded newer image for mariadb:11
docker.io/library/mariadb:11
```

Hinter dem Doppelpunkt steht der Tag, hier die Hauptversion 11. Ohne Tag lädt Docker `latest`, wie in Kapitel 3 beschrieben. `docker pull` ist auch nützlich, um

ein vorhandenes Image zu aktualisieren: Erscheint unter demselben Tag eine neue Fassung, lädt der Befehl sie herunter. Die lokal vorhandenen Images zeigt `docker images`.

```
$ docker images
IMAGE                ID             DISK USAGE   CONTENT SIZE   EXTRA
alpine:latest        a8560b36e8b8       13.6MB         4.28MB    U
hello-world:latest   f7931603f70e       20.3kB         3.96kB    U
mariadb:11           3f9c6d2a1b7e        329MB          109MB
nginx:latest         d4918ca78576        225MB         59.8MB    U
```

Wenn Sie ältere Anleitungen im Internet kennen, wundern Sie sich vielleicht über diese Ausgabe. Mit Docker 29 hat sich die Darstellung geändert. Früher hatte die Ausgabe die Spalten `REPOSITORY`, `TAG`, `IMAGE ID`, `CREATED` und `SIZE`. Jetzt steht der vollständige Name mit Tag in der ersten Spalte. `DISK USAGE` gibt den Platz an, den das Image auf der Platte belegt, `CONTENT SIZE` die Größe der komprimierten Daten. Ein `U` in der Spalte `EXTRA` markiert Images, die gerade von einem Container verwendet werden. Alle Größen in dieser Ausgabe sind Beispiele und weichen bei Ihnen ab. Die alte Darstellung lässt sich bei Bedarf mit einer Vorlage nachbilden:

```
$ docker images --format "table {{.Repository}}\t{{.Tag}}\t{{.ID}}\t
    ↪ {{.Size}}"
REPOSITORY   TAG      IMAGE ID       SIZE
alpine       latest   a8560b36e8b8   13.6MB
hello-world  latest   f7931603f70e   20.3kB
mariadb      11       3f9c6d2a1b7e   329MB
nginx        latest   d4918ca78576   225MB
```

6.5 docker stop, start und restart

Diese drei Befehle steuern den Zustand eines vorhandenen Containers. Sie nehmen den Namen oder die Kennung entgegen. Ein paar Anfangszeichen der Kennung genügen, solange sie eindeutig sind.

```
$ docker stop web
web
$ docker ps -q
$ docker start web
web
$ docker restart web
web
```

`docker stop` beendet den Container geordnet. Docker schickt dem Hauptprozess zuerst ein Signal mit der Bitte aufzuhören (SIGTERM). Reagiert der Prozess nicht innerhalb von zehn Sekunden, erzwingt Docker das Ende mit SIGKILL. Mit `-t` lässt sich die Wartezeit ändern, zum Beispiel `docker stop -t 30 web` für Programme, die länger zum Aufräumen brauchen. Der gestoppte Container bleibt erhalten und taucht in `docker ps -a` auf. `docker start` startet ihn wieder, und `docker restart` erledigt beides hintereinander. Die Daten in der beschreibbaren Schicht überstehen das, die Containerkennung bleibt dieselbe.

Wollen Sie mehrere Container auf einmal ansprechen, geben Sie die Namen nacheinander an, etwa `docker stop web db`. Ein Container, der sich nicht beenden lässt, wird mit `docker kill web` sofort abgebrochen. Das sollte die Ausnahme bleiben, denn das Programm hat keine Chance, offene Dateien oder Datenbanken sauber zu schließen.

6.6 docker rm und docker rmi

Gestoppte Container belegen weiter Platz. `docker rm` entfernt sie.

```
$ docker rm sweet_ritchie
sweet_ritchie
$ docker rm web
Error response from daemon: cannot remove container "/web": container
    ↪ is running: stop the container before removing or force
    ↪ remove
$ docker stop web
web
$ docker rm web
```

```
web
```

Docker verweigert das Löschen eines laufenden Containers und weist darauf hin, dass er zuerst gestoppt werden sollte. Mit `docker rm -f web` wird der Container zuerst beendet und dann entfernt. Das Löschen ist endgültig. Alle Daten in der beschreibbaren Schicht sind danach weg.

`docker rmi` (*remove image*) löscht Images, die nicht mehr gebraucht werden.

```
$ docker rmi mariadb:11
Untagged: mariadb:11
Deleted: sha256:3f9c6d2a1b7e...
```

Verwendet noch ein Container das Image, und sei er nur gestoppt, verweigert Docker das Löschen. Die Meldung lautet sinngemäß `conflict: unable to delete <ID> (must be forced) - container <ID> is using its reference to image`. Löschen Sie dann zuerst den Container, danach lässt sich das Image entfernen. Ein erzwungenes Löschen mit `-f` ist selten sinnvoll, denn es lässt den Container ohne Image zurück.

Aufräumen im Großen

Wer viele Container ausprobiert hat, löscht nicht jeden einzeln. Der Befehl `docker container prune` entfernt alle gestoppten Container auf einmal. Kapitel 9 zeigt die sicheren Wege, ein System aufzuräumen.

6.7 docker logs

Alles, was der Hauptprozess eines Containers auf die Standardausgabe oder die Standardfehlerausgabe schreibt, fängt Docker auf. `docker logs` zeigt es an. Damit Sie etwas zu sehen haben, starten wir den Webserver neu und rufen die Seite einmal ab.

```
$ docker run -d --name web -p 8080:80 nginx
$ curl -s http://localhost:8080 > /dev/null
```

```
$ docker logs web
/docker-entrypoint.sh: Configuration complete; ready for start up
...
172.17.0.1 - - [11/Nov/2026:10:45:12 +0000] "GET / HTTP/1.1" 200 615
    ↪ "-" "curl/8.x" "-"
```

Die letzte Zeile ist der Zugriff, den `curl` ausgelöst hat. Die Adresse `172.17.0.1` gehört zum Docker-Netzwerk auf dem Host. Warum sie dort statt Ihrer eigenen Adresse steht, erklärt Kapitel 11. Drei Optionen machen `docker logs` im Alltag nützlich.

```
$ docker logs --tail 20 web
$ docker logs -f web
$ docker logs --since 10m -t web
```

`--tail 20` zeigt nur die letzten 20 Zeilen, was bei langen Protokollen unverzichtbar ist. `-f` (*follow*) hängt sich an das Protokoll an und zeigt neue Zeilen laufend an, bis Sie [Strg]+[C] drücken. Der Container läuft dabei weiter. `--since 10m` beschränkt die Ausgabe auf die letzten zehn Minuten, und `-t` setzt vor jede Zeile einen Zeitstempel.

6.8 docker inspect

`docker inspect` gibt alle Einzelheiten eines Docker-Objekts als JSON-Text aus: Konfiguration, Zustand, Netzwerkdaten, eingebundene Verzeichnisse und vieles mehr.

```
$ docker inspect web
[
    {
        "Id": "
    ↪ c41e8d0b6f2a8b1c0d7e5a9f3b2c4d6e8f0a1b3c5d7e9f1a2b4c6d8e0f2a4b6c
    ↪ ",
        "Created": "2026-11-11T10:41:03.512Z",
        "Path": "/docker-entrypoint.sh",
```

```
        "State": {
            "Status": "running",
            ...
```

Der vollständige Text umfasst mehrere hundert Zeilen. Einzelne Werte pickt man mit `--format` heraus. Die Vorlage greift mit Punkten auf die Ebenen des JSON zu.

```
$ docker inspect --format '{{.State.Status}}' web
running
$ docker inspect --format '{{range .NetworkSettings.Networks}}{{.
    ↪ IPAddress}}{{end}}' web
172.17.0.2
```

Der erste Aufruf liest den Zustand, der zweite die IP-Adresse des Containers im Docker-Netzwerk. Die zweite Vorlage ist etwas länger, weil ein Container in mehreren Netzwerken sein kann und `range` sie der Reihe nach durchgeht. Für den Anfang genügt es zu wissen, dass `docker inspect` die Antwort auf fast jede Frage nach der Konfiguration eines Containers enthält. Der Befehl funktioniert auch für Images, Netzwerke und Volumes.

6.9 docker exec

`docker exec` führt einen Befehl in einem bereits laufenden Container aus. Das unterscheidet ihn von `docker run`, der einen neuen Container erzeugt.

```
$ docker exec web nginx -v
nginx version: nginx/1.x.y
$ docker exec -it web sh
# ls /usr/share/nginx/html
50x.html  index.html
# exit
$ docker ps --format "{{.Names}}: {{.Status}}"
web: Up 12 minutes
```

Der erste Aufruf führt einen einzelnen Befehl im Container aus und gibt das Ergebnis zurück. Der zweite öffnet mit `-it` eine Shell im laufenden Container. Dort

können Sie umsehen, Dateien prüfen und Fehler suchen. Verlassen Sie die Shell mit `exit`, läuft der Container weiter, denn die Shell war nur ein zusätzlicher Prozess und nicht der Hauptprozess des Containers. Das ist ein wesentlicher Unterschied zu `docker run -it`, wo die Shell der Hauptprozess war.

Änderungen per exec sind flüchtig

Wer sich per `docker exec` in einen Container einloggt und dort Dateien ändert oder Software installiert, ändert nur die beschreibbare Schicht dieses einen Containers. Löschen oder ersetzen Sie den Container, ist die Änderung verloren. Dauerhafte Änderungen gehören in ein Image oder ein Volume, wie die Kapitel 7 und 10 zeigen.

6.10 Aufräumen nach den Übungen

Zum Schluss entfernen wir die Container und Images dieses Kapitels, damit die Übungsumgebung sauber in die nächsten Kapitel startet.

```
$ docker rm -f web
web
$ docker container prune -f
Deleted Containers:
d2f4a6b8c0e1a3b5c7d9e1f2a4b6c8d0e2f4a6b8c0d2e4f6a8b0c2d4e6f8a0b2

Total reclaimed space: 0B
$ docker rmi nginx alpine hello-world
Untagged: nginx:latest
Deleted: sha256:d4918ca78576...
$ docker ps -a
CONTAINER ID   IMAGE   COMMAND   CREATED   STATUS   PORTS   NAMES
$ docker images
```

`docker rm -f web` beendet den Webserver und löscht ihn. `docker container prune -f` entfernt alle übrigen gestoppten Container, hier den Testcontainer aus Kapitel 5. Die Option `-f` unterdrückt die Rückfrage. Erst danach lassen sich die Images löschen, denn solange ein Container ein Image verwendet, verweigert Do-

cker das Löschen. `docker images` zeigt am Ende keine Zeile mehr, die Umgebung ist leer.

Praxisprojekt 1: Docker-Grundinstallation

In diesem Projekt bauen Sie von Grund auf einen Docker-Server und weisen anschließend mit einem Prüfskript nach, dass er funktioniert. Das Projekt fasst die Kapitel 4 bis 6 zu einer durchgehenden Aufgabe zusammen. Planen Sie dafür etwa zwei Stunden ein.

Ausgangslage und Ziel

Ihr Team braucht einen Übungsserver, auf dem später Webserver und Datenbanken in Containern laufen sollen. Sie bekommen eine leere virtuelle Maschine und richten sie so ein, dass sich alle folgenden Projekte darauf durchführen lassen. Am Ende steht ein Server mit folgenden Eigenschaften.

Anforderung	Sollzustand
Betriebssystem	Debian 13 (alternativ Ubuntu Server LTS), vollständig aktualisiert
Hostname	`docker01`
Administrator	Benutzer `admin` mit `sudo`-Recht und Mitglied der Gruppe `docker`
Docker	Docker Engine aus dem offiziellen Repository, Compose-Plugin vorhanden
Dienste	`docker` und `containerd` laufen und starten automatisch
Protokolle	Rotation eingeschaltet (10 MB, drei Dateien)
Arbeitsverzeichnis	`/srv/docker/projekt-1`, Besitzer `admin`

Schritt 1: Server vorbereiten

Führen Sie die Schritte aus Kapitel 4 durch: System prüfen, `sudo` sicherstellen, Hostname setzen, aktualisieren, bei Bedarf neu starten, Netzwerk und Speicherplatz kontrollieren. Legen Sie danach das Projektverzeichnis an.

```
$ mkdir -p /srv/docker/projekt-1
$ cd /srv/docker/projekt-1
```

Schritt 2: Docker installieren

Richten Sie das Repository ein und installieren Sie die fünf Pakete, wie in Kapitel 5 beschrieben. Nehmen Sie `admin` in die Gruppe `docker` auf, melden Sie sich neu an und legen Sie die Datei `/etc/docker/daemon.json` mit der Protokollrotation an. Starten Sie den Daemon danach neu.

Schritt 3: Erste Container starten

Starten Sie nacheinander drei Container und beobachten Sie jeweils, was geschieht. Der Test mit `hello-world` prüft den ganzen Weg von der Registry bis zur Ausgabe. Der Alpine-Container zeigt, dass ein Container nur einen Prozess hat. Der nginx-Container zeigt, dass ein veröffentlichter Port von außen erreichbar ist.

```
$ docker run --rm hello-world
$ docker run --rm alpine cat /etc/os-release
$ docker run -d --name p1-web -p 8080:80 nginx
$ curl -s http://localhost:8080 | grep title
<title>Welcome to nginx!</title>
```

Rufen Sie die Seite zusätzlich im Browser Ihres Rechners auf, mit der Adresse Ihres Servers und dem Port 8080. Entfernen Sie den Webserver danach mit `docker rm -f p1-web`.

Schritt 4: Funktionsprüfung mit einem Skript

Statt jeden Punkt von Hand zu prüfen, schreiben Sie ein kleines Bash-Skript. Es führt alle Prüfungen nacheinander aus und meldet für jede, ob sie bestanden wurde. Solche Skripte sind in der Praxis üblich, weil man sie nach jedem Update wiederholen kann.

/srv/docker/projekt-1/check.sh

```
#!/bin/bash
# Funktionsprüfung für Projekt 1: Docker-Grundinstallation
FEHLER=0
ok()   { echo "[ OK ]     $1"; }
fail() { echo "[FEHLER]   $1"; FEHLER=1; }

[ "$(hostname)" = "docker01" ] && ok "Hostname ist docker01" \
  || fail "Hostname ist nicht docker01"
systemctl is-active --quiet docker && ok "Docker-Dienst läuft" \
  || fail "Docker-Dienst läuft nicht"
systemctl is-enabled --quiet docker && ok "Docker startet beim
    ↪ Systemstart" \
  || fail "Docker startet nicht automatisch"
id -nG | grep -qw docker && ok "Benutzer ist in der Gruppe docker" \
  || fail "Benutzer ist nicht in der Gruppe docker"
docker version --format '{{.Server.Version}}' >/dev/null 2>&1 \
  && ok "Client erreicht den Daemon" || fail "Client erreicht den
    ↪ Daemon nicht"
docker compose version >/dev/null 2>&1 && ok "Compose-Plugin ist
    ↪ vorhanden" \
  || fail "Compose-Plugin fehlt"
[ "$(docker info --format '{{.LoggingDriver}}' 2>/dev/null)" = "json-
    ↪ file" ] \
  && ok "Protokolltreiber json-file" || fail "Protokolltreiber nicht
    ↪ json-file"
docker run --rm hello-world >/dev/null 2>&1 && ok "hello-world läuft"
    ↪  \
  || fail "hello-world läuft nicht"

docker run -d --name p1-web -p 8080:80 nginx >/dev/null 2>&1
sleep 3
curl -s http://localhost:8080 | grep -q "Welcome to nginx" \
  && ok "nginx antwortet auf Port 8080" || fail "nginx antwortet
    ↪ nicht"
docker rm -f p1-web >/dev/null 2>&1

echo
```

```
if [ "$FEHLER" -eq 0 ]; then
  echo "Alle Prüfungen bestanden."
else
  echo "Mindestens eine Prüfung ist fehlgeschlagen."
fi
exit $FEHLER
```

Die Datei legen Sie mit einem Editor an, zum Beispiel `nano check.sh`. Danach machen Sie sie ausführbar und starten sie.

```
$ chmod +x check.sh
$ ./check.sh
[ OK ]     Hostname ist docker01
[ OK ]     Docker-Dienst läuft
[ OK ]     Docker startet beim Systemstart
[ OK ]     Benutzer ist in der Gruppe docker
[ OK ]     Client erreicht den Daemon
[ OK ]     Compose-Plugin ist vorhanden
[ OK ]     Protokolltreiber json-file
[ OK ]     hello-world läuft
[ OK ]     nginx antwortet auf Port 8080

Alle Prüfungen bestanden.
```

Wie das Skript arbeitet, lässt sich zeilenweise nachvollziehen. Die Funktionen `ok` und `fail` geben eine Meldung aus, `fail` merkt sich außerdem, dass etwas schiefgegangen ist. Jede Prüfung folgt dem Muster `Bedingung && ok || fail`: Ist die Bedingung erfüllt, wird `ok` aufgerufen, sonst `fail`. Der Rückgabewert am Ende (`exit $FEHLER`) ist `0` bei Erfolg und `1` bei einem Fehler. Damit lässt sich das Skript später in automatische Abläufe einbauen.

Schritt 5: Abnahme und Aufräumen

Gehen Sie zum Abschluss die Abnahmeliste durch. Jeder Punkt muss mit „ja“ beantwortet werden können.

Prüfpunkt	Erfüllt
Das System ist vollständig aktualisiert, und ein Kernel-Update wurde mit einem Neustart wirksam.	☐
Der Hostname lautet `docker01`.	☐
`docker version` zeigt Client und Server.	☐
`docker compose version` gibt eine Version aus.	☐
Das Prüfskript meldet „Alle Prüfungen bestanden.“	☐
Nach einem Neustart des Servers läuft Docker ohne Eingriff.	☐
Die Protokollrotation ist mit `docker info` bestätigt.	☐

Starten Sie den Server als letzten Test neu und führen Sie das Prüfskript noch einmal aus. Läuft es durch, ist die Grundinstallation abgeschlossen. Entfernen Sie danach nicht benötigte Container und Images mit den Befehlen aus Kapitel 6. Das Skript lassen Sie liegen, denn es leistet bei jedem späteren Projekt gute Dienste.

Wenn etwas nicht klappt

Symptom	Ursache und Abhilfe
`permission denied` beim Zugriff auf den Docker-Socket	`admin` ist nicht in der Gruppe `docker`, oder die Anmeldung wurde nach der Änderung nicht erneuert. Mit `id` prüfen, abmelden und neu anmelden.
`port is already allocated`	Der Port 8080 ist belegt. Mit `sudo ss -tlnp | grep 8080` den Verursacher suchen oder das Skript auf einen anderen Port umstellen.
Der Docker-Dienst startet nach der Änderung der `daemon.json` nicht	Meist ein Tippfehler im JSON. `journalctl -u docker` nennt die Stelle.
`apt update` meldet einen Fehler beim Docker-Repository	Prüfen, ob die Datei `docker.sources` echte Werte enthält und der Schlüssel unter `/etc/apt/keyrings/docker.asc` lesbar ist.
`hello-world` lädt nicht	Namensauflösung oder Internetverbindung prüfen, wie in Kapitel 4 beschrieben. Auch eine falsch gehende Uhr kann die Ursache sein.

Lernkontrolle zu Teil II

Die Aufgaben prüfen die Kapitel 4 bis 6. Bearbeiten Sie die praktischen Aufgaben auf Ihrem Übungsserver. Die Lösungen stehen im Anhang.

Verständnisfragen

V1. Warum sollten Sie den Server nach einem Kernel-Update neu starten, bevor Sie Container betreiben?

V2. Wozu dient der Schlüssel, den Sie beim Einrichten des Docker-Repositorys herunterladen?

V3. Nennen Sie die fünf Pakete der Docker-Installation und beschreiben Sie in einem Satz, welches Paket `containerd` und `runc` enthält und welches das Compose-Plugin.

V4. Worin unterscheiden sich `docker run` und `docker exec`?

V5. Warum ist die Mitgliedschaft in der Gruppe `docker` sicherheitsrelevant?

V6. Was geschieht mit dem Hauptprozess eines Containers, wenn Sie `docker stop` aufrufen?

V7. Was bedeutet die Option `-p 8080:80`? Welche Zahl steht für den Host, welche für den Container?

V8. Warum kann ein Container-Port von außen erreichbar sein, obwohl er in `ufw` gesperrt wurde?

Multiple-Choice-Aufgaben

M1. Welcher Befehl nimmt den Benutzer `admin` in die Gruppe `docker` auf, ohne ihn aus anderen Gruppen zu entfernen?

(a) `sudo usermod -G docker admin`
(b) `sudo usermod -aG docker admin`
(c) `sudo groupadd docker admin`
(d) `sudo adduser docker`

M2. In welcher Datei stehen die Einstellungen des Docker-Daemons?

(a) `/etc/docker/daemon.json`
(b) `/etc/docker.conf`
(c) `/var/lib/docker/config`
(d) `/usr/bin/dockerd.cfg`

M3. Welche Option startet einen Container im Hintergrund?

(a) `-i`
(b) `-p`
(c) `-d`
(d) `-t`

M4. Der Status eines Containers lautet `Exited (0)`. Was bedeutet das?

(a) Der Container wurde durch Docker abgebrochen.
(b) Der Container läuft ohne Prozess.
(c) Der Container wurde gelöscht.
(d) Der Container hat sich ohne Fehler beendet.

M5. In welchem Paket befindet sich das Compose-Plugin?

(a) `docker-ce`
(b) `docker-compose-plugin`
(c) `containerd.io`
(d) `docker-buildx-plugin`

M6. Welcher Befehl gibt nur die Kennungen aller Container aus, auch der beendeten?

(a) `docker ps -q`
(b) `docker ps -a`
(c) `docker ps -aq`

(d) `docker images -q`

Praktische Aufgaben

P1. Starten Sie einen Alpine-Container mit dem Namen `lernen`, der den Inhalt der Datei `/etc/os-release` ausgibt. Prüfen Sie danach mit `docker ps -a`, welchen Status er hat, und entfernen Sie ihn.

P2. Starten Sie einen nginx-Container mit dem Namen `web2` auf dem Host-Port 8081. Rufen Sie die Seite mit `curl` ab, zeigen Sie die letzten fünf Zeilen des Protokolls an und entfernen Sie den Container.

P3. Ändern Sie die `daemon.json` so, dass jede Protokolldatei höchstens 5 MB groß wird und zwei Dateien behalten werden. Starten Sie den Daemon neu und bestätigen Sie den Protokolltreiber mit `docker info`.

Fehlersuche

F1. `docker run -d --name web -p 8080:80 nginx` bricht mit `port is already allocated` ab. Nennen Sie Ursache und zwei Möglichkeiten zur Abhilfe.

F2. `docker rmi alpine` meldet einen Konflikt, weil ein Container das Image verwendet. Wie gehen Sie vor?

F3. Nach einer Änderung der `daemon.json` startet der Docker-Dienst nicht mehr. Welche Befehle helfen bei der Suche?

Szenario aus dem Betrieb

Eine Kollegin meldet, dass sie auf dem Docker-Server die Meldung `permission denied while trying to connect to the Docker daemon socket` erhält. Sie hat Docker gestern installieren lassen, ihr Benutzer wurde heute Morgen in die Gruppe `docker` aufgenommen. Beschreiben Sie, was Sie prüfen und was Sie ihr raten. Überlegen Sie außerdem, ob es sicherheitstechnisch vertretbar ist, jedem Teammitglied diese Gruppenmitgliedschaft zu geben.

TEIL III

Images und Container verwalten

KAPITEL 7

Docker Images

Ein Container ist nur so gut wie das Image, aus dem er entsteht. In diesem Kapitel öffnen wir das Image und sehen nach, woraus es besteht. Sie lernen, wie Schichten, Tags und Prüfsummen zusammenspielen, wie man Images herunterlädt, hochlädt und baut und warum ein zweiter Build oft in Sekundenbruchteilen fertig ist. Am Ende wissen Sie, wie man Images klein hält und sinnvoll beschriftet.

7.1 Aufbau eines Images

Ein Image ist kein einzelnes Gebilde, sondern ein Bündel aus mehreren Teilen. Den Kern bilden die *Layer*, also Schichten. Jede Schicht enthält die Dateien, die ein einzelner Schritt beim Bau des Images hinzugefügt, geändert oder gelöscht hat. Dazu kommen Metadaten, die beschreiben, wie ein Container aus dem Image starten soll: welcher Befehl läuft, welche Umgebungsvariablen gesetzt sind, welcher Benutzer verwendet wird und welcher Port gemeint ist. Ein dritter Teil, das *Manifest*, verzeichnet, aus welchen Schichten und welcher Konfiguration das Image besteht.

Die Schichten liegen übereinander wie Transparentfolien. Die unterste enthält ein minimales Grundsystem, etwa Alpine Linux. Die nächste fügt vielleicht einen Webserver hinzu, die dritte eine Konfigurationsdatei, die vierte die eigene Webseite. Betrachtet man den Stapel von oben, sieht man das Zusammenspiel aller Schichten als ein einziges Dateisystem. Das ist die Arbeit des Union-Dateisystems aus Kapitel 2.

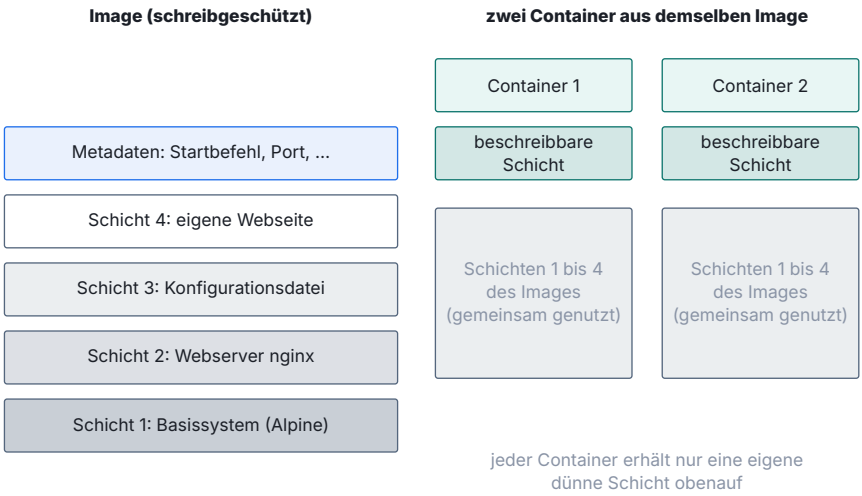


Abbildung 7.1: Schichten eines Images und die beschreibbare Schicht der Container

Die Abbildung zeigt auch, warum Docker so sparsam mit Platz umgeht. Zwei Container aus demselben Image teilen sich alle Schichten des Images. Jeder bekommt nur seine eigene dünne, beschreibbare Schicht, die erst wächst, wenn im Container etwas geschrieben wird. Zehn nginx-Container belegen deshalb nicht das Zehnfache eines einzelnen.

Fachbegriff: Layer

Ein Layer ist eine schreibgeschützte Schicht aus Dateisystemänderungen. Layer werden über eine Prüfsumme ihres Inhalts identifiziert und können von mehreren Images gemeinsam genutzt werden. Wenn zwei Images dieselbe Basis verwenden, liegt diese Basis nur einmal auf der Platte.

Wie ein vorhandenes Image aufgebaut ist, zeigt der Befehl `docker history`. Er listet die Schichten von der obersten zur untersten auf und nennt zu jeder den Befehl, mit dem sie entstanden ist.

```
$ docker pull nginx:stable-alpine
$ docker history --format "{{.Size}}\t{{.CreatedBy}}" nginx:stable-
    ↪ alpine
```

```
0B      CMD ["nginx" "-g" "daemon off;"
0B      STOPSIGNAL SIGQUIT
0B      EXPOSE map[80/tcp:{}]
0B      ENTRYPOINT ["/docker-entrypoint.sh"]
1.2kB   COPY docker-entrypoint.sh /
...
8.3MB   ADD alpine-minirootfs-... / # buildkit
```

Die Zeilen lesen sich von oben nach unten rückwärts: ganz unten steht die Basis, ganz oben die letzte Anweisung. Einträge mit `0B` fügen keine Dateien hinzu, sie schreiben nur Metadaten, etwa den Startbefehl oder den Port. Die genaue Ausgabe hängt von der aktuellen Fassung des Images ab. Verkürzt ist sie hier nur wiedergegeben.

7.2 Der Image-Speicher

Wo Docker die Schichten ablegt, hat sich mit Docker 29 geändert. Das lohnt einen genauen Blick, weil Sie bei Anleitungen im Internet oft die alte Beschreibung finden.

Bis Docker 28 verwaltete Docker die Images mit eigenen Speichertreibern. Der verbreitetste hieß `overlay2` und legte die Schichten unter `/var/lib/docker/overlay2` ab. Ab Docker 29 ist bei *frischen Installationen* der *containerd Image Store* die Voreinstellung. Dabei übernimmt `containerd`, das wir aus Kapitel 3 kennen, auch die Verwaltung der Images. Es verwendet dazu sogenannte Snapshotter. Für die Praxis bleibt der Mechanismus gleich, denn OverlayFS stapelt die Schichten wie in Kapitel 2 beschrieben. Geändert haben sich der Speicherort und einige Eigenheiten: Die Images liegen bei frischen Installationen unter `/var/lib/containerd`. Auf Systemen, die von einer älteren Version aktualisiert wurden, bleibt der alte Speicher bestehen, bis man den neuen aktiviert. Die älteren Treiber gelten als veraltet und sollen in einer künftigen Version entfallen. Der containerd-Speicher belegt außerdem mehr Platz als der alte, weil er jede Schicht sowohl komprimiert als auch entpackt vorhält.

Welchen Speicher Ihr Server verwendet, verrät `docker info`.

```
$ docker info --format '{{.Driver}}'
overlayfs
$ docker info | grep -E "Storage Driver|driver-type|Docker Root Dir"
 Storage Driver: overlayfs
  driver-type: io.containerd.snapshotter.v1
 Docker Root Dir: /var/lib/docker
```

Die Ausgabe `overlayfs` zusammen mit dem Typ `io.containerd.snapshotter.v1` zeigt den containerd-Speicher an. Steht dort dagegen `overlay2`, arbeitet der klassische Speicher. Das Buch beschreibt beide Fälle, wo sie sich unterscheiden. Die Befehle für Images funktionieren in beiden gleich.

Der Image-Speicher ist nicht austauschbar

Wer auf dem Server von einem Speicher zum anderen wechselt, sieht die bisherigen Images zunächst nicht mehr, denn jeder Speicher hat seinen eigenen Bestand. Sie lassen sich neu laden. Containerdaten und Volumes bleiben dabei erhalten. Wer unbedingt beim klassischen Speicher bleiben möchte, trägt in der `daemon.json` ein: `"features": {"containerd-snapshotter": false}`. Das ist nur als Übergangslösung gedacht. Das Daemon-Feature `userns-remap` lässt sich mit dem neuen Speicher derzeit nicht verbinden.

7.3 Image-Tags und Prüfsummen

Ein Image trägt einen Namen, und dieser Name hat einen Tag, wie Kapitel 3 gezeigt hat. Ein Tag ist ein Zeiger auf ein bestimmtes Image. Wichtig ist: Er lässt sich jederzeit auf ein anderes Image umsetzen. Das Image, das heute unter `nginx:stable-alpine` liegt, ist in einem halben Jahr ein anderes, weil die Entwickler eine neue Fassung unter demselben Tag veröffentlichen. Auch `latest` hat keine besondere Magie. Es ist nur ein Tag, der meist auf die zuletzt veröffentlichte Fassung zeigt, aber nichts garantiert.

Wer ganz sicher sein will, genau dasselbe Image zu bekommen, verwendet die Prüfsumme, den *Digest*. Er wird aus dem Inhalt des Images berechnet und ändert sich bei jeder noch so kleinen Änderung.

```
$ docker images --digests nginx:stable-alpine
$ docker pull nginx@sha256:<Prüfsumme>
```

Eine Adresse mit `@sha256:` verweist auf genau eine Fassung, die sich nicht mehr verändern kann. Die Prüfsumme entnehmen Sie der Ausgabe des ersten Befehls.

Mit `docker tag` geben Sie einem vorhandenen Image einen zusätzlichen Namen, ohne es zu kopieren.

```
$ docker tag nginx:stable-alpine meinserver/web:1.0
$ docker tag meinserver/web:1.0 meinserver/web:latest
```

Beide Namen zeigen anschließend auf dasselbe Image. Das Löschen eines Namens mit `docker rmi meinserver/web:latest` entfernt nur den Zeiger. Das Image bleibt, solange noch ein anderer Name oder ein Container darauf verweist.

7.4 Images laden und hochladen

Das Herunterladen kennen Sie bereits von `docker pull`. Dasselbe geschieht automatisch, wenn `docker run` ein Image nicht lokal findet. Das Hochladen heißt *Push*. Es setzt ein Konto bei der Registry voraus, bei Docker Hub also ein kostenloses Benutzerkonto. Das Anmelden erledigt `docker login`.

```
$ docker login
$ docker tag meinserver/web:1.0 benutzername/web:1.0
$ docker push benutzername/web:1.0
```

Der Namensraum vor dem Schrägstrich muss Ihrem Konto bei Docker Hub entsprechen. `benutzername` steht hier für Ihren eigenen Anmeldenamen. Docker lädt dabei nur die Schichten hoch, die die Registry noch nicht kennt. Aktualisiert man ein Image, wandert deshalb meist nur die oberste Schicht über das Netz.

> **Zugangsdaten gehören nicht in Images**
>
> Wer Images veröffentlicht, sollte vorher prüfen, dass weder Passwörter noch Schlüssel noch private Konfigurationsdateien darin stecken. Was ein-

> mal in einer Schicht steht, bleibt auch dann lesbar, wenn eine spätere Schicht die Datei löscht. Jeder, der das Image herunterlädt, kann die ältere Schicht öffnen.

Für den Betrieb im eigenen Netz muss es nicht Docker Hub sein. Eine eigene Registry lässt sich selbst mit einem Container betreiben. Das ist eine gute Übung, die ohne Konto bei einem Anbieter auskommt.

```
$ docker run -d --name registry -p 5000:5000 registry:2
$ docker tag meinserver/web:1.0 localhost:5000/web:1.0
$ docker push localhost:5000/web:1.0
The push refers to repository [localhost:5000/web]
...: Pushed
1.0: digest: sha256:... size: ...
$ docker rmi localhost:5000/web:1.0
$ docker pull localhost:5000/web:1.0
```

Der erste Befehl startet die Registry als Container auf Port 5000. Der Name `localhost:5000` im Image-Namen sagt Docker, dass die Registry der eigene Rechner ist und Docker Hub außen vor bleibt. Für diese Adresse darf Docker ausnahmsweise unverschlüsselt arbeiten. Eine Registry, die im Netzwerk erreichbar sein soll, braucht dagegen eine verschlüsselte Verbindung und eine Anmeldung. Mehr dazu in Kapitel 16. Räumen Sie danach mit `docker rm -f registry` auf.

7.5 Images bauen

Bisher haben wir fertige Images verwendet. Der Bau eines eigenen Images beginnt mit einem Rezept, dem *Dockerfile*, das in Kapitel 8 ausführlich behandelt wird. Hier genügt ein erstes kleines Beispiel, damit die folgenden Abschnitte konkret werden. Legen Sie ein Arbeitsverzeichnis an.

```
$ mkdir -p /srv/docker/images-demo && cd /srv/docker/images-demo
$ nano index.html
```

Die Datei `index.html` enthält eine einfache Seite.

index.html

```
<!DOCTYPE html>
<html lang="de">
<head><meta charset="utf-8"><title>Mein erstes Image</title></head>
<body><h1>Hallo aus meinem eigenen Image</h1></body>
</html>
```

Daneben legen Sie die Datei `Dockerfile` an. Der Name schreibt sich genau so, mit großem D und ohne Endung.

Dockerfile

```
FROM nginx:stable-alpine
COPY index.html /usr/share/nginx/html/index.html
```

Zwei Zeilen genügen. `FROM` wählt das Basisimage, hier nginx auf Alpine. `COPY` kopiert die Datei in das Image und ersetzt dort die Standardseite. Der Bau startet mit `docker build`.

```
$ docker build -t meine-seite:1.0 .
[+] Building 1.9s (7/7) FINISHED
 => [internal] load build definition from Dockerfile
 => [internal] load metadata for docker.io/library/nginx:stable-
    ↪ alpine
 => [1/2] FROM docker.io/library/nginx:stable-alpine
 => [2/2] COPY index.html /usr/share/nginx/html/index.html
 => exporting to image
 => => naming to docker.io/library/meine-seite:1.0
```

Die Ausgabe ist hier gekürzt. `-t meine-seite:1.0` gibt dem neuen Image Namen und Tag. Der Punkt am Ende ist wichtig: Er nennt den *Build-Kontext*, also das Verzeichnis, dessen Inhalt Docker für den Bau bekommt. Alles, was `COPY` kopieren soll, muss dort liegen. Der Punkt steht für das aktuelle Verzeichnis. Dass der Bau gelungen ist, zeigt ein Start.

```
$ docker run -d --name seite -p 8080:80 meine-seite:1.0
$ curl -s http://localhost:8080 | grep h1
```

```
<body><h1>Hallo aus meinem eigenen Image</h1></body>
```

Damit haben Sie Ihr erstes eigenes Image gebaut und gestartet.

7.6 Der Image-Cache

Wiederholen Sie den Bau, ohne etwas zu ändern. Er ist erstaunlich schnell.

```
$ docker build -t meine-seite:1.0 .
[+] Building 0.3s (7/7) FINISHED
 => CACHED [2/2] COPY index.html /usr/share/nginx/html/index.html
```

Das Wort `CACHED` bedeutet, dass Docker die frühere Schicht wiederverwendet und nichts neu erzeugt hat. Jeder Schritt im Dockerfile ergibt eine Schicht, und Docker merkt sich das Ergebnis. Beim nächsten Bau prüft es, ob sich seit damals etwas geändert hat. Für `COPY` vergleicht Docker dazu den Inhalt der kopierten Dateien. Ist alles gleich, nimmt es die gespeicherte Schicht.

Ändern Sie nun etwas in der `index.html` und bauen Sie neu. Docker erkennt, dass die kopierte Datei anders ist, und führt diesen Schritt neu aus. Eine wichtige Eigenschaft folgt daraus: Ändert sich ein Schritt, werden *alle Schritte danach* ebenfalls neu ausgeführt, auch wenn sich dort nichts geändert hat. Die Schritte davor kommen weiter aus dem Cache. Daraus lässt sich eine Faustregel ableiten, die Kapitel 8 aufgreift: Was sich selten ändert, gehört im Dockerfile nach oben, was sich oft ändert, nach unten.

Den Cache umgeht man mit `--no-cache`, etwa wenn ein Download im Dockerfile unbedingt frisch sein soll.

```
$ docker build --no-cache -t meine-seite:1.0 .
```

7.7 Image-Größen optimieren

Kleine Images laden schneller, belegen weniger Platz und enthalten weniger Programme, die Sicherheitslücken haben können. Fünf Maßnahmen bringen den größten Nutzen.

Die erste ist die Wahl einer kleinen Basis. Viele offizielle Images gibt es in mehreren Varianten. Die Variante mit `-alpine` baut auf Alpine Linux auf und ist oft um ein Vielfaches kleiner als die Standardvariante auf Debian-Basis. Die Variante mit `-slim` ist eine abgespeckte Debian-Fassung. Vergleichen Sie selbst.

```
$ docker pull nginx:stable
$ docker pull nginx:stable-alpine
$ docker images nginx
```

Die Ausgabe zeigt, dass die Alpine-Variante deutlich weniger Platz belegt. Die Zahlen unterscheiden sich je nach Version und werden hier nicht abgedruckt. Alpine verwendet eine andere C-Bibliothek (musl statt glibc). Einige Programme, die auf glibc angewiesen sind, laufen darauf nicht oder verhalten sich anders. In solchen Fällen ist `-slim` die Alternative.

Die zweite Maßnahme betrifft die Anzahl der Schichten und das Aufräumen. Wer in einem Schritt Pakete installiert und die Paketlisten im nächsten löscht, spart nichts, denn die Paketlisten bleiben in der früheren Schicht erhalten. Aufräumen muss im selben Schritt geschehen, in dem etwas entsteht. Das zeigt Kapitel 8 bei `RUN`.

Die dritte Maßnahme ist, nur das Nötige in den Build-Kontext zu legen. Eine Datei `.dockerignore` im Kontext-Verzeichnis schließt Dateien aus, die Docker nicht bekommen soll, etwa Projektverwaltungsordner oder Zugangsdaten.

.dockerignore

```
.git
*.log
secrets/
node_modules
```

Die vierte Maßnahme sind mehrstufige Builds, bei denen Werkzeuge zum Kompilieren nicht im fertigen Image landen. Sie bekommen in Kapitel 8 einen eigenen

Abschnitt. Die fünfte Maßnahme ist, nichts zu installieren, was die Anwendung nicht braucht: keine Editoren, keine Diagnoseprogramme und keine Compiler im endgültigen Image.

7.8 Image-Versionierung

Wer Images im Betrieb einsetzt, braucht eine klare Regel für die Tags. Bewährt hat sich die Anlehnung an eine Versionsnummer aus drei Teilen: Hauptversion, Nebenversion und Korrektur, zum Beispiel `1.4.2`. Veröffentlicht man ein Image, vergibt man gleich mehrere Tags auf dasselbe Image.

```
$ docker tag meine-seite:1.4.2 meine-seite:1.4
$ docker tag meine-seite:1.4.2 meine-seite:1
$ docker tag meine-seite:1.4.2 meine-seite:latest
```

Wer die ganz genaue Fassung braucht, nimmt `1.4.2`. Wer automatisch alle Korrekturen innerhalb von 1.4 bekommen möchte, nimmt `1.4`. Für Produktivsysteme gilt: Verwenden Sie nie `latest`. Damit wissen Sie nach einem Neustart nicht mehr, welche Fassung läuft, und ein Update kann unbemerkt einen Fehler mitbringen. Besser ist ein fester Tag, und für maximale Sicherheit der Digest. Aktualisieren Sie die Fassung dann bewusst und testen Sie sie vorher.

KAPITEL 8

Dockerfile

Das Dockerfile ist das Rezept für ein Image. Es steht in einer einfachen Textdatei, lässt sich mit jedem Editor schreiben und in einer Versionsverwaltung ablegen. Wer das Dockerfile hat, kann das Image jederzeit und auf jedem Rechner neu bauen. Dieses Kapitel erklärt Aufbau und Syntax, geht alle wichtigen Anweisungen einzeln durch und zeigt am Ende, wie man Images kleiner und sicherer macht.

8.1 Aufbau und Syntax

Ein Dockerfile besteht aus Anweisungen, von denen jede in einer eigenen Zeile beginnt. Zuerst kommt der Name der Anweisung, danach ihre Argumente. Üblich ist die Schreibweise in Großbuchstaben, damit man Anweisungen leicht von den Argumenten unterscheidet. Zeilen, die mit einem Rautenzeichen beginnen, sind Kommentare. Lange Zeilen darf man mit einem Backslash am Zeilenende umbrechen.

Dockerfile (Beispiel)

```
# Basisimage wählen
FROM alpine:3
# Ein Programm installieren
RUN apk add --no-cache curl \
    && curl --version
```

Docker liest das Dockerfile von oben nach unten. Jede Anweisung, die das Dateisystem verändert, erzeugt eine neue Schicht, wie in Kapitel 7 beschrieben. Die erste Anweisung muss `FROM` sein. Nur `ARG` darf davor stehen.

Zwei Schreibweisen gibt es bei Anweisungen, die einen Befehl enthalten, nämlich bei `RUN`, `CMD` und `ENTRYPOINT`. In der *Shell-Form* steht der Befehl als gewöhnlicher Text. Docker führt ihn dann über eine Shell aus, genauer `/bin/sh -c`. In der *Exec-Form* steht der Befehl als Liste in eckigen Klammern mit doppelten Anführungszeichen. Docker startet das Programm dann direkt, ohne Shell dazwischen.

Shell-Form und Exec-Form

```
CMD python app.py
CMD ["python", "app.py"]
```

Der Unterschied ist bei `CMD` und `ENTRYPOINT` wichtig. In der Shell-Form wird die Shell zum Hauptprozess des Containers, und das Programm läuft als ihr Kindprozess. Signale wie das SIGTERM von `docker stop` erreichen das Programm dann nicht zuverlässig, und der Container braucht für das Beenden die vollen zehn Sekunden. In der Exec-Form ist das Programm selbst der Hauptprozess und bekommt die Signale direkt. Für `CMD` und `ENTRYPOINT` verwendet man deshalb fast immer die Exec-Form.

8.2 FROM

`FROM` legt das Basisimage fest, auf dem alles aufbaut.

```
FROM nginx:stable-alpine
FROM debian:13-slim
FROM scratch
```

Das Basisimage bestimmt, welche Programme zur Verfügung stehen und wie groß das Ergebnis wird. Der Name `scratch` ist ein Sonderfall: Er bezeichnet ein vollkommen leeres Image ohne Betriebssystem. Darauf kann man ein einzelnes, vollständig eigenständiges Programm legen, wie es unser Beispiel im Abschnitt über mehrstufige Builds zeigt. Mit `AS` bekommt eine Stufe einen Namen, auf den spätere Stufen verweisen können.

8.3 RUN

`RUN` führt beim Bau einen Befehl aus und speichert das Ergebnis als neue Schicht. Typisch ist die Installation von Paketen.

RUN auf Alpine und auf Debian

```
# Alpine
RUN apk add --no-cache curl

# Debian
RUN apt-get update \
    && apt-get install -y --no-install-recommends curl \
    && rm -rf /var/lib/apt/lists/*
```

Bei Alpine verhindert `--no-cache`, dass der Paketindex im Image zurückbleibt. Bei Debian muss man das Aufräumen selbst erledigen: `apt-get update` lädt die Paketlisten, `--no-install-recommends` verzichtet auf empfohlene Zusatzpakete, und `rm -rf /var/lib/apt/lists/*` löscht die Listen wieder. Alles steht in einer einzigen `RUN`-Anweisung, verbunden mit `&&`. Das hat einen Grund. Würde man das Löschen in einer eigenen `RUN`-Anweisung erledigen, bliebe die Schicht mit den Paketlisten im Image erhalten, und das Image würde nicht kleiner. Zusammengehörige Schritte gehören daher in dieselbe Anweisung.

8.4 COPY und ADD

`COPY` kopiert Dateien aus dem Build-Kontext in das Image.

```
COPY index.html /usr/share/nginx/html/
COPY site/ /usr/share/nginx/html/
COPY --chown=app:app config.ini /etc/app/
```

Die erste Zeile kopiert eine Datei, die zweite den Inhalt eines Verzeichnisses. Die Option `--chown` setzt Besitzer und Gruppe der kopierten Dateien. Ohne sie gehören die Dateien Root. Das ist ein häufiger Grund, warum ein Programm, das als normaler Benutzer läuft, seine Dateien nicht lesen oder beschreiben kann.

`ADD` tut dasselbe, kann aber zwei Dinge mehr: Es entpackt lokale Archive (`.tar`, `.tar.gz`) automatisch und kann Dateien von einer Webadresse laden. Gerade diese Zusatzfunktionen machen das Verhalten schwerer vorhersehbar. Die Docker-Dokumentation empfiehlt, `COPY` zu verwenden, wo immer es genügt. `ADD` ist erst dann sinnvoll, wenn ein lokales Archiv automatisch entpackt werden soll. In der Regel ist es aber besser, einen Download mit `RUN curl` zu erledigen und zu prüfen.

Eine dritte Variante von `COPY` benutzt man bei mehrstufigen Builds: `COPY --from=build /hallo /hallo` holt Dateien aus einer früheren Stufe.

8.5 WORKDIR

`WORKDIR` legt das Arbeitsverzeichnis fest, in dem alle folgenden Anweisungen (`RUN`, `COPY`, `CMD` und andere) ausgeführt werden. Existiert das Verzeichnis nicht, legt Docker es an.

```
WORKDIR /app
COPY . .
```

Das ist übersichtlicher als lange Pfade in jeder Anweisung und besser als `RUN cd /app && …`, denn der Wechsel mit `cd` gilt nur innerhalb dieser einen `RUN`-Anweisung. Das Arbeitsverzeichnis gilt auch später für den laufenden Container.

8.6 ENV und ARG

Beide Anweisungen setzen Variablen, aber zu verschiedenen Zeitpunkten. `ENV` setzt eine Umgebungsvariable, die im fertigen Image erhalten bleibt und im laufenden Container zur Verfügung steht. `ARG` definiert eine Variable, die nur während des Baus existiert und danach verschwindet.

ENV und ARG

```
ARG VERSION=1.0
ENV APP_VERSION=${VERSION}
```

Der Standardwert von `ARG` lässt sich beim Bau überschreiben.

```
$ docker build --build-arg VERSION=2.0 -t pyseite:2.0 .
```

Im Beispiel gibt `ARG` den Wert beim Bau vor, und `ENV` macht daraus eine dauerhafte Variable des Images. Umgebungsvariablen eines Containers lassen sich außerdem beim Start mit `docker run -e` ändern, wie Kapitel 9 zeigt.

Keine Geheimnisse in ENV und ARG

Werte aus `ENV` und `ARG` stehen in den Metadaten des Images und sind mit `docker history` oder `docker inspect` lesbar. Passwörter, Schlüssel und Zugangstoken dürfen weder dort noch in einer `COPY`-Anweisung auftauchen. Für Zugangsdaten gibt es Secrets, die Kapitel 18 behandelt.

8.7 EXPOSE

`EXPOSE` vermerkt im Image, auf welchem Port die Anwendung lauscht.

```
EXPOSE 80
EXPOSE 8000/tcp
```

Wichtig ist, was `EXPOSE` *nicht* tut: Es veröffentlicht den Port nicht. Es ist eine Dokumentation für Menschen und Werkzeuge. Erreichbar wird der Port von außen erst mit `-p` beim Start. Mit `docker run -P` (großes P) veröffentlicht Docker alle mit `EXPOSE` vermerkten Ports auf zufälligen Host-Ports.

8.8 USER

Standardmäßig läuft der Prozess in einem Container als Root. Das ist im Container zwar weniger gefährlich als auf dem Host, aber unnötig riskant. `USER` legt fest, mit welchem Benutzer die folgenden Anweisungen und der Hauptprozess laufen. Der Benutzer muss im Image vorhanden sein, und meist legt man ihn mit `RUN` selbst an.

Benutzer anlegen und verwenden

```
# Debian-basierte Images
RUN useradd --create-home --uid 10001 app
# Alpine
RUN adduser -D -u 10001 app

USER app
```

Die feste Nummer 10001 ist Absicht. Sie liegt weit über den Nummern von Systembenutzern und ist unabhängig vom Namen eindeutig. Nach `USER` laufen alle weiteren Befehle ohne Root-Rechte, auch `RUN`, das deshalb keine Pakete mehr installieren kann. Die Reihenfolge ist daher wichtig: Pakete zuerst, dann `USER`. Kapitel 18 zeigt, wie man Non-Root-Container überprüft.

8.9 CMD und ENTRYPOINT

Beide Anweisungen legen fest, was beim Start des Containers läuft, aber auf verschiedene Weise. `CMD` gibt den Standardbefehl an, den man beim Start einfach ersetzen kann. `ENTRYPOINT` legt das Programm fest, das den Container ausmacht und das man nicht aus Versehen ersetzt. Kombiniert man beide, ist `ENTRYPOINT` das Programm und `CMD` liefert dessen Standardargumente.

Dockerfile für das Beispiel hallo

```
FROM alpine:3
ENTRYPOINT ["echo", "Hallo,"]
CMD ["Welt"]
```

```
$ docker build -t hallo .
$ docker run --rm hallo
Hallo, Welt
$ docker run --rm hallo Docker
Hallo, Docker
```

Beim zweiten Aufruf ersetzt das Argument `Docker` nur den `CMD`-Teil, der `ENTRYPOINT` bleibt. Wollen Sie ihn trotzdem austauschen, etwa um in den Con-

tainer hineinzusehen, geht das mit `docker run --entrypoint sh`. Die Wirkung der Kombinationen fasst die Tabelle zusammen.

Im Dockerfile	Ohne Argumente	Mit Argumenten bei `docker run`
nur `CMD`	führt `CMD` aus	Argumente ersetzen `CMD` vollständig
nur `ENTRYPOINT`	führt `ENTRYPOINT` aus	Argumente werden angehängt
beide	`ENTRYPOINT` mit `CMD` als Argumenten	Argumente ersetzen nur `CMD`

Als Faustregel gilt: Wer ein Image baut, das ein bestimmtes Programm ausführt, nimmt `ENTRYPOINT` für das Programm und `CMD` für Standardoptionen. Wer ein Image baut, das jemand nach Belieben mit verschiedenen Befehlen starten soll, nimmt nur `CMD`.

8.10 HEALTHCHECK

Ein Container gilt als laufend, solange sein Hauptprozess existiert. Das sagt nichts darüber, ob die Anwendung auch funktioniert. Ein Webserver kann hängen und trotzdem als Prozess vorhanden sein. `HEALTHCHECK` gibt Docker einen Befehl, mit dem es regelmäßig prüfen kann, ob die Anwendung gesund ist.

HEALTHCHECK

```
HEALTHCHECK --interval=10s --timeout=3s --start-period=5s --retries=3
    ↪ \
  CMD python -c "import urllib.request; urllib.request.urlopen('http
    ↪ ://localhost:8000/')"
```

Der Befehl nach `CMD` läuft im Container. Beendet er sich mit dem Wert 0, gilt der Container als gesund, mit 1 als krank. Die Optionen steuern den Ablauf: `--interval` ist der Abstand zwischen zwei Prüfungen, `--timeout` die Zeit, nach der eine Prüfung als fehlgeschlagen gilt, `--start-period` eine Schonfrist beim Start, in der Fehlschläge noch nicht zählen, und `--retries` die Zahl der Fehlschlä-

ge in Folge, nach der der Zustand auf krank wechselt. Dieser Zustand erscheint in `docker ps` und `docker inspect`.

```
$ docker inspect --format '{{.State.Health.Status}}' pyseite
healthy
```

Die möglichen Zustände sind `starting`, `healthy` und `unhealthy`. Wichtig zu wissen ist, dass Docker einen kranken Container nicht von selbst neu startet. Der Zustand dient als Information für Sie, für Docker Compose (Kapitel 12) und für Überwachungswerkzeuge. Ein Programm, das auf den Zustand achten soll, muss ihn selbst abfragen.

8.11 Ein vollständiges Beispiel

Die bisherigen Anweisungen fügen wir nun in einem Image zusammen. Es soll eine statische Webseite mit dem Python-Webserver ausliefern, als normaler Benutzer, mit Gesundheitsprüfung und einer Versionsangabe. Legen Sie ein Verzeichnis an.

```
$ mkdir -p /srv/docker/dockerfile-demo/site && cd /srv/docker/
    ↪ dockerfile-demo
$ echo '<h1>Hallo aus Python</h1>' > site/index.html
```

Dazu das Dockerfile.

Dockerfile

```
FROM python:3-slim

ARG VERSION=1.0
ENV APP_VERSION=${VERSION}

RUN useradd --create-home --uid 10001 app

WORKDIR /home/app/site
COPY --chown=app:app site/ .
```

```
USER app
EXPOSE 8000

HEALTHCHECK --interval=10s --timeout=3s --start-period=5s --retries=3
    ↪ \
  CMD python -c "import urllib.request; urllib.request.urlopen('http
    ↪ ://localhost:8000/')"

CMD ["python", "-m", "http.server", "8000"]
```

Jede Zeile entspricht einer Anweisung dieses Kapitels. Das Basisimage ist die schlanke Python-Variante auf Debian-Basis. `ARG` und `ENV` halten die Version fest. Der Benutzer `app` wird angelegt, bevor die Dateien kopiert werden, und die Anweisung `COPY --chown` übergibt sie an ihn. Erst danach schaltet `USER` auf den neuen Benutzer um. Der Webserver startet in der Exec-Form von `CMD`. Bauen und starten Sie das Image.

```
$ docker build -t pyseite:1.0 .
$ docker run -d --name pyseite -p 8081:8000 pyseite:1.0
$ curl -s http://localhost:8081
<h1>Hallo aus Python</h1>
$ docker exec pyseite id
uid=10001(app) gid=10001(app) groups=10001(app)
$ docker exec pyseite printenv APP_VERSION
1.0
```

Der Webserver antwortet, der Prozess läuft nicht als Root, und die Variable aus dem Dockerfile ist im Container gesetzt. Nach etwa einer Minute zeigt `docker inspect` den Zustand `healthy`, wie oben gezeigt. Räumen Sie mit `docker rm -f pyseite` auf.

8.12 Mehrstufige Builds

Wer ein Programm in einer Sprache wie Go oder C schreibt, braucht zum Bau einen Compiler. Zum Ausführen genügt das fertige Programm. Würde man beides in einem Image zusammenpacken, schleppte das Endprodukt hunderte Megabyte an Werkzeugen mit, die es nie wieder braucht. Ein *mehrstufiger Build* löst das Problem. Das Dockerfile enthält mehrere `FROM`-Anweisungen, jede beginnt eine neue Stufe, und die letzte Stufe wird zum fertigen Image. Aus früheren Stufen holt man mit `COPY --from` nur die Ergebnisse.

Das Beispiel ist ein winziges Go-Programm. Legen Sie ein Verzeichnis mit drei Dateien an.

main.go

```
package main

import "fmt"

func main() {
^^Ifmt.Println("Hallo aus einem winzigen Container")
}
```

go.mod

```
module hallo

go 1.22
```

Dockerfile

```
FROM golang:1-alpine AS build
WORKDIR /src
COPY go.mod main.go ./
RUN CGO_ENABLED=0 go build -o /hallo .

FROM scratch
COPY --from=build /hallo /hallo
ENTRYPOINT ["/hallo"]
```

Die erste Stufe heißt `build`. Sie enthält den Go-Compiler und übersetzt das Programm in eine einzelne Datei. Die Variable `CGO_ENABLED=0` sorgt dafür, dass das Programm vollständig eigenständig ist und keine Systembibliotheken braucht. Die zweite Stufe beginnt mit `scratch`, dem leeren Image, und nimmt aus der ersten nur die fertige Datei mit.

```
$ docker build -t hallo-go .
$ docker run --rm hallo-go
Hallo aus einem winzigen Container
$ docker images hallo-go
```

Das fertige Image umfasst nur wenige Megabyte, während das Image der Bauphase mehrere hundert Megabyte groß ist. Die Compiler-Stufe verschwindet nicht aus dem Cache, sie wird aber nicht ausgeliefert. Das spart Platz, beschleunigt Downloads und verringert die Angriffsfläche, denn im Endprodukt gibt es weder eine Shell noch einen Paketmanager.

8.13 Best Practices

Die folgenden Regeln haben sich im Alltag bewährt. Sie ergeben sich aus dem, was in diesem und im vorigen Kapitel erklärt wurde.

Wählen Sie ein kleines, gepflegtes Basisimage und legen Sie den Tag fest. Eine Angabe wie `python:3-slim` reicht zum Lernen. Im Betrieb verwenden Sie eine genaue Fassung oder den Digest, damit der Bau reproduzierbar bleibt. Ordnen Sie die Anweisungen so, dass sich selten ändernde Schritte oben stehen. Zuerst kommen Pakete und Abhängigkeiten, zuletzt der eigene Code. So bleiben die oberen Schichten im Cache, und der Bau nach einer Codeänderung dauert Sekunden.

Fassen Sie zusammengehörige Schritte in einer `RUN`-Anweisung zusammen und räumen Sie im selben Schritt auf. Verwenden Sie `COPY` statt `ADD`, wo es geht. Legen Sie eine Datei `.dockerignore` an, damit nur Notwendiges im Build-Kontext landet. Lassen Sie den Prozess als normaler Benutzer laufen und verwenden Sie bei `CMD` und `ENTRYPOINT` die Exec-Form. Betreiben Sie in einem Container möglichst einen Prozess. Muss etwas dauerhaft zusammenlaufen, gehört es in getrennte Container, die Compose verbindet.

Verzichten Sie auf Geheimnisse im Image, denn jede Schicht lässt sich auslesen. Und prüfen Sie jedes Dockerfile mit dem eingebauten Werkzeug, das Fehler und Verbesserungsmöglichkeiten meldet.

```
$ docker build --check .
```

Die Option `--check` führt keine Builds durch, sondern prüft das Dockerfile gegen die Empfehlungen von Docker und gibt Hinweise aus, etwa zu unklaren Variablen oder zur Schreibweise der Anweisungen.

KAPITEL 9

Containerverwaltung

Container zu starten ist einfach. Sie im Betrieb zu führen, erfordert ein paar weitere Werkzeuge: Man muss sie vorbereiten, mit Einstellungen versorgen, in ihrem Verbrauch begrenzen, untersuchen und am Ende wieder aufräumen. Dieses Kapitel zeigt alle diese Handgriffe an Beispielen. Es setzt die Kenntnis der Grundbefehle aus Kapitel 6 voraus.

9.1 Container erstellen, starten und stoppen

Bisher haben wir Container mit `docker run` erzeugt und gleichzeitig gestartet. Beide Schritte lassen sich trennen. `docker create` legt einen Container an, ohne ihn zu starten. Er steht danach im Zustand *created*, wie Abbildung 2.1 zeigt.

```
$ docker create --name vorbereitet -p 8082:80 nginx
$ docker ps -a --filter name=vorbereitet --format "table {{.Names}}\t
    ↪ {{.Status}}"
NAMES         STATUS
vorbereitet   Created
$ docker start vorbereitet
vorbereitet
```

Das Trennen hat Vorteile, wenn man einen Container erst konfigurieren will, bevor er loslegt, oder wenn man viele Container vorbereitet und später gemeinsam startet. Im Alltag nimmt man trotzdem meist `docker run`. Zum Stoppen gibt es

die Befehle aus Kapitel 6: `docker stop` für das geordnete Beenden, `docker kill` für den sofortigen Abbruch und `docker restart` für beides nacheinander.

9.2 Interaktive Container und Detached Mode

Container laufen auf zwei grundlegend verschiedene Arten. Im *interaktiven Modus* (`-it`) ist Ihr Terminal mit dem Container verbunden. Sie tippen, der Container antwortet, und mit dem Ende der Sitzung endet auch der Container. Das eignet sich für Experimente, für kurze Werkzeuge und zum Nachsehen in einem Image.

```
$ docker run -it --rm debian:13-slim bash
root@8f2a1c3d9b41:/# cat /etc/os-release | head -n 2
PRETTY_NAME="Debian GNU/Linux 13 (trixie)"
NAME="Debian GNU/Linux"
root@8f2a1c3d9b41:/# exit
```

Im *Detached Mode* (`-d`) läuft der Container im Hintergrund. Das Terminal bleibt frei, und der Container arbeitet weiter, auch wenn Sie sich vom Server abmelden. So betreibt man Dienste wie Webserver und Datenbanken. Wer später doch einen Blick in den laufenden Container werfen will, nutzt `docker logs` und `docker exec`.

Ein dritter Weg liegt dazwischen. Mit `docker attach` verbindet man sich mit der Standardein- und -ausgabe des Hauptprozesses eines laufenden Containers. Das ist selten sinnvoll, denn ein versehentliches [Strg]+[C] kann den Container beenden. `docker exec` ist fast immer die bessere Wahl.

9.3 Umgebungsvariablen

Viele Images lassen sich über Umgebungsvariablen einstellen, ohne dass man das Image ändern muss. Die Datenbank-Images tun das für Passwörter und Benutzernamen, wie Kapitel 14 zeigt. Mit `-e` setzt man eine Variable beim Start.

```
$ docker run --rm -e GRUSS=Hallo alpine printenv GRUSS
Hallo
```

Bei mehreren Variablen wird die Befehlszeile schnell lang. Dann legt man sie in eine Datei, eine Variable pro Zeile, und übergibt sie mit `--env-file`.

app.env

```
GRUSS=Hallo
SPRACHE=de
```

```
$ docker run --rm --env-file app.env alpine printenv
GRUSS=Hallo
SPRACHE=de
...
```

Zusätzlich zu den eigenen Variablen setzt Docker einige selbst, zum Beispiel `HOSTNAME` und `PATH`. Daher zeigt `printenv` mehr Zeilen, als hier abgedruckt sind. Variablen aus `--env-file` und `-e` überschreiben gleichnamige `ENV`-Angaben im Dockerfile.

Passwörter in Variablen

Umgebungsvariablen sind bequem, aber nicht geheim. Wer `docker inspect` ausführen darf, sieht alle Werte, und viele Programme schreiben ihre Umgebung bei einem Absturz ins Protokoll. Dateien mit Passwörtern gehören nicht in ein Verzeichnis, das in einer Versionsverwaltung landet. Sicherere Wege zeigt Kapitel 18.

9.4 Ressourcenlimits

Ohne Einschränkung darf ein Container so viel Prozessorzeit und Arbeitsspeicher verbrauchen, wie der Host hergibt. Ein fehlerhaftes Programm kann dann den ganzen Server lahmlegen. Docker setzt dazu die Control Groups ein, die Kapitel 2 vorgestellt hat. Die wichtigsten Optionen sind in der Tabelle zusammengefasst.

Option	Wirkung
`--memory 256m`	begrenzt den Arbeitsspeicher auf 256 Megabyte
`--memory-swap`	legt die Obergrenze für Arbeitsspeicher und Auslagerung zusammen fest
`--cpus 0.5`	erlaubt höchstens eine halbe Prozessorkern-Leistung
`--pids-limit 100`	begrenzt die Zahl der Prozesse im Container

Ein Beispiel mit zwei Limits.

```
$ docker run -d --name begrenzt --memory 128m --cpus 0.5 nginx
$ docker stats --no-stream --format "table {{.Name}}\t{{.CPUPerc}}\t
    ↪ {{.MemUsage}}"
NAME        CPU %    MEM USAGE / LIMIT
begrenzt    0.00%    4.5MiB / 128MiB
```

Die Spalte `MEM USAGE / LIMIT` zeigt den Verbrauch und das gesetzte Limit. Dass Docker dieses Limit tatsächlich im Kernel hinterlegt, lässt sich nachprüfen. Auf einem Host mit cgroup v2 und `systemd` liegt die Einstellung in einer Datei im Verzeichnis der cgroup des Containers.

```
$ ID=$(docker inspect --format '{{.Id}}' begrenzt)
$ cat /sys/fs/cgroup/system.slice/docker-$ID.scope/memory.max
134217728
$ cat /sys/fs/cgroup/system.slice/docker-$ID.scope/cpu.max
50000 100000
```

Der Wert `134217728` sind genau 128 Mebibyte in Byte. `50000 100000` bedeutet, dass der Container in jedem Zeitfenster von 100 000 Mikrosekunden höchstens

50 000 davon Prozessorzeit bekommt, also eine halbe CPU. Hier schließt sich der Kreis zu Kapitel 2: Docker erzeugt keine eigene Technik für Limits, es schreibt die Werte in cgroup-Dateien.

Überschreitet ein Prozess sein Speicherlimit, beendet der Kernel ihn. Man spricht vom *OOM-Kill* (Out of Memory). Der Container endet dann mit dem Rückgabewert 137. Der Zustand lässt sich nachlesen.

```
$ docker inspect --format '{{.State.OOMKilled}} {{.State.ExitCode}}'
    ↪ begrenzt
false 0
```

Bei einem Container, der durch das Limit beendet wurde, steht dort `true 137`. Zu knappe Limits sind eine häufige Ursache für Container, die ohne erkennbaren Grund ausfallen. Wählen Sie Werte, die zum tatsächlichen Verbrauch des Programms passen, und beobachten Sie ihn zuerst. Kapitel 19 vertieft das. Das Limit eines laufenden Containers ändert `docker update`, zum Beispiel `docker update --memory 256m --memory-swap 256m begrenzt`.

9.5 Container untersuchen

Für das Untersuchen laufender Container gibt es neben `docker logs`, `docker inspect` und `docker exec` drei weitere Befehle. `docker stats` zeigt den Verbrauch an Prozessor, Arbeitsspeicher, Netzwerk und Platten in Echtzeit, bis Sie [Strg]+[C] drücken. Mit `--no-stream` gibt es nur eine Momentaufnahme, wie oben. `docker top` listet die Prozesse im Container auf, vom Host aus betrachtet.

```
$ docker top begrenzt
UID    PID    PPID   C   STIME   TTY   TIME       CMD
root   4721   4698   0   10:52   ?     00:00:00   nginx: master
    ↪ process nginx -g daemon off;
101    4770   4721   0   10:52   ?     00:00:00   nginx: worker
    ↪ process
```

Die Prozessnummern sind die, die der Host vergibt, nicht die im Container. Im Container trägt der Hauptprozess die Nummer 1. Wer sie auf dem Host mit `ps` sucht, findet dieselben Prozesse mit anderen Nummern. Das ist der pid-

Namespace aus Kapitel 2. Der dritte Befehl, `docker diff`, zeigt, was sich im Dateisystem des Containers gegenüber dem Image geändert hat.

```
$ docker exec begrenzt touch /tmp/test.txt
$ docker diff begrenzt
C /tmp
A /tmp/test.txt
...
```

Ein `A` steht für eine hinzugefügte Datei (*added*), ein `C` für ein geändertes Verzeichnis (*changed*) und ein `D` für eine gelöschte Datei (*deleted*). Das ist die beschreibbare Schicht des Containers aus Abbildung 7.1. Die weiteren Zeilen ergeben sich aus dem Betrieb von nginx, etwa Änderungen an temporären Verzeichnissen. Ein Container, bei dem `docker diff` viele Einträge in Datenverzeichnissen zeigt, ist ein Hinweis darauf, dass dort eigentlich ein Volume hingehört.

9.6 Container bereinigen

Wer viel experimentiert, sammelt gestoppte Container, ungenutzte Images und Reste an. Docker hat dafür eigene Befehle. Zuerst verschafft man sich einen Überblick, wie viel Platz belegt ist.

```
$ docker system df
TYPE            TOTAL   ACTIVE   SIZE      RECLAIMABLE
Images          4       2        512MB     340MB (66%)
Containers      3       1        1.2MB     800kB (66%)
Local Volumes   1       0        45MB      45MB (100%)
Build Cache     12      0        120MB     120MB
```

Die Zahlen sind ein Beispiel. Die Spalte `RECLAIMABLE` nennt, wie viel sich freigeben ließe. Zum Aufräumen dienen die `prune`-Befehle, die jeweils eine Art von Objekten bereinigen.

Befehl	Entfernt
`docker container prune`	alle gestoppten Container
`docker image prune`	Images ohne Tag, auf die nichts mehr verweist (mit `-a` alle ungenutzten)
`docker volume prune`	unbenutzte *anonyme* Volumes (mit `-a` auch unbenutzte benannte)
`docker network prune`	ungenutzte Netzwerke
`docker builder prune`	den Build-Cache
`docker system prune`	gestoppte Container, ungenutzte Netzwerke, Images ohne Tag und ungenutzten Build-Cache in einem Schritt

Jeder dieser Befehle fragt vor dem Löschen nach, was sich mit `-f` abschalten lässt. Lesen Sie die Rückfrage, denn bei `prune` gibt es kein Zurück.

> **Volumes können Daten enthalten**
>
> `docker volume prune` löscht Volumes, die kein Container verwendet. Seit Docker 23 trifft das in der Grundeinstellung nur *anonyme* Volumes. Mit

der Option `-a` (`--all`) gehören auch benannte dazu. Auch anonyme Volumes können wichtige Daten enthalten: Die Datenbank-Images legen sie an, wenn man ihnen kein eigenes Volume gibt (Kapitel 10). Hat man den Container einer solchen Datenbank gelöscht, ist ihr anonymes Volume ungenutzt, und `prune` löscht es samt Daten. Die Daten sind danach unwiederbringlich verloren. Der Befehl `docker system prune --volumes` räumt ebenfalls anonyme Volumes ab. Prüfen Sie vorher mit `docker volume ls`, was vorhanden ist, und sichern Sie Wichtiges, wie Kapitel 10 beschreibt.

9.7 Automatische Neustartregeln

Ein Container, der abstürzt oder nach einem Neustart des Servers nicht mehr läuft, ist im Betrieb ein Problem. Neustartregeln (*restart policies*) sagen Docker, wann es einen Container selbst wieder starten soll. Sie werden mit `--restart` gesetzt.

Regel	Verhalten
`no`	kein automatischer Neustart (Voreinstellung)
`on-failure[:n]`	Neustart nur, wenn der Container mit einem Fehlerwert endet, höchstens `n`-mal
`always`	Neustart immer, auch nach einem Neustart des Docker-Daemons
`unless-stopped`	wie `always`, aber nicht, wenn der Container von Hand gestoppt wurde

Die Wirkung von `on-failure` lässt sich mit einem Container zeigen, der absichtlich fehlschlägt.

```
$ docker run -d --name kaputt --restart on-failure:3 alpine sh -c "
    ↪ exit 1"
$ sleep 5
$ docker inspect --format '{{.RestartCount}} {{.State.Status}}'
    ↪ kaputt
```

```
3 exited
```

Docker hat den Container dreimal neu gestartet und danach aufgegeben. Zwischen den Versuchen wartet es mit wachsendem Abstand. Der Unterschied zwischen `always` und `unless-stopped` zeigt sich beim Neustart des Daemons: Ein von Hand gestoppter Container mit `always` startet danach trotzdem wieder, mit `unless-stopped` nicht. Für Dienste, die dauerhaft laufen sollen, ist `unless-stopped` die gängige Wahl. Die Regel eines vorhandenen Containers lässt sich nachträglich ändern.

```
$ docker update --restart unless-stopped begrenzt
```

Eine Neustartregel ersetzt keine Fehlersuche. Ein Container, der in einer Schleife abstürzt und neu startet, verbirgt das Problem nur. Prüfen Sie bei einem solchen Container mit `docker logs`, warum er endet, und beachten Sie den Zähler `RestartCount`. Räumen Sie die Beispiele dieses Kapitels mit den folgenden Befehlen auf.

```
$ docker rm -f vorbereitet begrenzt kaputt
```

Lernkontrolle zu Teil III

Die Aufgaben prüfen die Kapitel 7 bis 9. Die praktischen Aufgaben führen Sie auf Ihrem Übungsserver durch. Die Lösungen stehen im Anhang.

Verständnisfragen

V1. Was ist ein Layer, und warum belegen zehn Container aus demselben Image nicht das Zehnfache an Platz?

V2. Worin unterscheiden sich ein Tag und ein Digest? Warum sollte man `latest` im Produktivbetrieb meiden?

V3. Was bedeutet der Punkt am Ende von `docker build -t meine-seite:1.0 .`?

V4. Warum werden nach einer Änderung in der Mitte eines Dockerfiles alle folgenden Schritte neu ausgeführt?

V5. Worin unterscheiden sich `ENV` und `ARG`?

V6. Warum verwendet man bei `CMD` und `ENTRYPOINT` die Exec-Form?

V7. Was bewirkt `EXPOSE 80`, und was bewirkt es nicht?

V8. Nennen Sie zwei Vorteile mehrstufiger Builds.

Multiple-Choice-Aufgaben

M1. Welche Anweisung legt das Basisimage fest?

(a) `BASE`
(b) `FROM`
(c) `IMAGE`
(d) `USE`

M2. Mit welcher Option begrenzt man den Arbeitsspeicher eines Containers auf 256 Megabyte?

(a) `--ram 256m`
(b) `--mem 256`
(c) `--memory 256m`
(d) `--limit 256`

M3. Wodurch wird ein Port im laufenden Container von außen erreichbar?

(a) Durch `EXPOSE` im Dockerfile
(b) Durch `PUBLISH` im Dockerfile
(c) Durch `PORT` im Dockerfile
(d) Durch `-p` beim Start des Containers

M4. Ein Container mit `--restart unless-stopped` wurde von Hand gestoppt. Dann startet der Docker-Daemon neu. Was geschieht?

(a) Der Container bleibt gestoppt.
(b) Der Container startet neu.
(c) Der Container startet nur, wenn er gesund ist.
(d) Es hängt vom Image ab.

M5. Was bedeutet der Buchstabe `A` in der Ausgabe von `docker diff`?

(a) archived
(b) attached
(c) aborted
(d) added

M6. Welcher Befehl entfernt alle gestoppten Container?

(a) `docker rm --all`
(b) `docker container prune`
(c) `docker clean`

(d) `docker system stop`

Praktische Aufgaben

P1. Bauen Sie ein Image mit dem Namen `meine-seite:2.0`, das auf `nginx:stable-alpine` aufsetzt und eine eigene `index.html` enthält. Starten Sie es auf Host-Port 8083, prüfen Sie die Seite mit `curl` und entfernen Sie Container und Image wieder.

P2. Bauen Sie dasselbe Image zweimal hintereinander und beobachten Sie die Ausgabe. Ändern Sie dann die `index.html` und bauen Sie ein drittes Mal. Notieren Sie, welche Schritte jeweils aus dem Cache kommen.

P3. Starten Sie einen nginx-Container mit `--memory 64m --cpus 0.25`. Lesen Sie die Werte aus den Dateien `memory.max` und `cpu.max` in der cgroup des Containers aus und vergleichen Sie sie mit Ihren Angaben.

Fehlersuche

F1. `docker build -t meine-seite:1.0` bricht ab und meldet, es werde ein Argument erwartet. Was fehlt?

F2. Beim Bau meldet `COPY index.html /usr/share/nginx/html/`, die Datei sei nicht gefunden worden, obwohl sie im Verzeichnis liegt, in dem Sie arbeiten. Nennen Sie zwei mögliche Ursachen.

F3. Ein Container, der mit `USER app` läuft, meldet `Permission denied` beim Schreiben in `/data`. Woran liegt das, und wie beheben Sie es?

Szenario aus dem Betrieb

Ein Kollege hat ein Image gebaut, das 1,2 GB groß ist. Jede kleine Codeänderung führt zu einem Bau von mehreren Minuten, und auf dem Produktivserver läuft das Image mit dem Tag `latest`. Welche Verbesserungen schlagen Sie vor, und in welcher Reihenfolge gehen Sie vor?

TEIL IV

Speicher und Netzwerke

KAPITEL 10

Docker Storage

Container sind vergänglich. Man startet sie, ersetzt sie durch neue Fassungen und löscht sie, wenn sie nicht mehr gebraucht werden. Die Daten, die eine Anwendung erzeugt, sollen dabei meist erhalten bleiben: die Datenbank, die hochgeladenen Dateien, die Konfiguration. Dieses Kapitel zeigt die Möglichkeiten, die Docker dafür bietet, und erklärt, wann man welche wählt. Am Ende beherrschen Sie auch Sicherung, Wiederherstellung und den Umgang mit Zugriffsrechten.

10.1 Das Dateisystem des Containers

Jeder Container hat, wie in Kapitel 2 und 7 gezeigt, eine dünne beschreibbare Schicht über den schreibgeschützten Schichten des Images. Alles, was der Hauptprozess in das Dateisystem schreibt, landet dort. Das funktioniert ohne weiteres Zutun, hat aber drei Nachteile.

Erstens ist die Schicht an den Container gebunden. Löscht man ihn mit `docker rm`, verschwinden die Daten mit. Zweitens ist sie schwer zugänglich. Andere Container und der Host kommen nicht ohne Umwege an die Dateien heran. Drittens ist das Schreiben in die Schicht langsamer als in ein gewöhnliches Verzeichnis, weil das Union-Dateisystem jede Änderung über den Copy-on-Write-Mechanismus abwickelt. Für Datenbanken ist das spürbar.

Wie schnell dabei Daten verloren gehen, zeigt ein kurzer Versuch.

```
$ docker run -d --name fluechtig alpine sh -c "echo wichtig > /notiz.
    ↪ txt && sleep 300"
$ docker exec fluechtig cat /notiz.txt
wichtig
$ docker rm -f fluechtig
$ docker run --rm alpine cat /notiz.txt
cat: can't open '/notiz.txt': No such file or directory
```

Der erste Container hat die Datei geschrieben, und sie war lesbar, solange er existierte. Der zweite Container, aus demselben Image gestartet, findet sie nicht, denn sie lag in der beschreibbaren Schicht des ersten.

10.2 Drei Wege, Daten zu speichern

Docker kennt drei Mechanismen, um Daten außerhalb der beschreibbaren Schicht abzulegen. Alle drei binden ein Verzeichnis in den Container ein (man sagt: sie mounten es), und der Container sieht dort ganz normale Dateien.

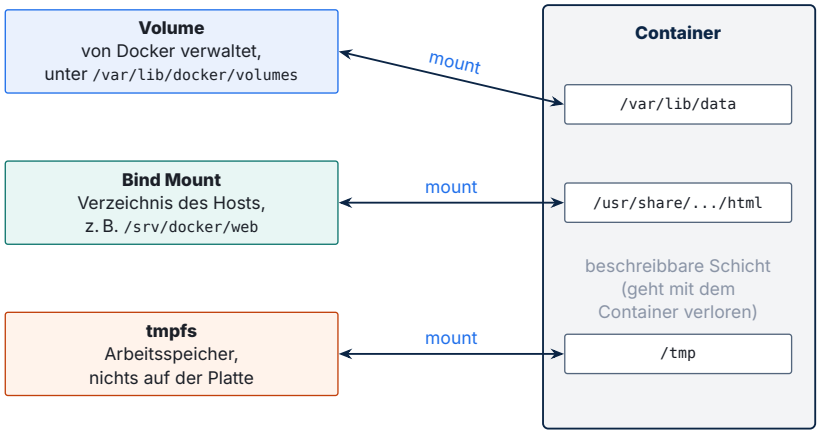


Abbildung 10.1: Volume, Bind Mount und tmpfs binden Speicher in den Container ein

Die Tabelle fasst die Unterschiede zusammen.

	Volume	Bind Mount	tmpfs
Speicherort	von Docker verwaltet (`/var/lib/docker/volumes`)	beliebiges Verzeichnis des Hosts	Arbeitsspeicher
Überlebt den Container	ja	ja	nein
Typischer Einsatz	Datenbanken, Anwendungsdaten	Konfiguration, Webinhalte, Entwicklung	kurzlebige oder geheime Daten
Sicherung	über Docker-Befehle	mit gewöhnlichen Dateiwerkzeugen	nicht vorgesehen

10.3 Volumes

Ein *Volume* ist ein von Docker verwaltetes Verzeichnis. Man muss sich nicht darum kümmern, wo es liegt, und kann es mehreren Containern gleichzeitig zur Verfügung stellen. Das ist der empfohlene Weg für Daten, die ein Container erzeugt und die dauerhaft bleiben sollen.

Fachbegriff: Volume

Ein Volume ist ein benannter Datenspeicher, den Docker anlegt und verwaltet. Er existiert unabhängig von jedem Container und bleibt erhalten, wenn Container gelöscht werden.

Die Befehle dafür beginnen mit `docker volume`.

```
$ docker volume create daten
daten
$ docker volume ls
DRIVER    VOLUME NAME
local     daten
$ docker volume inspect daten
[
    {
```

```
        "CreatedAt": "2026-11-11T11:20:41+01:00",
        "Driver": "local",
        "Labels": null,
        "Mountpoint": "/var/lib/docker/volumes/daten/_data",
        "Name": "daten",
        "Options": null,
        "Scope": "local"
    }
]
```

Der Eintrag `Mountpoint` verrät, wo auf dem Host die Dateien liegen. Dort sollte man nur nachsehen und nichts von Hand ändern, denn die Verwaltung gehört Docker. Verwendet wird ein Volume mit der Option `-v` (Kurzform) oder `--mount` (ausführliche Form). Das Beispiel wiederholt den Versuch von eben, diesmal mit einem Volume.

```
$ docker run --rm -v daten:/ablage alpine sh -c "echo wichtig > /
    ↪ ablage/notiz.txt"
$ docker run --rm -v daten:/ablage alpine cat /ablage/notiz.txt
wichtig
```

Beide Container sind längst gelöscht, aber die Datei ist noch da. Hinter dem Doppelpunkt steht der Pfad im Container. Dieselbe Angabe lautet in der ausführlichen Schreibweise so:

```
$ docker run --rm --mount type=volume,source=daten,target=/ablage
    ↪ alpine ls /ablage
notiz.txt
```

Die ausführliche Form ist länger, aber selbsterklärend. Sie meldet außerdem Fehler genauer. Welche Schreibweise Sie verwenden, ist Geschmackssache. In Compose-Dateien begegnet Ihnen eine dritte Variante, die Kapitel 12 zeigt.

Zwei Eigenheiten sind gut zu kennen. Die erste: Wenn Sie ein *leeres* Volume in ein Verzeichnis einbinden, das im Image schon Dateien enthält, kopiert Docker diese Dateien beim ersten Einbinden in das Volume. Das ist bequem, denn der Container findet seine Standarddateien vor. Bei einem Bind Mount passiert das

nicht, dort verdeckt das eingebundene Verzeichnis den Inhalt des Images. Die zweite: Gibt man beim Start keinen Namen an, legt Docker ein *anonymes Volume* mit zufälligem Namen an. Das geschieht auch, wenn ein Image eine `VOLUME`-Anweisung im Dockerfile enthält. Die Datenbank-Images tun das. Solche Volumes sind schwer wiederzufinden, und sie sammeln sich an. Geben Sie deshalb wichtigen Daten immer einen eigenen Namen.

```
$ docker volume rm daten
daten
```

`docker volume rm` löscht ein Volume und alle Daten darin. Solange ein Container es verwendet, auch ein gestoppter, verweigert Docker das Löschen.

10.4 Bind Mounts

Ein *Bind Mount* bindet ein bestimmtes Verzeichnis oder eine bestimmte Datei des Hosts in den Container ein. Der Container und der Host sehen dann dieselben Dateien, und Änderungen sind auf beiden Seiten sofort sichtbar. Das macht Bind Mounts ideal für Konfigurationsdateien, die Sie auf dem Host pflegen, und für Webinhalte, die Sie mit einem Editor bearbeiten.

```
$ mkdir -p /srv/docker/bind-demo/html
$ echo '<h1>Vom Host eingebunden</h1>' > /srv/docker/bind-demo/html/
    ↪ index.html
$ docker run -d --name bind-web -p 8084:80 \
    -v /srv/docker/bind-demo/html:/usr/share/nginx/html:ro nginx
$ curl -s http://localhost:8084
<h1>Vom Host eingebunden</h1>
```

Der Pfad links vom Doppelpunkt ist ein Verzeichnis auf dem Host, der Pfad rechts der Ort im Container. Das Anhängsel `:ro` steht für *read-only*: Der Container darf die Dateien lesen, aber nicht verändern. Für Inhalte und Konfigurationen, die der Container nur braucht, ist das eine sinnvolle Absicherung. Jetzt ändern wir die Datei auf dem Host.

```
$ echo '<h1>Geändert, ohne den Container anzufassen</h1>' > /srv/
    ↪ docker/bind-demo/html/index.html
$ curl -s http://localhost:8084
<h1>Geändert, ohne den Container anzufassen</h1>
```

Die Änderung ist ohne Neustart sichtbar. Ein Bind Mount unterscheidet sich von einem Volume in einem Punkt, der in der Praxis viele Fehler verursacht: Der Pfad muss auf dem Host ein *absoluter Pfad* sein, der mit einem Schrägstrich beginnt. Ein Name ohne Schrägstrich (wie `daten`) gilt als Volume.

-v legt fehlende Verzeichnisse an

Verweist `-v` auf ein Verzeichnis, das auf dem Host nicht existiert, legt Docker es still und leise an, und zwar mit Root als Besitzer. Ein Tippfehler im Pfad führt dann zu einem leeren Verzeichnis und löst keine Fehlermeldung aus. Die ausführliche Schreibweise `--mount type=bind,source=…,target=…` bricht in diesem Fall mit einem Fehler ab und ist deshalb sicherer.

10.5 tmpfs

Ein *tmpfs-Mount* legt ein Verzeichnis im Arbeitsspeicher an. Es ist schnell und schreibt nichts auf die Platte. Beim Beenden des Containers sind die Daten weg. Geeignet ist es für temporäre Dateien, Zwischenspeicher und Daten, die nie auf einen Datenträger gelangen sollen, wie Sitzungsschlüssel.

```
$ docker run --rm --tmpfs /tmp:size=64m alpine df -h /tmp
Filesystem                Size      Used Available Use% Mounted on
tmpfs                    64.0M         0     64.0M   0% /tmp
```

Die Angabe `size=64m` begrenzt die Größe auf 64 Megabyte. Ohne sie darf das tmpfs so groß werden, wie der Arbeitsspeicher des Hosts hergibt. Beachten Sie, dass tmpfs-Daten Arbeitsspeicher belegen, der für andere Prozesse fehlt. Kapitel 18 greift tmpfs bei Containern mit schreibgeschütztem Dateisystem wieder auf.

10.6 Persistente Datenspeicherung in der Praxis

Die Wahl zwischen den Varianten folgt einer einfachen Faustregel. Daten, die der Container erzeugt und die bleiben sollen, gehören in ein *Volume*. Dateien, die Sie auf dem Host bearbeiten und dem Container zur Verfügung stellen, gehören in einen *Bind Mount*. Alles Kurzlebige und Geheime gehört in ein *tmpfs*. Das folgende Beispiel zeigt den ganzen Ablauf an einem Container, dessen Daten bei einem Austausch erhalten bleiben sollen.

```
$ docker run -d --name notizen -v notizen-daten:/ablage alpine sleep
    ↪ 3600
$ docker exec notizen sh -c "date > /ablage/start.txt"
$ docker rm -f notizen
$ docker run -d --name notizen-neu -v notizen-daten:/ablage alpine
    ↪ sleep 3600
$ docker exec notizen-neu cat /ablage/start.txt
Wed Nov 11 11:31:08 UTC 2026
```

Der erste Container wurde gelöscht und durch einen neuen ersetzt. Das Volume, und damit die Datei, hat den Wechsel überstanden. Genauso tauscht man später den Container einer Datenbank gegen eine neuere Fassung aus, ohne die Daten anzutasten.

10.7 Sicherung und Wiederherstellung

Ein Volume lässt sich mit einem kleinen Hilfscontainer sichern. Der Container bindet das Volume und ein Verzeichnis des Hosts ein und packt den Inhalt in ein Archiv.

```
$ mkdir -p /srv/docker/backup && cd /srv/docker/backup
$ docker run --rm -v notizen-daten:/daten:ro -v "$PWD":/backup alpine
    ↪ \
    tar czf /backup/notizen-daten.tar.gz -C /daten .
$ tar tzf notizen-daten.tar.gz
```

```
./
./start.txt
```

Das Volume ist hier mit `:ro` schreibgeschützt eingebunden, damit die Sicherung nichts verändern kann. `"$PWD"` steht für das aktuelle Verzeichnis und sorgt dafür, dass das Archiv auf dem Host landet. Die Anführungszeichen schützen vor Problemen mit Leerzeichen im Pfad. Der Befehl `tar tzf` listet den Inhalt des Archivs auf und bestätigt, dass die Sicherung die Datei enthält.

Zum Wiederherstellen legt man ein neues Volume an und entpackt das Archiv hinein.

```
$ docker volume create notizen-wiederhergestellt
$ docker run --rm -v notizen-wiederhergestellt:/daten -v "$PWD":/
    ↪ backup alpine \
    tar xzf /backup/notizen-daten.tar.gz -C /daten
$ docker run --rm -v notizen-wiederhergestellt:/daten alpine cat /
    ↪ daten/start.txt
Wed Nov 11 11:31:08 UTC 2026
```

Die Datei ist im neuen Volume vorhanden. Eine Sicherung ist erst dann etwas wert, wenn man die Wiederherstellung ausprobiert hat, wie hier geschehen.

Dateikopien von Datenbanken sind riskant

Dieses Verfahren sichert Dateien, und das ist nur dann zuverlässig, wenn niemand gleichzeitig schreibt. Bei Datenbanken ist das im laufenden Betrieb nicht der Fall. Eine kopierte Datenbankdatei kann mitten in einer Änderung entstehen und unbrauchbar sein. Für Datenbanken gibt es eigene Sicherungswerkzeuge, die in Kapitel 14 vorgestellt werden. Alternativ stoppt man den Container vor dem Kopieren.

10.8 Rechte und Berechtigungen

Unter Linux gehören Dateien Benutzern, die durch Nummern bezeichnet werden. Der Kernel kennt nur diese Nummern, die Namen löst erst ein Programm anhand von `/etc/passwd` auf. Das hat bei Containern eine Folge, die Einsteiger verwirrt: Die Nummer zählt, nicht der Name. Der Benutzer `admin` mit der Nummer 1000 auf dem Host und ein Benutzer mit der Nummer 1000 im Container sind für den Kernel dieselbe Person, auch wenn der Container ihn anders nennt.

Ein Versuch macht das sichtbar.

```
$ mkdir -p ~/rechte && cd ~/rechte
$ docker run --rm -v "$PWD":/daten alpine touch /daten/von-root.txt
$ docker run --rm --user 1000:1000 -v "$PWD":/daten alpine touch /
    ↪ daten/von-admin.txt
$ ls -ln
total 0
-rw-r--r-- 1 1000 1000 0 Nov 11 11:40 von-admin.txt
-rw-r--r-- 1    0    0 0 Nov 11 11:40 von-root.txt
```

Der erste Container läuft standardmäßig als Root (Nummer 0) und erzeugt eine Datei, die auf dem Host Root gehört. Der Benutzer `admin` darf sie nicht ändern und nicht einmal löschen. Der zweite läuft mit `--user 1000:1000`, und seine Datei gehört auf dem Host `admin`. Das erklärt zwei typische Probleme. Schreibt ein Container als Root in einen Bind Mount, entstehen Dateien, die der normale Benutzer auf dem Host nicht mehr anfassen kann. Und läuft ein Container als Nicht-Root-Benutzer, darf er nur in Verzeichnisse schreiben, die dieser Nummer gehören oder anderweitig freigegeben sind.

Die Abhilfe hat zwei Wege. Bei Bind Mounts passt man die Rechte auf dem Host an oder startet den Container mit der passenden Benutzernummer. Bei Volumes setzt man die Rechte im Image, zum Beispiel mit `chown` im Dockerfile, bevor `USER` auf den Benutzer wechselt, wie Kapitel 8 zeigt. Nachträglich kann ein kurzer Hilfscontainer die Rechte ändern.

```
$ docker run --rm -v notizen-daten:/daten alpine chown -R 10001:10001
    ↪ /daten
```

Aufräumen können Sie die Beispieldatei von Root mit `sudo rm von-root.txt`.

10.9 Speicherverwaltung

Volumes und Images wachsen unbemerkt. Wie viel Platz Docker belegt, zeigt `docker system df` mit der Option `-v` auch im Einzelnen.

```
$ docker system df -v
...
Local Volumes space usage:

VOLUME NAME            LINKS   SIZE
notizen-daten          0       29B
notizen-wiederherg...  0       29B
```

In der Spalte `LINKS` steht, wie viele Container das Volume verwenden. Volumes mit `0` Verbindungen sind Kandidaten zum Aufräumen. Die Warnung aus Kapitel 9 gilt hier besonders: `docker volume prune` löscht in der Grundeinstellung unbenutzte anonyme Volumes, mit `-a` auch unbenutzte benannte, und beides kann wichtige Daten enthalten. Prüfen Sie die Liste mit `docker volume ls` und löschen Sie lieber gezielt. Ein einzelnes Volume entfernt `docker volume rm NAME`. Wer anonyme Volumes aufspüren will, die ein Image angelegt hat, filtert mit `docker volume ls -f dangling=true`.

Räumen Sie die Beispiele dieses Kapitels auf.

```
$ docker rm -f notizen-neu bind-web
$ docker volume rm notizen-daten notizen-wiederhergestellt
```

KAPITEL 11

Docker Networking

Ein Container ohne Netzwerk ist ein Programm in einer verschlossenen Kammer. Fast jeder Dienst muss mit anderen Containern sprechen, von Benutzern erreichbar sein oder selbst ins Internet. Docker baut dafür virtuelle Netzwerke, die nach denselben Regeln arbeiten wie echte: mit Schnittstellen, Adressen, Switches und einer Firewall. Dieses Kapitel erklärt den Aufbau, die verschiedenen Netzwerkarten, die Namensauflösung und den Umgang mit Portfreigaben. Es endet mit dem Thema, das in der Praxis die meisten Überraschungen bereithält: dem Zusammenspiel von Docker und Firewall.

11.1 Netzwerkarchitektur

Jeder Container bekommt, wie Kapitel 2 beschrieben hat, einen eigenen net-Namespace mit eigenen Schnittstellen und eigener IP-Adresse. Damit die Container miteinander und mit der Außenwelt reden können, verbindet Docker diese Namespaces über virtuelle Netzwerkkabel mit einem virtuellen Switch auf dem Host. Diesen Switch nennt man *Bridge*, was auf Deutsch Brücke heißt. Das Kabel ist ein sogenanntes *veth-Paar*: zwei Schnittstellen, von denen eine im Container (dort `eth0`) und die andere auf dem Host liegt. Was in eine Schnittstelle hineingeht, kommt aus der anderen heraus.

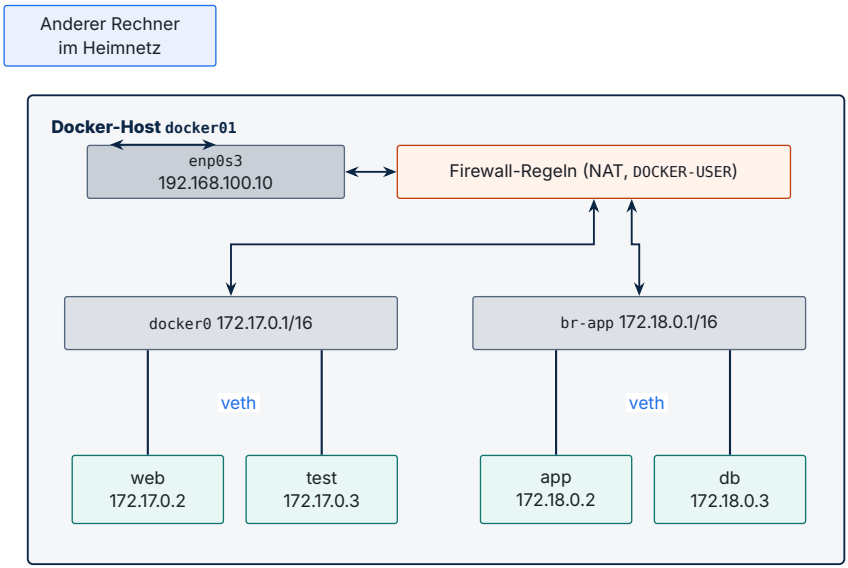


Abbildung 11.1: Docker-Netzwerkarchitektur mit zwei Bridges auf einem Host

Die Abbildung zeigt einen Host mit zwei Bridges. `docker0` ist die Bridge, die Docker bei der Installation anlegt, und `br-app` eine selbst erzeugte. An jeder hängen zwei Container. Die Container einer Bridge erreichen sich gegenseitig. Zwischen den Bridges gibt es in der Grundeinstellung keine Verbindung. Nach draußen läuft der Verkehr über die Firewall-Regeln des Hosts, die dabei die Adressen umschreiben (NAT). Ein Container im Netz `172.17.0.0/16` wird für das Heimnetz

zur Adresse des Hosts, `192.168.100.10`. Auf dem Host sind diese Bausteine mit gewöhnlichen Werkzeugen sichtbar.

```
$ docker run -d --name web nginx
$ ip -br address
lo               UNKNOWN        127.0.0.1/8 ::1/128
enp0s3           UP             192.168.100.10/24
docker0          UP             172.17.0.1/16
veth3a1b2c4@if7  UP
```

Die Bridge `docker0` hat die Adresse `172.17.0.1`. Sie dient den Containern als Gateway. Zu jedem laufenden Container gehört auf dem Host eine `veth`-Schnittstelle, deren Name bei Ihnen anders lautet. Starten Sie weitere Container, kommen weitere hinzu.

11.2 Netzwerkarten im Überblick

Docker unterscheidet mehrere Netzwerktreiber, die sich im Grad der Isolation und der Anbindung an das Heimnetz unterscheiden. Die Treiber bridge, host und none gibt es von Anfang an.

```
$ docker network ls --format "table {{.Name}}\t{{.Driver}}\t{{.Scope
    ↪ }}"
NAME      DRIVER    SCOPE
bridge    bridge    local
host      host      local
none      null      local
```

Treiber	Eigenschaft	Typischer Einsatz
`bridge`	eigenes Netz hinter einer Bridge, NAT nach außen, Portfreigaben nötig	Standard für fast alle Anwendungen
`host`	der Container teilt das Netzwerk des Hosts, keine Isolation	Dienste mit sehr vielen Ports oder höchsten Leistungsanforderungen
`none`	nur die Loopback-Schnittstelle, kein Netzwerk	Rechenaufgaben ohne Netz, maximale Isolation
`macvlan`	der Container erhält eine eigene Adresse im Heimnetz	Anwendungen, die wie ein eigener Rechner im LAN erscheinen sollen

11.3 Bridge Networks

Ohne Angabe landet jeder Container im Netz `bridge`, das zur Bridge `docker0` gehört. Man nennt es auch das *Standard-Bridge-Netzwerk*. Es funktioniert, hat aber Einschränkungen. Containern in diesem Netz kann man keine Namen auflösen, und alle Container teilen sich dasselbe Netz, auch solche, die nichts voneinander wissen sollen. Für eigene Anwendungen legt man deshalb *benutzerdefinierte Bridges* an, wie der folgende Abschnitt zeigt.

11.4 Benutzerdefinierte Netzwerke

Ein benutzerdefiniertes Netzwerk legt man mit `docker network create` an. Es bekommt eine eigene Bridge, einen eigenen Adressbereich und, anders als das Standardnetz, eine automatische Namensauflösung.

```
$ docker network create lab
7f4c1d8e2a9b...
$ docker network inspect lab --format '{{range .IPAM.Config}}{{.
    ↪ Subnet}} {{.Gateway}}{{end}}'
172.18.0.0/16 172.18.0.1
```

Docker wählt für das neue Netzwerk den nächsten freien Adressbereich, hier `172.18.0.0/16`. Wer den Bereich selbst festlegen will, etwa um Überschneidungen mit dem Firmennetz zu vermeiden, übergibt ihn beim Anlegen.

```
$ docker network create --subnet 10.50.0.0/24 --gateway 10.50.0.1 app-
    ↪ netz
```

Container verbinden Sie mit `--network` beim Start. Mit `--ip` lässt sich eine feste Adresse innerhalb des selbst definierten Bereichs vergeben. Ein vorhandener Container lässt sich nachträglich mit `docker network connect` an ein weiteres Netz anschließen und mit `docker network disconnect` wieder trennen.

```
$ docker run -d --name db1 --network app-netz --ip 10.50.0.10 alpine
    ↪ sleep 3600
$ docker network connect lab db1
$ docker network disconnect lab db1
$ docker network rm app-netz
Error response from daemon: error while removing network: network app-
    ↪ netz id ... has active endpoints
```

Ein Netzwerk lässt sich nur entfernen, solange kein Container mehr angeschlossen ist. Die Fehlermeldung am Ende zeigt, was geschieht, wenn man es vorher versucht. Mit `docker rm -f db1` und danach `docker network rm app-netz` räumen Sie auf.

11.5 Containerkommunikation

In einem benutzerdefinierten Netzwerk erreichen sich Container über ihren Namen. Das ist der Normalfall in jeder Anwendung: Die Webanwendung sagt der Datenbank nicht „verbinde dich mit 172.18.0.3“, sondern „verbinde dich mit `db`“. Die Adresse darf sich ändern, der Name bleibt.

```
$ docker run -d --name web --network lab nginx
$ docker run --rm --network lab alpine ping -c 2 web
PING web (172.18.0.2): 56 data bytes
64 bytes from 172.18.0.2: seq=0 ttl=64 time=0.112 ms
64 bytes from 172.18.0.2: seq=1 ttl=64 time=0.087 ms
```

Der Alpine-Container hat den Namen `web` in eine Adresse umgewandelt. Auch Anwendungsprotokolle funktionieren so, etwa HTTP.

```
$ docker run --rm --network lab alpine wget -qO- http://web | head -n
    ↪ 4
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
```

Beachten Sie, dass hier kein Port veröffentlicht wurde. Container im selben Netzwerk erreichen sich ohne Portfreigabe, und zwar auf den Ports, auf denen die Dienste tatsächlich lauschen. Die Portfreigabe mit `-p` brauchen nur Zugriffe von außerhalb des Docker-Netzes.

Im Standard-Bridge-Netzwerk funktioniert das nicht.

```
$ docker run -d --name web-alt nginx
$ docker run --rm alpine ping -c 1 web-alt
ping: bad address 'web-alt'
```

Ohne selbst definiertes Netz bleibt nur die IP-Adresse, die sich bei jedem Neustart ändern kann. Auch das ist ein Grund, immer eigene Netzwerke zu verwenden. Container in verschiedenen Netzen sehen sich nicht.

```
$ docker network create anders
$ docker run --rm --network anders alpine ping -c 1 web
ping: bad address 'web'
```

Ein Container kann in mehreren Netzen gleichzeitig hängen. Das nutzt man für Entwürfe mit Vermittler: Ein Reverse Proxy, der im Netz der Webanwendungen und im Netz nach außen steht, die Datenbank dagegen nur im inneren Netz. Kapitel 16 baut so eine Anordnung auf.

11.6 DNS innerhalb von Docker

Wie kommt der Name `web` zu seiner Adresse? In benutzerdefinierten Netzen betreibt Docker einen eigenen kleinen DNS-Server, der in jedem Container unter der festen Adresse `127.0.0.11` erreichbar ist. Das lässt sich nachlesen.

```
$ docker run --rm --network lab alpine cat /etc/resolv.conf
nameserver 127.0.0.11
options ndots:0
```

Fragt ein Programm im Container nach einem Namen, landet die Anfrage bei diesem Docker-DNS. Kennt er den Namen als Container oder *Alias* im selben Netz, antwortet er direkt. Andernfalls leitet er die Anfrage an die Namensserver des Hosts weiter, sodass auch Namen aus dem Internet aufgelöst werden. Neben dem Containernamen lassen sich weitere Namen vergeben.

```
$ docker run -d --name db2 --network lab --network-alias datenbank
    ↪ alpine sleep 3600
$ docker run --rm --network lab alpine ping -c 1 datenbank
```

Beide Namen führen zum selben Container. Aliasse sind praktisch, wenn mehrere Container denselben Dienstnamen tragen sollen, oder wenn ein Programm einen festen Namen erwartet. Im Standardnetz gibt es diesen Dienst nicht. Dort übernimmt der Container die Einstellungen des Hosts aus dessen `/etc/resolv.conf`. Möchten Sie einem Container bestimmte Namensserver mitgeben, geht das

mit `--dns 192.168.100.1`. Für alle Container gemeinsam stellt man sie in der `daemon.json` ein.

/etc/docker/daemon.json (Ausschnitt)

```
{
  "dns": ["192.168.100.1", "1.1.1.1"]
}
```

11.7 Host Network und None

Mit `--network host` nutzt der Container direkt das Netzwerk des Hosts. Es gibt keinen eigenen net-Namespace, keine eigene Adresse und keine Portfreigabe. Lauscht der Webserver im Container auf Port 80, lauscht er damit auf Port 80 des Hosts.

```
$ docker run -d --name host-web --network host nginx
$ curl -sI http://localhost | head -n 1
HTTP/1.1 200 OK
$ docker rm -f host-web
```

Der Vorteil ist, dass die Umleitung über Bridge und NAT entfällt, was bei sehr hohem Datenaufkommen Leistung spart. Der Preis ist hoch: Es gibt keine Isolation des Netzwerks mehr. Der Container sieht alle Schnittstellen des Hosts und kann jeden Port belegen. Zwei Container, die denselben Port benutzen, kollidieren. Docker legt für diese Betriebsart keine Firewall-Regeln an. Verwenden Sie `host` nur, wenn es einen guten Grund gibt.

Mit `--network none` bekommt der Container nur die Loopback-Schnittstelle, wie in Kapitel 2 beim net-Namespace gezeigt. Das passt für Aufgaben, die ohne Netz auskommen sollen, etwa das Verarbeiten von Dateien. Ein Netz ist dann nicht erreichbar, auch keine Namensauflösung.

```
$ docker run --rm --network none alpine ip -br address
lo               UNKNOWN        127.0.0.1/8
```

11.8 Macvlan

Der Treiber `macvlan` gibt dem Container eine eigene MAC- und IP-Adresse im Heimnetz. Für andere Rechner im Netz sieht er dann aus wie ein eigenständiges Gerät, das ohne Portfreigabe direkt erreichbar ist. Das passt für Anwendungen, die feste Ports im Netz brauchen oder wie ein physisches Gerät auftreten sollen.

```
$ docker network create -d macvlan \
    --subnet 192.168.100.0/24 --gateway 192.168.100.1 \
    --ip-range 192.168.100.192/28 -o parent=enp0s3 lan
$ docker run -d --name lan-web --network lan --ip 192.168.100.200
    ↪ nginx
```

Die Option `parent` nennt die Netzwerkkarte des Hosts, über die der Verkehr läuft. Mit `--ip-range` grenzt man den Bereich ein, aus dem Docker Adressen vergibt, damit sie nicht mit denen des DHCP-Servers kollidieren. Drei Eigenheiten müssen Sie kennen. Der Host selbst kann den Container über die Macvlan-Verbindung nicht erreichen, andere Rechner im Netz aber schon. Läuft der Docker-Host selbst in einer virtuellen Maschine, muss der Hypervisor mehrere MAC-Adressen an der Netzwerkkarte der VM erlauben, bei VirtualBox heißt das „Promiscuous Mode“. Und der Container ist ohne weitere Regeln direkt im Heimnetz sichtbar, die Firewall-Regeln von Docker greifen dafür nicht. Ein verwandter Treiber namens `ipvlan` funktioniert ähnlich, verwendet aber die MAC-Adresse des Hosts.

11.9 Portfreigaben

Ein Container in einem Bridge-Netz ist von außen nicht erreichbar, bis man einen Port veröffentlicht. Das tut die Option `-p`, die das Format `-p HOST-PORT:CONTAINER-PORT` hat. Es gibt mehrere Schreibweisen.

Angabe	Bedeutung
`-p 8080:80`	Host-Port 8080 auf allen Adressen des Hosts führt zu Port 80 im Container
`-p 127.0.0.1:8080:80`	wie oben, aber nur über die Loopback-Adresse des Hosts erreichbar
`-p 192.168.100.10:8080:80`	nur über die genannte Adresse des Hosts erreichbar
`-p 53:53/udp`	Port für UDP statt TCP (TCP ist die Voreinstellung)
`-p 80`	der Host wählt einen freien Port
`-P`	veröffentlicht alle mit `EXPOSE` markierten Ports auf zufälligen Host-Ports

Die zweite Schreibweise ist die wichtigste für die Sicherheit, wie der nächste Abschnitt zeigt. Welche Ports ein laufender Container veröffentlicht, zeigt `docker port`.

```
$ docker run -d --name pub -p 8080:80 nginx
$ docker port pub
80/tcp -> 0.0.0.0:8080
80/tcp -> [::]:8080
$ sudo ss -tlnp | grep 8080
LISTEN 0 4096 0.0.0.0:8080 0.0.0.0:* users:(("docker-proxy",pid=3141,
    ↪ fd=7))
```

Auf dem Host lauscht ein Hilfsprogramm namens `docker-proxy` auf dem Port. Es nimmt Verbindungen entgegen, die vom Host selbst kommen, etwa über `localhost`, und leitet sie zum Container weiter. Verbindungen von anderen Rechnern fängt dagegen die Firewall des Kernels ab, ohne das Hilfsprogramm. Dieser Unterschied erklärt, warum in Kapitel 6 im Protokoll von nginx die Adresse `172.17.0.1` als Absender stand: Der Zugriff über `localhost` lief über den Proxy

und das Docker-Netz, deshalb sieht der Container als Absender das Gateway statt Ihrer eigenen Adresse.

11.10 Firewall und Netzwerkisolation

Hier lauert die Überraschung, die Kapitel 5 angekündigt hat. Sie ist wichtig genug für eine genaue Betrachtung.

11.10.1 Docker umgeht ufw

Sobald Sie einen Port mit `-p` veröffentlichen, trägt Docker eigene Regeln in die Firewall des Kernels ein. Sie greifen, bevor die Regeln von `ufw` oder `firewalld` zum Zug kommen. Die Docker-Dokumentation sagt es so: Der Verkehr zu veröffentlichten Ports wird umgeleitet, bevor er die Einstellungen von `ufw` durchläuft. Ein Port, den Sie in `ufw` gesperrt haben, ist deshalb von außen erreichbar, sobald ein Container ihn veröffentlicht.

> **Ein veröffentlichter Port ist offen**
>
> `-p 5432:5432` bei einer Datenbank macht sie für das ganze Netz erreichbar, unabhängig davon, was in `ufw status` steht. Veröffentlichen Sie nur, was von außen erreichbar sein soll. Die Datenbank einer Anwendung gehört in ein Docker-Netz ohne Portfreigabe, wie im vorigen Abschnitt gezeigt.

Das Verhalten lässt sich von einem zweiten Rechner aus nachprüfen. Rufen Sie von einem anderen Gerät im Heimnetz die Adresse des Servers auf.

```
$ curl -m 3 http://192.168.100.10:8080
<!DOCTYPE html>
...
```

Die Antwort kommt, obwohl `ufw` den Port nicht freigegeben hat.

11.10.2 Drei Gegenmaßnahmen

Die einfachste Maßnahme ist, Ports nur auf der Loopback-Adresse zu veröffentlichen. Ein solcher Dienst ist nur vom Host selbst erreichbar. Das passt, wenn davor ein Reverse Proxy auf dem Host sitzt oder wenn nur Sie selbst per SSH-Tunnel zugreifen wollen.

```
$ docker run -d --name intern -p 127.0.0.1:8085:80 nginx
$ curl -s http://localhost:8085 | head -n 1
<!DOCTYPE html>
```

Von einem anderen Rechner aus liefert derselbe Aufruf auf die Adresse des Servers keine Antwort. Die zweite Maßnahme ist, den Port gar nicht zu veröffentlichen und ihn nur in einem Docker-Netz anzubieten. Container im selben Netz erreichen ihn trotzdem, wie oben gezeigt. Die dritte Maßnahme sind eigene Regeln in der Kette `DOCKER-USER`. Docker wertet sie vor seinen eigenen Regeln aus und lässt sie unangetastet. Ein Beispiel erlaubt Zugriffe auf Container nur aus dem Heimnetz.

```
$ sudo iptables -I DOCKER-USER -i enp0s3 ! -s 192.168.100.0/24 -j
    ↪ DROP
$ sudo iptables -I DOCKER-USER -m conntrack --ctstate ESTABLISHED,
    ↪ RELATED -j ACCEPT
$ sudo iptables -L DOCKER-USER -n --line-numbers
```

Die erste Regel verwirft alle Pakete, die über die Netzwerkkarte `enp0s3` hereinkommen und nicht aus dem Netz `192.168.100.0/24` stammen. Das Argument `-I` setzt eine Regel an den Anfang der Kette. Weil die zweite Regel danach eingefügt wird, steht sie *vor* der ersten, und diese Reihenfolge ist entscheidend. Die zweite Regel lässt alle Pakete passieren, die zu einer bereits bestehenden Verbindung gehören. Ohne sie würde die erste auch die Antworten auf Verbindungen verwerfen, die ein Container selbst nach draußen aufgebaut hat. Ein Container könnte dann zwar eine Anfrage ins Internet senden, aber die Antwort käme nie an, und Updates oder Image-Downloads aus dem Container heraus schlügen fehl. Neue Verbindungen aus dem Heimnetz dagegen passieren die erste Regel unbehelligt.

Zwei weitere Eigenheiten sind zu beachten. Erstens kommen die Pakete in `DOCKER-USER` erst *nach* der Adressumschreibung an. Eine Regel sieht dort die

Adresse und den Port des Containers und nicht den veröffentlichten Port des Hosts. Möchte man auf den ursprünglichen Port prüfen, braucht man die Erweiterung `conntrack`, zum Beispiel `-m conntrack --ctorigdstport 8080 --ctdir ORIGINAL`. Zweitens bleiben die Regeln nicht über einen Neustart erhalten. Wie man sie dauerhaft macht, zeigt Kapitel 18.

Docker schützt außerdem Container, deren Ports nicht veröffentlicht sind. Ein anderer Rechner kann sie in der Grundeinstellung nicht erreichen, selbst wenn er die Adresse im Docker-Netz kennt. Für die Fälle, in denen das gewollt ist, gibt es die Netzwerkoption `com.docker.network.bridge.trusted_host_interfaces` und die Daemon-Option `allow-direct-routing`.

11.10.3 Netzwerke ohne Zugang nach außen

Ein Netzwerk mit der Option `--internal` hat keinen Zugang zur Außenwelt. Container darin erreichen sich untereinander, aber nicht das Internet und nicht den Host.

```
$ docker network create --internal nur-intern
$ docker run --rm --network nur-intern alpine ping -c 1 8.8.8.8
ping: sendto: Network unreachable
```

Das ist die richtige Wahl für Datenbanken, die nur von der Anwendung angesprochen werden. Wer eine solche Anordnung baut, verbindet die Anwendung mit zwei Netzen: dem inneren, in dem die Datenbank steht, und einem äußeren, aus dem der Reverse Proxy sie erreicht.

11.10.4 Ausblick: das nftables-Backend

Seit Docker 29 gibt es als Alternative ein Backend, das statt `iptables` die neuere Technik `nftables` verwendet. Es wird in der `/etc/docker/daemon.json` eingeschaltet:

/etc/docker/daemon.json

```
{
  "firewall-backend": "nftables"
}
```

Die Docker-Dokumentation bezeichnet das Backend als experimentell. Es kennt keine `DOCKER-USER`-Kette, stattdessen schreibt man eigene Tabellen mit passender Priorität. Docker erwartet außerdem, dass es seine nftables-Tabellen allein verwaltet. Es lässt sich nicht zusammen mit dem Swarm-Modus betreiben. In diesem Buch bleiben wir beim bewährten `iptables`-Backend, und die Anleitungen setzen es voraus.

> **Backend nicht mitten im Betrieb wechseln**
>
> Wer das Backend umstellt, ändert, wie Docker Ports veröffentlicht und welche Regeln es dafür anlegt. Außerdem schaltet Docker mit nftables die IP-Weiterleitung nicht mehr selbst ein. Testen Sie einen Wechsel zuerst auf einem Übungssystem und lesen Sie die Hinweise zur Migration in der Docker-Dokumentation.

Räumen Sie die Beispiele dieses Kapitels auf.

```
$ docker rm -f web web-alt db2 pub intern lan-web 2>/dev/null
$ docker network rm lab anders nur-intern lan 2>/dev/null
$ sudo iptables -D DOCKER-USER -i enp0s3 ! -s 192.168.100.0/24 -j
    ↪ DROP
$ sudo iptables -D DOCKER-USER -m conntrack --ctstate ESTABLISHED,
    ↪ RELATED -j ACCEPT
```

Praxisprojekt 2: Eigener Webserver

In diesem Projekt betreiben Sie einen eigenen Webserver mit nginx im Container. Die Webseite liegt auf dem Host und wird als Bind Mount eingebunden, die Protokolle landen in einem Volume und überstehen jeden Austausch des Containers. Das Projekt verbindet die Kapitel 7 bis 11. Planen Sie etwa anderthalb Stunden ein.

Ausgangslage und Ziel

Ein Verein möchte seine Seite selbst betreiben. Der Server soll die Seite ausliefern, bei unbekannten Adressen eine eigene Fehlerseite zeigen und jeden Zugriff protokollieren. Die Seite soll sich ohne Eingriff in den Container ändern lassen. Der Sollzustand sieht so aus.

Anforderung	Sollzustand
Container	`p2-web` aus `nginx:stable-alpine`, startet nach einem Neustart automatisch
Webseite	Dateien unter `/srv/docker/projekt-2/html`, schreibgeschützt eingebunden
Konfiguration	`/srv/docker/projekt-2/conf/default.conf`, schreibgeschützt eingebunden
Protokolle	Volume `p2-logs`, Datei `zugriffe.log`
Zugriff	Browser, `http://docker01:8080` oder Adresse des Servers mit Port 8080
Fehlerseite	`404.html` für unbekannte Adressen

Schritt 1: Verzeichnisse und Dateien anlegen

```
$ mkdir -p /srv/docker/projekt-2/html /srv/docker/projekt-2/conf
$ cd /srv/docker/projekt-2
```

Die Startseite bekommt eine eigene Stildatei.

html/index.html

```
<!DOCTYPE html>
<html lang="de">
<head>
  <meta charset="utf-8">
  <title>Projekt 2: Eigener Webserver</title>
  <link rel="stylesheet" href="stil.css">
```

```
</head>
<body>
  <h1>Projekt 2: Eigener Webserver</h1>
  <p>Diese Seite liefert nginx aus einem Container.</p>
</body>
</html>
```

html/stil.css

```
body { font-family: sans-serif; max-width: 40rem; margin: 3rem auto;
    ↪ }
h1   { color: #0b2545; }
```

html/404.html

```
<!DOCTYPE html>
<html lang="de">
<head><meta charset="utf-8"><title>Nicht gefunden</title></head>
<body><h1>Seite nicht gefunden</h1><p>Diese Adresse gibt es hier
    ↪ nicht.</p></body>
</html>
```

Die Konfiguration ersetzt die Standardeinstellung des Images.

conf/default.conf

```
server {
    listen 80;
    server_name _;
    charset utf-8;

    root /usr/share/nginx/html;
    index index.html;

    access_log /var/log/nginx/zugriffe.log;

    location / {
        try_files $uri $uri/ =404;
    }
```

```
    error_page 404 /404.html;
}
```

Die Zeile `try_files` sucht erst die angefragte Datei, dann ein gleichnamiges Verzeichnis und liefert sonst den Fehler 404. `error_page` verweist für diesen Fehler auf die eigene Seite. `access_log` legt die Zugriffsprotokolle in einer eigenen Datei ab, die im Volume landet.

Schritt 2: Container starten

Der Start besteht aus einem langen Befehl. Damit Sie ihn nicht jedes Mal neu tippen, legen Sie ihn in ein kleines Skript.

/srv/docker/projekt-2/start.sh

```
#!/bin/bash
# Startet den Webserver für Projekt 2 (ersetzt einen vorhandenen
    ↪ Container)
docker rm -f p2-web >/dev/null 2>&1
docker run -d --name p2-web --restart unless-stopped -p 8080:80 \
  -v /srv/docker/projekt-2/html:/usr/share/nginx/html:ro \
  -v /srv/docker/projekt-2/conf/default.conf:/etc/nginx/conf.d/
    ↪ default.conf:ro \
  -v p2-logs:/var/log/nginx \
  nginx:stable-alpine
```

```
$ chmod +x start.sh
$ ./start.sh
$ docker ps --format "table {{.Names}}\t{{.Status}}\t{{.Ports}}"
NAMES     STATUS         PORTS
p2-web    Up 3 seconds   0.0.0.0:8080->80/tcp, [::]:8080->80/tcp
```

Das Skript löscht zuerst einen eventuell vorhandenen Container gleichen Namens. Danach startet es den neuen. Das Volume `p2-logs` legt Docker beim ers-

ten Start von selbst an. Beide Bind Mounts sind mit `:ro` schreibgeschützt, der Container darf Webseite und Konfiguration also nicht verändern.

Schritt 3: Zugriff und Änderungen

Rufen Sie die Seite mit `curl` und im Browser auf. Im Browser geben Sie die Adresse Ihres Servers mit dem Port ein, also etwa `http://192.168.100.10:8080`.

```
$ curl -s http://localhost:8080 | grep title
  <title>Projekt 2: Eigener Webserver</title>
$ curl -s -o /dev/null -w '%{http_code}\n' http://localhost:8080/gibt-
    ↪ es-nicht
404
```

Ändern Sie nun den Text in `html/index.html` und laden Sie die Seite im Browser neu. Die Änderung erscheint sofort, ohne dass der Container neu gestartet werden muss. Das ist die Wirkung des Bind Mounts. Zeigen Sie außerdem die Fehlerseite im Browser an, indem Sie eine unbekannte Adresse aufrufen.

Schritt 4: Protokolle und der Austausch des Containers

Jeder Zugriff steht in der Protokolldatei im Volume. Dort lesen Sie nach.

```
$ docker exec p2-web tail -n 3 /var/log/nginx/zugriffe.log
172.17.0.1 - - [11/Nov/2026:12:05:44 +0000] "GET / HTTP/1.1" 200 214
    ↪ "-" "curl/8.x" "-"
172.17.0.1 - - [11/Nov/2026:12:05:51 +0000] "GET /gibt-es-nicht HTTP
    ↪ /1.1" 404 146 "-" "curl/8.x" "-"
$ docker volume inspect p2-logs --format '{{.Mountpoint}}'
/var/lib/docker/volumes/p2-logs/_data
```

Die Größen in den Zeilen und die Zeitstempel sind Beispiele. Jetzt kommt der entscheidende Test: Tauschen Sie den Container aus und prüfen Sie, ob die Protokolle überleben.

```
$ docker exec p2-web sh -c 'wc -l < /var/log/nginx/zugriffe.log'
2
$ ./start.sh
$ docker exec p2-web sh -c 'wc -l < /var/log/nginx/zugriffe.log'
2
```

Der Container ist ein neuer, die Zeilen sind noch da. Auf dem Volume liegen die Daten, im Container nur die Verbindung dorthin.

Schritt 5: Sicherung des Volumes

Sichern Sie das Protokoll-Volume mit dem Verfahren aus Kapitel 10 in ein Archiv.

```
$ mkdir -p /srv/docker/backup
$ docker run --rm -v p2-logs:/daten:ro -v /srv/docker/backup:/backup
    ↪ alpine \
    tar czf /backup/p2-logs.tar.gz -C /daten .
$ tar tzf /srv/docker/backup/p2-logs.tar.gz
```

Die Ausgabe von `tar tzf` kann neben der Datei `zugriffe.log` weitere Einträge enthalten, denn beim ersten Einbinden eines leeren Volumes kopiert Docker den vorhandenen Inhalt des Verzeichnisses hinein (Kapitel 10). Wichtig ist die Zeile mit `zugriffe.log`.

Schritt 6: Funktionsprüfung und Abnahme

Das Prüfskript fasst die wichtigsten Punkte zusammen.

/srv/docker/projekt-2/check.sh

```
#!/bin/bash
# Funktionsprüfung für Projekt 2: Eigener Webserver
FEHLER=0
ok()   { echo "[ OK ]     $1"; }
fail() { echo "[FEHLER]   $1"; FEHLER=1; }
```

```
URL=http://localhost:8080

[ -n "$(docker ps -q --filter name=p2-web --filter status=running)" ]
    ↪ \
  && ok "Container p2-web läuft" || fail "Container p2-web läuft
    ↪ nicht"
[ "$(curl -s -o /dev/null -w '%{http_code}' $URL/)" = "200" ] \
  && ok "Startseite liefert 200" || fail "Startseite liefert nicht
    ↪ 200"
curl -s $URL/ | grep -q "Projekt 2" \
  && ok "Startseite enthält den erwarteten Text" || fail "Text fehlt"
[ "$(curl -s -o /dev/null -w '%{http_code}' $URL/gibt-es-nicht)" =
    ↪ "404" ] \
  && ok "Unbekannte Adresse liefert 404" || fail "404 fehlt"
curl -s $URL/gibt-es-nicht | grep -q "nicht gefunden" \
  && ok "Eigene Fehlerseite wird gezeigt" || fail "Fehlerseite fehlt"
docker exec p2-web sh -c 'touch /usr/share/nginx/html/test 2>/dev/
    ↪ null' \
  && fail "Webseiten-Verzeichnis ist beschreibbar" \
  || ok "Webseiten-Verzeichnis ist schreibgeschützt"

VOR=$(docker exec p2-web sh -c 'wc -l < /var/log/nginx/zugriffe.log')
/srv/docker/projekt-2/start.sh >/dev/null 2>&1
sleep 3
NACH=$(docker exec p2-web sh -c 'wc -l < /var/log/nginx/zugriffe.log
    ↪ ')
[ "$VOR" -gt 0 ] && [ "$NACH" -ge "$VOR" ] \
  && ok "Protokoll überlebt den Austausch ($VOR Zeilen)" \
  || fail "Protokoll ist nach dem Austausch nicht mehr da"

echo
[ "$FEHLER" -eq 0 ] && echo "Alle Prüfungen bestanden." \
  || echo "Mindestens eine Prüfung ist fehlgeschlagen."
exit $FEHLER
```

```
$ chmod +x check.sh && ./check.sh
[ OK ]     Container p2-web läuft
```

```
[ OK ]     Startseite liefert 200
[ OK ]     Startseite enthält den erwarteten Text
[ OK ]     Unbekannte Adresse liefert 404
[ OK ]     Eigene Fehlerseite wird gezeigt
[ OK ]     Webseiten-Verzeichnis ist schreibgeschützt
[ OK ]     Protokoll überlebt den Austausch (4 Zeilen)

Alle Prüfungen bestanden.
```

Weil das Skript die Zugriffe selbst auslöst, enthält das Protokoll vor dem Austausch mindestens die Zeilen aus den ersten Prüfungen. Die Zahl in Ihrer Ausgabe weicht ab. Zur Abnahme gehören außerdem drei Handgriffe: Starten Sie den Server neu und prüfen Sie, ob `p2-web` ohne Eingriff wieder läuft. Rufen Sie die Seite von einem anderen Rechner im Netz auf. Und ändern Sie die Startseite und kontrollieren Sie, dass die Änderung ohne Neustart erscheint.

Wenn etwas nicht klappt

Symptom	Ursache und Abhilfe
nginx zeigt die Standardseite statt der eigenen	Der Bind Mount stimmt nicht. Prüfen Sie den Pfad im Skript. Ein Pfad ohne führenden Schrägstrich wird als Volume gedeutet.
`403 Forbidden`	nginx darf die Dateien nicht lesen. Mit `chmod -R a+rX /srv/docker/projekt-2/html` setzen Sie lesbare Rechte.
Der Container beendet sich sofort	Meist ein Fehler in der `default.conf`. `docker logs p2-web` nennt die Zeile.
`port is already allocated`	Port 8080 ist belegt, etwa durch einen Container aus einem früheren Kapitel. `docker ps` zeigt es.
Die Seite ist von außen nicht erreichbar	Läuft eine Firewall auf dem Server oder im Netz? Zugriffe über `localhost` sagen nichts über andere Rechner.

Lernkontrolle zu Teil IV

Die Aufgaben prüfen die Kapitel 10 und 11. Die praktischen Aufgaben führen Sie auf Ihrem Übungsserver durch. Die Lösungen stehen im Anhang.

Verständnisfragen

V1. Warum gehen Daten in der beschreibbaren Schicht eines Containers verloren, und wie vermeidet man das?

V2. Nennen Sie zwei Unterschiede zwischen einem Volume und einem Bind Mount.

V3. Was geschieht, wenn man ein leeres Volume in ein Verzeichnis einbindet, in dem das Image bereits Dateien mitbringt? Und was geschieht bei einem Bind Mount?

V4. Wozu dient der Zusatz `:ro` bei einem Mount?

V5. Warum kann ein Container Dateien erzeugen, die der Benutzer `admin` auf dem Host nicht löschen darf?

V6. Warum erreichen sich Container im Standard-Bridge-Netz nicht über ihre Namen, in einem selbst angelegten Netz aber schon?

V7. Wie löst Docker in einem benutzerdefinierten Netz Namen auf? Unter welcher Adresse ist der Namensdienst im Container erreichbar?

V8. Warum ist ein mit `-p` veröffentlichter Port auch dann von außen erreichbar, wenn `ufw` ihn sperrt? Nennen Sie zwei Gegenmaßnahmen.

Multiple-Choice-Aufgaben

M1. Welche Schreibweise bindet das Volume `daten` nach `/ablage` im Container ein?

(a) `-v /daten:/ablage`
(b) `-v daten:/ablage`
(c) `-p daten:/ablage`
(d) `--tmpfs daten:/ablage`

M2. Welcher Netzwerktreiber gibt einem Container eine eigene Adresse im Heimnetz?

(a) `bridge`
(b) `host`
(c) `none`
(d) `macvlan`

M3. Mit welcher Angabe wird ein Port nur auf der Loopback-Adresse des Hosts veröffentlicht?

(a) `-p 127.0.0.1:8080:80`
(b) `-p lo:8080:80`
(c) `--local 8080`
(d) `-p 8080:80/local`

M4. Unter welcher Adresse läuft der eingebaute DNS-Server von Docker im Container?

(a) `172.17.0.1`
(b) `8.8.8.8`
(c) `127.0.0.11`
(d) `192.168.100.1`

M5. Was bewirkt `docker network create --internal`?

(a) Es verschlüsselt den Verkehr im Netz.
(b) Es verwehrt den Containern den Zugang nach außen.
(c) Es verbirgt das Netz in der Liste.
(d) Es macht das Netz zum Standardnetz.

M6. Welche Art von Mount geht beim Beenden des Containers verloren?

(a) Ein benanntes Volume

(b) Ein Bind Mount
(c) Ein anonymes Volume
(d) Ein tmpfs-Mount

Praktische Aufgaben

P1. Legen Sie ein Volume `uebung` an und schreiben Sie mit einem Alpine-Container eine Datei hinein. Löschen Sie den Container, lesen Sie die Datei mit einem neuen Container aus, sichern Sie das Volume als Archiv und stellen Sie es in einem zweiten Volume wieder her.

P2. Legen Sie ein Netzwerk `uebung-netz` an und starten Sie darin einen nginx-Container `web`. Rufen Sie die Seite aus einem Alpine-Container im selben Netz über den Namen ab. Zeigen Sie dann, dass derselbe Name im Standardnetz nicht aufgelöst wird.

P3. Veröffentlichen Sie einen nginx-Container so, dass er nur über die Loopback-Adresse erreichbar ist (Port 8090). Weisen Sie mit `ss -tlnp` nach, auf welcher Adresse der Port lauscht.

Fehlersuche

F1. Ein Administrator startet nach einem Tippfehler (`dten` statt `daten`) einen Container mit dem falschen Volume-Namen. Der Container ist leer, alle Daten scheinen verloren. Wie erkennen und beheben Sie das?

F2. nginx zeigt nach `-v html:/usr/share/nginx/html` die Standardseite statt der eigenen Dateien aus dem Verzeichnis `html` neben dem Dockerfile. Woran liegt das?

F3. `ping web` im Container `alpine` auf einem selbst angelegten Netz meldet `bad address`. Nennen Sie drei Ursachen, die Sie prüfen.

Szenario aus dem Betrieb

Auf einem Server läuft ein Datenbank-Container, der mit `-p 3306:3306` gestartet wurde. Auf dem Server sperrt `ufw` den Port 3306, trotzdem erreicht ihn ein Rechner aus einem fremden Netz. Erklären Sie, warum das so ist, und beschreiben Sie, wie Sie das Problem dauerhaft lösen, wenn die Anwendung, die die Datenbank nutzt, ebenfalls in einem Container läuft.

TEIL V

Docker Compose und Datenbanken

KAPITEL 12

Einführung in Docker Compose

Mit den bisherigen Befehlen lässt sich jeder einzelne Container starten. Eine echte Anwendung besteht aber meist aus mehreren: einem Webserver, einer Anwendung, einer Datenbank. Wer jeden Container von Hand mit einem langen `docker run` startet, muss sich Optionen, Netzwerke und Reihenfolgen merken und macht dabei Fehler. Docker Compose löst das Problem, indem es die ganze Anwendung in einer einzigen Textdatei beschreibt. Dieses Kapitel erklärt, wie diese Datei aufgebaut ist.

12.1 Zweck und Funktionsweise

Docker Compose ist ein Werkzeug, mit dem man eine Anwendung aus mehreren Containern beschreibt und mit einem Befehl startet, stoppt und verwaltet. Die Beschreibung steht in einer Datei im YAML-Format, die in diesem Buch durchgehend `compose.yaml` heißt. Compose liest die Datei, legt die nötigen Netzwerke und Volumes an und startet die Container in der richtigen Reihenfolge. Das Werkzeug arbeitet *deklarativ*: Sie beschreiben den gewünschten Endzustand, nicht die einzelnen Schritte dorthin. Ändern Sie die Datei und starten neu, vergleicht Compose den Ist- mit dem Sollzustand und ändert nur, was sich unterscheidet.

Fachbegriff: Compose-Projekt

Alle Container, Netzwerke und Volumes, die zu einer `compose.yaml` gehören, bilden ein Projekt. Der Projektname ist standardmäßig der Name des Verzeichnisses, in dem die Datei liegt. Er steht als Vorsatz vor den Namen aller erzeugten Objekte.

Das Werkzeug ist als Plugin Teil von Docker und wird mit `docker compose` aufgerufen, wie Kapitel 5 beschrieben hat. Es erkennt die Dateinamen `compose.yaml`, `compose.yml`, `docker-compose.yaml` und `docker-compose.yml`. Wir verwenden den ersten, denn er ist der von der Spezifikation bevorzugte.

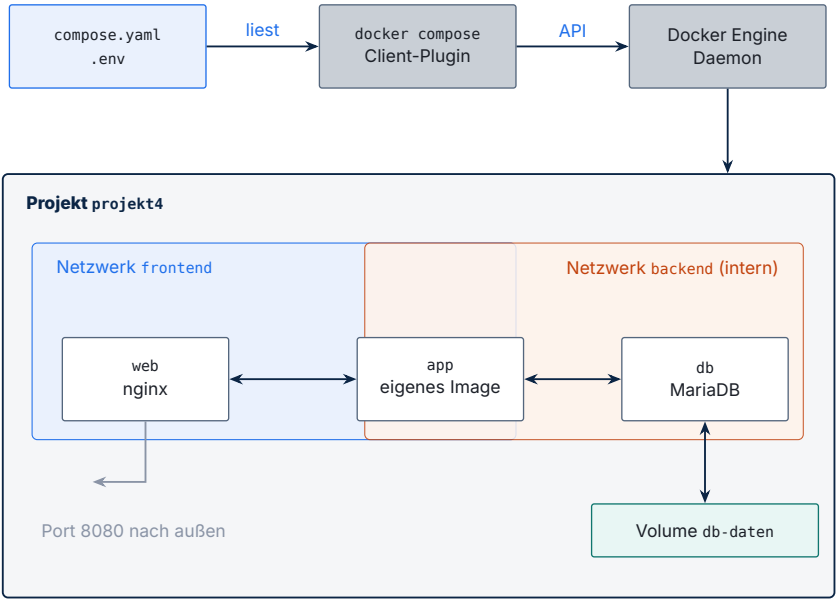


Abbildung 12.1: Aufbau eines Compose-Projekts: Datei, Client, Engine und die erzeugten Objekte

Die Abbildung zeigt das Zusammenspiel. Compose ist nur der Client. Es liest die Datei, übersetzt sie in Anfragen und schickt diese an die Docker Engine, die die Container, Netzwerke und Volumes tatsächlich anlegt. Compose selbst hat keinen eigenen Dienst im Hintergrund. Das Beispiel der Abbildung baut auf dem Praxisprojekt 4 auf, in dem Sie es nachbauen.

12.2 YAML-Grundlagen

YAML ist ein Textformat für strukturierte Daten, das man gut lesen kann. Compose-Dateien nutzen es, und deshalb hängen ihre Fehler fast immer mit den wenigen Regeln von YAML zusammen. Die Struktur entsteht durch *Einrückung*. Eingerückt wird mit Leerzeichen, nie mit Tabulatoren. Üblich sind zwei Leerzeichen pro Ebene.

Die Grundbausteine sind *Zuordnungen* und *Listen*. Eine Zuordnung besteht aus Schlüssel und Wert, getrennt durch einen Doppelpunkt mit Leerzeichen. Eine Liste besteht aus Zeilen, die mit einem Bindestrich beginnen.

YAML in Kürze

```
# Ein Kommentar beginnt mit einem Rautenzeichen
name: projekt4            # Zuordnung: Schlüssel und Wert
ports:                    # Schlüssel mit einer Liste darunter
  - "8080:80"
  - "8443:443"
environment:              # Schlüssel mit einer Zuordnung darunter
  DB_HOST: db
  DB_NAME: besucher
```

Einige Eigenheiten verursachen regelmäßig Fehler. Erstens: Ein Tabulator statt Leerzeichen macht die Datei ungültig. Stellen Sie Ihren Editor entsprechend ein. Zweitens: Der Doppelpunkt braucht ein Leerzeichen dahinter. `DB_HOST:db` ist keine Zuordnung. Drittens: Portangaben wie `8080:80` gehören in Anführungszeichen. Mit zwei Zahlen und einem Doppelpunkt kann YAML sie als Zeitangabe lesen und das Ergebnis verfälschen. Viertens: Werte wie `yes`, `no`, `on` und `off` liest YAML in älteren Fassungen als Wahrheitswerte. Setzen Sie solche Texte in Anführungszeichen, wenn sie als Text gemeint sind. Fünftens: Ein Kommentar braucht vor dem Rautenzeichen ein Leerzeichen, wenn er hinter einem Wert steht.

12.3 Die Datei compose.yaml

Ein erstes Beispiel zeigt den Aufbau an einem einzigen Webserver. Legen Sie ein Verzeichnis an.

```
$ mkdir -p /srv/docker/compose-erste && cd /srv/docker/compose-erste
$ nano compose.yaml
```

compose.yaml

```
services:
  web:
    image: nginx:stable-alpine
    ports:
      - "8080:80"
```

Die Datei hat nur ein Element auf oberster Ebene: `services`. Darunter steht ein Dienst namens `web`. Ein *Dienst* (englisch *service*) ist in Compose die Beschreibung eines Containers, aus der Compose bei Bedarf einen oder mehrere Container erzeugt. Starten Sie das Projekt.

```
$ docker compose up -d
[+] Running 2/2
 ✔ Network compose-erste_default  Created
 ✔ Container compose-erste-web-1  Started
$ docker compose ps
NAME                    IMAGE                SERVICE   STATUS
    ↪ PORTS
compose-erste-web-1     nginx:stable-alpine  web       Up 3 seconds
    ↪ 0.0.0.0:8080->80/tcp, ...
$ docker compose down
```

Compose hat selbstständig ein Netzwerk angelegt und den Container gestartet. Beachten Sie die Namen: Der Projektname `compose-erste` stammt vom Verzeichnis, danach folgen der Dienstname und eine Nummer. Das Netzwerk heißt `compose-erste_default`. In dieses Standardnetz kommen alle Dienste des Projekts, und sie finden sich darin über ihren Dienstnamen, wie in Kapitel 11 beschrieben. Die vollständige Datei kann auf oberster Ebene diese Elemente enthalten:

Element	Inhalt
`name`	der Projektname (optional, sonst der Verzeichnisname)
`services`	die Container, aus denen die Anwendung besteht
`networks`	selbst definierte Netzwerke
`volumes`	benannte Volumes
`secrets`	Zugangsdaten, die als Dateien in Container gelangen (Kapitel 18)
`configs`	Konfigurationsdateien, die Compose bereitstellt

Das Feld version entfällt

In älteren Anleitungen steht am Anfang der Datei eine Zeile wie `version: "3.8"`. Die heutige Compose-Spezifikation braucht sie nicht mehr. Das Feld wird ignoriert und führt zu einer Warnung. Schreiben Sie es nicht in neue Dateien. Findet sich in alten Dateien, die Sie übernehmen, eine solche Zeile, können Sie sie löschen.

12.4 Services

Ein Dienst entspricht im Kern den Optionen von `docker run`. Wer weiß, wie man einen Container mit `docker run` startet, kann jede Option in Compose wiederfinden. Die Tabelle stellt die wichtigsten gegenüber.

Compose-Schlüssel	`docker run`	Bedeutung
`image`	Image-Name	das zu startende Image
`build`	(`docker build`)	Image aus einem Dockerfile bauen
`ports`	`-p`	Portfreigaben
`volumes`	`-v`	Volumes und Bind Mounts
`environment`	`-e`	Umgebungsvariablen
`env_file`	`--env-file`	Variablen aus einer Datei
`networks`	`--network`	Netzwerke des Dienstes
`restart`	`--restart`	Neustartregel
`command`	Befehl hinter dem Image	ersetzt `CMD` des Images
`entrypoint`	`--entrypoint`	ersetzt `ENTRYPOINT` des Images
`user`	`--user`	Benutzer im Container
`healthcheck`	`--health-cmd`	Gesundheitsprüfung
`mem_limit`, `cpus`	`--memory`, `--cpus`	Ressourcenlimits

Ein Dienst kann entweder ein fertiges Image verwenden (`image`) oder eines aus einem Dockerfile bauen (`build`). Beides lässt sich kombinieren: Mit `build` und `image` zusammen baut Compose das Image und gibt ihm den angegebenen Namen.

Dienst mit eigenem Image

```
services:
  app:
    build: ./app
    image: projekt4-app:1.0
```

Hier liegt ein Dockerfile im Verzeichnis `app` neben der `compose.yaml`. Mit `docker compose build` oder `docker compose up --build` wird es gebaut.

> **container_name sparsam einsetzen**
>
> Compose erlaubt es, mit `container_name` einen festen Namen zu vergeben. Das ist verlockend, hat aber einen Preis: Mit festem Namen lässt sich der Dienst nicht mehr mehrfach starten, und zwei Projekte mit demselben Namen stoßen zusammen. Überlassen Sie die Namen im Normalfall Compose.

12.5 Networks

Ohne weitere Angaben legt Compose für jedes Projekt ein Netzwerk namens `projektname_default` an und hängt alle Dienste daran. Für einfache Anwendungen genügt das. Für mehr Kontrolle definiert man eigene Netzwerke auf oberster Ebene und weist sie den Diensten zu.

Eigene Netzwerke

```
services:
  web:
    image: nginx:stable-alpine
    networks:
      - frontend
  db:
    image: mariadb:11.8
    networks:
      - backend

networks:
  frontend:
  backend:
    internal: true
```

Der Dienst `web` hängt nur im Netz `frontend`, der Dienst `db` nur im Netz `backend`. Die beiden sehen sich nicht. Das Netz `backend` ist mit `internal: true` zusätzlich

vom Internet abgeschnitten, wie in Kapitel 11 beschrieben. Ein Dienst, der beide Netze erreichen soll, bekommt beide in seine Liste.

12.6 Volumes

Auch Volumes werden auf oberster Ebene deklariert und in den Diensten verwendet. Daneben sind Bind Mounts möglich, bei denen Compose relative Pfade vom Verzeichnis der `compose.yaml` aus auflöst.

Volume und Bind Mount

```
services:
  db:
    image: mariadb:11.8
    volumes:
      - db-daten:/var/lib/mysql
  web:
    image: nginx:stable-alpine
    volumes:
      - ./html:/usr/share/nginx/html:ro
      - ./nginx/default.conf:/etc/nginx/conf.d/default.conf:ro

volumes:
  db-daten:
```

Der Eintrag `db-daten:` unter `volumes` legt ein Volume an. Compose stellt den Projektnamen voran, das Volume heißt auf dem Host also `projektname_db-daten`. Die Schreibweise `./html` ist ein relativer Bind Mount. Sie ist in Compose ausdrücklich erlaubt und löst die Stolperfalle aus Kapitel 10, wo ein Name ohne Schrägstrich als Volume gedeutet wurde: Ein Pfad, der mit `./` beginnt, gilt immer als Verzeichnis.

12.7 Environment

Umgebungsvariablen lassen sich auf drei Wegen setzen. Direkt im Dienst, entweder als Zuordnung oder als Liste. Aus einer Datei mit `env_file`. Oder, am häufigsten, durch Einsetzen von Werten aus der Umgebung oder aus einer Datei namens `.env`.

environment und env_file

```
services:
  app:
    image: projekt4-app:1.0
    environment:
      DB_HOST: db
      DB_NAME: ${DB_NAME}
    env_file:
      - ./app.env
```

Der Ausdruck `${DB_NAME}` ist eine *Variable zum Einsetzen* (Interpolation). Compose ersetzt ihn beim Einlesen der Datei durch den Wert der Variablen. Seine Quelle ist entweder die Umgebung der Shell oder eine Datei namens `.env`, die neben der `compose.yaml` liegt. Compose liest sie automatisch.

.env

```
DB_NAME=besucher
DB_USER=zaehler
DB_PASSWORD=bitte-ersetzen
```

Mit zwei Zusätzen lassen sich Vorgaben und Pflichtangaben festlegen. `${VARIABLE:-wert}` setzt einen Standardwert ein, wenn die Variable fehlt oder leer ist. `${VARIABLE:?Fehlermeldung}` bricht mit der angegebenen Meldung ab, wenn sie fehlt.

Standardwerte und Pflichtangaben

```
ports:
  - "${HTTP_PORT:-8080}:80"
environment:
```

```
    DB_PASSWORD: ${DB_PASSWORD:?Bitte DB_PASSWORD in der Datei .env
      ↪ setzen}
```

Die zweite Form schützt vor einem häufigen Fehler: Ohne sie startet ein Container mit leerem Passwort, wenn die Variable vergessen wurde. Ein Dollarzeichen, das Compose nicht als Variable deuten soll, verdoppelt man: `$$`.

Die .env-Datei gehört nicht in die Versionsverwaltung

Die Datei `.env` enthält oft Passwörter. Legen Sie sie nie in ein öffentliches Git-Repository und schränken Sie die Rechte ein: `chmod 600 .env`. Legen Sie stattdessen eine Datei `.env.beispiel` mit Platzhaltern ab, die zeigt, welche Variablen nötig sind.

12.8 depends_on und Healthchecks

Eine Anwendung braucht ihre Datenbank, bevor sie starten kann. Mit `depends_on` legt man die Reihenfolge fest. In der einfachen Form wartet Compose nur darauf, dass der andere Container *gestartet* ist. Das genügt oft nicht, denn eine Datenbank braucht nach dem Start noch einige Sekunden, bis sie Verbindungen annimmt. Die ausführliche Form kann auf einen *gesunden* Container warten.

depends_on mit Bedingung

```
services:
  app:
    image: projekt4-app:1.0
    depends_on:
      db:
        condition: service_healthy
  db:
    image: mariadb:11.8
    healthcheck:
      test: ["CMD", "healthcheck.sh", "--connect", "--
    ↪ innodb_initialized"]
      interval: 10s
```

```
      timeout: 5s
      retries: 5
      start_period: 20s
```

Die Bedingung `service_healthy` setzt voraus, dass der Dienst, auf den gewartet wird, einen `healthcheck` hat. Fehlt er, meldet Compose einen Fehler. Die Schlüssel entsprechen den Optionen von `HEALTHCHECK` im Dockerfile aus Kapitel 8: `interval` ist der Abstand der Prüfungen, `timeout` die Höchstdauer einer Prüfung, `retries` die Zahl der Fehlschläge bis zum Zustand *unhealthy* und `start_period` die Schonfrist nach dem Start. Der Befehl unter `test` steht in der Listenform `["CMD", …]` (ohne Shell) oder in der Form `["CMD-SHELL", "Befehl"]`, die ihn von einer Shell ausführen lässt. Außer `service_healthy` gibt es `service_started` (nur gestartet) und `service_completed_successfully` (ein einmaliger Auftrag ist erfolgreich beendet).

Healthcheck prüft nicht die Anwendung

Ein Healthcheck sagt nur, was der Befehl prüft. Der Befehl `healthcheck.sh` aus dem MariaDB-Image prüft, ob der Server Verbindungen annimmt und die Speicherengine bereit ist. Er sagt nichts darüber, ob Ihre Tabellen vorhanden sind. Eine Anwendung sollte trotzdem Verbindungsfehler behandeln und es nach einer Pause erneut versuchen.

12.9 Neustartregeln

Auch Neustartregeln stehen in der Datei, und zwar je Dienst. Die Werte sind dieselben wie bei `docker run` in Kapitel 9.

restart

```
services:
  web:
    image: nginx:stable-alpine
    restart: unless-stopped
```

Für Dienste, die dauerhaft laufen sollen, ist `unless-stopped` die gängige Wahl. Ohne Angabe gilt `no`, ein Container kommt nach einem Neustart des Servers nicht von selbst wieder.

KAPITEL 13

Mehrere Container gemeinsam betreiben

Nach der Theorie des letzten Kapitels folgt der Alltag mit Compose. Sie bauen ein kleines Projekt aus zwei Containern, starten und beenden es, lesen die Protokolle, ändern die Konfiguration und spielen Updates ein. Alle Befehle dieses Kapitels gelten genauso für große Anwendungen mit zehn oder mehr Diensten.

13.1 Docker Compose installieren und prüfen

Eine eigene Installation ist nicht nötig. Das Compose-Plugin kam in Kapitel 5 mit dem Paket `docker-compose-plugin`. Ob es vorhanden ist, prüft ein einzelner Befehl.

```
$ docker compose version
Docker Compose version vX.Y.Z
```

Steht dort statt einer Version die Meldung `'compose' is not a docker command`, fehlt das Paket, und die Installation aus Kapitel 5 ist unvollständig. Das alte Einzelprogramm `docker-compose` mit Bindestrich sollten Sie nicht mehr verwenden. Es wird nicht mehr weiterentwickelt, und viele Anleitungen im Internet beziehen sich noch darauf. Wo dort `docker-compose` steht, schreiben Sie `docker compose` mit Leerzeichen.

Zum bequemeren Arbeiten kann die Shell Befehle, Optionen und Namen mit der Tabulatortaste vervollständigen, auch Unterbefehle wie `docker compose`. Das Skript dafür erzeugt der Docker-Client selbst. Für Bash installieren Sie zuerst das Paket `bash-completion` und legen das Skript danach im Benutzerverzeichnis ab, wie die Docker-Dokumentation es beschreibt:

```
$ sudo apt install bash-completion
$ mkdir -p ~/.local/share/bash-completion/completions
$ docker completion bash > ~/.local/share/bash-completion/completions
    ↪ /docker
```

Damit Bash die Vervollständigung lädt, muss sie in der Datei `~/.bashrc` eingebunden sein. Die Docker-Dokumentation nennt dafür die folgenden Zeilen. Danach laden Sie die Konfiguration neu:

```
$ cat <<'EOT' >> ~/.bashrc
if [ -f /etc/bash_completion ]; then
    . /etc/bash_completion
fi
EOT
$ source ~/.bashrc
```

Ob es wirkt, prüfen Sie in einem neuen Terminal: Tippen Sie `docker comp` und drücken Sie die Tabulatortaste. Skripte für andere Shells beschreibt die Docker-Dokumentation.

13.2 Eine Anwendung definieren

Das Beispielprojekt soll zwei Dienste enthalten: einen Webserver (nginx), der Anfragen von außen annimmt, und eine Anwendung, die dahinter arbeitet. Als Anwendung verwenden wir das Image aus Kapitel 8, also den kleinen Python-Webserver. Legen Sie das Projekt an und kopieren Sie das Dockerfile-Beispiel hinein.

```
$ mkdir -p /srv/docker/compose-demo/nginx
$ cp -r /srv/docker/dockerfile-demo /srv/docker/compose-demo/app
$ cd /srv/docker/compose-demo
```

Der Webserver soll Anfragen an die Anwendung weiterreichen. Dazu braucht er eine kleine Konfiguration.

nginx/default.conf

```
server {
    listen 80;

    location / {
        proxy_pass http://app:8000;
    }
}
```

Die Adresse `http://app:8000` enthält den Dienstnamen `app` als Rechnernamen. Das funktioniert, weil Compose beide Dienste in dasselbe Netzwerk hängt und der Docker-DNS aus Kapitel 11 den Namen auflöst. Die Datei, die beide Dienste beschreibt, sieht so aus.

compose.yaml

```
services:
  web:
    image: nginx:stable-alpine
    ports:
      - "8080:80"
    volumes:
      - ./nginx/default.conf:/etc/nginx/conf.d/default.conf:ro
    depends_on:
      app:
        condition: service_healthy
    restart: unless-stopped

  app:
    build: ./app
    image: compose-demo-app:1.0
```

```
    restart: unless-stopped
```

Der Dienst `app` wird aus dem Verzeichnis `app` gebaut. Weil das Dockerfile dort eine `HEALTHCHECK`-Anweisung enthält, kann der Dienst `web` mit `service_healthy` darauf warten. Vor dem ersten Start lässt sich die Datei prüfen.

```
$ docker compose config --quiet && echo "Datei ist gültig"
Datei ist gültig
```

`docker compose config` liest die Datei, setzt alle Variablen ein und gibt das Ergebnis aus. Mit `--quiet` unterbleibt die Ausgabe, und nur Fehler erscheinen. Das ist ein guter erster Schritt, wenn eine Datei sich nicht wie erwartet verhält.

13.3 Container starten

Zum Starten genügt ein Befehl. Die Option `-d` schickt die Container in den Hintergrund, so wie bei `docker run`.

```
$ docker compose up -d
[+] Building 6.2s (9/9) FINISHED
 ...
[+] Running 3/3
 ✔ Network compose-demo_default  Created
 ✔ Container compose-demo-app-1  Healthy
 ✔ Container compose-demo-web-1  Started
```

Compose hat zuerst das Image für `app` gebaut, dann das Netzwerk angelegt, den Container `app` gestartet und gewartet, bis er gesund war, und danach `web` gestartet. Dass `app` vor `web` kommt, folgt aus `depends_on`. Ob alles läuft, zeigt `docker compose ps`.

```
$ docker compose ps --format "table {{.Service}}\t{{.Status}}\t{{.
    ↪ Ports}}"
SERVICE   STATUS                   PORTS
```

```
app       Up 25 seconds (healthy)   8000/tcp
web       Up 24 seconds             0.0.0.0:8080->80/tcp,
     ↪ [::]:8080->80/tcp
$ curl -s http://localhost:8080
<h1>Hallo aus Python</h1>
```

Die Anfrage geht an nginx, wird an die Anwendung weitergereicht und kommt von dort beantwortet zurück. Nur `web` veröffentlicht einen Port. Der Dienst `app` ist von außen nicht erreichbar. Zwei Optionen von `up` sind wichtig. `--build` baut die Images vor dem Start neu, was nach Änderungen am Dockerfile oder am Programmcode nötig ist. `--wait` lässt Compose warten, bis alle Dienste laufen oder gesund sind, und gibt dann erst die Eingabe zurück. Das ist praktisch in Skripten.

```
$ docker compose up -d --build --wait
```

13.4 Container stoppen

Für das Beenden gibt es zwei Befehle mit unterschiedlicher Wirkung. `docker compose stop` hält die Container an, entfernt sie aber nicht, `docker compose start` startet sie wieder. `docker compose down` dagegen stoppt und *entfernt* Container und Netzwerke.

```
$ docker compose stop
$ docker compose start
$ docker compose down
[+] Running 3/3
 ✔ Container compose-demo-web-1  Removed
 ✔ Container compose-demo-app-1  Removed
 ✔ Network compose-demo_default  Removed
```

Beachten Sie, was `down` nicht entfernt: die Volumes und die gebauten Images. Das ist Absicht. Daten in benannten Volumes überstehen ein `down`, und genau deshalb gehören Datenbanken in Volumes. Soll auch das gelöscht werden, gibt es zwei Zusätze.

Befehl	Zusätzlich entfernt
`docker compose down -v`	die im Projekt deklarierten Volumes (Daten gehen verloren)
`docker compose down --rmi local`	die für das Projekt gebauten Images

down -v löscht Daten

Die Option `-v` löscht die Volumes des Projekts, auch das einer Datenbank. Die Daten sind danach unwiederbringlich verloren, und Docker fragt nicht nach. Verwenden Sie die Option nur in Testumgebungen oder nach einer Sicherung.

13.5 Protokolle auswerten

Die Protokolle aller Dienste eines Projekts zeigt `docker compose logs`. Die Ausgabe ist farbig und mit dem Dienstnamen gekennzeichnet, sodass man sie auseinanderhalten kann.

```
$ docker compose logs --tail 3
app-1  | 172.18.0.3 - - [11/Nov/2026 12:41:07] "GET / HTTP/1.0" 200 -
web-1  | 172.18.0.1 - - [11/Nov/2026:12:41:07 +0000] "GET / HTTP/1.1"
     ↪ 200 25 "-" "curl/8.x" "-"
$ docker compose logs -f web
$ docker compose logs -t --since 10m app
```

Die Optionen kennen Sie von `docker logs`: `--tail` begrenzt die Zeilenzahl, `-f` folgt dem Protokoll, `-t` setzt Zeitstempel, `--since` beschränkt auf einen Zeitraum. Hängt man am Ende einen Dienstnamen an, erscheint nur dessen Ausgabe. Das ist beim Suchen eines Fehlers in einer größeren Anwendung der wichtigste Handgriff.

Im Protokoll der Anwendung sieht man hier als Absender die Adresse `172.18.0.3`, also die des Containers `web`. Daran erkennt man, wie die Anfrage gelaufen ist: von außen zu nginx, von nginx zur Anwendung.

13.6 Konfiguration ändern

Ändern Sie die `compose.yaml` oder eine Datei, die sie einbindet, und führen Sie `docker compose up -d` erneut aus. Compose vergleicht, was sich geändert hat, und ersetzt nur die betroffenen Dienste.

```
$ nano nginx/default.conf
$ docker compose up -d
[+] Running 2/2
 ✔ Container compose-demo-app-1  Running
 ✔ Container compose-demo-web-1  Started
```

Hier wurde nur die nginx-Konfiguration geändert, die in den Container eingebunden ist, und deshalb bleibt `app` unberührt. Ändert sich dagegen die Definition

eines Dienstes selbst, etwa das Image oder ein Port, ersetzt Compose den Container (`Recreated`). Eine Änderung in einer eingebundenen Datei erkennt Compose nicht von selbst. Dann hilft ein gezielter Neustart des Dienstes.

```
$ docker compose restart web
```

Wer nginx die neue Konfiguration ohne Neustart des Containers laden lassen will, kann stattdessen `docker compose exec web nginx -s reload` verwenden.

> **nginx merkt sich Adressen**
>
> nginx löst den Namen in `proxy_pass` beim Start einmal auf und behält die Adresse. Wird der Dienst `app` neu erzeugt und bekommt eine neue Adresse im Docker-Netz, ruft nginx weiter die alte auf, und Sie erhalten Fehler vom Typ `502 Bad Gateway`. Starten Sie nginx dann mit `docker compose restart web` neu. Kapitel 16 zeigt Wege, das dauerhaft zu vermeiden.

Einzelne Befehle in einem laufenden Dienst führt `docker compose exec` aus, so wie `docker exec`, nur mit dem Dienstnamen statt des Containernamens. Für einmalige Aufgaben in einem neuen Container nutzt man `docker compose run --rm`.

```
$ docker compose exec app id
uid=10001(app) gid=10001(app) groups=10001(app)
$ docker compose run --rm app python --version
Python 3.x.y
```

13.7 Updates durchführen

Ein Update bedeutet bei Compose, neue Images zu holen und die Container damit neu zu starten. Für Dienste mit einem fertigen Image holt `docker compose pull` die neuesten Fassungen der angegebenen Tags.

```
$ docker compose pull
$ docker compose up -d
```

```
[+] Running 2/2
 ✔ Container compose-demo-app-1  Running
 ✔ Container compose-demo-web-1  Recreated
```

Das `pull` lädt nur herunter, das `up -d` erzeugt die Container neu, deren Image sich geändert hat. Dienste mit eigenem Image baut man neu. Mit `--pull` holt Compose dabei auch das Basisimage frisch.

```
$ docker compose build --pull
$ docker compose up -d
```

Ein Update ändert Software, und das kann schiefgehen. Drei Gewohnheiten machen es beherrschbar. Erstens: Verwenden Sie feste Tags, wie in Kapitel 7 beschrieben, und ändern Sie den Tag bewusst. Zweitens: Sichern Sie vor dem Update die Daten, besonders bei Datenbanken. Drittens: Notieren Sie den bisherigen Tag. Geht das Update schief, tragen Sie ihn wieder in die `compose.yaml` ein und führen `docker compose up -d` aus. Compose erzeugt die Container mit dem alten Image neu.

Nach mehreren Updates sammeln sich alte Images an. `docker image prune` entfernt solche ohne Tag. Der Befehl `docker compose images` zeigt, welche Images das Projekt verwendet.

13.8 Projekte verwalten

Auf einem Server laufen oft mehrere Projekte. `docker compose ls` listet sie auf, auch wenn man nicht im richtigen Verzeichnis steht.

```
$ docker compose ls
NAME           STATUS       CONFIG FILES
compose-demo   running(2)   /srv/docker/compose-demo/compose.yaml
projekt4       running(3)   /srv/docker/projekt-4/compose.yaml
```

Befehle wirken immer auf das Projekt, dessen `compose.yaml` im aktuellen Verzeichnis liegt. Aus einem anderen Verzeichnis kann man mit `-f` die Datei oder mit `-p` den Projektnamen angeben.

```
$ docker compose -f /srv/docker/projekt-4/compose.yaml ps
$ docker compose -p projekt4 logs --tail 5
```

Mehrere Dateien lassen sich zusammenführen. Eine Datei namens `compose.override.yaml` liest Compose neben der `compose.yaml` automatisch ein. Einzelne Werte der Hauptdatei überschreibt sie, Zuordnungen wie `environment` führt sie zusammen, und Listen wie `ports` ergänzt sie, ohne die vorhandenen Einträge zu ersetzen. Das nutzt man für Einstellungen, die sich zwischen Entwicklung und Betrieb unterscheiden.

compose.override.yaml

```
services:
  web:
    restart: "no"
    environment:
      DEBUG: "1"
```

Hier bekommt der Dienst `web` in der Entwicklungsumgebung keine Neustartregel, und eine zusätzliche Variable kommt hinzu. Die Anführungszeichen um `"no"` sind Absicht, denn ohne sie läse YAML das Wort als Wahrheitswert `false`.

Dienste, die nur bei Bedarf laufen sollen, etwa ein Werkzeug zur Datenbankverwaltung, versieht man mit einem Profil.

Dienst mit Profil

```
services:
  adminer:
    image: adminer
    ports:
      - "127.0.0.1:8082:8080"
    profiles: ["debug"]
```

```
$ docker compose up -d
$ docker compose --profile debug up -d
```

Der erste Aufruf lässt den Dienst `adminer` aus, der zweite startet ihn mit. Räumen Sie das Beispielprojekt am Ende auf.

```
$ docker compose down --rmi local
```

KAPITEL 14

Datenbankumgebungen

Fast jede Anwendung speichert ihre Daten in einer Datenbank. Container eignen sich gut dafür, solange man einige Eigenheiten kennt: Die Daten müssen außerhalb des Containers liegen, die Zugangsdaten gehören nicht ins Image, und eine Sicherung muss auf die Datenbank zugeschnitten sein. Dieses Kapitel zeigt den Betrieb von MariaDB und PostgreSQL mit den offiziellen Images. Es behandelt Einrichtung, dauerhafte Speicherung, Benutzer, Sicherung, Wiederherstellung und die Anbindung anderer Container.

14.1 MariaDB mit Docker

MariaDB ist ein freies relationales Datenbanksystem und ein Abkömmling von MySQL. Das offizielle Image heißt `mariadb`. Für den produktiven Einsatz verwendet man einen Tag mit fester Hauptversion. In diesem Buch ist das `mariadb:11.8`, eine Version mit Langzeitunterstützung. Der Tag `lts` zeigt ebenfalls auf eine solche Version, ändert sich aber irgendwann auf eine neuere.

Das Image richtet sich beim ersten Start über Umgebungsvariablen ein.

Variable	Wirkung
`MARIADB_ROOT_PASSWORD`	Passwort des Administrators `root`
`MARIADB_RANDOM_ROOT_PASSWORD`	erzeugt ein zufälliges Passwort und schreibt es ins Protokoll
`MARIADB_DATABASE`	legt beim ersten Start eine Datenbank an
`MARIADB_USER`	legt einen Benutzer an, der auf diese Datenbank alle Rechte hat
`MARIADB_PASSWORD`	Passwort dieses Benutzers
`MARIADB_ROOT_PASSWORD_FILE`	liest das Root-Passwort aus einer Datei (ebenso für die anderen Passwörter)

Eine der Möglichkeiten für das Root-Passwort muss gesetzt sein, sonst bricht der Container beim ersten Start mit einer Fehlermeldung ab. Ein erster Versuch mit `docker run` zeigt den Ablauf.

```
$ docker run -d --name mdb-test \
    -e MARIADB_ROOT_PASSWORD=nur-zum-testen \
    -e MARIADB_DATABASE=testdb \
    mariadb:11.8
$ docker logs mdb-test 2>&1 | tail -n 3
... [Note] mariadbd: ready for connections.
$ docker exec -it mdb-test mariadb -uroot -pnur-zum-testen
MariaDB [(none)]> SHOW DATABASES;
MariaDB [(none)]> exit
$ docker rm -f mdb-test
```

Der Befehl `mariadb` im Container ist der Kommandozeilen-Client. Die Ausgabe von `SHOW DATABASES;` listet neben `testdb` die Systemdatenbanken auf. Das Passwort auf der Kommandozeile ist hier nur für den Test gedacht. Es steht dort in der Prozessliste und im Verlauf der Shell. Wie man es vermeidet, zeigt der Abschnitt über Sicherungen.

Die Variablen wirken nur beim ersten Start

`MARIADB_ROOT_PASSWORD`, `MARIADB_DATABASE`, `MARIADB_USER` und `MARIADB_PASSWORD` werden nur ausgewertet, wenn das Datenverzeichnis leer ist. Bei einer bestehenden Datenbank ändert eine neue Einstellung nichts. Wer später ein Passwort in der `compose.yaml` ändert, ändert damit nicht das Passwort der Datenbank. Dazu braucht es einen SQL-Befehl, wie im Abschnitt über Benutzer gezeigt.

Im Betrieb beschreibt man den Dienst in einer `compose.yaml`. Das folgende Beispiel enthält alles Wichtige: die Zugangsdaten aus der `.env`, ein Volume für die Daten, einen Healthcheck und eine Neustartregel.

compose.yaml (MariaDB)

```
services:
  db:
    image: mariadb:11.8
    restart: unless-stopped
    environment:
      MARIADB_ROOT_PASSWORD: ${DB_ROOT_PASSWORD:?DB_ROOT_PASSWORD
    ↪ setzen}
      MARIADB_DATABASE: ${DB_NAME:-shop}
      MARIADB_USER: ${DB_USER:-shopuser}
      MARIADB_PASSWORD: ${DB_PASSWORD:?DB_PASSWORD setzen}
    volumes:
      - db-daten:/var/lib/mysql
    healthcheck:
      test: ["CMD", "healthcheck.sh", "--connect", "--
    ↪ innodb_initialized"]
      interval: 10s
      timeout: 5s
      retries: 5
```

```
      start_period: 20s

volumes:
  db-daten:
```

Der Befehl `healthcheck.sh` gehört zum Image. Er legt beim ersten Start einen eigenen Benutzer ohne Rechte an, mit dem er prüft, ob der Server Verbindungen annimmt und die Speicherengine InnoDB fertig initialisiert ist. Das Datenverzeichnis des Servers ist `/var/lib/mysql`, und genau dort gehört das Volume hin.

14.2 PostgreSQL mit Docker

PostgreSQL ist das zweite verbreitete freie Datenbanksystem. Das Image heißt `postgres`. Seine Variablen sind ähnlich, die Namen beginnen mit `POSTGRES_`.

Variable	Wirkung
`POSTGRES_PASSWORD`	Passwort des Superusers (Pflicht)
`POSTGRES_USER`	Name des Superusers (Standard `postgres`)
`POSTGRES_DB`	Name der ersten Datenbank (Standard wie der Benutzername)
`POSTGRES_PASSWORD_FILE`	liest das Passwort aus einer Datei

Wie bei MariaDB gelten die Variablen nur, solange das Datenverzeichnis leer ist. Ein wichtiger Unterschied zu MariaDB betrifft den Speicherort der Daten, und er hat sich mit der Version 18 geändert.

> **PostgreSQL 18: anderer Pfad für das Volume**
>
> Bis PostgreSQL 17 mounten Sie das Volume auf `/var/lib/postgresql/data`. Ab Version 18 ist der Datenpfad versionsabhängig (`/var/lib/postgresql/18/docker`), und das Volume gehört auf das übergeordnete Verzeichnis `/var/lib/postgresql`. Der Grund ist, dass sich Hauptversionen so leichter aktualisieren lassen. Wer den alten Pfad auf einem 18er-

> Image verwendet, erhält je nach System eine Fehlermeldung beim Start. Umgekehrt gilt: Mountet man bei einem 17er-Image das übergeordnete Verzeichnis, legt Docker für den Datenpfad ein anonymes Volume an, und die Daten gehen beim Löschen des Containers verloren. Prüfen Sie deshalb in jeder Anleitung, für welche Version sie geschrieben ist.

Die Beispiele in diesem Buch verwenden `postgres:18` und das Volume auf `/var/lib/postgresql`.

compose.yaml (PostgreSQL)

```
services:
  pg:
    image: postgres:18
    restart: unless-stopped
    environment:
      POSTGRES_PASSWORD: ${PG_PASSWORD:?PG_PASSWORD setzen}
      POSTGRES_USER: ${PG_USER:-appuser}
      POSTGRES_DB: ${PG_DB:-appdb}
    volumes:
      - pg-daten:/var/lib/postgresql
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U $${POSTGRES_USER} -d $${
      ↪ POSTGRES_DB}"]
      interval: 10s
      timeout: 5s
      retries: 5

volumes:
  pg-daten:
```

Das Programm `pg_isready` prüft, ob der Server Verbindungen annimmt. Die doppelten Dollarzeichen sind wichtig: `$$` verhindert, dass Compose die Variable selbst einsetzt. Sie soll erst in der Shell im Container ausgewertet werden. Den Client `psql` startet man so.

```
$ docker compose exec pg psql -U appuser -d appdb
appdb=# \l
appdb=# \q
```

Die Meta-Befehle von `psql` beginnen mit einem Backslash. `\l` listet die Datenbanken auf, `\dt` die Tabellen, `\q` beendet das Programm.

14.3 Persistente Datenspeicherung

Das Verzeichnis, in dem die Datenbank ihre Dateien ablegt, muss ein Volume sein. Andernfalls liegen die Daten in der beschreibbaren Schicht des Containers und verschwinden mit ihm, wie Kapitel 10 gezeigt hat. Die Datenbank-Images haben dafür eine `VOLUME`-Anweisung. Gibt man beim Start kein eigenes Volume an, legt Docker ein anonymes an. Das funktioniert, bis man den Container löscht und neu erzeugt. Dann liegt ein neues, leeres anonymes Volume im neuen Container, und die alte Datenbank ist zwar noch auf der Platte, aber unter einem Zufallsnamen versteckt.

Die Regel lautet daher: Jedes Datenbank-Image bekommt ein *benanntes* Volume auf dem Datenverzeichnis des Servers.

Image	Datenverzeichnis, auf das das Volume gehört
`mariadb`	`/var/lib/mysql`
`postgres` bis Version 17	`/var/lib/postgresql/data`
`postgres` ab Version 18	`/var/lib/postgresql`

Ob das Volume tatsächlich verwendet wird, lässt sich nachprüfen.

```
$ docker compose ps -q db | xargs docker inspect \
    --format '{{range .Mounts}}{{.Type}} {{.Name}} -> {{.Destination
    ↪ }}{{"\n"}}{{end}}'
volume projekt3_db-daten -> /var/lib/mysql
```

Steht dort statt eines sprechenden Namens eine lange Zeichenkette aus Ziffern und Buchstaben, ist das Volume anonym. Dann stimmt in der Datei etwas nicht, meist der Pfad.

14.4 Benutzer und Passwörter

Beim ersten Start legt das Image zwei Arten von Konten an. Der Administrator (`root` bei MariaDB, der Superuser bei PostgreSQL) darf alles. Der Anwendungsbenutzer aus `MARIADB_USER` oder `POSTGRES_USER` hat Rechte auf seine Datenbank. Die Anwendung soll sich nie als Administrator anmelden. Wird sie gehackt, wären sonst alle Datenbanken verloren. Das Prinzip heißt *geringste Rechte* (*least privilege*): Jedes Konto bekommt nur, was es braucht.

Weitere Konten legt man mit SQL an. Ein Beispiel für MariaDB, das einen Benutzer mit reinem Leserecht einrichtet.

```
$ docker compose exec db sh -c 'mariadb -uroot -p"
    ↪ $MARIADB_ROOT_PASSWORD"'
MariaDB [(none)]> CREATE USER 'bericht'@'%' IDENTIFIED BY 'ein-
    ↪ starkes-passwort';
MariaDB [(none)]> GRANT SELECT ON shop.* TO 'bericht'@'%';
MariaDB [(none)]> FLUSH PRIVILEGES;
MariaDB [(none)]> SHOW GRANTS FOR 'bericht'@'%';
MariaDB [(none)]> exit
```

Der Teil `'%'` nach dem Benutzernamen bedeutet, dass der Benutzer sich von jedem Rechner aus verbinden darf, hier also von jedem Container im selben Netz. Das Passwort `"$MARIADB_ROOT_PASSWORD"` steht in einfachen Anführungszeichen für die äußere Shell. Dadurch wertet es erst die Shell *im Container* aus, wo die Variable gesetzt ist. Das Passwort erscheint so weder in Ihrer Befehlsübersicht noch in der Prozessliste des Hosts.

Möchte man ein Passwort später ändern, geht das ebenfalls per SQL, nicht über die `compose.yaml`.

```
MariaDB [(none)]> ALTER USER 'bericht'@'%' IDENTIFIED BY 'neues-
    ↪ passwort';
```

Passwörter in der `.env` sind besser als Passwörter in der `compose.yaml`, aber nicht ideal. Eine sicherere Möglichkeit bieten Docker Secrets, bei denen das Passwort als Datei in den Container gelangt. Die Datenbank-Images unterstützen das über die Variablen mit der Endung `_FILE`, zum Beispiel `MARIADB_ROOT_PASSWORD_FILE`. Kapitel 18 zeigt, wie man das einrichtet.

Sonderzeichen in Passwörtern

Passwörter mit Zeichen wie `$`, `#`, Anführungszeichen oder Leerzeichen machen in `.env`-Dateien und Shell-Befehlen Ärger, weil diese Zeichen eine Sonderbedeutung haben. Für Passwörter in diesem Buch genügen Buchstaben und Ziffern, mindestens 20 Zeichen lang. Ein solches Passwort erzeugt `tr -dc A-Za-z0-9 </dev/urandom | head -c 24; echo`.

14.5 Datenbank-Backups

Die Dateikopie eines Volumes, wie in Kapitel 10 gezeigt, ist für Datenbanken im laufenden Betrieb unzuverlässig. Besser ist ein *Dump*, eine Textdatei mit den SQL-Befehlen, die die Datenbank neu aufbauen. Beide Datenbanken haben dafür ein eigenes Programm: `mariadb-dump` und `pg_dump`.

14.5.1 MariaDB sichern

```
$ docker compose exec -T db sh -c \
    'mariadb-dump -uroot -p"$MARIADB_ROOT_PASSWORD" --single-
    ↪ transaction --databases shop' \
    > shop.sql
$ grep -m1 "MariaDB dump" shop.sql
-- MariaDB dump 10.19  Distrib 11.8.x-MariaDB, for debian-linux-gnu (
    ↪ x86_64)
```

Die Option `-T` schaltet bei `docker compose exec` das Terminal ab. Sie ist nötig, wenn die Ausgabe in eine Datei umgeleitet wird, sonst vermischt sich die Ausgabe mit Steuerzeichen. `--single-transaction` liest die Datenbank konsistent, ohne sie zu sperren, und funktioniert bei Tabellen mit der Engine InnoDB.

`--databases shop` schreibt zusätzlich die Befehle zum Anlegen der Datenbank in den Dump. Alle Datenbanken sichert `--all-databases`. Im Image gibt es nur noch die neuen Programmnamen `mariadb` und `mariadb-dump`. Die älteren Namen `mysql` und `mysqldump` sind seit MariaDB 11 nicht mehr vorhanden.

Komprimieren lässt sich der Dump gleich beim Schreiben.

```
$ docker compose exec -T db sh -c \
    'mariadb-dump -uroot -p"$MARIADB_ROOT_PASSWORD" --single-
    ↪ transaction --databases shop' \
    | gzip > shop-$(date +%F).sql.gz
```

14.5.2 PostgreSQL sichern

```
$ docker compose exec -T pg pg_dump -U appuser -d appdb > appdb.sql
$ docker compose exec -T pg pg_dump -U appuser -Fc -d appdb > appdb.
    ↪ dump
```

Die erste Form schreibt SQL-Text. Die zweite (`-Fc`) erzeugt ein komprimiertes Archiv im eigenen Format, das man mit `pg_restore` einspielt und aus dem sich einzelne Tabellen auswählen lassen. Für alle Datenbanken samt Benutzern gibt es `pg_dumpall`.

14.6 Wiederherstellung

Das Einspielen ist die Umkehrung. Der Dump wird in den Client der Datenbank geleitet.

```
$ docker compose exec -T db sh -c 'mariadb -uroot -p"
    ↪ $MARIADB_ROOT_PASSWORD"' < shop.sql
$ gunzip -c shop-2026-11-11.sql.gz | docker compose exec -T db sh -c
    ↪ 'mariadb -uroot -p"$MARIADB_ROOT_PASSWORD"'
$ docker compose exec -T pg psql -U appuser -d appdb < appdb.sql
```

```
$ docker compose exec -T pg pg_restore -U appuser -d appdb --clean --
    ↪ if-exists < appdb.dump
```

Auch hier ist `-T` nötig, weil die Eingabe aus einer Datei kommt. `--clean --if-exists` bei `pg_restore` löscht vorhandene Objekte vor dem Einspielen. Bei einem Dump mit `--databases` legt MariaDB die Datenbank bei Bedarf selbst an.

Eine Sicherung gilt erst nach dem Wiederherstellungstest

Prüfen Sie jede Sicherung, indem Sie sie in eine frische Datenbank einspielen und nachsehen, ob die Daten vollständig sind. Viele Administratoren entdecken erst im Ernstfall, dass ihre Sicherung leer oder unvollständig war. In Praxisprojekt 3 üben Sie genau das. Wie man Sicherungen automatisiert und wie man den Ernstfall plant, zeigt Kapitel 19.

14.7 Verbindungen zwischen Containern

Eine Anwendung erreicht ihre Datenbank über das Docker-Netz. Als Rechnername dient der Dienstname, und der Port ist der Standardport der Datenbank: 3306 bei MariaDB und 5432 bei PostgreSQL. Ein Port muss dafür *nicht* veröffentlicht werden. Die Verbindungsdaten für eine Anwendung lauten damit zum Beispiel:

Verbindungsdaten einer Anwendung

```
DB_HOST=db
DB_PORT=3306
DB_NAME=shop
DB_USER=shopuser
DB_PASSWORD=<aus der .env>
```

Datenbank-Ports nicht veröffentlichen

Ein `ports`-Eintrag wie `"3306:3306"` macht die Datenbank für das ganze Netz erreichbar, und das gilt auch dann, wenn `ufw` den Port sperrt (Kapitel 11). Veröffentlichen Sie Datenbank-Ports nur, wenn es einen zwingen-

> den Grund gibt, und dann nur auf `127.0.0.1`. Für die Verwaltung genügt `docker compose exec`.

Typische Fehler beim Verbinden und ihre Ursachen:

Fehlermeldung	Ursache
`Can't connect` oder `Connection refused`	Die Datenbank ist noch nicht bereit. Mit `depends_on` und Healthcheck sicherstellen, dass die Anwendung erst danach startet.
`Unknown host` / `bad address`	Der Dienstname stimmt nicht oder die Container hängen nicht im selben Netz.
`Access denied for user`	Passwort oder Benutzername falsch, oder das Passwort in der `.env` wurde nach dem ersten Start geändert (die Datenbank kennt noch das alte).
`Unknown database`	Die Datenbank wurde nie angelegt. `MARIADB_DATABASE` wirkt nur beim ersten Start mit leerem Datenverzeichnis.

Die Verbindung lässt sich aus einem zweiten Container testen, ohne dass eine Anwendung läuft. Dazu startet man einen kurzlebigen Client im selben Netz.

```
$ docker run --rm -it --network projekt3_default mariadb:11.8 \
    mariadb -h db -ushopuser -p -e "SELECT 1"
```

Das Netz `projekt3_default` entsteht, wenn das Projekt `projekt3` läuft. Der Befehl fragt nach dem Passwort und gibt bei Erfolg die Zahl 1 aus. Das Image dient hier nur als Quelle für den Client.

Praxisprojekt 3: Datenbankserver

In diesem Projekt betreiben Sie einen MariaDB-Server im Container. Sie legen Daten und Benutzer an, belegen, dass die Daten einen Austausch des Containers überstehen, und richten eine Sicherung samt Wiederherstellung ein. Das Projekt verbindet die Kapitel 12 bis 14. Planen Sie etwa zwei Stunden ein.

Ausgangslage und Ziel

Ein kleiner Onlineshop braucht eine Datenbank, die zuverlässig läuft und gesichert wird. Der Sollzustand:

Anforderung	Sollzustand
Server	`mariadb:11.8` als Dienst `db` im Projekt `projekt3`
Zugangsdaten	in der Datei `.env`, Rechte `600`, zufällige Passwörter
Daten	benanntes Volume `db-daten` auf `/var/lib/mysql`
Datenbank und Benutzer	Datenbank `shop`, Benutzer `shopuser` (alle Rechte auf `shop`) und `bericht` (nur lesen)
Zugriff	nur im Docker-Netz, Port 3306 wird nicht veröffentlicht
Sicherung	Skript `backup.sh` erzeugt einen komprimierten Dump, `restore.sh` spielt ihn ein

Schritt 1: Dateien anlegen

```
$ mkdir -p /srv/docker/projekt-3 /srv/docker/backup/projekt-3
$ cd /srv/docker/projekt-3
$ {
    echo "DB_NAME=shop"
    echo "DB_USER=shopuser"
    echo "DB_PASSWORD=$(tr -dc A-Za-z0-9 </dev/urandom | head -c 24)"
    echo "DB_ROOT_PASSWORD=$(tr -dc A-Za-z0-9 </dev/urandom | head -c
    ↪ 24)"
  } > .env
$ chmod 600 .env
```

Die Klammern fassen die vier `echo`-Zeilen zusammen und leiten ihre Ausgabe gemeinsam in die Datei. Die Passwörter bestehen aus 24 zufälligen Buchstaben und Ziffern. Notieren Sie sich die Datei nicht auf Papier ab, sie bleibt auf dem Server. Die `compose.yaml` entspricht dem Beispiel aus Kapitel 14, ergänzt um den Projektnamen.

compose.yaml

```
name: projekt3

services:
  db:
    image: mariadb:11.8
    restart: unless-stopped
    environment:
      MARIADB_ROOT_PASSWORD: ${DB_ROOT_PASSWORD:?DB_ROOT_PASSWORD
    ↪ setzen}
      MARIADB_DATABASE: ${DB_NAME:-shop}
      MARIADB_USER: ${DB_USER:-shopuser}
      MARIADB_PASSWORD: ${DB_PASSWORD:?DB_PASSWORD setzen}
    volumes:
      - db-daten:/var/lib/mysql
    healthcheck:
      test: ["CMD", "healthcheck.sh", "--connect", "--
    ↪ innodb_initialized"]
```

```
      interval: 10s
      timeout: 5s
      retries: 5
      start_period: 20s

volumes:
  db-daten:
```

Schritt 2: Server starten

```
$ docker compose up -d --wait
[+] Running 3/3
 ✔ Network projekt3_default      Created
 ✔ Volume projekt3_db-daten      Created
 ✔ Container projekt3-db-1       Healthy
$ docker compose ps --format "table {{.Service}}\t{{.Status}}\t{{.
    ↪ Ports}}"
SERVICE   STATUS                    PORTS
db        Up 40 seconds (healthy)   3306/tcp
```

Beim ersten Start braucht MariaDB etwas länger, weil es das Datenverzeichnis anlegt. Die Spalte `PORTS` zeigt `3306/tcp` ohne Pfeil: Der Port ist im Image vermerkt, aber nicht veröffentlicht.

Schritt 3: Daten und Benutzer anlegen

Öffnen Sie den Client als Administrator. Das Passwort holt die Shell im Container aus der Umgebung, es erscheint nicht in Ihrer Befehlsübersicht.

```
$ docker compose exec db sh -c 'mariadb -uroot -p"
    ↪ $MARIADB_ROOT_PASSWORD"'
MariaDB [(none)]> USE shop;
MariaDB [shop]> CREATE TABLE gaeste (
```

```
    ->   id INT AUTO_INCREMENT PRIMARY KEY,
    ->   name VARCHAR(50) NOT NULL
    -> );
MariaDB [shop]> INSERT INTO gaeste (name) VALUES ('Anna'), ('Ben'),
    ↪ ('Chris');
MariaDB [shop]> SELECT * FROM gaeste;
+----+-------+
| id | name  |
+----+-------+
|  1 | Anna  |
|  2 | Ben   |
|  3 | Chris |
+----+-------+
```

Legen Sie in derselben Sitzung den Benutzer mit reinem Leserecht an. Ersetzen Sie das Passwort durch ein eigenes.

```
MariaDB [shop]> CREATE USER 'bericht'@'%' IDENTIFIED BY '
    ↪ LeseNurDieseDaten2026';
MariaDB [shop]> GRANT SELECT ON shop.* TO 'bericht'@'%';
MariaDB [shop]> SHOW GRANTS FOR 'bericht'@'%';
MariaDB [shop]> exit
```

Prüfen Sie nun, dass der neue Benutzer lesen, aber nicht schreiben darf.

```
$ docker compose exec db mariadb -ubericht -p shop -e "SELECT COUNT
    ↪ (*) FROM gaeste"
Enter password:
+----------+
| COUNT(*) |
+----------+
|        3 |
+----------+
$ docker compose exec db mariadb -ubericht -p shop -e "INSERT INTO
    ↪ gaeste (name) VALUES ('X')"
Enter password:
```

```
ERROR 1142 (42000) at line 1: INSERT command denied to user 'bericht'
    ↪ @'...' for table `shop`.`gaeste`
```

Schritt 4: Daten überstehen den Austausch

Entfernen Sie den Container und starten Sie ihn neu. Das Volume bleibt bestehen.

```
$ docker compose down
$ docker compose up -d --wait
$ docker compose exec db sh -c 'mariadb -uroot -p"
    ↪ $MARIADB_ROOT_PASSWORD" -N -e "SELECT COUNT(*) FROM shop.
    ↪ gaeste"'
3
```

Die drei Zeilen sind noch da. Der Container ist ein neuer, die Datenbank gehört dem Volume.

Schritt 5: Sicherung und Wiederherstellung

Die beiden Skripte arbeiten mit `mariadb-dump` und `mariadb`, wie in Kapitel 14 beschrieben.

/srv/docker/projekt-3/backup.sh

```
#!/bin/bash
# Sichert die Datenbank shop als komprimierten SQL-Dump
set -eo pipefail
cd /srv/docker/projekt-3
ZIEL=/srv/docker/backup/projekt-3
mkdir -p "$ZIEL"
DATEI="$ZIEL/shop-$(date +%Y-%m-%d_%H%M%S).sql.gz"
docker compose exec -T db sh -c \
  'mariadb-dump -uroot -p"$MARIADB_ROOT_PASSWORD" --single-
    ↪ transaction --databases shop' \
  | gzip > "$DATEI"
```

```
echo "Sicherung: $DATEI"
```

/srv/docker/projekt-3/restore.sh

```
#!/bin/bash
# Spielt einen Dump aus backup.sh wieder ein: ./restore.sh DATEI.sql.
    ↪ gz
set -eo pipefail
cd /srv/docker/projekt-3
[ -f "$1" ] || { echo "Aufruf: $0 DATEI.sql.gz"; exit 1; }
gunzip -c "$1" | docker compose exec -T db sh -c 'mariadb -uroot -p"
    ↪ $MARIADB_ROOT_PASSWORD"'
echo "Wiederhergestellt aus: $1"
```

```
$ chmod +x backup.sh restore.sh
$ ./backup.sh
Sicherung: /srv/docker/backup/projekt-3/shop-2026-11-11_144502.sql.gz
```

Die Option `set -eo pipefail` bricht das Skript ab, wenn irgendein Teil der Befehlskette fehlschlägt, nicht nur der letzte. Ohne `pipefail` würde ein fehlgeschlagener Dump unbemerkt eine leere Datei hinterlassen, weil `gzip` erfolgreich endet. Jetzt der entscheidende Test: Zerstören Sie die Tabelle und stellen Sie sie wieder her.

```
$ docker compose exec db sh -c 'mariadb -uroot -p"
    ↪ $MARIADB_ROOT_PASSWORD" -e "DROP TABLE shop.gaeste"'
$ ./restore.sh /srv/docker/backup/projekt-3/shop-2026-11-11_144502.
    ↪ sql.gz
Wiederhergestellt aus: /srv/docker/backup/projekt-3/shop-2026-11-11
    ↪ _144502.sql.gz
$ docker compose exec db sh -c 'mariadb -uroot -p"
    ↪ $MARIADB_ROOT_PASSWORD" -N -e "SELECT COUNT(*) FROM shop.
    ↪ gaeste"'
3
```

Der Dateiname enthält Datum und Uhrzeit Ihrer Sicherung. Die Tabelle ist wieder da, mit allen drei Zeilen.

Schritt 6: Funktionsprüfung und Abnahme

/srv/docker/projekt-3/check.sh

```
#!/bin/bash
# Funktionsprüfung für Projekt 3 (stellt dabei die Tabelle gaeste
    ↪ wieder her)
cd /srv/docker/projekt-3 || exit 1
FEHLER=0
ok()   { echo "[ OK ]     $1"; }
fail() { echo "[FEHLER]   $1"; FEHLER=1; }
sql() {
  docker compose exec -T db sh -c \
    'mariadb -uroot -p"$MARIADB_ROOT_PASSWORD" -N -e "$1"' sh "$1" \
    2>/dev/null
}

[ "$(docker compose ps --format '{{.Health}}' db)" = "healthy" ] \
  && ok "Datenbankserver ist gesund" || fail "Datenbankserver ist
    ↪ nicht gesund"
[ -z "$(docker compose port db 3306 2>/dev/null)" ] \
  && ok "Port 3306 ist nicht veröffentlicht" || fail "Port 3306 ist
    ↪ veröffentlicht"
[ "$(sql 'SELECT COUNT(*) FROM shop.gaeste')" -ge 1 ] 2>/dev/null \
  && ok "Tabelle gaeste enthält Daten" || fail "Tabelle gaeste fehlt
    ↪ oder ist leer"
GRANTS=$(sql "SHOW GRANTS FOR 'bericht'@'%'")
echo "$GRANTS" | grep -q "SELECT ON" && ! echo "$GRANTS" | grep -q "
    ↪ INSERT" \
  && ok "Benutzer bericht darf nur lesen" || fail "Rechte von bericht
    ↪  stimmen nicht"

VOR=$(sql 'SELECT COUNT(*) FROM shop.gaeste')
DATEI=$(./backup.sh | tail -n 1 | sed 's/^Sicherung: //')
[ -s "$DATEI" ] && ok "Sicherung wurde erzeugt" || fail "Sicherung
    ↪ fehlt oder ist leer"
sql 'DROP TABLE shop.gaeste' >/dev/null
./restore.sh "$DATEI" >/dev/null 2>&1
NACH=$(sql 'SELECT COUNT(*) FROM shop.gaeste')
```

```
[ -n "$VOR" ] && [ "$VOR" = "$NACH" ] \
  && ok "Wiederherstellung ergibt $NACH Zeilen" || fail "
     ↪ Wiederherstellung unvollständig"

echo
[ "$FEHLER" -eq 0 ] && echo "Alle Prüfungen bestanden." \
  || echo "Mindestens eine Prüfung ist fehlgeschlagen."
exit $FEHLER
```

```
$ chmod +x check.sh && ./check.sh
[ OK ]     Datenbankserver ist gesund
[ OK ]     Port 3306 ist nicht veröffentlicht
[ OK ]     Tabelle gaeste enthält Daten
[ OK ]     Benutzer bericht darf nur lesen
[ OK ]     Sicherung wurde erzeugt
[ OK ]     Wiederherstellung ergibt 3 Zeilen

Alle Prüfungen bestanden.
```

Zur Abnahme gehört außerdem: Starten Sie den Server neu und prüfen Sie, dass die Datenbank ohne Eingriff wieder gesund ist. Die Rechte der Datei `.env` müssen `600` lauten (`ls -l .env`). Und die Sicherungsdatei muss vorhanden und nicht leer sein.

Nicht mit down -v aufräumen, solange die Daten wichtig sind

`docker compose down -v` löscht das Volume und damit die Datenbank. Wollen Sie das Projekt am Ende ausdrücklich entfernen, tun Sie es erst nach der Sicherung. Mit `docker compose down` ohne `-v` bleibt das Volume erhalten.

Wenn etwas nicht klappt

Symptom	Ursache und Abhilfe
`DB_ROOT_PASSWORD setzen` beim Start	Die Datei `.env` fehlt oder liegt im falschen Verzeichnis. Sie muss neben der `compose.yaml` liegen.
`Access denied for user 'root'`	Das Passwort in der `.env` wurde nach dem ersten Start geändert. Die Datenbank kennt noch das alte. Siehe Kapitel 14.
Der Container bleibt auf `starting`	Beim ersten Start dauert die Einrichtung. Mit `docker compose logs db` den Fortschritt ansehen.
Die Sicherungsdatei ist sehr klein	Der Dump ist fehlgeschlagen. Das Skript mit `bash -x ./backup.sh` laufen lassen und die Meldung lesen.
`Unknown database 'shop'`	`MARIADB_DATABASE` wirkt nur bei leerem Datenverzeichnis. Entweder die Datenbank per SQL anlegen oder das Volume (nach Sicherung) löschen und neu starten.

Praxisprojekt 4: Multi-Container-Anwendung

In diesem Projekt bauen Sie eine kleine Webanwendung aus drei Containern: einem Webserver, einer eigenen Anwendung und einer Datenbank. Die Anwendung zählt Seitenaufrufe und speichert den Zähler in der Datenbank. Das Projekt verbindet Compose, Dockerfile, Netzwerke, Volumes und Datenbank aus den Kapiteln 7 bis 14. Es ist die Anwendung, die Abbildung 12.1 zeigt. Planen Sie etwa zweieinhalb Stunden ein.

Ausgangslage und Ziel

Anforderung	Sollzustand
Dienste	`web` (nginx), `app` (eigenes Image, Python), `db` (MariaDB 11.8)
Zugang von außen	nur über `web` auf Port 8080
Netzwerke	`frontend` (web und app), `backend` (app und db, ohne Zugang nach außen)
Daten	Volume `db-daten`, übersteht Neustarts und Austausch der Container
Start	Reihenfolge db, app, web durch Healthchecks gesichert
Zugangsdaten	in der `.env`, Rechte `600`

Schritt 1: Verzeichnisse und Anwendung

```
$ mkdir -p /srv/docker/projekt-4/app /srv/docker/projekt-4/nginx /srv
    ↪ /docker/backup/projekt-4
$ cd /srv/docker/projekt-4
```

Die Anwendung ist ein kleines Python-Programm, das mit der Bibliothek PyMySQL auf die Datenbank zugreift. Bei jedem Aufruf der Startseite erhöht es einen Zähler in der Tabelle `besuche` und zeigt den Stand an. Unter `/health` meldet es nur, dass es läuft, ohne die Datenbank zu fragen. Das ist die Adresse für den Healthcheck.

app/app.py

```
import html
import os
from http.server import BaseHTTPRequestHandler, HTTPServer

import pymysql


def verbinden():
    return pymysql.connect(
        host=os.environ["DB_HOST"],
        user=os.environ["DB_USER"],
        password=os.environ["DB_PASSWORD"],
        database=os.environ["DB_NAME"],
        autocommit=True,
    )


def zaehler_erhoehen():
    db = verbinden()
    try:
        cursor = db.cursor()
        cursor.execute(
            "CREATE TABLE IF NOT EXISTS besuche "
            "(id INT PRIMARY KEY, anzahl INT NOT NULL)"
```

```
        )
        cursor.execute(
            "INSERT INTO besuche (id, anzahl) VALUES (1, 1) "
            "ON DUPLICATE KEY UPDATE anzahl = anzahl + 1"
        )
        cursor.execute("SELECT anzahl FROM besuche WHERE id = 1")
        return cursor.fetchone()[0]
    finally:
        db.close()


class Handler(BaseHTTPRequestHandler):
    def antworten(self, status, inhalt, typ="text/html; charset=utf
    ↪ -8"):
        daten = inhalt.encode("utf-8")
        self.send_response(status)
        self.send_header("Content-Type", typ)
        self.send_header("Content-Length", str(len(daten)))
        self.end_headers()
        self.wfile.write(daten)

    def do_GET(self):
        if self.path == "/health":
            self.antworten(200, "ok", "text/plain; charset=utf-8")
        elif self.path == "/":
            try:
                anzahl = zaehler_erhoehen()
                text = f"Diese Seite wurde {anzahl} Mal aufgerufen."
                status = 200
            except Exception as fehler:
                text = "Datenbankfehler: " + html.escape(str(fehler))
                status = 500
            self.antworten(status, f"<h1>Besucherzähler</h1><p>{text
    ↪ }</p>")
        else:
            self.antworten(404, "Nicht gefunden", "text/plain;
    ↪ charset=utf-8")
```

```
if __name__ == "__main__":
    HTTPServer(("", 8000), Handler).serve_forever()
```

Zwei Einzelheiten sind wichtig. Die Zugangsdaten kommen aus Umgebungsvariablen, nicht aus dem Programm. Und nur die Startseite erhöht den Zähler: Ein Browser fragt zusätzlich nach `/favicon.ico`, das ohne Sonderbehandlung den Zähler verfälschen würde. Das Dockerfile baut das Image und folgt den Regeln aus Kapitel 8.

app/Dockerfile

```
FROM python:3-slim

RUN useradd --create-home --uid 10001 app \
    && pip install --no-cache-dir pymysql

WORKDIR /home/app
COPY --chown=app:app app.py .

USER app
EXPOSE 8000

HEALTHCHECK --interval=10s --timeout=3s --start-period=5s --retries=3
    ↪ \
  CMD python -c "import urllib.request; urllib.request.urlopen('http
    ↪ ://localhost:8000/health')"

CMD ["python", "app.py"]
```

Die Konfiguration des Webservers reicht Anfragen an die Anwendung weiter.

nginx/default.conf

```
server {
    listen 80;

    location / {
        proxy_pass http://app:8000;
```

```
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
    }
}
```

Schritt 2: Zugangsdaten und Compose-Datei

```
$ {
    echo "DB_NAME=besucher"
    echo "DB_USER=zaehler"
    echo "DB_PASSWORD=$(tr -dc A-Za-z0-9 </dev/urandom | head -c 24)"
    echo "DB_ROOT_PASSWORD=$(tr -dc A-Za-z0-9 </dev/urandom | head -c
    ↪ 24)"
  } > .env
$ chmod 600 .env
```

compose.yaml

```
name: projekt4

services:
  web:
    image: nginx:stable-alpine
    restart: unless-stopped
    ports:
      - "8080:80"
    volumes:
      - ./nginx/default.conf:/etc/nginx/conf.d/default.conf:ro
    depends_on:
      app:
        condition: service_healthy
    networks:
      - frontend

  app:
```

```
    build: ./app
    image: projekt4-app:1.0
    restart: unless-stopped
    environment:
      DB_HOST: db
      DB_NAME: ${DB_NAME}
      DB_USER: ${DB_USER}
      DB_PASSWORD: ${DB_PASSWORD}
    depends_on:
      db:
        condition: service_healthy
    networks:
      - frontend
      - backend

  db:
    image: mariadb:11.8
    restart: unless-stopped
    environment:
      MARIADB_ROOT_PASSWORD: ${DB_ROOT_PASSWORD}
      MARIADB_DATABASE: ${DB_NAME}
      MARIADB_USER: ${DB_USER}
      MARIADB_PASSWORD: ${DB_PASSWORD}
    volumes:
      - db-daten:/var/lib/mysql
    healthcheck:
      test: ["CMD", "healthcheck.sh", "--connect", "--
    ↪ innodb_initialized"]
      interval: 10s
      timeout: 5s
      retries: 5
      start_period: 20s
    networks:
      - backend

networks:
  frontend:
  backend:
```

```
    internal: true

volumes:
  db-daten:
```

Die Datei fasst zusammen, was Sie in den letzten Kapiteln gelernt haben. Die Anwendung hängt in beiden Netzen, der Webserver nur im vorderen und die Datenbank nur im hinteren. Der Webserver kann die Datenbank daher nicht erreichen, und die Datenbank hat durch `internal: true` keine Verbindung ins Internet. Veröffentlicht wird nur der Port 8080 von `web`.

Schritt 3: Starten und ausprobieren

```
$ docker compose config --quiet && echo "Datei ist gültig"
Datei ist gültig
$ docker compose up -d --build --wait
[+] Running 6/6
 ✔ Network projekt4_frontend  Created
 ✔ Network projekt4_backend   Created
 ✔ Volume projekt4_db-daten   Created
 ✔ Container projekt4-db-1    Healthy
 ✔ Container projekt4-app-1   Healthy
 ✔ Container projekt4-web-1   Started
$ curl -s http://localhost:8080
<h1>Besucherzähler</h1><p>Diese Seite wurde 1 Mal aufgerufen.</p>
$ curl -s http://localhost:8080
<h1>Besucherzähler</h1><p>Diese Seite wurde 2 Mal aufgerufen.</p>
```

Jeder Aufruf erhöht den Zähler, und der Stand steht in der Datenbank. Rufen Sie die Seite auch im Browser auf, mit der Adresse Ihres Servers und dem Port 8080. Die Statusmeldungen zeigen außerdem, dass Compose die Reihenfolge eingehalten hat: erst die Datenbank, dann die Anwendung, zuletzt der Webserver.

Schritt 4: Netzwerk und Isolation prüfen

Die Netzwerkaufteilung lässt sich nachprüfen. Die Anwendung findet die Datenbank, der Webserver nicht.

```
$ docker compose exec app getent hosts db
172.19.0.2      db
$ docker compose exec web ping -c 1 db
ping: bad address 'db'
$ docker compose port db 3306
$ docker compose port web 80
0.0.0.0:8080
```

Die Datenbank ist für die Anwendung unter dem Namen `db` sichtbar. Der Webserver kennt sie nicht, weil er nur im Netz `frontend` hängt. Die Abfrage des Datenbank-Ports liefert nichts zurück, denn er wird nicht veröffentlicht. Ihre Adresse kann von der hier gezeigten abweichen.

Schritt 5: Daten überstehen den Austausch

```
$ docker compose down
$ docker compose up -d --wait
$ curl -s http://localhost:8080
<h1>Besucherzähler</h1><p>Diese Seite wurde 3 Mal aufgerufen.</p>
```

Alle drei Container sind neu, aber der Zähler zählt weiter, denn das Volume hat die Datenbank bewahrt. Bei `docker compose down -v` würde er wieder bei 1 beginnen.

Schritt 6: Sicherung

/srv/docker/projekt-4/backup.sh

```
#!/bin/bash
# Sichert die Datenbank des Projekts als komprimierten SQL-Dump
set -eo pipefail
cd /srv/docker/projekt-4
ZIEL=/srv/docker/backup/projekt-4
mkdir -p "$ZIEL"
DATEI="$ZIEL/besucher-$(date +%Y-%m-%d_%H%M%S).sql.gz"
docker compose exec -T db sh -c \
  'mariadb-dump -uroot -p"$MARIADB_ROOT_PASSWORD" --single-
    ↪ transaction --databases "$MARIADB_DATABASE"' \
  | gzip > "$DATEI"
echo "Sicherung: $DATEI"
```

```
$ chmod +x backup.sh && ./backup.sh
Sicherung: /srv/docker/backup/projekt-4/besucher-2026-11-11_153012.
    ↪ sql.gz
```

Hier steht `"$MARIADB_DATABASE"` in der Befehlskette, die die Shell im Container auswertet. Der Name der Datenbank kommt also aus derselben Quelle wie beim Anlegen.

Schritt 7: Funktionsprüfung und Abnahme

/srv/docker/projekt-4/check.sh

```
#!/bin/bash
# Funktionsprüfung für Projekt 4: Multi-Container-Anwendung
cd /srv/docker/projekt-4 || exit 1
FEHLER=0
ok()   { echo "[ OK ]     $1"; }
fail() { echo "[FEHLER]   $1"; FEHLER=1; }
URL=http://localhost:8080
```

```
zahl() { curl -s $URL/ | sed -n 's/.*wurde \([0-9]*\) Mal.*/\1/p'; }

[ "$(docker compose ps --status running --services | sort | tr '\n' '
    ↪  ')" = "app db web " ] \
  && ok "Alle drei Dienste laufen" || fail "Nicht alle Dienste laufen
    ↪ "
[ "$(curl -s -o /dev/null -w '%{http_code}' $URL/)" = "200" ] \
  && ok "Startseite liefert 200" || fail "Startseite liefert nicht
    ↪ 200"
A=$(zahl); B=$(zahl)
[ -n "$A" ] && [ "$B" -eq $((A + 1)) ] 2>/dev/null \
  && ok "Zähler steigt ($A, dann $B)" || fail "Zähler steigt nicht"
[ -z "$(docker compose port db 3306 2>/dev/null)" ] \
  && ok "Datenbank-Port ist nicht veröffentlicht" || fail "Datenbank-
    ↪ Port ist veröffentlicht"
docker compose exec -T web ping -c 1 -W 2 db >/dev/null 2>&1 \
  && fail "web erreicht die Datenbank" || ok "web erreicht die
    ↪ Datenbank nicht"
docker compose exec -T app getent hosts db >/dev/null 2>&1 \
  && ok "app erreicht die Datenbank" || fail "app erreicht die
    ↪ Datenbank nicht"

VOR=$(zahl)
docker compose down >/dev/null 2>&1
docker compose up -d --wait >/dev/null 2>&1
NACH=$(zahl)
[ -n "$VOR" ] && [ "$NACH" -eq $((VOR + 1)) ] 2>/dev/null \
  && ok "Zähler übersteht den Austausch ($VOR, dann $NACH)" || fail "
    ↪ Daten sind verloren"

echo
[ "$FEHLER" -eq 0 ] && echo "Alle Prüfungen bestanden." \
  || echo "Mindestens eine Prüfung ist fehlgeschlagen."
exit $FEHLER
```

```
$ chmod +x check.sh && ./check.sh
[ OK ]     Alle drei Dienste laufen
```

```
[ OK ]     Startseite liefert 200
[ OK ]     Zähler steigt (4, dann 5)
[ OK ]     Datenbank-Port ist nicht veröffentlicht
[ OK ]     web erreicht die Datenbank nicht
[ OK ]     app erreicht die Datenbank
[ OK ]     Zähler übersteht den Austausch (6, dann 7)

Alle Prüfungen bestanden.
```

Die Zahlen hängen davon ab, wie oft Sie die Seite schon aufgerufen haben. Zur Abnahme gehören zusätzlich: Ein Neustart des Servers, nach dem alle Dienste ohne Eingriff wieder laufen. Die Rechte `600` der `.env`. Und ein Blick mit `docker compose logs` in die Protokolle aller drei Dienste.

Nach dem Neuerzeugen von app den Webserver neu starten

Erzeugen Sie nur den Dienst `app` neu, etwa nach einer Codeänderung mit `docker compose up -d --build app`, bekommt er unter Umständen eine neue Adresse, und nginx ruft weiter die alte auf. Starten Sie dann mit `docker compose restart web` den Webserver neu, wie in Kapitel 13 beschrieben.

Wenn etwas nicht klappt

Symptom	Ursache und Abhilfe
`502 Bad Gateway`	nginx erreicht die Anwendung nicht. `docker compose ps` prüfen, ob `app` gesund ist, und `docker compose restart web` versuchen.
Die Seite meldet `Datenbankfehler`	`docker compose logs app db` lesen. Meist stimmen Benutzer oder Passwort nicht mit dem überein, was beim ersten Start in die Datenbank kam.
`app` bleibt `unhealthy`	`docker compose logs app` zeigt den Fehler. Häufig fehlt die Variable `DB_HOST` oder die `.env` wurde nicht gelesen.
`pip` kann PyMySQL nicht laden	Der Server erreicht das Internet nicht. Netzwerk und Namensauflösung wie in Kapitel 4 prüfen.
Der Zähler beginnt nach einem Neustart bei 1	Das Volume wurde gelöscht (`down -v`) oder die Datenbank hat ein anonymes Volume benutzt. `docker volume ls` zeigt es.

Lernkontrolle zu Teil V

Die Aufgaben prüfen die Kapitel 12 bis 14. Die praktischen Aufgaben führen Sie auf Ihrem Übungsserver durch. Die Lösungen stehen im Anhang.

Verständnisfragen

V1. Was bedeutet es, dass Docker Compose *deklarativ* arbeitet? Was geschieht, wenn Sie die `compose.yaml` ändern und `docker compose up -d` erneut ausführen?

V2. Warum steht das Feld `version` nicht mehr in neuen Compose-Dateien?

V3. Worin unterscheidet sich `docker compose down` von `docker compose stop`, und was bewirkt zusätzlich die Option `-v`?

V4. Warum genügt `depends_on` in der einfachen Form oft nicht für eine Datenbank? Was leistet die Bedingung `service_healthy`?

V5. Warum wirken die Variablen `MARIADB_ROOT_PASSWORD` und `MARIADB_DATABASE` nur beim ersten Start? Was folgt daraus für eine spätere Passwortänderung?

V6. Auf welches Verzeichnis gehört das Volume bei `mariadb`, bei `postgres:17` und bei `postgres:18`?

V7. Warum ist eine Sicherung per Dateikopie des Volumes bei einer laufenden Datenbank riskant? Welche Alternative gibt es?

V8. Welche Rechnernamen und Ports verwendet eine Anwendung im Docker-Netz, um eine Datenbank zu erreichen? Muss dafür ein Port veröffentlicht werden?

Multiple-Choice-Aufgaben

M1. Welcher Dateiname wird für Compose in diesem Buch verwendet?

(a) `docker-compose.yaml`
(b) `compose.json`
(c) `compose.yaml`
(d) `Composefile`

M2. Wie schreibt man in einer Compose-Datei einen Standardwert für eine Variable?

(a) `${PORT:-8080}`
(b) `${PORT=8080}`
(c) `$PORT(8080)`
(d) `{{PORT|8080}}`

M3. Welche Option von `docker compose exec` braucht man, wenn die Ausgabe in eine Datei umgeleitet wird?

(a) `-d`
(b) `-T`
(c) `-it`
(d) `-q`

M4. Welche Bedingung lässt Compose auf einen gesunden Dienst warten?

(a) `service_started`
(b) `service_ready`
(c) `service_completed_successfully`
(d) `service_healthy`

M5. Welches Programm erstellt in einem MariaDB-11-Container den Dump einer Datenbank?

(a) `mysqldump`
(b) `mariadb-dump`
(c) `pg_dump`

(d) `dbexport`

M6. Was bewirkt `internal: true` bei einem Compose-Netzwerk?
(a) Das Netz ist nur für den Host sichtbar.
(b) Das Netz hat keinen Zugang nach außen.
(c) Das Netz wird verschlüsselt.
(d) Das Netz wird nicht angelegt.

Praktische Aufgaben

P1. Schreiben Sie eine `compose.yaml` für einen nginx-Dienst, der Port 8090 nach außen anbietet und das Verzeichnis `html` schreibgeschützt einbindet. Starten Sie das Projekt, rufen Sie die Seite ab, lesen Sie die Protokolle und entfernen Sie das Projekt wieder.

P2. Erweitern Sie die Datei um einen Dienst `db` mit MariaDB, dessen Passwort aus einer `.env` kommt und der ein benanntes Volume nutzt. Warten Sie mit `depends_on` und Healthcheck, bis die Datenbank bereit ist, bevor nginx startet.

P3. Sichern Sie eine Datenbank mit `mariadb-dump` in eine komprimierte Datei, löschen Sie eine Tabelle und stellen Sie sie aus der Sicherung wieder her. Prüfen Sie die Zeilenzahl vor und nach dem Test.

Fehlersuche

F1. `docker compose up` meldet, die Datei sei ungültig (`yaml: line 7: mapping values are not allowed`). Welche typischen Ursachen prüfen Sie?

F2. Eine Anwendung meldet beim Start `Connection refused` zur Datenbank, obwohl beide im selben Compose-Projekt laufen. Was ist die wahrscheinlichste Ursache, und wie beheben Sie sie?

F3. Der Administrator hat in der `.env` das Datenbankpasswort geändert und den Stack neu gestartet. Die Anwendung meldet `Access denied`. Warum, und wie lösen Sie es?

Szenario aus dem Betrieb

Ein Team betreibt einen Stack aus Webserver, Anwendung und PostgreSQL 17 mit einem Volume auf `/var/lib/postgresql/data`. Der Entwickler möchte auf `postgres:18` aktualisieren und ändert nur den Image-Tag. Beschreiben Sie, was dabei schiefgehen kann, und wie Sie das Update sicher vorbereiten.

TEIL VI

Webdienste und Serveranwendungen

KAPITEL 15

Webserver mit Docker

In Praxisprojekt 2 haben Sie einen nginx-Container zum Laufen gebracht. Dieses Kapitel geht dahinter: Es erklärt, wie das nginx-Image aufgebaut ist, wie man die Konfiguration versteht und prüft, wie man Webinhalte am besten bereitstellt und wie man Fehler anhand der Protokolle findet. Es setzt voraus, dass Sie die Grundlagen aus dem Projekt kennen. Auch hier gilt: Alle Befehle laufen auf Debian 13 oder Ubuntu Server mit Docker 29.

15.1 nginx installieren

Eine Installation im herkömmlichen Sinn gibt es nicht. Der Webserver steckt im Image, und ein `docker pull` genügt. Das offizielle Image gibt es in mehreren Zweigen, die man am Tag erkennt.

Tag	Bedeutung
`nginx:stable`	der stabile Zweig, bekommt nur Fehlerkorrekturen, Debian-Basis
`nginx:stable-alpine`	wie oben, aber auf Alpine Linux und dadurch deutlich kleiner
`nginx:mainline`	der Hauptzweig mit neuen Funktionen, Debian-Basis
`nginx:mainline-alpine`	wie oben auf Alpine

Für den Betrieb ist der stabile Zweig die sichere Wahl, und wir verwenden durchgehend `nginx:stable-alpine`. Welche Fassung dahinter steckt, fragt man im Container ab.

```
$ docker run --rm nginx:stable-alpine nginx -v
nginx version: nginx/1.x.y
$ docker run --rm nginx:stable-alpine nginx -V 2>&1 | head -n 2
nginx version: nginx/1.x.y
built by gcc ...
```

Die Ausgabe von `-V` mit großem V nennt zusätzlich die Module und Übersetzungsoptionen. Die Versionsnummer weicht bei Ihnen ab. Wichtiger als die Version ist, wo im Image welche Dateien liegen.

Pfad im Container	Inhalt
`/etc/nginx/nginx.conf`	Hauptkonfiguration, bindet `conf.d` ein
`/etc/nginx/conf.d/`	Konfigurationsdateien der einzelnen Server (`default.conf`)
`/usr/share/nginx/html/`	Standardverzeichnis für die Webinhalte
`/var/log/nginx/`	Protokolle, im Image Verweise auf Standardausgabe und Standardfehlerausgabe
`/docker-entrypoint.d/`	Skripte, die beim Start vor nginx laufen

Diese Pfade braucht man ständig, wenn man Dateien einbindet. Die Hauptdatei `nginx.conf` muss man selten ändern. Man ergänzt oder ersetzt stattdessen Dateien in `conf.d`.

15.2 Webserver-Container betreiben

Zum Betrieb gehört mehr als der Start. Die drei Handgriffe, die man dabei am häufigsten braucht, sind das Prüfen der Konfiguration, das Neuladen ohne Unterbrechung und ein Gesundheitstest.

Vor jeder Änderung an der Konfiguration prüft man sie, ohne den laufenden Server anzufassen. Das erledigt `nginx -t` im Container.

```
$ docker exec p2-web nginx -t
nginx: the configuration file /etc/nginx/nginx.conf syntax is ok
nginx: configuration file /etc/nginx/nginx.conf test is successful
```

Enthält die Konfiguration einen Fehler, nennt die Meldung die Datei und die Zeile. Erst wenn der Test erfolgreich ist, lädt man die Konfiguration neu.

```
$ docker exec p2-web nginx -s reload
```

Beim Neuladen starten neue Arbeitsprozesse mit der neuen Konfiguration, während die alten ihre laufenden Anfragen noch beenden. Besucher merken nichts davon. Das ist dem Neustart des Containers vorzuziehen. Ist die neue Konfiguration fehlerhaft, bleibt die alte aktiv. Das Neuladen gilt für Änderungen an Dateien, die im Container sichtbar sind, etwa über einen Bind Mount. Ändert sich dagegen das Image oder die Compose-Definition, erzeugt Compose den Container neu.

Der Gesundheitstest nutzt das im Alpine-Image enthaltene Programm `wget`. Es prüft, ob der Server auf einer Anfrage antwortet. In einer `compose.yaml` sieht das so aus.

Healthcheck für nginx

```
services:
  web:
    image: nginx:stable-alpine
    healthcheck:
      test: ["CMD", "wget", "-q", "--spider", "http://127.0.0.1/"]
      interval: 15s
      timeout: 3s
      retries: 3
      start_period: 5s
```

Die Option `--spider` sorgt dafür, dass `wget` nur anfragt und die Seite nicht herunterlädt. Der Befehl beendet sich mit dem Wert 0, wenn der Server antwortet. Die Adresse `127.0.0.1` ist im Container die des Containers selbst, nicht des Hosts.

Laufen als normaler Benutzer

Das Standard-Image startet nginx als Root, damit es den privilegierten Port 80 belegen kann. Die Arbeitsprozesse laufen danach als Benutzer `nginx`. Wer strengere Vorgaben hat, nimmt das Image `nginxinc/nginx-unprivileged`. Es läuft komplett ohne Root-Rechte und lauscht deshalb auf Port 8080 statt 80. Kapitel 18 greift das Thema auf.

15.3 Eigene Webseiten bereitstellen

Für die Webinhalte gibt es zwei Wege, die beide ihre Berechtigung haben. Beim ersten liegen die Dateien auf dem Host und werden in den Container eingebunden. Beim zweiten backt man sie in ein eigenes Image.

	Bind Mount	Eigenes Image
Änderungen	sofort sichtbar, kein Neustart nötig	neues Image bauen, Container ersetzen
Wiederholbarkeit	hängt vom Zustand des Hosts ab	Image enthält alles, überall gleich
Verteilung	Dateien müssen auf jeden Host kopiert werden	Image genügt, kommt aus der Registry
Typischer Einsatz	Entwicklung, häufig wechselnde Inhalte	Betrieb, feste Fassungen

Der zweite Weg ist mit dem Wissen aus Kapitel 8 schnell gebaut.

Dockerfile für eine feste Webseite

```
FROM nginx:stable-alpine
COPY nginx/default.conf /etc/nginx/conf.d/default.conf
COPY html/ /usr/share/nginx/html/
```

```
$ docker build -t verein-web:1.0 .
$ docker run -d --name verein -p 8080:80 verein-web:1.0
```

Das Image enthält die Seite samt Konfiguration. Wer es auf einem anderen Server startet, bekommt dieselbe Seite. Ein Update der Seite ist ein neues Image mit neuem Tag, und ein Zurücksetzen ist ein Start mit dem alten Tag. Für kleine Projekte und zum Lernen ist der Bind Mount bequemer. Für den dauerhaften Betrieb auf mehreren Servern ist das Image robuster.

15.3.1 Die Konfiguration verstehen

Die Datei `conf.d/default.conf` beschreibt einen oder mehrere *Server*, jeweils mit einem Block `server`. Darin stehen Ort und Verhalten.

conf.d/default.conf

```
server {
    listen 80;
    server_name _;
    charset utf-8;

    root /usr/share/nginx/html;
    index index.html;

    gzip on;
    gzip_types text/css application/javascript application/json image
    ↪ /svg+xml;

    location / {
        try_files $uri $uri/ =404;
    }

    location /assets/ {
        expires 7d;
        add_header Cache-Control "public";
    }

    error_page 404 /404.html;
}
```

Jede Zeile hat eine feste Bedeutung. `listen` nennt den Port im Container. `server_name` legt fest, für welchen Namen der Block zuständig ist, der Unterstrich steht für „jeden“. `root` und `index` bestimmen das Verzeichnis und

die Standarddatei. `gzip` komprimiert Textdateien vor dem Senden, was die Übertragung beschleunigt. Ein `location`-Block gilt für eine bestimmte Adresse: Der erste fängt alles auf und sucht die Datei oder das Verzeichnis, der zweite gilt nur unter `/assets/` und weist Browser an, die Dateien sieben Tage lang zwischenzuspeichern. `error_page` verweist für den Fehler 404 auf eine eigene Seite.

> **Semikolon nicht vergessen**
>
> Jede Anweisung in nginx endet mit einem Semikolon. Ein fehlendes Semikolon ist der häufigste Konfigurationsfehler, und nginx meldet ihn mit einer Meldung wie `unexpected "}"`, die auf die Zeile hinter dem Fehler zeigt. `nginx -t` fängt das ab.

15.4 Bind Mounts für Webinhalte

Bei der Arbeit mit Bind Mounts gibt es drei Eigenheiten, die in der Praxis regelmäßig Verwirrung stiften.

Die erste betrifft Zugriffsrechte. Der nginx-Arbeitsprozess läuft als Benutzer `nginx` und muss die Dateien lesen dürfen. Die Dateien brauchen deshalb Leserechte für andere, die Verzeichnisse zusätzlich das Recht zum Betreten. Bei einem `403 Forbidden` ist das der erste Verdacht. Abhilfe schafft:

```
$ chmod -R a+rX /srv/docker/projekt-2/html
```

Das große `X` setzt das Ausführungsrecht nur bei Verzeichnissen und bei Dateien, die es schon haben. Es ist die sichere Schreibweise.

Die zweite Eigenheit: Verweise (symbolische Links) im eingebundenen Verzeichnis, die auf Orte außerhalb davon zeigen, funktionieren im Container nicht. Der Container sieht nur das eingebundene Verzeichnis. Ein Link auf `/home/admin/bilder` führt im Container ins Leere.

Die dritte betrifft einzelne eingebundene Dateien. Viele Editoren und Werkzeuge ersetzen eine Datei beim Speichern durch eine neue, statt sie zu überschreiben. Für den Bind Mount einer *einzelnen Datei* hat das Folgen: Der Container hält weiter die alte Datei fest und sieht Ihre Änderung nicht. Ein Neustart des Containers

behebt das. Wer häufig Konfigurationsdateien ändert, bindet deshalb besser das *Verzeichnis* ein, in dem sie liegen. Bei Verzeichnissen tritt das Problem nicht auf.

> **Browser speichern zwischen**
>
> Sieht man nach einer Änderung noch die alte Seite, liegt es oft am Browser und nicht am Server. Ein hartes Neuladen mit [Strg]+[F5] oder die Entwicklerwerkzeuge mit abgeschaltetem Zwischenspeicher zeigen, ob die neue Fassung ausgeliefert wird. Auch `curl` umgeht den Zwischenspeicher des Browsers.

15.5 Portfreigaben

Ein Webserver im Container lauscht auf Port 80 oder 8080 und wird mit `-p` nach außen geführt, wie in Kapitel 11 gezeigt. Einige Besonderheiten treffen speziell Webserver.

Der Port auf dem Host muss frei sein. Ein Host-Port 80 gehört häufig schon einem anderen Dienst, etwa einem vorhandenen Webserver. Mit `sudo ss -tlnp | grep -E ":80 |:443 "` sieht man, wer ihn belegt. Auf einem Server, der *nur* Docker-Webdienste betreibt, kann der Container die Ports 80 und 443 direkt bekommen.

Mehrere Webserver brauchen verschiedene Host-Ports. Zwei Container können nicht beide den Host-Port 80 belegen. Für einen Besucher, der `http://server:8081` eintippen muss, ist das unbequem. Die Lösung ist ein Reverse Proxy, der auf Port 80 und 443 lauscht und die Anfragen nach Namen an die Container verteilt. Das ist das Thema des nächsten Kapitels.

Wer einen Port nur lokal anbieten will, bindet ihn an die Loopback-Adresse (`127.0.0.1:8080:80`), und wer ihn für das Heimnetz öffnen will, lässt die Adresse weg. Nicht vergessen: Ein veröffentlichter Port umgeht `ufw`.

15.6 Logs und Fehleranalyse

nginx führt zwei Protokolle. Das *Zugriffsprotokoll* (*access log*) hat für jede Anfrage eine Zeile. Das *Fehlerprotokoll* (*error log*) enthält Probleme des Servers. Im offiziellen Image sind beide so eingestellt, dass sie an die Standardausgabe und Standardfehlerausgabe gehen, und damit sind sie ganz normal mit `docker logs` lesbar. Das gilt für einen Container mit der Standardkonfiguration, im Beispiel unten ein Container namens `web`. Im Projekt 2 haben wir das Zugriffsprotokoll bewusst in eine eigene Datei umgeleitet, dort zeigt `docker logs` deshalb keine Zugriffe.

```
$ docker logs --tail 3 web
172.17.0.1 - - [11/Nov/2026:15:20:01 +0000] "GET / HTTP/1.1" 200 214
    ↪ "-" "Mozilla/5.0 ..." "-"
172.17.0.1 - - [11/Nov/2026:15:20:01 +0000] "GET /stil.css HTTP/1.1"
    ↪ 200 120 "http://docker01:8080/" "Mozilla/5.0 ..." "-"
2026/11/11 15:20:09 [error] 30#30: *4 open() "/usr/share/nginx/html/
    ↪ gibt-es-nicht" failed (2: No such file or directory), client:
    ↪  172.17.0.1, ...
```

Die beiden ersten Zeilen sind Zugriffe. Die Felder lesen sich von links: Adresse des Besuchers, Zeitpunkt, die Anfrage in Anführungszeichen, der Statuscode (200 für Erfolg), die Größe der Antwort in Byte, die Herkunftsseite und der Browser. Die dritte Zeile stammt aus dem Fehlerprotokoll und nennt die Ursache im Klartext: Die Datei gibt es nicht. Die Schwere der Fehlerstufe steht in eckigen Klammern. Die Stufe lässt sich in der Konfiguration verändern, zum Beispiel `error_log /var/log/nginx/error.log warn;`.

Den Statuscode erkennt man schnell wieder. Die Tabelle nennt die häufigsten und ihre übliche Ursache.

Code	Bedeutung	Übliche Ursache
200	in Ordnung	alles gut
301, 302	Weiterleitung	beabsichtigt (zum Beispiel auf HTTPS)
403	verboten	fehlende Leserechte, kein `index` und kein Verzeichnislisting
404	nicht gefunden	Datei fehlt, falscher Pfad, falscher Bind Mount
499	Besucher hat abgebrochen	Browser hat die Verbindung geschlossen, meist harmlos
502	fehlerhaftes Gateway	Reverse Proxy erreicht die Anwendung nicht (Kapitel 16)
504	Zeitüberschreitung	Anwendung antwortet zu langsam

Möchte man die Protokolle als Dateien aufbewahren, etwa für eine Auswertung, legt man sie in ein Volume. Das Beispiel dafür liefert Praxisprojekt 2 mit `zugriffe.log`. Wichtig ist, dass solche Dateien nicht unbegrenzt wachsen. Bei Ausgabe an `docker logs` begrenzt die Rotation aus Kapitel 5 die Größe. Bei eigenen Dateien braucht es eine Rotation, die Kapitel 19 behandelt.

Bei einem Fehler hilft eine feste Reihenfolge. Zuerst: Läuft der Container (`docker ps`)? Dann: Was steht im Protokoll (`docker logs`)? Ist die Konfiguration gültig (`docker exec NAME nginx -t`)? Ist die Datei im Container sichtbar (`docker exec NAME ls -l /usr/share/nginx/html`)? Kommt die Anfrage überhaupt an (Test mit `curl` auf dem Server selbst, dann von einem anderen Rechner)? Wer so vorgeht, grenzt die Ursache schnell auf Container, Konfiguration, Dateien oder Netzwerk ein.

KAPITEL 16

Reverse Proxy mit Docker

Sobald auf einem Server mehr als eine Webanwendung läuft, entsteht ein Problem: Alle wollen Port 443 haben, und nur eine kann ihn bekommen. Ein Reverse Proxy löst das. Er steht als einziger Eingang vor allen Anwendungen, nimmt die Anfragen entgegen und verteilt sie. Dieses Kapitel erklärt das Prinzip, richtet nginx als Reverse Proxy ein, behandelt die Namensauflösung im Docker-Netz und führt in die Grundlagen von TLS ein, der Verschlüsselung für Webverbindungen.

16.1 Grundlagen eines Reverse Proxy

Ein Proxy ist ein Vermittler. Ein *Forward Proxy* steht auf der Seite der Benutzer und fragt stellvertretend für sie im Internet an, wie ein Firmen-Proxy. Ein *Reverse Proxy* steht dagegen auf der Seite der Server. Der Besucher spricht mit ihm und glaubt, er rede mit der Anwendung. Der Reverse Proxy leitet die Anfrage an die richtige Anwendung im Hintergrund weiter und gibt deren Antwort zurück.

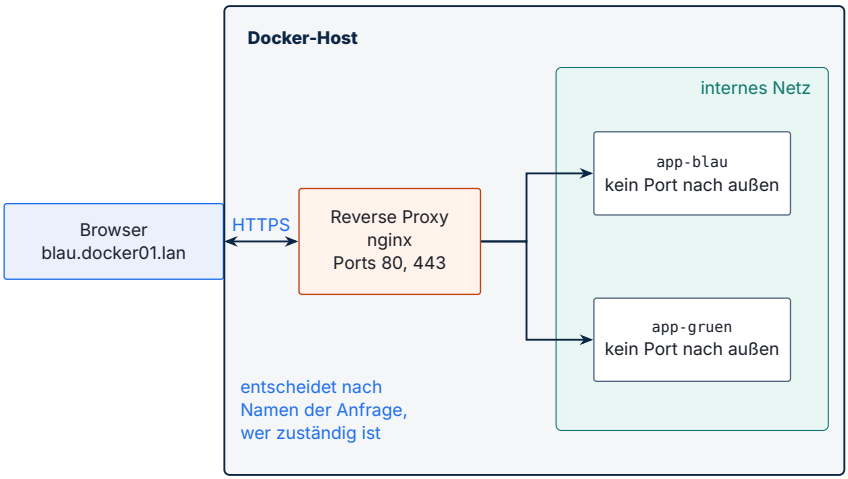


Abbildung 16.1: Ein Reverse Proxy als einziger Eingang vor mehreren Anwendungen

Das Bild zeigt die Anordnung, die dieses Kapitel aufbaut. Der Browser fragt nach `blau.docker01.lan`, der Proxy sieht den Namen, schaut in seine Konfiguration und reicht die Anfrage an den Container `app-blau` weiter. Eine Anfrage nach `gruen.docker01.lan` geht an `app-gruen`. Beide Anwendungen sind im internen Docker-Netz unsichtbar für alle außer dem Proxy. Das bringt mehrere Vorteile.

Ein einziger Eingang: Nur Port 80 und 443 sind offen, egal wie viele Anwendungen dahinterstehen. Die Verschlüsselung (TLS) wird an einer Stelle erledigt und nicht in jeder Anwendung einzeln. Die Anwendungen selbst brauchen keine öffentlichen Ports und sind vor direktem Zugriff geschützt. Neue Anwendungen kommen hinzu, ohne dass sich für Besucher etwas ändert. Und der Proxy kann Zusatzaufgaben übernehmen: Zugriffe protokollieren, Anfragen begrenzen, Antworten zwischenspeichern oder Last verteilen.

16.2 nginx als Reverse Proxy

nginx ist als Webserver bekannt, wird aber sehr oft als Reverse Proxy eingesetzt. Dafür zuständig ist die Anweisung `proxy_pass`, die den Zielort angibt.

Einfacher Reverse Proxy

```
server {
    listen 80;
    server_name blau.docker01.lan;

    location / {
        proxy_pass http://app-blau:80;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
}
```

Alle Anfragen an `blau.docker01.lan` gehen an den Container `app-blau`, Port 80. Die `proxy_set_header`-Zeilen sind wichtig. Ohne sie sähe die Anwendung als Absender immer nur die Adresse des Proxys und als Rechnernamen den Namen des internen Containers. Mit ihnen erfährt sie, wer tatsächlich angefragt hat.

Kopfzeile	Inhalt
`Host`	der Name, den der Besucher eingegeben hat (`$host`)
`X-Real-IP`	die Adresse des Besuchers
`X-Forwarded-For`	Kette aller Adressen, die die Anfrage durchlaufen hat
`X-Forwarded-Proto`	`http` oder `https`, je nachdem, wie der Besucher gekommen ist

Weil dieselben Zeilen in jedem `location`-Block gebraucht werden, legt man sie in eine eigene Datei und bindet sie ein.

snippets/proxy-header.conf

```
proxy_set_header Host $host;
proxy_set_header X-Real-IP $remote_addr;
proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
proxy_set_header X-Forwarded-Proto $scheme;
```

Einbinden

```
location / {
    proxy_pass http://app-blau:80;
    include /etc/nginx/snippets/proxy-header.conf;
}
```

Bei `proxy_pass` gibt es eine Eigenheit, die oft Fehler verursacht. Steht hinter dem Rechnernamen ein Pfad, auch nur ein Schrägstrich, ersetzt nginx den passenden Teil der Adresse. Steht dort nichts, reicht es die Adresse unverändert weiter.

Konfiguration	Anfrage	Ziel
`location /api/ { proxy_pass http://app:8000; }`	`/api/info`	`/api/info`
`location /api/ { proxy_pass http://app:8000/; }`	`/api/info`	`/info`

Im zweiten Fall verschwindet der Teil `/api/`, weil hinter dem Ziel ein Schrägstrich steht. Die Anwendung bekommt die Anfrage dann ohne den Präfix. Der Unterschied ist winzig und folgenreich. Sie sollten ihn in jeder neuen Konfiguration bewusst wählen.

16.3 Mehrere Webanwendungen

Ein Reverse Proxy unterscheidet Anwendungen am Namen. Das geht, weil jeder Browser in der Kopfzeile `Host` mitschickt, welche Adresse der Besucher eingegeben hat. nginx vergleicht sie mit den `server_name`-Angaben und wählt den passenden `server`-Block aus. Man spricht von *virtuellen Hosts*.

Zwei virtuelle Hosts

```
server {
    listen 80;
    server_name blau.docker01.lan;
    location / { proxy_pass http://app-blau:80; include /etc/nginx/
    ↪ snippets/proxy-header.conf; }
}

server {
    listen 80;
    server_name gruen.docker01.lan;
    location / { proxy_pass http://app-gruen:80; include /etc/nginx/
    ↪ snippets/proxy-header.conf; }
}
```

Es gibt auch die Möglichkeit, nach dem *Pfad* zu unterscheiden. Dann liegen alle Anwendungen unter einem Namen, etwa `docker01.lan/blau/` und `docker01.lan/gruen/`. Das ist einfacher einzurichten, bereitet aber Probleme, weil viele Anwendungen davon ausgehen, dass sie auf der Wurzel liegen, und ihre Links entsprechend schreiben. Unterscheiden nach Namen ist in der Regel die robustere Wahl.

Was geschieht, wenn jemand einen Namen eingibt, den die Konfiguration nicht kennt, oder nur die IP-Adresse des Servers? nginx nimmt dann den ersten `server`-Block oder den, den man mit `default_server` kennzeichnet. Es ist ratsam, diesen bewusst festzulegen und ihm keine Anwendung zu geben.

Auffangblock für unbekannte Namen

```
server {
    listen 80 default_server;
    return 444;
}
```

Die Antwort `444` ist eine Besonderheit von nginx. Der Server schließt die Verbindung, ohne etwas zu senden. So erfahren Fremde, die nur die IP-Adresse kennen, nichts über Ihre Anwendungen.

Für HTTPS braucht der Auffangblock eine andere Schreibweise. Ein Server mit `listen 443 ssl` verlangt sonst ein Zertifikat, auch wenn er nichts ausliefern

soll. Die Anweisung `ssl_reject_handshake on;` lehnt den Verbindungsaufbau für unbekannte Namen von vornherein ab.

Auffangblock für HTTPS

```
server {
    listen 443 ssl default_server;
    ssl_reject_handshake on;
}
```

16.3.1 Namen auflösen

Die Namen `blau.docker01.lan` und `gruen.docker01.lan` gibt es nirgends im Internet. Damit der Browser sie findet, muss man sie bekannt machen. Im Heimnetz geschieht das am besten im Router oder auf einem DNS-Server. Für den Test genügt ein Eintrag in der Datei `/etc/hosts` des Rechners, an dem Sie arbeiten. Unter Windows heißt sie `C:\Windows\System32\drivers\etc\hosts`.

/etc/hosts auf dem Arbeitsrechner

```
192.168.100.10   docker01.lan blau.docker01.lan gruen.docker01.lan
```

Auf dem Server selbst kann man sich den Eintrag sparen und `curl` den Namen mitgeben.

```
$ curl -s --resolve blau.docker01.lan:80:127.0.0.1 http://blau.
    ↪ docker01.lan/
$ curl -s -H "Host: gruen.docker01.lan" http://127.0.0.1/
```

Die Option `--resolve` sagt `curl`, welche Adresse es für den Namen verwenden soll. Die zweite Form schickt die Kopfzeile `Host` von Hand mit. Beide sind zum Testen der Namenslogik unverzichtbar.

16.4 Interne Docker-Netzwerke

Der Reverse Proxy ist die Tür, die Anwendungen sind die Räume dahinter. Das Docker-Netz legt fest, wer in welchen Raum darf. Die Anordnung aus Abbildung 16.1 braucht zwei Netze. Das eine, `frontend`, ist ein gewöhnliches Bridge-Netz, über das der Proxy seine Ports nach außen führt. Das andere, `intern`, ist mit `internal: true` vom Internet getrennt. Dort hängen die Anwendungen. Der Proxy ist in beiden Netzen, die Anwendungen nur im inneren.

Ausschnitt compose.yaml

```
services:
  proxy:
    image: nginx:stable-alpine
    ports:
      - "80:80"
      - "443:443"
    networks:
      - frontend
      - intern
  app-blau:
    image: nginx:stable-alpine
    networks:
      - intern

networks:
  frontend:
  intern:
    internal: true
```

Zwei Dinge folgen daraus. Die Anwendungen veröffentlichen keine Ports und sind von außen nicht erreichbar, auch nicht versehentlich. Und sie können nicht selbst ins Internet telefonieren, was sie bei einer Kompromittierung deutlich weniger gefährlich macht. Der Proxy muss in *beiden* Netzen stehen. Ein Container, der nur an einem internen Netz hängt, kann keine Ports nach außen führen.

16.5 Namensauflösung im Proxy

Hier lauert die Falle, auf die Kapitel 13 hingewiesen hat. nginx löst den Rechnernamen in `proxy_pass` beim Start einmal auf und merkt sich die Adresse. Wird der Anwendungs-Container später ersetzt und bekommt eine neue Adresse im Docker-Netz, ruft nginx weiter die alte auf und liefert `502 Bad Gateway`. Fehlt der Container beim Start ganz, weigert sich nginx sogar zu starten (`host not found in upstream`).

Die Lösung nutzt einen Dienst, den wir schon kennen: den DNS-Server von Docker unter `127.0.0.11`. Mit der Anweisung `resolver` sagt man nginx, dass es ihn befragen soll, und mit einer *Variablen* in `proxy_pass` erzwingt man, dass die Auflösung bei jeder Anfrage neu geschieht.

Dynamische Namensauflösung

```
resolver 127.0.0.11 valid=10s;

server {
    listen 80;
    server_name blau.docker01.lan;

    location / {
        set $ziel http://app-blau:80;
        proxy_pass $ziel;
        include /etc/nginx/snippets/proxy-header.conf;
    }
}
```

`valid=10s` legt fest, wie lange nginx eine Antwort zwischenspeichert. Das `set` legt die Zieladresse in eine Variable. Weil `proxy_pass` mit einer Variablen arbeitet, löst nginx den Namen zur Laufzeit auf. Zwei Vorteile ergeben sich. Der Proxy startet auch, wenn die Anwendung gerade fehlt, und liefert dann `502`, bis sie wieder da ist. Und nach einem Austausch des Containers findet er die neue Adresse spätestens nach zehn Sekunden von selbst. Ein Neustart des Proxys wird überflüssig. Bei einer Variablen in `proxy_pass` ohne Pfadteil gibt nginx die Adresse unverändert weiter, wie in der Tabelle oben.

16.6 Grundlagen von TLS

Eine Verbindung über `http://` läuft unverschlüsselt. Jeder, der im Netz mitliest, sieht Passwörter und Inhalte im Klartext. *TLS* (Transport Layer Security) verschlüsselt sie. Webseiten, die mit `https://` beginnen, verwenden es. TLS leistet drei Dinge: Es *verschlüsselt* den Verkehr, es sichert die *Unversehrtheit* (niemand kann die Daten unbemerkt verändern) und es weist die *Echtheit* der Gegenstelle nach (man spricht wirklich mit dem Server, den man meint).

Die Echtheit beruht auf *Zertifikaten*. Ein Zertifikat ist eine Datei, die den Namen des Servers und einen öffentlichen Schlüssel enthält und von einer Zertifizierungsstelle (*Certificate Authority*, CA) digital unterschrieben ist. Der Browser vertraut einer Liste von Zertifizierungsstellen und prüft die Unterschrift. Passt der Name im Zertifikat zur Adresse und ist die Unterschrift gültig, baut er die verschlüsselte Verbindung auf.

Fachbegriff: Zertifikat und Schlüssel

Ein TLS-Server braucht zwei Dateien: das *Zertifikat* (`.crt`), das öffentlich ist und jedem Besucher gezeigt wird, und den *privaten Schlüssel* (`.key`), der geheim bleiben muss. Wer den Schlüssel kennt, kann sich als der Server ausgeben.

Es gibt drei Arten von Zertifikaten, die man unterscheiden sollte.

Art	Eigenschaft	Einsatz
selbstsigniert	vom Betreiber selbst unterschrieben, Browser warnen	Tests, interne Dienste
von einer öffentlichen CA	Browser vertrauen ihm, braucht einen öffentlichen Namen	Dienste im Internet
von einer eigenen CA	Browser vertrauen, wenn die CA dort eingetragen ist	Firmennetze mit vielen internen Diensten

Öffentliche Zertifikate bekommt man kostenlos von der Zertifizierungsstelle *Let's Encrypt*. Sie arbeitet mit dem Protokoll ACME und prüft, ob der Antragsteller

den Namen kontrolliert, zum Beispiel indem sie eine Datei von einer bestimmten Adresse abruft. Das setzt einen *öffentlichen* Namen und einen aus dem Internet erreichbaren Server voraus. Ein Name wie `docker01.lan` kommt dafür nicht in Frage. Für das Übungsnetz verwenden wir deshalb ein selbstsigniertes Zertifikat, das die Technik genauso nutzt. Im Internetbetrieb ersetzt man es später durch ein öffentliches. Die Konfiguration von nginx ändert sich dabei nur im Pfad der Dateien.

16.6.1 Ein selbstsigniertes Zertifikat erzeugen

```
$ mkdir -p certs
$ openssl req -x509 -nodes -newkey rsa:2048 -days 365 \
    -keyout certs/server.key -out certs/server.crt \
    -subj "/CN=docker01.lan" \
    -addext "subjectAltName=DNS:docker01.lan,DNS:blau.docker01.lan,
    ↪ DNS:gruen.docker01.lan"
$ chmod 600 certs/server.key
```

Der Befehl erzeugt in einem Schritt einen Schlüssel (`rsa:2048`) und ein dazu passendes Zertifikat, das 365 Tage gilt. `-nodes` lässt den Schlüssel ohne Passwort, damit nginx ohne Eingabe starten kann. `-subj` nennt den Namen. Entscheidend ist die Erweiterung `subjectAltName`: Moderne Browser prüfen die Liste alternativer Namen und ignorieren das Feld `CN`. Jeder Name, unter dem der Server erreichbar sein soll, muss dort stehen. Der Schlüssel bekommt die Rechte `600`, nur der Besitzer darf ihn lesen. Den Inhalt prüft man so:

```
$ openssl x509 -in certs/server.crt -noout -subject -dates -ext
    ↪ subjectAltName
subject=CN = docker01.lan
notBefore=Nov 11 14:00:00 2026 GMT
notAfter=Nov 11 14:00:00 2027 GMT
X509v3 Subject Alternative Name:
    DNS:docker01.lan, DNS:blau.docker01.lan, DNS:gruen.docker01.lan
```

16.6.2 TLS in nginx einrichten

HTTPS-Server

```
server {
    listen 443 ssl;
    server_name blau.docker01.lan;

    ssl_certificate     /etc/nginx/certs/server.crt;
    ssl_certificate_key /etc/nginx/certs/server.key;
    ssl_protocols       TLSv1.2 TLSv1.3;

    location / {
        set $ziel http://app-blau:80;
        proxy_pass $ziel;
        include /etc/nginx/snippets/proxy-header.conf;
    }
}

server {
    listen 80;
    server_name blau.docker01.lan gruen.docker01.lan;
    return 301 https://$host$request_uri;
}
```

Der erste Block bedient HTTPS. `listen 443 ssl` schaltet die Verschlüsselung ein, die beiden `ssl_certificate`-Zeilen nennen Zertifikat und Schlüssel, und `ssl_protocols` erlaubt nur die Versionen 1.2 und 1.3. Ältere Versionen gelten als unsicher. Der zweite Block lenkt jede unverschlüsselte Anfrage mit der Antwort `301` auf die HTTPS-Adresse um. `$request_uri` behält den Pfad bei. Das Zertifikat prüft man von außen mit `openssl s_client` oder mit `curl`.

```
$ curl -sk --resolve blau.docker01.lan:443:127.0.0.1 -o /dev/null -w
    ↪ '%{http_code}\n' https://blau.docker01.lan/
200
$ curl -s --resolve blau.docker01.lan:443:127.0.0.1 https://blau.
    ↪ docker01.lan/
curl: (60) SSL certificate problem: self-signed certificate
```

Die Option `-k` sagt `curl`, dass es die Prüfung des Zertifikats überspringen soll. Ohne sie lehnt `curl` das selbstsignierte Zertifikat ab, wie es ein Browser mit einer Warnung tut. Bei einer Verbindung, die wirklich geschützt sein soll, verwendet man `-k` nie. Stattdessen gibt man die Zertifikatsdatei mit `--cacert certs/server.crt` an und überprüft damit die Verbindung ernsthaft.

HSTS nicht vorschnell einschalten

Mit der Kopfzeile `Strict-Transport-Security` verpflichtet man Browser, eine Domäne künftig nur noch per HTTPS aufzurufen, und zwar für die angegebene Zeit. Das schützt, kann aber aussperren, wenn das Zertifikat abläuft oder falsch ist. Schalten Sie sie erst ein, wenn die Zertifikatsverwaltung zuverlässig läuft, und beginnen Sie mit kurzer Dauer.

16.7 Sicherheitsaspekte

Ein Reverse Proxy ist die einzige Stelle, die aus dem Netz erreichbar ist. Seine Konfiguration trägt deshalb große Verantwortung. Einige Einstellungen haben sich bewährt.

Gehen Sie sparsam mit Informationen um: `server_tokens off;` verbirgt die nginx-Version in Fehlerseiten und Kopfzeilen. Erlauben Sie nur die nötigen Methoden und begrenzen Sie die Größe von Anfragen mit `client_max_body_size`, damit niemand Ihren Server mit Riesen-Uploads füllt. Mit `limit_req` lässt sich die Zahl der Anfragen je Besucher begrenzen und so ein einfacher Schutz gegen Überlastung einrichten.

Begrenzung von Anfragen und Größen

```
limit_req_zone $binary_remote_addr zone=pro_besucher:10m rate=10r/s;

server {
    listen 80;
    server_name blau.docker01.lan;
    server_tokens off;
    client_max_body_size 5m;
```

```
    location / {
        limit_req zone=pro_besucher burst=20 nodelay;
        proxy_pass http://app-blau:80;
        include /etc/nginx/snippets/proxy-header.conf;
    }
}
```

Die Zone `pro_besucher` merkt sich die Adressen der Besucher in 10 Megabyte Speicher. `rate=10r/s` erlaubt zehn Anfragen pro Sekunde, `burst=20` duldet kurze Spitzen darüber. Setzen Sie die Werte passend zu Ihrer Anwendung. Zu knappe Grenzen sperren echte Besucher aus.

Sicherheitskopfzeilen weisen den Browser an, vorsichtiger zu sein. Zwei einfache sind `add_header X-Content-Type-Options "nosniff" always;` und `add_header X-Frame-Options "SAMEORIGIN" always;`. Letztere verhindert, dass fremde Seiten Ihre Seite in einem Rahmen einbetten. Das Wort `always` sorgt dafür, dass die Kopfzeile auch bei Fehlerantworten gesendet wird.

Zum Schluss gehören drei Grundsätze zur Haltung eines Reverse Proxys. Der Proxy zeigt nur das, was zeigen soll: Verwaltungsoberflächen wie Portainer oder Datenbankwerkzeuge gehören nicht ungeschützt hinter denselben Eingang wie die Webseite. Zugangsdaten gehören nicht in die Konfiguration, die im Repository landet. Und der Proxy muss aktuell bleiben, denn er ist die Angriffsfläche Nummer eins. Die Update-Routine aus Kapitel 13 gilt für ihn besonders.

KAPITEL 17

Portainer

Alles, was Sie bisher getan haben, geschah auf der Kommandozeile. Das ist der Weg, den Administratoren im Alltag gehen, und er lässt sich in Skripten festhalten. Für Überblick und gelegentliche Handgriffe schätzen viele daneben eine grafische Oberfläche. Portainer ist die bekannteste für Docker. Dieses Kapitel zeigt Installation und Bedienung und widmet dem Absichern besonders viel Raum, denn Portainer bekommt Rechte, die man nicht verschenken sollte.

17.1 Einführung

Portainer ist eine Weboberfläche, mit der man Container, Images, Volumes und Netzwerke verwaltet. Es läuft selbst als Container und spricht über den Docker-Socket mit dem Daemon, genau wie das Programm `docker` (Kapitel 3). Es gibt zwei Ausgaben. Die *Community Edition* (CE) ist frei nutzbar und genügt für alles in diesem Buch. Die *Business Edition* ist kostenpflichtig und bietet unter anderem eine feinere Rechteverwaltung. Wir verwenden die Community Edition.

Der Aufbau besteht aus dem *Portainer Server*, der die Oberfläche bereitstellt, und optional einem *Agent*, der auf entfernten Docker-Hosts läuft und diese in die Oberfläche einbindet. Für einen einzelnen Server brauchen Sie nur den Server, der den lokalen Docker-Daemon verwaltet.

Wofür taugt Portainer, und wofür nicht? Es zeigt auf einen Blick, was auf dem Server läuft, wie viel Ressourcen die Container verbrauchen und welche Images und Volumes brachliegen. Es erlaubt, Container zu starten, zu stoppen und deren

Protokolle zu lesen, ohne eine Shell zu öffnen. Es ist weniger geeignet als Ersatz für Compose-Dateien in einer Versionsverwaltung. Was man in der Oberfläche anklickt, ist nirgends dokumentiert und lässt sich schwer wiederholen. Als Faustregel gilt: Beschreiben Sie Anwendungen in Dateien und nutzen Sie Portainer zum Ansehen und für kleine Eingriffe.

Portainer bekommt Root-Rechte über den Server

Damit Portainer Container verwalten kann, bekommt es den Docker-Socket eingebunden. Wer den Socket steuern kann, hat wie in Kapitel 3 beschrieben praktisch Root-Rechte auf dem Host. Das gilt für Portainer genauso: Wer sich an der Oberfläche als Administrator anmelden kann, kann damit den ganzen Server übernehmen. Die Oberfläche ist also ein besonders wertvolles Angriffsziel, und der Abschnitt über das Absichern am Ende dieses Kapitels ist keine Zugabe, sondern Teil der Installation.

17.2 Installation

Portainer läuft als Container, und wir beschreiben es wie jede andere Anwendung in einer `compose.yaml`. Der Tag `lts` bezeichnet die Fassung mit Langzeitunterstützung, die Portainer für Server empfiehlt. Er bewegt sich innerhalb dieses Zweigs mit, ohne dass man auf eine neue Hauptfassung springt.

Portainer muss zu Docker 29 passen

Mit Docker Engine 29 hat Docker die kleinste unterstützte Version seiner Programmierschnittstelle angehoben. Ältere Portainer-Fassungen kommen damit nicht zurecht, und die Umgebung lässt sich nicht laden. Die Korrektur steckt in Portainer 2.33.5 (LTS) und 2.36.0 (kurzlebigerer Zweig) sowie in allen neueren Fassungen. Der Tag `lts` liefert eine solche Fassung. Setzen Sie auf einem Docker-29-Server keinen älteren, fest eingetragenen Tag ein.

```
$ mkdir -p /srv/docker/portainer && cd /srv/docker/portainer
$ nano compose.yaml
```

compose.yaml

```
name: portainer

services:
  portainer:
    image: portainer/portainer-ce:lts
    restart: unless-stopped
    ports:
      - "127.0.0.1:9443:9443"
    volumes:
      - /var/run/docker.sock:/var/run/docker.sock
      - portainer-daten:/data
    security_opt:
      - no-new-privileges:true

volumes:
  portainer-daten:
```

Jede Zeile hat einen Grund. Der Port 9443 ist der verschlüsselte Zugang zur Oberfläche. Er ist ausdrücklich an `127.0.0.1` gebunden, also nur vom Server selbst erreichbar. Warum, erklärt der Abschnitt über das Absichern. Der erste Eintrag unter `volumes` ist der Docker-Socket, ohne den Portainer nichts verwalten könnte. Der zweite ist das Volume `portainer-daten`, das Konten und Einstellungen aufbewahrt. `no-new-privileges` verbietet Prozessen im Container, zusätzliche Rechte zu erlangen.

Port	Verwendung
9443	Weboberfläche über HTTPS (hier genutzt)
9000	Weboberfläche über unverschlüsseltes HTTP, wird nicht veröffentlicht
8000	Tunnel für Edge-Agents auf entfernten Standorten, für einen einzelnen Server nicht nötig

Beim Start in Docker 29 sollte der Server hochfahren, ohne dass etwas zu ändern ist.

```
$ docker compose up -d
[+] Running 3/3
 ✔ Network portainer_default           Created
 ✔ Volume portainer_portainer-daten    Created
 ✔ Container portainer-portainer-1     Started
$ docker compose ps --format "table {{.Service}}\t{{.Status}}\t{{.
    ↪ Ports}}"
SERVICE     STATUS         PORTS
portainer   Up 5 seconds   127.0.0.1:9443->9443/tcp
```

17.2.1 Zugriff von Ihrem Arbeitsplatz

Weil der Port nur an der Loopback-Adresse des Servers lauscht, erreichen Sie ihn von Ihrem Rechner aus nicht direkt. Das ist gewollt. Ein *SSH-Tunnel* schafft die Verbindung. Er leitet einen Port Ihres Arbeitsplatzes über die verschlüsselte SSH-Verbindung an den Port des Servers weiter. Auf Ihrem Arbeitsrechner rufen Sie auf:

```
$ ssh -L 9443:127.0.0.1:9443 admin@docker01
```

Solange diese SSH-Sitzung offen ist, erreichen Sie die Oberfläche in Ihrem Browser unter der Adresse `https://localhost:9443`. Der Teil `-L 9443:127.0.0.1:9443` bedeutet: Port 9443 hier führt zu Adresse 127.0.0.1, Port 9443 auf dem Server. Der Browser zeigt eine Warnung wegen des Zertifikats. Portainer hat beim ersten Start ein selbstsigniertes Zertifikat erzeugt, wie es in Kapitel 16 beschrieben ist. Bei einer Verbindung durch den Tunnel ist die Warnung hier vertretbar, denn Sie sprechen nachweislich mit Ihrem eigenen Server.

17.2.2 Das Administratorkonto anlegen

Beim ersten Öffnen verlangt Portainer, ein Administratorkonto anzulegen. Wählen Sie einen Benutzernamen, der nicht `admin` lautet, und ein langes Passwort, mindestens zwölf Zeichen. Danach wählen Sie die Umgebung aus, die Portainer verwalten soll: die lokale Docker-Umgebung über den Socket.

Nicht zu lange warten

Aus Sicherheitsgründen sperrt sich Portainer nach dem ersten Start, wenn nicht innerhalb weniger Minuten ein Administratorkonto angelegt wurde. Es erscheint die Meldung, die Instanz sei aus Sicherheitsgründen abgelaufen. Die Abhilfe ist ein Neustart mit `docker compose restart portainer`. Der Zweck der Sperre ist, dass kein Fremder das Konto anlegt, bevor Sie es tun.

17.3 Die Weboberfläche

Nach der Anmeldung zeigt die Startseite die verfügbaren Umgebungen. Bei einem einzelnen Server ist das `local`. Ein Klick darauf öffnet die Verwaltung dieser Umgebung. Die Menüpunkte links gliedern die Aufgaben. Je nach Version heißen sie leicht anders.

Menüpunkt	Inhalt
Dashboard	Überblick: Anzahl und Zustand der Container, Images, Volumes und Netzwerke, Ressourcen des Hosts
Stacks	Compose-Projekte, die Portainer selbst angelegt hat
Containers	alle Container mit Zustand und Aktionen
Images	lokale Images, Download und Bau
Networks	Docker-Netzwerke
Volumes	Volumes und ihre Nutzung
Events	Ereignisprotokoll des Docker-Daemons
Users, Settings	Verwaltung der Benutzer und Einstellungen (nur für Administratoren)

Das Dashboard liefert in Sekunden, wofür auf der Kommandozeile mehrere Befehle nötig wären: Wie viele Container laufen, wie viele sind gestoppt, welche melden *unhealthy*, und wie viel Platz belegen Images und Volumes. Es ist die beste Stelle, um nach Neustarts und Updates zu prüfen, ob alles wieder läuft.

17.4 Container verwalten

Unter *Containers* erscheint die Liste der Container, mit Namen, Image, Zustand, Erstellungszeitpunkt und veröffentlichten Ports. Die Zustände aus Kapitel 2 sehen Sie hier wieder: laufend, gestoppt, angehalten. Ist ein Healthcheck definiert, steht auch dessen Ergebnis da. Über Schaltflächen und die Mehrfachauswahl führt man dieselben Aktionen aus, die Sie von `docker start`, `docker stop`, `docker restart`, `docker kill`, `docker pause` und `docker rm` kennen.

Ein Klick auf den Namen öffnet die Einzelansicht. Dort liegen die Werkzeuge, die man bei der Fehlersuche braucht. Die Details zeigen Konfiguration, Umgebungsvariablen, eingebundene Volumes und die Netzwerke des Containers, also das, was `docker inspect` liefert. Die Protokolle entsprechen `docker logs`, einschließlich Zeitstempel und Suche. Die Statistik zeigt Prozessor, Arbeitsspeicher und Netzwerk wie `docker stats`. Die *Konsole* öffnet eine Shell im Container wie `docker exec -it`. Mit der Funktion zum *Neuerzeugen* ersetzt Portainer den Container, optional mit frisch geladenem Image.

Portainer	Kommandozeile
Liste der Container	`docker ps -a`
Logs	`docker logs`
Inspect	`docker inspect`
Stats	`docker stats`
Console	`docker exec -it NAME sh`
Recreate	`docker rm -f` und erneuter `docker run`

Auch neue Container lassen sich über ein Formular anlegen. Man trägt Namen, Image, Portfreigaben, Volumes, Netzwerk, Umgebungsvariablen und Neustartregel ein. Das entspricht den Optionen von `docker run`. Für einen schnellen Test ist das praktisch. Für alles, was bleiben soll, ist eine Compose-Datei besser, weil sie sich festhalten und wiederholen lässt.

> **Nicht an Containern fremder Projekte drehen**
>
> Löschen oder Ersetzen eines Containers aus einem Compose-Projekt über Portainer funktioniert, kann aber das Projekt durcheinanderbringen. Com-

pose geht davon aus, dass die Container so aussehen, wie die Datei sie beschreibt. Wer sie an Compose vorbei ändert, riskiert Überraschungen beim nächsten `docker compose up`. Nutzen Sie Portainer bei Compose-Projekten zum Ansehen und für Neustarts, nicht zum Umbauen.

17.5 Images verwalten

Unter *Images* stehen alle lokalen Images mit Tag, Größe und Alter. Ein Hinweis zeigt an, welche Images von keinem Container verwendet werden. Dort lässt sich ein Image aus einer Registry laden (entspricht `docker pull`), eines aus einem Dockerfile bauen, ein Image als Archiv exportieren und wieder importieren und ein ungenutztes entfernen (`docker rmi`). Vorhandene Images lassen sich in Registries hochladen, die man unter *Registries* einträgt, darunter Docker Hub und eigene Registries wie die aus Kapitel 7.

Zum Aufräumen hilft die Funktion, alle ungenutzten Images zu entfernen. Sie entspricht `docker image prune -a`. Die Warnung aus Kapitel 9 gilt auch hier: Ein Image, das kein Container verwendet, kann trotzdem ein wichtiger Rückfallpunkt für ein Update sein. Prüfen Sie die Liste, bevor Sie sie leeren.

17.6 Volumes verwalten

Die Liste unter *Volumes* zeigt alle Volumes, ihren Treiber, den Zeitpunkt der Erstellung und ob ein Container sie verwendet. Sie entspricht `docker volume ls`, ist aber übersichtlicher, weil man auf einen Blick sieht, welche Volumes ungenutzt herumliegen. Hier findet man schnell die anonymen Volumes, vor denen Kapitel 10 gewarnt hat: Sie tragen als Namen eine lange Zeichenkette.

Neue Volumes legt man mit Namen an, und ein Klick auf ein vorhandenes zeigt die Container, die es nutzen. Beim Löschen verweigert Docker, wenn ein Container das Volume noch verwendet, und Portainer gibt diese Meldung weiter.

Löschen ist endgültig

> Ein Klick auf *Remove* löscht das Volume samt aller Daten, ohne Papierkorb. Das gilt auch für das Volume der Datenbank. Sichern Sie wichtige Daten vorher, wie in Kapitel 14 beschrieben, und prüfen Sie den Namen des Volumes zweimal.

17.7 Netzwerke verwalten

Unter *Networks* stehen die Netzwerke mit Treiber, Adressbereich und Gateway. Die drei Standardnetze `bridge`, `host` und `none` lassen sich nicht löschen. Zu jedem Netzwerk zeigt die Einzelansicht die angeschlossenen Container mit ihren Adressen. Das ist besonders nützlich bei der Fehlersuche in Multi-Container-Anwendungen: Man sieht sofort, ob zwei Container, die miteinander reden sollen, im selben Netz hängen. Neue Netzwerke legt man mit Treiber, Adressbereich und Gateway an, entsprechend `docker network create`. Container lassen sich in der Einzelansicht eines Containers mit weiteren Netzen verbinden oder davon trennen.

17.8 Stacks

Neben einzelnen Containern kennt Portainer *Stacks*, also Compose-Projekte. Man fügt eine Compose-Datei in einen Editor ein, und Portainer startet sie. Das ist ein bequemer Weg, kleine Anwendungen auszuprobieren. Beachten Sie die Grenze: Portainer verwaltet nur die Stacks, die es selbst angelegt hat. Compose-Projekte, die Sie auf der Kommandozeile gestartet haben, erscheinen zwar mit ihren Containern, lassen sich aber in der Stack-Verwaltung nicht bearbeiten. Entscheiden Sie sich pro Projekt für einen Weg und mischen Sie nicht beide für dieselbe Anwendung.

17.9 Die Benutzeroberfläche absichern

Portainer ist ein Werkzeug mit der Macht eines Root-Zugangs. Folgende Maßnahmen schützen es.

Erreichbarkeit einschränken

Die wichtigste Maßnahme steckt schon in der Compose-Datei: Der Port ist an `127.0.0.1` gebunden. Aus dem Netz ist die Oberfläche nicht erreichbar, und niemand kann Passwörter durchprobieren. Der Zugang führt über einen SSH-Tunnel, der wiederum mit Schlüsseln und eigener Anmeldung geschützt ist. Wer Portainer im Heimnetz oder über das Internet erreichbar machen will, stellt es hinter einen Reverse Proxy mit TLS, wie ihn Praxisprojekt 5 aufbaut, und beschränkt zusätzlich den Zugang auf bestimmte Adressen.

Starke Zugangsdaten

Verwenden Sie einen eigenen Benutzernamen statt `admin`, ein langes, zufälliges Passwort und ein Konto je Person. Teilen Sie keine Konten. Portainer protokolliert Anmeldungen, und das nützt nur, wenn jede Person ein eigenes Konto hat.

Geringste Rechte

Legen Sie für Personen, die nur etwas ansehen müssen, normale Benutzerkonten statt Administratorkonten an. Beachten Sie, dass die feine Rechtevergabe nach Rollen der Business Edition vorbehalten ist. In der Community Edition ist sie eingeschränkt, und ein Benutzer mit Zugang zur Umgebung hat dort viel Spielraum. Vergeben Sie solche Zugänge sparsam.

Aktuell halten

Portainer ist als Teil der Angriffsfläche so wichtig wie der Reverse Proxy. Aktualisieren Sie regelmäßig auf die jeweils neueste Fassung des gewählten Zweigs. Das Volume mit den Daten bleibt dabei erhalten.

```
$ cd /srv/docker/portainer
$ docker compose pull
$ docker compose up -d
```

Sitzungsdauer

In den Einstellungen unter der Authentifizierung lässt sich die Dauer einer Anmeldung begrenzen. Kürzere Sitzungen verringern das Risiko, wenn ein Rechner unbeaufsichtigt bleibt.

Datenbestand sichern

Das Volume `portainer-daten` enthält Konten und Einstellungen. Sichern Sie es wie in Kapitel 10 beschrieben, am besten nach jeder größeren Änderung.

> **Ein schreibgeschützter Socket schützt nicht**
>
> Man liest gelegentlich den Rat, den Docker-Socket mit `:ro` einzubinden, um Portainer nur Lesezugriff zu geben. Das schützt nicht. Der Zusatz `:ro` betrifft nur die Socket-*Datei*, also ob man sie löschen oder ersetzen kann. Über den Socket laufen dagegen weiterhin alle Befehle der Docker-Schnittstelle, auch schreibende. Wer die Rechte tatsächlich beschränken will, braucht einen Vermittler, der die Schnittstelle filtert (einen sogenannten Socket-Proxy), oder muss auf die Verwaltung per Oberfläche verzichten. Kapitel 18 behandelt das Thema.

Praxisprojekt 5: Reverse Proxy

In diesem Projekt stellen Sie einen nginx als Reverse Proxy vor zwei Webanwendungen. Der Proxy ist der einzige Eingang, verteilt die Anfragen nach Namen, verschlüsselt mit einem selbstsignierten Zertifikat und leitet unverschlüsselte Zugriffe um. Die Anwendungen selbst sind von außen unsichtbar. Das Projekt verbindet die Kapitel 11, 12 und 15 bis 16 und bereitet den Einsatz von TLS vor. Planen Sie etwa zwei Stunden ein.

Ausgangslage und Ziel

Ein Verein betreibt zwei kleine Webanwendungen auf einem Server. Beide sollen unter eigenen Namen über HTTPS erreichbar sein, aber nur ein Eingang soll offen stehen.

Anforderung	Sollzustand
Dienste	`proxy` (nginx), `app-blau` und `app-gruen` (jeweils nginx mit eigener Seite)
Namen	`blau.docker01.lan` und `gruen.docker01.lan`
Eingang	Ports 80 und 443, nur am Proxy
Netzwerke	`frontend` (Proxy), `intern` (Proxy und Anwendungen, ohne Zugang nach außen)
Verschlüsselung	selbstsigniertes Zertifikat mit beiden Namen, nur TLS 1.2 und 1.3
Verhalten	HTTP leitet auf HTTPS um, unbekannte Namen und die reine IP-Adresse werden abgewiesen

Die Ports 80 und 443 des Servers müssen frei sein. Stoppen Sie dazu Container aus früheren Kapiteln, die sie belegen, und prüfen Sie mit `sudo ss -tlnp | grep -E ":80 |:443 "`.

Schritt 1: Verzeichnisse und Anwendungen

```
$ sudo apt install openssl
$ mkdir -p /srv/docker/projekt-5/apps/blau /srv/docker/projekt-5/apps
    ↪ /gruen \
          /srv/docker/projekt-5/nginx/conf.d /srv/docker/projekt-5/
    ↪ nginx/snippets
$ cd /srv/docker/projekt-5
```

Die erste Zeile stellt sicher, dass `openssl` vorhanden ist. Die beiden Anwendungen sind einfache Seiten.

apps/blau/index.html

```
<!DOCTYPE html>
<html lang="de">
<head><meta charset="utf-8"><title>Anwendung Blau</title></head>
<body style="background:#dbeafe;font-family:sans-serif"><h1>Anwendung
    ↪ Blau</h1></body>
</html>
```

apps/gruen/index.html

```
<!DOCTYPE html>
<html lang="de">
<head><meta charset="utf-8"><title>Anwendung Grün</title></head>
<body style="background:#dcfce7;font-family:sans-serif"><h1>Anwendung
    ↪ Grün</h1></body>
</html>
```

Schritt 2: Zertifikat erzeugen

Das Skript erzeugt Schlüssel und Zertifikat, wie in Kapitel 16 erklärt.

/srv/docker/projekt-5/zertifikat.sh

```
#!/bin/bash
# Erzeugt ein selbstsigniertes Zertifikat für das Übungsnetz
set -e
cd /srv/docker/projekt-5
mkdir -p certs
openssl req -x509 -nodes -newkey rsa:2048 -days 365 \
  -keyout certs/server.key -out certs/server.crt \
  -subj "/CN=docker01.lan" \
  -addext "subjectAltName=DNS:docker01.lan,DNS:blau.docker01.lan,DNS:
    ↪ gruen.docker01.lan" 2>/dev/null
chmod 600 certs/server.key
echo "Zertifikat erstellt: certs/server.crt"
```

```
$ chmod +x zertifikat.sh && ./zertifikat.sh
Zertifikat erstellt: certs/server.crt
$ openssl x509 -in certs/server.crt -noout -subject -dates -ext
    ↪ subjectAltName
subject=CN = docker01.lan
notBefore=Nov 11 14:00:00 2026 GMT
notAfter=Nov 11 14:00:00 2027 GMT
X509v3 Subject Alternative Name:
    DNS:docker01.lan, DNS:blau.docker01.lan, DNS:gruen.docker01.lan
```

Schritt 3: Konfiguration des Proxys

Die Kopfzeilen, die der Proxy an die Anwendungen weitergibt, stehen in einer eigenen Datei.

nginx/snippets/proxy-header.conf

```
proxy_set_header Host $host;
proxy_set_header X-Real-IP $remote_addr;
proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
proxy_set_header X-Forwarded-Proto $scheme;
```

Die Hauptkonfiguration enthält die Auffangblöcke, die Umleitung und je einen Block pro Anwendung.

nginx/conf.d/default.conf

```
# Docker-DNS: Namen der Anwendungen bei jeder Anfrage neu auflösen
resolver 127.0.0.11 valid=10s;

# Auffangblöcke: unbekannte Namen und reine IP-Adressen
server {
    listen 80 default_server;
    return 444;
}

server {
```

```
    listen 443 ssl default_server;
    ssl_reject_handshake on;
}

# HTTP leitet auf HTTPS um
server {
    listen 80;
    server_name blau.docker01.lan gruen.docker01.lan;
    return 301 https://$host$request_uri;
}

# Anwendung Blau
server {
    listen 443 ssl;
    server_name blau.docker01.lan;

    ssl_certificate     /etc/nginx/certs/server.crt;
    ssl_certificate_key /etc/nginx/certs/server.key;
    ssl_protocols       TLSv1.2 TLSv1.3;

    server_tokens off;
    client_max_body_size 5m;
    add_header X-Content-Type-Options "nosniff" always;

    location / {
        set $ziel http://app-blau:80;
        proxy_pass $ziel;
        include /etc/nginx/snippets/proxy-header.conf;
    }
}

# Anwendung Grün
server {
    listen 443 ssl;
    server_name gruen.docker01.lan;

    ssl_certificate     /etc/nginx/certs/server.crt;
    ssl_certificate_key /etc/nginx/certs/server.key;
```

```
    ssl_protocols       TLSv1.2 TLSv1.3;

    server_tokens off;
    client_max_body_size 5m;
    add_header X-Content-Type-Options "nosniff" always;

    location / {
        set $ziel http://app-gruen:80;
        proxy_pass $ziel;
        include /etc/nginx/snippets/proxy-header.conf;
    }
}
```

Die Datei fasst Kapitel 16 zusammen. Die Zeile `resolver` verbindet nginx mit dem Namensdienst von Docker. Die beiden ersten Blöcke weisen unbekannte Namen ab. Der dritte lenkt HTTP um. Die beiden letzten bedienen die Anwendungen, mit `set $ziel` und `proxy_pass $ziel` für die dynamische Auflösung. Zusätzlich verbergen sie die nginx-Version, begrenzen die Größe von Anfragen auf fünf Megabyte und setzen eine Sicherheitskopfzeile.

Schritt 4: Compose-Datei und Start

compose.yaml

```
name: projekt5

services:
  proxy:
    image: nginx:stable-alpine
    restart: unless-stopped
    ports:
      - "80:80"
      - "443:443"
    volumes:
      - ./nginx/conf.d:/etc/nginx/conf.d:ro
      - ./nginx/snippets:/etc/nginx/snippets:ro
```

```
      - ./certs:/etc/nginx/certs:ro
    networks:
      - frontend
      - intern

  app-blau:
    image: nginx:stable-alpine
    restart: unless-stopped
    volumes:
      - ./apps/blau:/usr/share/nginx/html:ro
    networks:
      - intern

  app-gruen:
    image: nginx:stable-alpine
    restart: unless-stopped
    volumes:
      - ./apps/gruen:/usr/share/nginx/html:ro
    networks:
      - intern

networks:
  frontend:
  intern:
    internal: true
```

Hier sind die Bind Mounts auf *Verzeichnisse* gelegt, nicht auf einzelne Dateien. Das vermeidet das Problem mit ersetzten Dateien aus Kapitel 15. Vor dem ersten Start prüft man die Konfiguration von nginx, ohne dass der Server läuft.

```
$ docker compose config --quiet && echo "Datei ist gültig"
Datei ist gültig
$ docker compose up -d
$ docker compose exec proxy nginx -t
nginx: the configuration file /etc/nginx/nginx.conf syntax is ok
nginx: configuration file /etc/nginx/nginx.conf test is successful
```

```
$ docker compose ps --format "table {{.Service}}\t{{.Status}}\t{{.
    ↪ Ports}}"
SERVICE     STATUS         PORTS
app-blau    Up 6 seconds   80/tcp
app-gruen   Up 6 seconds   80/tcp
proxy       Up 6 seconds   0.0.0.0:80->80/tcp, [::]:80->80/tcp,
    ↪ 0.0.0.0:443->443/tcp, [::]:443->443/tcp
```

Nur der Proxy veröffentlicht Ports. Bei den Anwendungen steht `80/tcp` ohne Pfeil.

Schritt 5: Von außen ausprobieren

Auf dem Server selbst testen Sie mit `curl` und dem Schalter `--resolve`, ohne etwas einzutragen. Die Option `-k` überspringt die Prüfung des selbstsignierten Zertifikats.

```
$ curl -sk --resolve blau.docker01.lan:443:127.0.0.1 https://blau.
    ↪ docker01.lan/ | grep h1
<body style="background:#dbeafe;font-family:sans-serif"><h1>Anwendung
    ↪  Blau</h1></body>
$ curl -s -o /dev/null -w '%{http_code} %{redirect_url}\n' \
   --resolve gruen.docker01.lan:80:127.0.0.1 http://gruen.docker01.
    ↪ lan/
301 https://gruen.docker01.lan/
$ curl -sk -o /dev/null https://127.0.0.1/ ; echo "Rückgabewert: $?"
Rückgabewert: 35
```

Der erste Aufruf liefert die blaue Anwendung, der zweite zeigt die Umleitung, der dritte die Abweisung einer Anfrage nur mit der IP-Adresse: `curl` meldet den Rückgabewert 35, weil der Server den Verbindungsaufbau ablehnt. Für den Browser Ihres Arbeitsplatzes tragen Sie die Namen in dessen `hosts`-Datei ein, wie in Kapitel 16 beschrieben:

/etc/hosts auf dem Arbeitsrechner

```
192.168.100.10   docker01.lan blau.docker01.lan gruen.docker01.lan
```

Danach öffnen Sie `https://blau.docker01.lan` und `https://gruen.docker01.lan`. Der Browser warnt vor dem selbstsignierten Zertifikat. Das ist bei diesem Aufbau zu erwarten. Wer das Zertifikat `certs/server.crt` im Betriebssystem als vertrauenswürdig einträgt, wird die Warnung los. Die Adresse `http://blau.docker01.lan` führt auf die HTTPS-Adresse.

Schritt 6: Isolation prüfen

Die Anwendungen dürfen weder von außen noch ins Internet erreichbar sein.

```
$ docker compose port app-blau 80
$ docker compose exec app-blau wget -q -T 3 -O /dev/null http://
    ↪ example.com ; echo "Rückgabewert: $?"
Rückgabewert: 1
$ docker compose exec proxy wget -q -T 3 -O /dev/null http://app-blau
    ↪ / ; echo "Rückgabewert: $?"
Rückgabewert: 0
```

Die erste Abfrage liefert nichts, denn der Port ist nicht veröffentlicht. Die zweite zeigt, dass die Anwendung nicht ins Internet kommt. Die dritte zeigt, dass der Proxy sie im internen Netz erreicht. Die Rückgabewerte hängen vom Programm ab, entscheidend ist der Unterschied zwischen null und nicht null.

Schritt 7: Funktionsprüfung und Abnahme

/srv/docker/projekt-5/check.sh

```
#!/bin/bash
# Funktionsprüfung für Projekt 5: Reverse Proxy
cd /srv/docker/projekt-5 || exit 1
FEHLER=0
```

```
ok()   { echo "[ OK ]     $1"; }
fail() { echo "[FEHLER]   $1"; FEHLER=1; }
H=${HTTP_PORT:-80}; S=${HTTPS_PORT:-443}
holen() { curl -sk --resolve "$1:$S:127.0.0.1" "https://$1:$S/"; }

[ "$(docker compose ps --status running --services | sort | tr '\n' '
    ↪  ')" = "app-blau app-gruen proxy " ] \
  && ok "Alle drei Dienste laufen" || fail "Nicht alle Dienste laufen
    ↪ "
holen blau.docker01.lan | grep -q "Anwendung Blau" \
  && ok "blau.docker01.lan liefert Anwendung Blau" || fail "Blau
    ↪ fehlt"
holen gruen.docker01.lan | grep -q "Anwendung Gr" \
  && ok "gruen.docker01.lan liefert Anwendung Grün" || fail "Grün
    ↪ fehlt"
[ "$(curl -s -o /dev/null -w '%{http_code}' --resolve blau.docker01.
    ↪ lan:$H:127.0.0.1 http://blau.docker01.lan:$H/)" = "301" ] \
  && ok "HTTP wird auf HTTPS umgeleitet" || fail "Umleitung fehlt"
curl -s -o /dev/null -H 'Host: unbekannt.lan' http://127.0.0.1:$H/ \
  && fail "Unbekannter Name wird bedient (HTTP)" || ok "Unbekannter
    ↪ Name wird abgewiesen (HTTP)"
curl -sk -o /dev/null https://127.0.0.1:$S/ \
  && fail "Reine IP-Adresse wird bedient (HTTPS)" || ok "Reine IP-
    ↪ Adresse wird abgewiesen (HTTPS)"
[ "$(curl -s --cacert certs/server.crt --resolve blau.docker01.lan:$S
    ↪ :127.0.0.1 -o /dev/null -w '%{http_code}' https://blau.
    ↪ docker01.lan:$S/)" = "200" ] \
  && ok "Zertifikat passt zum Namen und ist vertrauenswürdig" || fail
    ↪  "Zertifikat wird nicht akzeptiert"
curl -sk --tls-max 1.1 --resolve blau.docker01.lan:$S:127.0.0.1 -o /
    ↪ dev/null https://blau.docker01.lan:$S/ \
  && fail "TLS 1.1 wird akzeptiert" || ok "TLS 1.1 wird abgelehnt"
[ -z "$(docker compose port app-blau 80 2>/dev/null)" ] \
  && ok "Anwendungen veröffentlichen keine Ports" || fail "Eine
    ↪ Anwendung veröffentlicht einen Port"
docker compose exec -T app-blau wget -q -T 3 -O /dev/null http://
    ↪ example.com >/dev/null 2>&1 \
```

```
  && fail "Anwendung erreicht das Internet" || ok "Anwendung erreicht
     ↪ das Internet nicht"
docker compose exec -T proxy nginx -t >/dev/null 2>&1 \
  && ok "Konfiguration des Proxys ist gültig" || fail "Konfiguration
     ↪ des Proxys ist fehlerhaft"

docker compose up -d --force-recreate app-blau >/dev/null 2>&1
sleep 12
holen blau.docker01.lan | grep -q "Anwendung Blau" \
  && ok "Proxy findet app-blau nach dem Austausch ohne Neustart" ||
     ↪ fail "Proxy findet app-blau nicht mehr"

echo
[ "$FEHLER" -eq 0 ] && echo "Alle Prüfungen bestanden." \
  || echo "Mindestens eine Prüfung ist fehlgeschlagen."
exit $FEHLER
```

```
$ chmod +x check.sh && ./check.sh
[ OK ]     Alle drei Dienste laufen
[ OK ]     blau.docker01.lan liefert Anwendung Blau
[ OK ]     gruen.docker01.lan liefert Anwendung Grün
[ OK ]     HTTP wird auf HTTPS umgeleitet
[ OK ]     Unbekannter Name wird abgewiesen (HTTP)
[ OK ]     Reine IP-Adresse wird abgewiesen (HTTPS)
[ OK ]     Zertifikat passt zum Namen und ist vertrauenswürdig
[ OK ]     TLS 1.1 wird abgelehnt
[ OK ]     Anwendungen veröffentlichen keine Ports
[ OK ]     Anwendung erreicht das Internet nicht
[ OK ]     Konfiguration des Proxys ist gültig
[ OK ]     Proxy findet app-blau nach dem Austausch ohne Neustart

Alle Prüfungen bestanden.
```

Die letzte Prüfung dauert etwas, weil sie nach dem Austausch zwölf Sekunden wartet, mehr als die zehn Sekunden Zwischenspeicher des Resolvers. Das Skript prüft damit genau die Eigenschaft, die in Kapitel 16 versprochen wurde: Der Proxy findet eine ausgetauschte Anwendung von selbst wieder. Zur Abnahme gehö-

ren zusätzlich ein Test im Browser mit beiden Namen und ein Neustart des Servers, nach dem alle drei Dienste von selbst laufen.

> **Der Ablauf des Zertifikats**
>
> Das Zertifikat gilt 365 Tage. Danach melden Browser einen Fehler. Erzeugen Sie es dann mit dem Skript neu und laden Sie die Konfiguration mit `docker compose exec proxy nginx -s reload` neu. Im Internetbetrieb übernimmt diese Aufgabe die Automatik von Let's Encrypt, wie in Kapitel 16 beschrieben.

Wenn etwas nicht klappt

Symptom	Ursache und Abhilfe
`port is already allocated` für 80 oder 443	Ein anderer Dienst belegt den Port. `sudo ss -tlnp | grep -E ":80 |:443 "` nennt ihn.
Der Browser findet den Namen nicht	Der Eintrag in der `hosts`-Datei fehlt oder enthält die falsche Adresse.
`502 Bad Gateway`	Die Anwendung läuft nicht oder hängt in einem anderen Netz. `docker compose ps` und `docker network inspect projekt5_intern` prüfen.
nginx beendet sich beim Start	Die Konfiguration ist fehlerhaft. `docker compose logs proxy` nennt die Zeile, `nginx -t` prüft sie.
Zertifikatswarnung trotz `--cacert`	Der Name in der Anfrage steht nicht in der Liste der Namen im Zertifikat. Mit `openssl x509 -ext subjectAltName` prüfen.
`403` bei den Anwendungen	Die Dateien in `apps/` sind nicht lesbar. `chmod -R a+rX apps` setzt die Rechte.

Lernkontrolle zu Teil VI

Die Aufgaben prüfen die Kapitel 15 bis 17. Die praktischen Aufgaben führen Sie auf Ihrem Übungsserver durch. Die Lösungen stehen im Anhang.

Verständnisfragen

V1. Nennen Sie drei Vorteile eines Reverse Proxys gegenüber einzelnen, nach außen veröffentlichten Webanwendungen.

V2. Worin unterscheidet sich ein Forward Proxy von einem Reverse Proxy?

V3. Wozu dienen die Kopfzeilen `Host` und `X-Forwarded-For`, die der Proxy an die Anwendung weitergibt?

V4. Was ist der Unterschied zwischen `proxy_pass http://app:8000;` und `proxy_pass http://app:8000/;` in einem `location /api/`-Block?

V5. Warum liefert nginx nach dem Austausch eines Anwendungs-Containers unter Umständen `502 Bad Gateway`, und wie vermeidet man das?

V6. Was leisten bei TLS das Zertifikat und der private Schlüssel? Warum bekommt man für den Namen `docker01.lan` kein Zertifikat von Let's Encrypt?

V7. Warum wird Portainer in diesem Buch nur an `127.0.0.1` gebunden, und wie erreicht man die Oberfläche trotzdem?

V8. Warum schützt der Zusatz `:ro` beim Einbinden des Docker-Sockets nicht vor Änderungen am Server?

Multiple-Choice-Aufgaben

M1. Welches Verzeichnis im nginx-Container bindet man üblicherweise ein, um die Server-Konfiguration zu ersetzen?

(a) `/usr/bin/nginx`
(b) `/etc/nginx/conf.d/`
(c) `/var/log/nginx/`
(d) `/usr/share/nginx/html/`

M2. Was prüft der Befehl `nginx -t`?

(a) Die Netzwerkverbindung
(b) Die Versionsnummer
(c) Die Konfiguration auf Fehler
(d) Die Leistung des Servers

M3. Was bedeutet der Statuscode 502?

(a) Die Datei wurde nicht gefunden.
(b) Der Zugriff ist verboten.
(c) Die Anwendung hat zu lange gebraucht.
(d) Der Proxy kann die Anwendung nicht erreichen.

M4. Welche Anweisung lehnt in nginx den TLS-Verbindungsaufbau für unbekannte Namen ab?

(a) `ssl_reject_handshake on;`
(b) `deny all;`
(c) `ssl_protocols none;`
(d) `return 403;`

M5. Auf welchem Port bietet Portainer die verschlüsselte Weboberfläche an?

(a) 8000
(b) 9443
(c) 9000
(d) 443

M6. Mit welcher Option sagt man `curl`, welche Adresse es für einen Namen verwenden soll?

(a) `--dns`
(b) `--resolve`

(c) `--hosts`
(d) `--lookup`

Praktische Aufgaben

P1. Bauen Sie ein Image `verein-web:1.0` auf Basis von `nginx:stable-alpine`, das eine eigene Seite enthält. Starten Sie es mit einem Healthcheck über `wget` und prüfen Sie, dass der Zustand `healthy` erreicht wird. Prüfen Sie dann die Konfiguration mit `nginx -t` und laden Sie sie mit `nginx -s reload` neu.

P2. Erweitern Sie Projekt 5 um eine dritte Anwendung `rot.docker01.lan`. Erzeugen Sie das Zertifikat mit dem zusätzlichen Namen neu, ergänzen Sie Konfiguration und Compose-Datei und prüfen Sie den Zugriff mit `curl --resolve`.

P3. Installieren Sie Portainer wie in Kapitel 17, mit dem Port nur auf `127.0.0.1`. Weisen Sie mit `ss -tlnp` nach, dass er nur dort lauscht. Öffnen Sie die Oberfläche über einen SSH-Tunnel, legen Sie das Administratorkonto an und lesen Sie die Protokolle eines Containers.

Fehlersuche

F1. Eine mit Bind Mount eingebundene Seite liefert `403 Forbidden`. Welche Ursachen prüfen Sie, und wie beheben Sie sie?

F2. Der Reverse Proxy beendet sich beim Start mit der Meldung `host not found in upstream "app"`. Was ist die Ursache, und wie lösen Sie das Problem?

F3. Der Browser meldet bei `https://blau.docker01.lan` einen Namensfehler im Zertifikat, obwohl das Zertifikat mit `CN=docker01.lan` erstellt wurde. Woran liegt das?

Szenario aus dem Betrieb

Ein Verein möchte Portainer vom Urlaub aus nutzen und schlägt vor, den Port 9443 im Router direkt ins Internet freizugeben. Beschreiben Sie, warum Sie davon abraten, und nennen Sie zwei sicherere Wege.

TEIL VII

Sicherheit und professioneller Betrieb

KAPITEL 18

Docker Security

Die Warnungen der letzten Kapitel haben ein Thema vorbereitet, das nun im Zusammenhang folgt: die Sicherheit eines Docker-Servers. Container sind kein Sicherheitsmerkmal an sich. Sie trennen Anwendungen, aber sie teilen den Kernel, und ihre Werkzeuge verlangen Rechte, die man sorgfältig vergeben muss. Dieses Kapitel zeigt, wo ein Docker-Server angreifbar ist und welche Einstellungen die Angriffsfläche verkleinern. Es endet mit einem Thema, das 2026 durch einen konkreten Vorfall besonders anschaulich wurde: der Sicherheit der Lieferkette.

18.1 Angriffsflächen

Eine *Angriffsfläche* ist die Summe aller Stellen, an denen ein Angreifer ansetzen kann. Bei einem Docker-Server liegen sie auf mehreren Ebenen übereinander, ähnlich wie die Schalen einer Zwiebel.

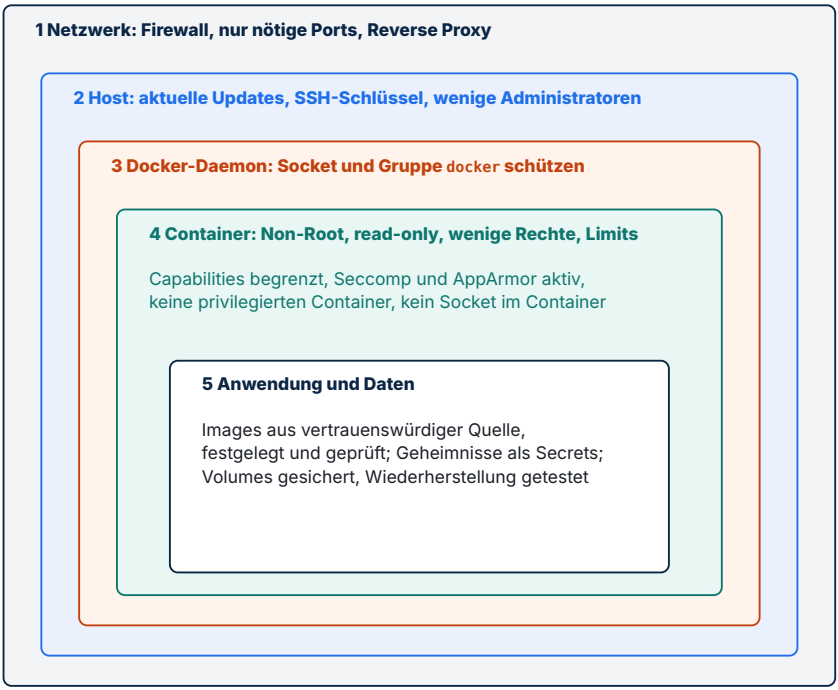


Abbildung 18.1: Sicherheitsarchitektur eines Docker-Servers in fünf Schichten

Von außen nach innen sind es fünf Schichten. Das Netzwerk entscheidet, wer überhaupt an den Server herankommt. Der Host ist das Betriebssystem, auf dem alles läuft. Ein Fehler dort betrifft jeden Container. Der Docker-Daemon hat weitreichende Rechte und ist wegen seines Sockets besonders heikel. Die Container sind die Hülle um die Anwendungen, und ihre Einstellungen entscheiden, wie viel ein ausgebrochener Angreifer erreicht. Im Kern liegen die Anwendung und ihre Daten, also das, was zu schützen ist. Jede Schicht fängt Fehler der äußeren

auf, und deshalb gilt der Grundsatz der *Verteidigung in der Tiefe*: Man verlässt sich nie auf eine einzige Maßnahme.

Typische Angriffswege lassen sich einordnen. Eine Sicherheitslücke in der Anwendung ermöglicht es, im Container Befehle auszuführen. Läuft der Container als Root mit vielen Rechten, kann der Angreifer von dort versuchen, auf den Host auszubrechen, weil beide denselben Kernel teilen. Ein eingebundener Docker-Socket macht den Ausbruch trivial. Ein manipuliertes Image bringt Schadcode schon mit. Und ein offen erreichbarer Verwaltungsport führt direkt an die Steuerung des Servers. Die folgenden Abschnitte nehmen diese Wege der Reihe nach vor.

18.2 Root-Rechte

Standardmäßig läuft der Prozess in einem Container als Root. Das ist weniger gefährlich, als es klingt, denn der Container-Root hat auf dem Host zunächst nur die eingeschränkten Rechte, die Namespaces, cgroups und die Standardsperren des Docker-Daemons ihm lassen (Kapitel 2). Er bleibt trotzdem Root im Sinne des Kernels: Gelingt ein Ausbruch aus dem Container, etwa über eine Lücke im Kernel oder eine Fehlkonfiguration, ist der Angreifer auf dem Host ebenfalls Root.

Es gibt drei Wege, das zu entschärfen. Der erste ist, im Container gar nicht als Root zu laufen (Abschnitt über Non-Root-Container). Der zweite ist, den *Daemon* ohne Root-Rechte zu betreiben. Der dritte ist, die Benutzernummern des Containers auf unprivilegierte Nummern des Hosts abzubilden.

18.2.1 Docker im Rootless-Modus

Im *Rootless-Modus* laufen der Daemon und alle Container als gewöhnlicher Benutzer innerhalb eines User-Namespaces. Root im Container entspricht dann einer unprivilegierten Benutzernummer auf dem Host. Ein Ausbruch führt nicht zu Root-Rechten, und der Docker-Socket gehört dem Benutzer und ist keine Root-Schnittstelle mehr. Die Voraussetzungen sind wenige Pakete und ein eigener Nummernbereich für den Benutzer.

```
$ sudo apt install uidmap dbus-user-session docker-ce-rootless-extras
$ grep "^$USER:" /etc/subuid /etc/subgid
/etc/subuid:admin:100000:65536
/etc/subgid:admin:100000:65536
```

Die Datei `/etc/subuid` weist dem Benutzer einen Bereich von mindestens 65 536 Nummern zu, die der Daemon für die Container verwendet. Auf Debian und Ubuntu legt der Installer den Eintrag normalerweise selbst an. Die Einrichtung erfolgt als *normaler Benutzer*, nicht mit `sudo`.

```
$ dockerd-rootless-setuptool.sh install
$ systemctl --user enable --now docker
$ export DOCKER_HOST=unix://$XDG_RUNTIME_DIR/docker.sock
$ docker run --rm hello-world
```

Das Werkzeug legt einen Dienst im Benutzerkonto an. Damit er auch ohne angemeldete Sitzung läuft, schaltet man mit `sudo loginctl enable-linger admin` das sogenannte Lingering ein. Die Einschränkungen sollten Sie kennen: Container dürfen nicht mit `--privileged` laufen, Ports unter 1024 sind für unprivilegierte Benutzer gesperrt (es sei denn, man senkt `net.ipv4.ip_unprivileged_port_start`), und die Netzwerkanbindung ist etwas langsamer. Der Daemon verwaltet im Rootless-Modus eigene Images und Volumes im Heimatverzeichnis des Benutzers. Er ist eine Alternative zum bisher beschriebenen *Root-Daemon*, kein Zusatz. Die Beispiele dieses Buches laufen mit dem Root-Daemon, weil er den Ports 80 und 443 für den Reverse Proxy direkt bedienen kann.

userns-remap

Daneben gibt es die Daemon-Einstellung `userns-remap`. Sie lässt den Daemon als Root laufen, bildet die Benutzernummern der Container aber auf einen unprivilegierten Bereich ab. Sie ist mit dem containerd-Image-Speicher aus Kapitel 7 derzeit nicht kombinierbar, und für viele Fälle ist der Rootless-Modus der geradlinigere Weg.

18.2.2 Privilegierte Container

Die Option `--privileged` hebt fast alle Schutzmaßnahmen auf. Der Container bekommt alle Capabilities, Zugriff auf alle Geräte des Hosts, und die Filter von Seccomp und AppArmor entfallen. Ein privilegierter Container ist praktisch ein gewöhnlicher Prozess mit Root-Rechten auf dem Host, nur mit einer eigenen Dateisicht. Verwenden Sie die Option nicht, außer für Werkzeuge, die wirklich Gerätezugriff brauchen, und dann nur bewusst und befristet. In Compose entspricht ihr der Eintrag `privileged: true`.

18.3 Der Docker-Socket

Der Docker-Socket `/var/run/docker.sock` ist die Steuerzentrale des Daemons. Wer ihn bedienen kann, kann einen neuen Container starten, der das Hauptverzeichnis des Hosts einbindet, und damit den ganzen Server übernehmen. Das lässt sich in einem Befehl zeigen. Führen Sie ihn nur auf Ihrem Übungsserver aus und verstehen Sie ihn als Warnung.

```
$ docker run --rm -v /:/host alpine cat /host/etc/shadow | head -n 2
root:$y$j9T$...:20400:0:99999:7:::
daemon:*:20400:0:99999:7:::
```

Der Container liest die Passwort-Datei des *Hosts*, obwohl man eigentlich nur Docker benutzen wollte. Daraus folgen drei Regeln.

Erstens: Die Gruppe `docker` ist ein Root-Zugang, wie in Kapitel 5 beschrieben. Nehmen Sie nur Personen auf, denen Sie Root-Rechte anvertrauen würden. Wer der Gruppe angehört, zeigt `getent group docker`.

Zweitens: Binden Sie den Socket nicht in Container ein, außer es ist unvermeidlich. Dazu gehören manche Verwaltungswerkzeuge wie Portainer (Kapitel 17) oder Überwachungsprogramme. Jede solche Ausnahme ist eine Tür zum Root-Zugang. Wenn es nicht anders geht, begrenzen Sie die Erreichbarkeit des Werkzeugs und halten es aktuell. Ein `:ro` an der Einbindung schützt nicht, wie Kapitel 17 gezeigt hat. Wer den Zugriff wirklich beschränken will, setzt einen *Socket-Proxy* dazwischen, einen kleinen Container, der nur ausgewählte Anfragen an die Docker-Schnittstelle durchlässt.

Drittens: Machen Sie den Daemon nie unverschlüsselt über das Netzwerk erreichbar. Fernzugriff gehört über SSH, wie Kapitel 3 gezeigt hat. Die Rechte der Socket-Datei prüft man so:

```
$ ls -l /var/run/docker.sock
srw-rw---- 1 root docker 0 Nov 11 09:12 /var/run/docker.sock
```

Besitzer ist Root, Gruppe ist `docker`, und andere haben keinen Zugriff. Ändern Sie diese Rechte nicht auf `666`, wie es in manchen Foren als Lösung für `permission denied` steht. Damit bekäme jeder Benutzer des Systems Root-Rechte.

18.4 Das Least-Privilege-Prinzip

Das Prinzip der *geringsten Rechte* (*least privilege*) besagt, dass jede Komponente nur die Rechte bekommen soll, die sie für ihre Aufgabe braucht, und nichts darüber hinaus. Wenn ein Container, der eine Webseite ausliefert, nur Dateien lesen und auf einem Port lauschen muss, braucht er keine Root-Rechte, kein beschreibbares Dateisystem, keine Netzwerkverwaltung und keinen Zugriff auf Geräte. Alles, was man ihm nicht gibt, kann ein Angreifer nicht missbrauchen. Die Tabelle stellt die Maßnahmen dieses Kapitels zusammen.

Bereich	Einstellung	Wirkung
Benutzer	`USER`, `--user`	Prozess läuft ohne Root-Rechte
Dateisystem	`--read-only`, `--tmpfs`	Container kann sich nicht verändern
Capabilities	`--cap-drop ALL`	Kernel-Sonderrechte entfallen
Rechte erweitern	`no-new-privileges`	Prozesse bekommen keine zusätzlichen Rechte
Filter	Seccomp, AppArmor	Systemaufrufe und Zugriffe werden begrenzt
Ressourcen	`--memory`, `--cpus`, `--pids-limit`	ein Container kann den Host nicht lahmlegen
Netzwerk	interne Netze, `127.0.0.1`-Bindung	nur nötige Verbindungen
Einbindungen	kein Socket, `:ro` bei Bind Mounts	kein Zugriff auf Host und Daemon

18.5 Non-Root-Container

Die einfachste und wirksamste Maßnahme ist, den Prozess im Container als normaler Benutzer laufen zu lassen. Im Dockerfile geschieht das mit `USER`, wie Kapitel 8 gezeigt hat. Beim Start lässt sich der Benutzer außerdem mit `--user` (in Compose `user:`) festlegen.

```
$ docker run --rm --user 10001:10001 alpine id
uid=10001 gid=10001 groups=10001
```

Wer ein fremdes Image verwendet, prüft, als wer es läuft.

```
$ docker run -d --name test-web nginx:stable-alpine
$ docker exec test-web id
uid=0(root) gid=0(root) groups=0(root)
```

```
$ docker rm -f test-web
```

Das Standard-Image von nginx startet den Hauptprozess als Root (nur die Arbeitsprozesse laufen als `nginx`). Für strenge Anforderungen gibt es das Image `nginxinc/nginx-unprivileged`, das komplett ohne Root läuft. Aus der Umstellung ergeben sich vier Folgen, die man im Voraus bedenken sollte.

Ports unter 1024 sind für Nicht-Root-Prozesse nicht frei. Der Webserver lauscht auf 8080, und die Portfreigabe führt `80:8080`. Dateien, in die der Prozess schreiben soll, müssen ihm gehören oder für ihn freigegeben sein, wie in Kapitel 10 gezeigt. Verzeichnisse für Zwischendateien müssen beschreibbar sein, etwa über ein `tmpfs`. Und Programme, die beim Start Pakete installieren oder Konfigurationen schreiben, funktionieren nicht mehr, ein weiterer Grund, solche Schritte in das Image zu verlegen.

18.6 Read-only-Dateisysteme

Ein Container, der sein eigenes Dateisystem nicht verändern kann, ist deutlich schwerer zu missbrauchen. Ein Angreifer kann dort keine Programme ablegen, keine Konfiguration manipulieren und keine Hintertür einbauen. Die Option `--read-only` macht das Wurzeldateisystem schreibgeschützt.

```
$ docker run --rm --read-only alpine touch /test.txt
touch: /test.txt: Read-only file system
$ docker run --rm --read-only --tmpfs /tmp alpine sh -c "touch /tmp/
    ↪ test.txt && echo geschrieben"
geschrieben
```

Viele Programme brauchen trotzdem einige Verzeichnisse zum Schreiben: für Zwischendateien, Prozesskennungen oder Zwischenspeicher. Diese stellt man gezielt mit `tmpfs` (Arbeitsspeicher) oder Volumes (dauerhaft) bereit. Alles andere bleibt geschützt. Für den normalen nginx-Container sieht das so aus.

Ausschnitt compose.yaml

```
services:
  web:
    image: nginx:stable-alpine
    read_only: true
    tmpfs:
      - /tmp
      - /var/run
      - /var/cache/nginx
```

Welche Verzeichnisse ein Programm braucht, findet man durch Ausprobieren. Man startet den Container schreibgeschützt, liest die Fehlermeldung im Protokoll (`Read-only file system` nennt den Pfad) und gibt genau diesen Pfad frei. Mit `docker diff` aus Kapitel 9 sieht man bei einem normal laufenden Container, wohin das Programm schreibt.

18.7 Linux Capabilities

Früher kannte Unix nur zwei Arten von Benutzern: Root darf alles, alle anderen fast nichts. Der Linux-Kernel hat die Allmacht von Root in einzelne Sonderrechte zerlegt, die *Capabilities*. Das Recht, Ports unter 1024 zu belegen, ist eine, das Recht, Dateibesitzer zu ändern, eine andere, das Recht, die Netzwerkkonfiguration zu verändern, wieder eine andere. Ein Prozess kann nur einen Teil davon haben.

Docker gibt Containern eine eingeschränkte Standardauswahl von 14 Capabilities, auch wenn der Prozess als Root läuft. Dazu gehören `CHOWN`, `DAC_OVERRIDE`, `FOWNER`, `FSETID`, `KILL`, `SETGID`, `SETUID`, `SETPCAP`, `SETFCAP`, `NET_BIND_SERVICE`, `NET_RAW`, `SYS_CHROOT`, `MKNOD` und `AUDIT_WRITE`. Gefährliche Rechte wie `SYS_ADMIN` (fast Root-Rechte im Kernel) oder `NET_ADMIN` (Netzwerkverwaltung) sind nicht dabei und müssen ausdrücklich hinzugefügt werden. Das ist ein Grund, warum man diese Optionen nur mit gutem Grund einsetzt.

Das Prinzip der geringsten Rechte verlangt, auch die Standardauswahl zu verkleinern: alle Capabilities entfernen und nur die nötigen hinzufügen.

```
$ docker run --rm alpine sh -c "grep CapEff /proc/self/status"
CapEff:^^I00000000a80425fb
$ docker run --rm --cap-drop ALL alpine sh -c "grep CapEff /proc/self
    ↪ /status"
CapEff:^^I0000000000000000
$ docker run --rm --cap-drop ALL --cap-add NET_BIND_SERVICE alpine sh
    ↪  -c "grep CapEff /proc/self/status"
CapEff:^^I0000000000000400
```

Die Zeile `CapEff` nennt die aktuell wirksamen Capabilities als Hexadezimalzahl. Beim Standard sind es viele Bits, nach `--cap-drop ALL` keine, und nach dem Hinzufügen von `NET_BIND_SERVICE` genau ein Bit. Die genauen Zahlen hängen von der Docker-Version ab. Entschlüsseln lassen sie sich mit `capsh --decode=00000000a80425fb` aus dem Paket `libcap2-bin`. Die Wirkung zeigt sich am Beispiel `ping`, das die Capability `NET_RAW` braucht.

```
$ docker run --rm --cap-drop NET_RAW alpine ping -c 1 127.0.0.1
ping: permission denied (are you root?)
```

In Compose heißt das `cap_drop: [ALL]` und `cap_add: [NET_BIND_SERVICE]`. Zusammen mit der Option `no-new-privileges` schließt man den letzten Weg, auf dem ein Prozess nachträglich Rechte erlangen könnte.

Ausschnitt compose.yaml

```
services:
  web:
    image: nginx:stable-alpine
    cap_drop:
      - ALL
    cap_add:
      - NET_BIND_SERVICE
      - CHOWN
      - SETUID
      - SETGID
    security_opt:
      - no-new-privileges:true
```

Welche Capabilities ein Programm braucht, ist nicht immer offensichtlich. Der nginx-Container beispielsweise wechselt für seine Arbeitsprozesse den Benutzer und braucht dafür `SETUID` und `SETGID`. Man beginnt mit `ALL` entfernt, liest die Fehlermeldungen und fügt zurück, was tatsächlich fehlt.

18.8 Seccomp

Jedes Programm ruft den Kernel über *Systemaufrufe* (*system calls*) auf: Dateien öffnen, Prozesse starten, Netzwerkverbindungen aufbauen. Der Kernel bietet mehrere hundert davon an, und die meisten Programme brauchen nur einen Bruchteil. *Seccomp* (secure computing) ist ein Filter, der dem Prozess nur eine Auswahl erlaubt. Ein gefährlicher Aufruf, der gesperrt ist, kann auch bei einer Lücke im Programm nicht ausgenutzt werden.

Docker wendet auf jeden Container standardmäßig ein Seccomp-Profil an, das einige Dutzend gefährliche Systemaufrufe sperrt. Ob es aktiv ist, steht in den Sicherheitsoptionen des Daemons.

```
$ docker info --format '{{.SecurityOptions}}'
[name=apparmor name=seccomp,profile=builtin name=cgroupns]
```

Der Eintrag `seccomp,profile=builtin` bedeutet, dass das eingebaute Standardprofil gilt. Es lässt sich durch ein eigenes ersetzen (`--security-opt seccomp=profil.json`), was für Anfänger selten nötig ist. Wichtiger ist, was man nicht tun soll: Die Option `--security-opt seccomp=unconfined` schaltet den Schutz ab. Sie taucht in Foren als Lösung für Fehlermeldungen auf und gehört höchstens als kurzer Test auf einen Übungsserver, nie in eine dauerhafte Konfiguration.

18.9 AppArmor

AppArmor ist ein weiteres Schutzsystem des Linux-Kernels. Es begrenzt, auf welche Dateien und Funktionen ein Programm zugreifen darf, und legt das in Profilen fest. Auf Debian und Ubuntu ist es in der Regel aktiv. Docker lädt ein Standardprofil mit dem Namen `docker-default` und wendet es auf Container an.

```
$ sudo aa-status | head -n 3
apparmor module is loaded.
...
$ docker run -d --name aa-test nginx:stable-alpine
$ docker inspect --format '{{.AppArmorProfile}}' aa-test
docker-default
$ docker rm -f aa-test
```

Das Programm `aa-status` gehört zum Paket `apparmor-utils`. Steht beim Container `docker-default`, schützt AppArmor ihn. Die Option `--security-opt apparmor=unconfined` schaltet den Schutz aus und gehört, wie bei Seccomp, nicht in eine dauerhafte Konfiguration. Sie müssen im Alltag kein eigenes Profil schreiben. Wichtig ist, dass AppArmor auf dem Host aktiv ist und Sie die Schutzfunktion nicht ohne Not umgehen.

18.10 Secrets

Passwörter, Schlüssel und Zugangstoken sind Geheimnisse und gehören nicht in Dockerfiles, nicht in Images und nicht in die Compose-Datei, die in einer Versionsverwaltung liegt. Umgebungsvariablen sind bequem, aber sie stehen in `docker inspect`, können in Protokolle geraten und werden an Kindprozesse weitergegeben. Besser ist es, ein Geheimnis als *Datei* in den Container zu geben. Compose bietet dafür das Element `secrets`.

compose.yaml mit Secrets

```
services:
  db:
    image: mariadb:11.8
    environment:
      MARIADB_ROOT_PASSWORD_FILE: /run/secrets/db_root_password
      MARIADB_PASSWORD_FILE: /run/secrets/db_password
      MARIADB_DATABASE: shop
      MARIADB_USER: shopuser
    secrets:
      - db_root_password
      - db_password
```

```
    volumes:
      - db-daten:/var/lib/mysql

secrets:
  db_root_password:
    file: ./secrets/db_root_password
  db_password:
    file: ./secrets/db_password

volumes:
  db-daten:
```

Die Dateien liegen auf dem Host im Verzeichnis `secrets`. Compose bindet jede als schreibgeschützte Datei unter `/run/secrets/NAME` in den Container ein. Die Variablen mit der Endung `_FILE` lesen MariaDB und viele andere offizielle Images aus der Datei. Das Passwort steht dann weder in der Compose-Datei noch in der Umgebung des Containers.

```
$ mkdir -p secrets && chmod 700 secrets
$ tr -dc A-Za-z0-9 </dev/urandom | head -c 24 > secrets/
    ↪ db_root_password
$ tr -dc A-Za-z0-9 </dev/urandom | head -c 24 > secrets/db_password
$ chmod 644 secrets/*
$ docker compose exec db ls -l /run/secrets
```

Zwei Dinge sind zu bedenken. Das Verzeichnis `secrets` auf dem Host schützen Sie mit den Rechten `700`, und die Dateien darin müssen für den Benutzer im Container lesbar sein, was bei einem Nicht-Root-Benutzer die Rechte `644` verlangt. Der Schutz vor anderen Benutzern des Hosts kommt dann vom Verzeichnis. Und das Verzeichnis gehört in die Datei `.gitignore`, damit es nicht in einem Repository landet. Ein Geheimnis zu wechseln heißt: neue Datei schreiben, Container neu starten, und bei einer Datenbank zusätzlich das Passwort per SQL ändern, wie in Kapitel 14 gezeigt.

Dateibasierte Secrets sind kein Secret-Dienst

Bei lokalem Docker Compose stammen dateibasierte Secrets aus Dateien auf dem Host. Compose bindet diese Dateien in den Container ein. Sie werden dadurch nicht automatisch verschlüsselt auf dem Datenträger gespeichert, und ein zentral verwalteter Secret-Dienst steht auch nicht dahinter. Den Schutz bieten Sie selbst: Sichern Sie das Verzeichnis und die Dateien mit geeigneten Rechten auf dem Host, nehmen Sie sie nicht in die Versionsverwaltung auf und legen Sie Sicherungen nur geschützt ab. Docker-Swarm-Secrets und zentrale Secret-Dienste gehen weiter, sie behandelt dieses Buch nicht.

18.11 Image-Sicherheit

Ein Image bringt eigene Software mit, und jede Software kann Lücken haben. Die Sicherheit beginnt deshalb bei der Auswahl. Bevorzugen Sie die *offiziellen Images* von Docker Hub (*Docker Official Images*) oder Images, die vom Hersteller der Software selbst gepflegt werden, und meiden Sie anonyme Images mit wenigen Downloads. Wählen Sie kleine Basisimages wie `-alpine` oder `-slim`. Was nicht im Image steckt, kann keine Lücke haben, wie Kapitel 7 gezeigt hat.

Legen Sie Tags fest und aktualisieren Sie bewusst. Ein Image, das Monate alt ist, enthält bekannte, inzwischen behobene Lücken. Bauen Sie eigene Images regelmäßig neu und mit `--pull`, damit sie auf einer aktuellen Basis aufsetzen. Legen Sie keine Geheimnisse in Schichten. Betreiben Sie Anwendungen als normaler Benutzer. Und nehmen Sie nur Software in das Image auf, die gebraucht wird.

18.12 Schwachstellenanalyse

Bekannte Sicherheitslücken in Software erhalten eine Kennung nach dem Schema `CVE-Jahr-Nummer` (Common Vulnerabilities and Exposures) und eine Einstufung nach Schwere von niedrig bis kritisch. Werkzeuge zur *Schwachstellenanalyse* vergleichen die Pakete und Bibliotheken in einem Image mit Datenbanken bekannter Lücken und listen die Treffer auf. Bekannte Werkzeuge sind Trivy, Grype und Docker Scout.

Das folgende Beispiel verwendet Trivy. Das Programm wird, wie andere Werkzeuge, als Container betrieben, aber mit zwei Vorsichtsmaßnahmen, die der Abschnitt über die Lieferkette begründet: Der Tag ist festgelegt, und der Docker-Socket wird *nicht* eingebunden. Stattdessen übergibt man das Image als Archiv.

```
$ TRIVY_VERSION=0.70.0
$ docker save nginx:stable-alpine -o /tmp/nginx.tar
$ docker run --rm -v /tmp:/work aquasec/trivy:$TRIVY_VERSION \
    image --input /work/nginx.tar --severity HIGH,CRITICAL
$ rm /tmp/nginx.tar
```

Die Nummer im Beispiel stammt aus der Zeit nach dem unten beschriebenen Vorfall. Prüfen Sie auf der Projektseite des Herstellers, welche Fassung aktuell empfohlen wird, und verwenden Sie eine überprüfte Version. Die Ausgabe ist eine Tabelle: Für jedes betroffene Paket stehen die Kennung der Lücke, die Schwere, die installierte Version und die Version, in der der Fehler behoben ist. Mit `--severity HIGH,CRITICAL` beschränkt man sie auf die schweren Fälle.

Treffer zu lesen will gelernt sein. Nicht jeder hat praktische Bedeutung: Eine Lücke in einem Programm, das im Container zwar vorhanden ist, aber nie aufgerufen wird, ist weniger dringlich als eine im Webserver selbst. Meldet das Werkzeug eine behobene Version, lautet die Antwort meist, das Image auf die neueste Fassung seines Tags zu aktualisieren oder die Basis zu wechseln. Meldet es keine behobene Version, gibt es keine Korrektur, und man muss abwägen. Wiederholen Sie die Prüfung regelmäßig, denn jeden Tag kommen neue Einträge in die Datenbanken. Ein Image, das gestern sauber war, kann heute Treffer haben.

18.13 Supply-Chain-Sicherheit

Mit *Lieferkette* (*Supply Chain*) meint man alles, was in eine Anwendung hineinfließt, ohne dass man es selbst geschrieben hat: Basisimages, Bibliotheken, Werkzeuge zum Bauen, Registries und Aktualisierungswege. Ein Angriff auf die Lieferkette zielt auf etwas, dem Sie vertrauen, und trifft Ihren Server erst über diesen Umweg. Er erreicht viele Opfer auf einmal, weil sie dasselbe Teil beziehen.

Ein Fall aus dem März 2026 zeigt das beispielhaft, und er betrifft ein Werkzeug aus diesem Kapitel. Angreifer verschafften sich Zugang zur Bauumgebung von

Trivy, dem verbreiteten Scanner für Schwachstellen. Am 19. März 2026 veröffentlichten sie eine verseuchte Fassung, die Zugangsdaten stahl. Über Docker Hub verbreiteten sich verseuchte Images unter den Tags `0.69.4`, später `0.69.5` und `0.69.6`, und auch der Tag `latest` wurde auf verseuchte Inhalte umgesetzt. Docker und der Hersteller entfernten die Images am 23. März. Die letzte saubere Fassung davor war `0.69.3`. Wer das Image mit eingebundenem Docker-Socket gestartet hatte, musste den ganzen Host als kompromittiert behandeln.

Der Vorfall belegt mehrere Grundsätze, die in diesem Buch schon vorkamen. Ein Tag ist veränderlich, auch ein vermeintlich feststehender wie `0.69.6`. Nur der Digest bezeichnet unveränderlich eine Fassung (Kapitel 7). Ein Werkzeug mit Zugriff auf den Docker-Socket hat Root-Rechte, auch ein Sicherheitswerkzeug (Kapitel 3). Und ein Vertrauensanker wie ein Sicherheitsprodukt ist ein besonders lohnendes Ziel. Daraus ergeben sich Maßnahmen.

Maßnahme	Wirkung
Digest statt Tag für wichtige Images	verhindert, dass ein umgesetzter Tag unbemerkt andere Inhalte liefert
Neue Fassungen mit Abstand einspielen	Verseuchte Veröffentlichungen werden meist innerhalb von Tagen entdeckt und zurückgezogen. Wer nicht am ersten Tag aktualisiert, ist seltener betroffen.
Signaturen prüfen	Werkzeuge wie Cosign (Sigstore) prüfen, ob ein Image vom Hersteller stammt, sofern er es signiert
Keinen Socket in Werkzeug-Container einbinden	Auch ein verseuchtes Werkzeug erreicht dann nicht den Host
Weniger Fremdsoftware	jedes zusätzliche Image ist ein zusätzlicher Vertrauensanker
Kritische Images in eigener Registry spiegeln	Sie bestimmen den Zeitpunkt der Aktualisierung
Meldungen der Hersteller verfolgen	Sicherheitsmitteilungen abonnieren, damit man betroffene Fassungen kennt
Zugangsdaten von Bauumgebungen begrenzen	kurzlebige Schlüssel und möglichst geringe Rechte

Digest-Angaben holt man sich aus dem lokalen Bestand und trägt sie in die Compose-Datei ein.

```
$ docker image inspect nginx:stable-alpine --format '{{index .
    ↪ RepoDigests 0}}'
nginx@sha256:<Prüfsumme>
```

compose.yaml mit festem Digest

```
services:
  web:
    image: nginx:stable-alpine@sha256:<Prüfsumme>
```

Bei einem Digest trägt man den Tag zusätzlich ein, der beim Lesen die Orientierung gibt, die Prüfsumme bestimmt die tatsächliche Fassung. Der Preis des Verfahrens: Man muss Updates bewusst durchführen und den neuen Digest eintragen. Das ist gewollt, denn damit entscheiden Sie, wann Neues in Ihren Server kommt.

Die Prüfliste dieses Kapitels

Wer die Absicherung eines bestehenden Containers durchgeht, kann sich an diesen Fragen orientieren: Läuft er als Nicht-Root? Hat er ein schreibgeschütztes Dateisystem? Sind alle überflüssigen Capabilities entfernt? Ist `no-new-privileges` gesetzt? Sind Speicher, Prozessorzeit und Prozesse begrenzt? Sind nur nötige Ports veröffentlicht, und gebunden an die richtige Adresse? Ist kein Socket eingebunden? Stammen die Geheimnisse aus Dateien? Ist das Image festgelegt und frisch geprüft? Das Praxisprojekt 6 bündelt diese Fragen in einem Prüfskript.

KAPITEL 19

Wartung und Monitoring

Ein Server, der einmal eingerichtet wurde und danach unbeachtet läuft, wird früher oder später ausfallen. Platten füllen sich, Software veraltet, ein Container stirbt in der Nacht. Professioneller Betrieb heißt, solche Ereignisse zu bemerken, bevor jemand anruft, und für den Ernstfall vorbereitet zu sein. Dieses Kapitel behandelt die regelmäßigen Aufgaben: überwachen, Ressourcen im Blick behalten, Protokolle und Speicherplatz verwalten, aktualisieren, sichern und die Wiederherstellung planen.

19.1 Containerüberwachung

Die erste Frage lautet immer: Läuft alles, was laufen soll? Antworten liefern die Zustände, die Sie aus den früheren Kapiteln kennen. Ein Container ist laufend, gestoppt, am Neustarten oder, wenn er einen Healthcheck hat, gesund oder ungesund. `docker ps` zeigt das auf einen Blick, und mit einem Filter beschränkt man die Ausgabe auf die auffälligen Fälle.

```
$ docker ps -a --filter "status=exited" --filter "status=restarting"
    ↪ \
    --format "table {{.Names}}\t{{.Status}}"
NAMES       STATUS
alt-test    Exited (1) 2 days ago
$ docker ps --filter "health=unhealthy" --format "{{.Names}}"
```

Zwei Kennzahlen sind besonders aufschlussreich. Der Rückgabewert eines beendeten Containers (`Exited (1)`) verrät, ob er sauber oder mit einem Fehler endete, Kapitel 20 erklärt die Werte. Und der Zähler der Neustarts zeigt Container, die in einer Schleife abstürzen und wieder starten.

```
$ docker inspect --format '{{.Name}}: {{.RestartCount}} Neustarts,
    ↪ Zustand {{.State.Status}}' $(docker ps -aq)
/projekt4-web-1: 0 Neustarts, Zustand running
/projekt4-app-1: 0 Neustarts, Zustand running
/projekt4-db-1: 0 Neustarts, Zustand running
```

Wer nicht ständig selbst nachsehen will, lässt ein Skript prüfen. Das folgende meldet jeden Container, der nicht läuft oder als ungesund gilt, und beendet sich mit einem Fehlerwert, wenn es etwas findet.

/usr/local/sbin/docker-gesundheit.sh

```
#!/bin/bash
# Meldet Container, die nicht laufen oder nicht gesund sind
PROBLEME=0
while read -r NAME STATUS; do
  case "$STATUS" in
    Up*"(unhealthy)"*)
      echo "UNGESUND:  $NAME ($STATUS)"; PROBLEME=1 ;;
    Up*)
      ;;
    *)
      echo "PROBLEM:   $NAME ($STATUS)"; PROBLEME=1 ;;
  esac
done < <(docker ps -a --format '{{.Names}} {{.Status}}')
[ "$PROBLEME" -eq 0 ] && echo "Alle Container sind in Ordnung."
exit $PROBLEME
```

Das Skript liest die Namen und Zustände aller Container und prüft den Anfang des Zustandstextes. `Up` bedeutet laufend, der Zusatz `(unhealthy)` einen fehlgeschlagenen Healthcheck, alles andere (`Exited`, `Restarting`, `Created`) gilt als Problem. Absichtlich gestoppte Container melden sich deshalb ebenfalls. Wer solche Container hat, löscht sie oder nimmt sie mit einem weiteren Muster aus.

Damit das Skript regelmäßig läuft, verwendet man auf Debian und Ubuntu einen *systemd-Timer*. Er besteht aus zwei kleinen Dateien: der Dienst beschreibt, was ausgeführt wird, der Timer, wann.

/etc/systemd/system/docker-gesundheit.service

```
[Unit]
Description=Prüfung der Docker-Container

[Service]
Type=oneshot
ExecStart=/usr/local/sbin/docker-gesundheit.sh
```

/etc/systemd/system/docker-gesundheit.timer

```
[Unit]
Description=Prüfung der Docker-Container alle fünf Minuten

[Timer]
OnBootSec=2min
OnUnitActiveSec=5min

[Install]
WantedBy=timers.target
```

```
$ sudo chmod +x /usr/local/sbin/docker-gesundheit.sh
$ sudo systemctl daemon-reload
$ sudo systemctl enable --now docker-gesundheit.timer
$ systemctl list-timers docker-gesundheit.timer
$ systemctl --failed
```

Der Timer startet den Dienst zwei Minuten nach dem Systemstart und danach alle fünf Minuten. Findet das Skript ein Problem, endet es mit einem Fehlerwert, und `systemd` markiert den Dienst als fehlgeschlagen. Das sieht man mit `systemctl --failed`, und die Ausgaben stehen im Journal (`journalctl -u docker-gesundheit`). Wer eine Benachrichtigung per E-Mail oder Chat will, hängt an den Dienst mit `OnFailure=` eine weitere Einheit, die sie verschickt. Das hängt von Ihrer Umgebung ab und wird hier nicht ausgeführt.

Einen Blick in das Geschehen auf dem Daemon bietet `docker events`. Der Befehl zeigt laufend alle Ereignisse: Container starten, stoppen, sterben, Healthchecks wechseln ihren Zustand. Mit `--since 1h` liest man die der letzten Stunde nach.

```
$ docker events --since 1h --until 0s --filter type=container --
    ↪ filter event=die
```

19.2 Ressourcenverbrauch: CPU und RAM

Ein Container, der die Rechenleistung oder den Arbeitsspeicher des Servers aufbraucht, nimmt die anderen mit. Dagegen helfen die Limits aus Kapitel 9, und damit man sinnvolle Grenzen wählen kann, muss man den tatsächlichen Verbrauch kennen.

Zwei Begriffe sollte man unterscheiden. *Prozessorlast* wird in Prozent eines Kerns angegeben. Ein Container, der zwei Kerne voll auslastet, zeigt 200 %. Der *Arbeitsspeicher* umfasst bei Containern den echten Verbrauch der Prozesse und zusätzlich den vom Kernel zwischengespeicherten Inhalt von Dateien. Dieser Zwischenspeicher gibt sich bei Bedarf frei, belegt in der Anzeige aber zunächst Platz. Der Kernel zählt ihn gegen das Limit, räumt ihn bei Knappheit aber zuerst ab. Ein Wert nahe am Limit ist deshalb nicht automatisch ein Problem.

Auf dem Host selbst zeigen gewöhnliche Werkzeuge den Gesamtzustand.

```
$ free -h
$ uptime
 14:02:11 up 12 days,  3:10,  1 user,  load average: 0.21, 0.18, 0.15
$ systemd-cgtop -n 1 --order=cpu | head -n 6
```

Die drei Zahlen hinter `load average` geben die mittlere Auslastung der letzten einer, fünf und fünfzehn Minuten an. Als Faustregel ist ein Wert unterhalb der Zahl der Prozessorkerne unkritisch. `systemd-cgtop` zeigt die cgroups nach Verbrauch geordnet, und die Container erscheinen dort als `docker-….scope`, ein Zusammenhang, den Kapitel 2 erklärt hat.

Reicht der Arbeitsspeicher nicht, beendet der Kernel Prozesse, den sogenannten OOM-Kill. Bei Containern mit Limit trifft das den Container selbst, bei erschöpf-

tem Host-Speicher irgendeinen Prozess. Ob es geschehen ist, verrät der Zustand des Containers und das Protokoll des Kernels.

```
$ docker inspect --format '{{.State.OOMKilled}}' projekt4-app-1
false
$ journalctl -k --since "1 day ago" | grep -i "out of memory"
```

19.3 docker stats

`docker stats` zeigt den Verbrauch aller laufenden Container und aktualisiert die Anzeige laufend. Mit `--no-stream` gibt es eine einzelne Momentaufnahme.

```
$ docker stats --no-stream --format "table {{.Name}}\t{{.CPUPerc}}\t
    ↪ {{.MemUsage}}\t{{.MemPerc}}"
NAME             CPU %    MEM USAGE / LIMIT    MEM %
projekt4-web-1   0.00%    4.2MiB / 3.8GiB      0.11%
projekt4-app-1   0.02%    21.5MiB / 3.8GiB     0.55%
projekt4-db-1    0.18%    92.7MiB / 3.8GiB     2.38%
```

Ohne gesetztes Limit zeigt die Spalte den gesamten Arbeitsspeicher des Hosts als Grenze, wie hier. Die Werte sind ein Beispiel. Die weiteren Spalten (`NET I/O` für den Netzwerkverkehr, `BLOCK I/O` für Plattenzugriffe und `PIDS` für die Zahl der Prozesse) kann man mit `--format` ergänzen. Gerade `PIDS` lohnt einen Blick: Ein stetig steigender Wert deutet auf einen Container hin, der Prozesse erzeugt, ohne sie zu beenden.

Wer Verlaufswerte braucht, statt nur Augenblicke, kommt mit `docker stats` an eine Grenze. Dafür gibt es Überwachungssysteme, die Messwerte sammeln und als Kurven darstellen, etwa Prometheus mit Grafana oder cAdvisor. Sie sind ein eigenes Thema und gehen über dieses Buch hinaus. Mit den Werkzeugen dieses Kapitels lässt sich ein einzelner Server aber gut betreuen.

19.4 Logs

Die Protokolle der Container sind die wichtigste Quelle bei Störungen. Wie Docker sie speichert, hängt vom *Logging-Treiber* ab. Der Standard heißt `json-file`: Docker schreibt die Ausgabe jedes Containers in eine Datei unter `/var/lib/docker/containers`. Mit der Rotation aus Kapitel 5 begrenzt man ihre Größe. Es gibt Alternativen. Der Treiber `local` speichert komprimiert und rotiert von Haus aus, und `journald` leitet die Ausgaben ins Journal des Systems, sodass man sie mit `journalctl` neben allen anderen Meldungen liest. Alle drei erlauben `docker logs`.

Pro Dienst lässt sich der Treiber in der Compose-Datei festlegen.

Ausschnitt compose.yaml

```
services:
  web:
    image: nginx:stable-alpine
    logging:
      driver: json-file
      options:
        max-size: "10m"
        max-file: "3"
```

Welche Einstellung ein laufender Container tatsächlich hat, zeigt `docker inspect`.

```
$ docker inspect --format '{{.HostConfig.LogConfig}}' projekt4-web-1
{json-file map[max-file:3 max-size:10m]}
```

Im Alltag helfen drei Handgriffe. Mit `docker logs --since 1h NAME` und `--tail` liest man gezielt den Ausschnitt, der interessiert. Mit `docker compose logs -f` verfolgt man alle Dienste eines Projekts. Und mit `grep` sucht man nach Stichworten, etwa `docker logs NAME 2>&1 | grep -i error`. Das `2>&1` leitet die Fehlerausgabe auf die normale Ausgabe um, denn `docker logs` gibt beide getrennt wieder, und `grep` liest nur die normale.

Bei mehreren Servern oder strengen Aufbewahrungsfristen sammelt man die Protokolle an einer zentralen Stelle. Das geht über weitere Logging-Treiber oder

über Sammelprogramme. Auch das ist ein eigenes Thema. Beachten Sie zumindest die rechtliche Seite: Protokolle enthalten Adressen von Besuchern und gelten als personenbezogene Daten. Bewahren Sie sie nicht länger auf, als Sie sie brauchen.

19.5 Speicherplatz

Das häufigste Betriebsproblem eines Docker-Servers ist eine volle Platte. Sie entsteht schleichend. Alte Images sammeln sich nach jedem Update, Protokolle wachsen, Build-Zwischenspeicher belegt Platz, und vergessene Volumes liegen herum. Wo Docker seine Daten ablegt, steht in Kapitel 3 und Kapitel 7: unter `/var/lib/docker`, bei frischen Installationen von Docker 29 die Images zusätzlich unter `/var/lib/containerd`.

Einen Überblick gibt `docker system df`.

```
$ docker system df
TYPE            TOTAL   ACTIVE   SIZE      RECLAIMABLE
Images          9       4        1.9GB     1.1GB (57%)
Containers      6       4        52MB      0B (0%)
Local Volumes   5       4        380MB     0B (0%)
Build Cache     21      0        640MB     640MB
```

Die Spalte `RECLAIMABLE` nennt, was sich freigeben ließe. Den Verbrauch einzelner Verzeichnisse zeigt `du`, und große Protokolldateien spürt `find` auf.

```
$ sudo du -sh /var/lib/docker/* 2>/dev/null | sort -h | tail -n 5
$ sudo find /var/lib/docker/containers -name "*-json.log" -size +50M
```

Zum Aufräumen taugen die `prune`-Befehle aus Kapitel 9, mit Augenmaß. Für einen Server hat sich bewährt, nur Dinge zu löschen, die ungenutzt *und* alt sind. Die Option `--filter until=` begrenzt auf ein Mindestalter.

```
$ docker image prune -a -f --filter "until=720h"
$ docker builder prune -f --filter "until=168h"
```

Der erste Befehl entfernt Images, die kein Container verwendet und die älter als 30 Tage (720 Stunden) sind. Der zweite räumt den Build-Zwischenspeicher auf, der älter als eine Woche ist. Volumes bereinigt man nicht automatisch. Die Warnung aus Kapitel 10 bleibt bestehen: Ein ungenutztes Volume kann die einzige Kopie wichtiger Daten sein.

Damit eine volle Platte nicht überrascht, prüft man den Füllstand regelmäßig. Das folgende Skript warnt, wenn ein Dateisystem von Docker eine Grenze überschreitet.

/usr/local/sbin/speicher-warnung.sh

```
#!/bin/bash
# Warnt, wenn Dateisysteme von Docker zu voll werden: speicher-
    ↪ warnung.sh [Grenze in Prozent]
GRENZE=${1:-80}
FEHLER=0
for PFAD in / /var/lib/docker /var/lib/containerd; do
  [ -d "$PFAD" ] || continue
  PROZENT=$(df --output=pcent "$PFAD" | tail -n 1 | tr -dc '0-9')
  if [ "$PROZENT" -ge "$GRENZE" ]; then
    echo "WARNUNG: $PFAD ist zu $PROZENT % belegt"; FEHLER=1
  else
    echo "OK:      $PFAD ist zu $PROZENT % belegt"
  fi
done
exit $FEHLER
```

Das Skript prüft die drei Pfade, die bei Docker wachsen können, überspringt nicht vorhandene und gibt den Füllstand aus. Mit einem Timer wie dem im Abschnitt über die Containerüberwachung lässt es sich stündlich ausführen.

19.6 Updates und Patches

Ein Docker-Server besteht aus mehreren Schichten, die verschieden aktualisiert werden: das Betriebssystem, die Docker-Software, die Images und die Anwendungen darin. Jede braucht ihren Rhythmus.

19.6.1 Das Betriebssystem

Sicherheitsupdates des Betriebssystems sollte der Server selbst einspielen. Auf Debian und Ubuntu übernimmt das das Paket `unattended-upgrades`.

```
$ sudo apt install unattended-upgrades
$ sudo dpkg-reconfigure -plow unattended-upgrades
$ sudo unattended-upgrade --dry-run --debug 2>&1 | tail -n 5
```

Die Grundeinstellung spielt Sicherheitsaktualisierungen der Distribution ein. Die Pakete aus dem Docker-Repository gehören nicht dazu. Das ist gewollt, denn ein Update von Docker startet den Daemon neu, und damit stehen die Container. Für den Kernel gilt: Er wird mit installiert, wirkt aber erst nach einem Neustart. Planen Sie dafür ein regelmäßiges Wartungsfenster ein.

19.6.2 Docker selbst

Updates der Docker-Pakete spielt man bewusst ein, zu einem Zeitpunkt, an dem ein kurzer Ausfall tragbar ist.

```
$ apt list --upgradable 2>/dev/null | grep -E "docker|containerd"
$ sudo apt update && sudo apt install --only-upgrade docker-ce docker-
    ↪ ce-cli containerd.io docker-compose-plugin
```

Wer den Ausfall vermeiden will, schaltet in der `daemon.json` die Option `"live-restore": true` ein. Dann laufen die Container weiter, während der Daemon neu startet. Die Option funktioniert nicht für alle Änderungen. Bei großen Versionssprüngen und bei bestimmten Einstellungen muss man die Container trotzdem neu starten. Lesen Sie vor jedem Hauptversionssprung die Hinweise des Herstellers.

19.6.3 Images und Anwendungen

Für die Container gilt das Vorgehen aus Kapitel 13. Aktuellere Images holen, Container neu erzeugen, prüfen. Vor jedem Update steht eine Sicherung, und danach eine Kontrolle. Eine bewährte Reihenfolge lautet:

1. Sicherung der Daten und der Konfiguration erstellen und prüfen.

2. Notieren, welche Image-Fassungen aktuell laufen (`docker compose images`).
3. Neue Fassungen auf einem Testsystem oder zumindest mit Abstand zur Veröffentlichung einspielen.
4. Images holen und Container erneuern (`docker compose pull` und `docker compose up -d`).
5. Zustand, Protokolle und die Anwendung prüfen.
6. Bei einem Fehler auf die notierten Fassungen zurückgehen.

Wie Kapitel 18 zeigt, ist ein gewisser Abstand zur Veröffentlichung dabei ein Sicherheitsgewinn. Ein Image, das erst Tage nach der Veröffentlichung eingespielt wird, hat die Chance, dass ein Fehler oder eine Manipulation bereits entdeckt wurde.

19.7 Backup-Strategien

Eine Sicherung ist die einzige Maßnahme, die vor jeder Art von Datenverlust schützt: Hardwaredefekt, Fehlbedienung, Schadsoftware, Brand. Für die Planung hilft die *3-2-1-Regel*: drei Kopien der Daten (das Original und zwei Sicherungen), auf zwei verschiedenen Arten von Datenträgern, davon eine an einem anderen Ort. Eine Sicherung auf derselben Platte wie das Original schützt nicht vor einem Plattenausfall.

Man muss festlegen, was zu sichern ist. Nicht alles ist gleich wichtig, und manches lässt sich neu herstellen.

Was	Sichern?	Weg
Daten in Volumes	ja	Archiv des Volumes (Kapitel 10)
Datenbanken	ja	Dump mit `mariadb-dump` oder `pg_dump` (Kapitel 14)
Compose-Dateien, Konfiguration	ja	Kopie oder Versionsverwaltung
`.env`, Secrets, Zertifikate, Schlüssel	ja, verschlüsselt	Archiv mit Verschlüsselung
`daemon.json`, Systemdateien, Firewall	ja	Kopie von `/etc/docker` und eigenen Dateien
Images	nein	lassen sich neu laden oder bauen, Dockerfiles sichern
Container	nein	werden aus Image und Konfiguration neu erzeugt

Das Leitprinzip: Alles, was man aus der Konfiguration neu erzeugen kann, braucht keine Sicherung. Gesichert werden die Daten und die Dateien, aus denen der Rest entsteht. Ein einfaches Sicherungsskript fasst die Teile zusammen. Hier ein Beispiel für ein Projekt mit Datenbank.

/srv/docker/sicherung.sh

```
#!/bin/bash
# Sichert Datenbank und Konfiguration eines Compose-Projekts
set -eo pipefail
```

```
PROJEKT=/srv/docker/projekt-4
ZIEL=/srv/docker/backup/projekt-4
STAMP=$(date +%Y-%m-%d_%H%M%S)
mkdir -p "$ZIEL"

cd "$PROJEKT"
docker compose exec -T db sh -c \
  'mariadb-dump -uroot -p"$MARIADB_ROOT_PASSWORD" --single-
    ↪ transaction --all-databases' \
  | gzip > "$ZIEL/db-$STAMP.sql.gz"

tar czf "$ZIEL/konfiguration-$STAMP.tar.gz" -C "$PROJEKT" \
  compose.yaml .env app nginx
chmod 600 "$ZIEL"/*-"$STAMP".*

# Sicherungen älter als 14 Tage löschen
find "$ZIEL" -type f -name "*.gz" -mtime +14 -delete
echo "Sicherung abgeschlossen: $STAMP"
```

Das Skript legt einen Dump der Datenbank und ein Archiv der Konfiguration ab, setzt die Rechte so, dass nur der Besitzer sie lesen darf, und löscht Sicherungen, die älter als 14 Tage sind. Die Dateien liegen allerdings noch auf demselben Server. Die dritte Kopie muss an einen anderen Ort, etwa auf einen zweiten Rechner.

```
$ rsync -a --delete /srv/docker/backup/ sicherung@nas.lan:/volume1/
    ↪ docker-backup/
```

Wer Sicherungen auf fremde Speicher legt, verschlüsselt sie vorher. Ein einfacher Weg ist `gpg --symmetric --cipher-algo AES256 DATEI`, das die Datei mit einem Passwort verschlüsselt. Das Passwort gehört nicht auf denselben Speicher wie die Sicherung.

Das Skript lässt sich mit einem Timer regelmäßig ausführen, wie im Abschnitt über die Containerüberwachung für das Gesundheitsskript gezeigt.

/etc/systemd/system/docker-sicherung.timer

```
[Unit]
Description=Tägliche Sicherung der Docker-Anwendungen

[Timer]
OnCalendar=*-*-* 02:30:00
Persistent=true
RandomizedDelaySec=300

[Install]
WantedBy=timers.target
```

Der zugehörige Dienst `docker-sicherung.service` sieht aus wie der des Gesundheitsskripts und führt `/srv/docker/sicherung.sh` aus. `Persistent=true` sorgt dafür, dass eine verpasste Sicherung, etwa weil der Server aus war, beim nächsten Start nachgeholt wird.

19.8 Wiederherstellungsplanung

Eine Sicherung ist erst dann etwas wert, wenn man sie wiederherstellen kann. Das klingt selbstverständlich, wird aber oft erst im Ernstfall geprüft und dann bereut. Zwei Kennzahlen helfen, den Bedarf zu klären. Der *Recovery Point Objective* (RPO) nennt, wie viele Daten man höchstens verlieren darf, gemessen in Zeit. Bei einer täglichen Sicherung sind es bis zu 24 Stunden. Der *Recovery Time Objective* (RTO) nennt, wie lange der Ausfall höchstens dauern darf. Beide folgen aus den Anforderungen des Betriebs, nicht aus der Technik. Ein Verein mit einer Webseite verträgt vermutlich einen Tag Ausfall und einen Tag Datenverlust, ein Onlineshop nicht.

Aus den Kennzahlen folgt, wie häufig man sichern muss und wie gut vorbereitet die Wiederherstellung sein muss. Für die Planung bewährt sich eine Übersicht der möglichen Ausfälle mit dem jeweiligen Weg.

Ausfall	Weg zur Wiederherstellung
Container defekt oder fehlerhaft	Container neu erzeugen (`docker compose up -d --force-recreate`), Daten bleiben im Volume
Volume beschädigt oder versehentlich gelöscht	Volume neu anlegen und Archiv einspielen
Datenbank beschädigt oder falsche Daten	Dump in eine frische Datenbank einspielen, Anwendung prüfen
Fehlerhaftes Update	Konfiguration auf den vorigen Image-Stand zurücksetzen, bei Bedarf Daten aus der Sicherung
Server ausgefallen oder zerstört	Neuinstallation von Host und Docker, Konfiguration und Daten aus der Sicherung

Der schwerste Fall ist der letzte, und für ihn braucht man ein Verfahren, das jemand ohne Vorwissen ausführen kann. Es gehört aufgeschrieben und an einem Ort abgelegt, der auch bei einem Serverausfall erreichbar ist.

1. Neues Debian-System installieren, Name und Netzwerk wie dokumentiert einrichten (Kapitel 4).
2. Docker Engine und Compose installieren (Kapitel 5), `daemon.json` aus der Sicherung einspielen.

3. Firewall und Dienste nach der Dokumentation einrichten (Kapitel 18).
4. Sicherungen vom entfernten Speicher holen und entschlüsseln.
5. Konfigurationsarchive entpacken, `.env` und Secrets einspielen.
6. Datenbanken starten und Dump einspielen, Volumes aus Archiven wiederherstellen.
7. Alle Dienste mit `docker compose up -d` starten.
8. Funktionsprüfung durchführen, die Prüfskripte der Projekte helfen dabei.

Das Wichtigste daran ist das Üben. Spielen Sie die Wiederherstellung mindestens einmal im Jahr auf einem Testsystem durch und messen Sie, wie lange sie dauert. Dann kennen Sie Ihren tatsächlichen RTO, und die Anleitung, die Sie nach dem Üben nachgebessert haben, funktioniert auch unter Druck.

> **Die Sicherung der Sicherung**
>
> Wenn der Server zerstört ist, sind auch die Sicherungen verloren, die auf ihm lagen. Das Passwort zur Entschlüsselung und die Wiederherstellungsanleitung müssen an einem anderen Ort liegen als die Sicherungen selbst. Ebenso die Zugangsdaten zu dem entfernten Speicher.

KAPITEL 20

Fehlerdiagnose

Früher oder später geht etwas schief: Ein Container startet nicht, ein Image lässt sich nicht laden, eine Anwendung erreicht ihre Datenbank nicht mehr. Wer dann ratlos Befehle ausprobiert, verliert Zeit und verschlimmert manchmal die Lage. Wer systematisch vorgeht, findet die Ursache meist in wenigen Minuten. Dieses Kapitel behandelt die häufigsten Störungen mit ihren Meldungen und Ursachen und beschreibt am Ende eine Methode, die für jede Störung funktioniert, auch für solche, die hier nicht vorkommen.

20.1 Der Container startet nicht

Ein Container, der sich sofort wieder beendet, ist die häufigste Störung. Der erste Handgriff ist immer der Blick auf Zustand und Protokoll.

```
$ docker ps -a --format "table {{.Names}}\t{{.Status}}"
NAMES    STATUS
meine-app   Exited (1) 3 seconds ago
$ docker logs meine-app
Error: DB_HOST is not set
```

Das Protokoll nennt in den meisten Fällen die Ursache im Klartext, hier eine fehlende Umgebungsvariable. Wer nichts darin findet, liest den Rückgabewert. Er verrät, *wie* der Container endete.

Wert	Bedeutung
0	Das Programm hat sich ordnungsgemäß beendet, es hatte nichts mehr zu tun
1	Allgemeiner Fehler der Anwendung, Details im Protokoll
125	Der Docker-Befehl selbst ist fehlgeschlagen, zum Beispiel wegen einer falschen Option
126	Der Befehl im Container ist nicht ausführbar (Rechte oder Format)
127	Der Befehl im Container existiert nicht (Tippfehler oder fehlendes Programm im Image)
137	Der Prozess wurde hart beendet (SIGKILL), oft wegen zu wenig Arbeitsspeicher oder nach `docker kill`
139	Der Prozess ist abgestürzt (Speicherzugriffsfehler)
143	Der Prozess wurde mit der Bitte aufzuhören beendet (SIGTERM), etwa von `docker stop`

Werte über 128 entstehen nach der Regel 128 plus Signalnummer. Aus 137 liest man ein Signal 9 heraus, aus 143 ein Signal 15. Bei 137 sollte man als Nächstes prüfen, ob der Kernel den Container wegen Speichermangels beendet hat.

```
$ docker inspect --format 'Wert={{.State.ExitCode}} OOM={{.State.
    ↪ OOMKilled}} Fehler={{.State.Error}}' meine-app
Wert=137 OOM=true Fehler=
```

Bei Rückgabewerten 126 und 127 hilft ein Blick in das Image. Man startet es mit einer Shell anstelle des vorgesehenen Programms und sucht nach der Ursache.

```
$ docker run --rm -it --entrypoint sh meine-app:1.0
/ # ls -l /app
/ # which python
```

Die Option `--entrypoint sh` ersetzt den Einstiegspunkt aus dem Dockerfile durch eine Shell. So sieht man, ob die Datei vorhanden und ausführbar ist und ob das Programm im Image existiert. Fehlt in einem schlanken Image selbst die Shell (etwa bei `scratch`-Images), hilft `docker create` mit `docker cp`, um Dateien herauszukopieren und zu untersuchen.

Typische Ursachen im Überblick: Eine Umgebungsvariable oder Konfigurationsdatei fehlt. Ein eingebundenes Verzeichnis ist leer oder hat falsche Rechte. Ein Programm im Image wurde mit dem falschen Dateiformat abgelegt, häufig mit Windows-Zeilenenden (`CRLF`) in einem Skript, was `exec format error` oder `no such file or directory` verursacht, obwohl die Datei da ist. Der Prozess läuft im Vordergrund nicht weiter, sodass der Container nach dem Start sofort endet, denn ein Container lebt nur so lange wie sein Hauptprozess.

20.2 Das Image kann nicht geladen werden

Beim Herunterladen eines Images unterscheidet man vier Arten von Ursachen: Das Image gibt es nicht, es ist nicht zugänglich, die Verbindung klappt nicht, oder lokal fehlt etwas. Die Fehlermeldung verrät, in welche Gruppe der Fall gehört.

Meldung	Ursache und Abhilfe
`pull access denied …repository does not exist or may require 'docker login'`	Tippfehler im Namen, oder das Image ist privat. Name prüfen, bei privaten Images mit `docker login` anmelden.
`manifest unknown` oder `not found`	Den Tag gibt es nicht. Die verfügbaren Tags stehen auf der Seite des Images bei Docker Hub.
`toomanyrequests`	Docker Hub begrenzt Abrufe, besonders für nicht angemeldete Nutzer. Mit `docker login` anmelden oder warten.
`no such host`, `i/o timeout`	Namensauflösung oder Netzwerk. Verbindung und DNS auf dem Host prüfen (Kapitel 4), bei Firmennetzen die Proxy-Einstellungen.
`x509: certificate signed by unknown authority` oder `has expired or is not yet valid`	Falsche Systemuhr oder ein Proxy, der Verbindungen entschlüsselt. Uhrzeit mit `timedatectl` prüfen.
`no matching manifest for linux/arm64`	Das Image gibt es nicht für die Architektur des Servers. Anderes Image wählen oder mit `--platform` testen.
`no space left on device`	Die Platte ist voll. `docker system df` und Kapitel 19.

Ein verwandter Fehler tritt erst beim Start auf: `exec format error`. Er bedeutet, dass das Programm im Image für eine andere Prozessorarchitektur gebaut wurde als die des Servers. Das passiert, wenn man ein Image auf einem Rechner mit anderer Architektur gebaut hat und auf dem Server startet. Mit `docker image inspect --format '{{.Architecture}}' NAME` sieht man, wofür das Image gebaut ist, und mit `uname -m` auf dem Server, was dort läuft.

20.3 Der Port ist bereits belegt

Die Meldung `Bind for 0.0.0.0:8080 failed: port is already allocated` oder `address already in use` bedeutet, dass der Host-Port von jemandem anderen genutzt wird. Man unterscheidet zwei Fälle. Entweder hält ein anderer

Docker-Container den Port, oder ein gewöhnliches *Programm auf dem Host*. Beide Fälle lassen sich schnell prüfen.

```
$ docker ps --filter "publish=8080" --format "{{.Names}}\t{{.Ports}}"
alt-web^^I0.0.0.0:8080->80/tcp
$ sudo ss -tlnp | grep ":8080 "
LISTEN 0 4096 0.0.0.0:8080 0.0.0.0:* users:(("docker-proxy",pid=3141,
    ↪ fd=7))
```

Die erste Abfrage listet Container, die den Port veröffentlichen. Die zweite zeigt, welches Programm auf dem Port lauscht, auch wenn es kein Container ist. Steht dort `docker-proxy`, ist es ein Container. Steht dort etwas anderes, etwa `apache2` oder `nginx`, läuft ein Dienst auf dem Host. Die Lösung hängt vom Fall ab: Den alten Container stoppen oder entfernen, den Hostdienst abschalten oder einen anderen Host-Port wählen. Bei Compose-Projekten ist ein vergessenes Projekt im Hintergrund die häufigste Ursache. `docker compose ls` zeigt alle laufenden.

20.4 Netzwerkprobleme

Bei Netzwerkstörungen hilft es, Schicht für Schicht vorzugehen, vom Einfachen zum Komplexen. Die Leitfrage lautet an jeder Stelle: Funktioniert es mit der IP-Adresse, und funktioniert es mit dem Namen? Wenn die Adresse klappt, der Name nicht, liegt es an der Namensauflösung. Wenn beides scheitert, am Netzwerk oder an einer Firewall.

Frage	Prüfung
Ist der Container im richtigen Netz?	`docker inspect --format '{{json .NetworkSettings.Networks}}' NAME` oder `docker network inspect NETZ`
Findet der Container den Namen?	`docker exec NAME getent hosts ZIEL`, `cat /etc/resolv.conf` im Container
Erreicht er die Adresse?	`docker exec NAME ping -c 1 ADRESSE` (wo `ping` existiert), `wget -q -O- http://ADRESSE:PORT`
Lauscht der Dienst auf dem Port?	`docker exec NAME netstat -tln` oder `ss -tln` (wo vorhanden), `docker logs NAME`
Erreicht der Host den Container?	`curl -v http://localhost:PORT` auf dem Server, `docker port NAME`
Blockt eine Firewall?	`sudo iptables -L DOCKER-USER -n`, `sudo ufw status`, Test von einem zweiten Rechner

Einige Fälle treten besonders oft auf. Ein Dienst im Container lauscht nur auf `127.0.0.1` statt auf `0.0.0.0`. Dann erreicht ihn zwar der Container selbst, aber kein anderer, und eine Portfreigabe führt ins Leere. Die Anwendung muss auf allen Adressen lauschen. Eine zweite Ursache: Die Container hängen in verschiedenen Netzen und sehen sich nicht (Kapitel 11). Eine dritte: Das Standardnetz hat keine Namensauflösung. Und die vierte betrifft Docker selbst. Die IP-Weiterleitung muss im Kernel eingeschaltet sein, damit Container ins Netz kommen. Docker erledigt das normalerweise selbst.

```
$ sysctl net.ipv4.ip_forward
net.ipv4.ip_forward = 1
```

Steht dort `0`, hat ein anderes Programm die Weiterleitung abgeschaltet. Das geschieht gelegentlich bei Firewall-Werkzeugen oder Systemeinstellungen.

Container sind oft so schlank, dass `ping`, `curl` und `ss` fehlen. Dann hilft ein Diagnose-Container, der sich in das Netzwerk eines laufenden Containers einklinkt. Dazu eignet sich das Image `nicolaka/netshoot`, das viele Werkzeuge mitbringt.

```
$ docker run --rm -it --network container:projekt4-app-1 nicolaka/
    ↪ netshoot
```

Die Option `--network container:NAME` setzt den neuen Container in dieselbe Netzwerkumgebung wie den genannten. Er sieht dieselben Schnittstellen, Adressen und Ports. Verwenden Sie auch hier die Vorsichtsregeln aus Kapitel 18: Legen Sie den Tag fest, verwenden Sie nur Images aus vertrauenswürdiger Quelle, und binden Sie keinen Docker-Socket ein.

20.5 Berechtigungsfehler

`Permission denied` und `Operation not permitted` gehören zu den häufigsten Meldungen. Sie haben verschiedene Ursachen, und die Meldung verrät sie meist, wenn man auf den Pfad achtet.

Betrifft die Meldung den Docker-Socket (`permission denied while trying to connect to the Docker daemon socket`), gehört der Benutzer nicht zur Gruppe `docker`, oder die Anmeldung wurde nach der Aufnahme nicht erneuert (Kapitel 5). Betrifft sie eine Datei oder ein Verzeichnis im Container, stimmen meist die Benutzernummern nicht überein. Die Regel dazu steht in Kapitel 10: Für den Kernel zählt die Nummer, nicht der Name. Um sie zu klären, vergleicht man die Nummer des Prozesses mit der des Besitzers.

```
$ docker exec meine-app id
uid=10001(app) gid=10001(app) groups=10001(app)
$ docker exec meine-app ls -ln /data
drwxr-xr-x 2 0 0 4096 Nov 11 10:00 /data
```

Hier läuft der Prozess als Benutzer 10001, das Verzeichnis `/data` gehört aber Root (Nummer 0) und erlaubt anderen nur Lesen. Das Schreiben scheitert. Abhilfe schafft, dem Verzeichnis den richtigen Besitzer zu geben, beim Bau des Images mit `chown` vor `USER`, bei Volumes nachträglich über einen Hilfscontainer (Kapitel 10).

`Operation not permitted` bei Befehlen wie `chown` oder beim Binden an einen Port unter 1024 deutet auf entfernte Capabilities hin. Mit `--cap-drop ALL` fehlen dem Prozess die dazugehörigen Rechte (Kapitel 18). Man fügt gezielt die benötigte hinzu, zum Beispiel `CHOWN` oder `NET_BIND_SERVICE`. Bei schreibgeschützten Dateisystemen lautet die Meldung `Read-only file system`, und der Pfad, den sie nennt, braucht ein `tmpfs` oder ein Volume.

> **Nicht mit chmod 777 und Root-Betrieb zudecken**
>
> Die schnelle Lösung vieler Anleitungen für Berechtigungsfehler lautet: `chmod -R 777` oder den Container als Root laufen lassen. Beides beseitigt die Meldung und öffnet gleichzeitig die Tür, die man eigentlich geschlossen halten wollte. Suchen Sie stattdessen die Ursache und geben Sie genau die Rechte, die nötig sind.

20.6 Volume-Probleme

Bei Volumes treten einige typische Fehler auf. Der erste ist ein leerer Datenbestand, wo Daten sein sollten. Meist hat der Container ein neues, anonymes Volume benutzt, statt des benannten. Ursachen sind ein Tippfehler im Volume-Namen, ein falscher Pfad im Container, oder bei PostgreSQL 18 der geänderte Datenpfad (Kapitel 14). Mit `docker volume ls` und `docker inspect` auf dem Container prüft man, welches Volume wo eingebunden ist.

```
$ docker volume ls
$ docker inspect --format '{{range .Mounts}}{{.Type}} {{.Name}} -> {{.
    ↪ Destination}}{{"\n"}}{{end}}' meine-db
```

Der zweite Fehler tritt beim Löschen auf: `volume is in use`. Ein Container, auch ein gestoppter, verwendet das Volume noch. `docker ps -a --filter`

`volume=NAME` nennt ihn. Der dritte betrifft Bind Mounts: Die Meldung `bind source path does not exist` zeigt an, dass das Verzeichnis auf dem Host fehlt, weil ein Tippfehler im Pfad steckt. Bei der Kurzschreibweise `-v` legt Docker das Verzeichnis dagegen leise selbst an, mit Root als Besitzer, wie in Kapitel 10 beschrieben. Der vierte betrifft den Platz: Ein volles Dateisystem lässt Datenbanken in Fehlermeldungen laufen. Der fünfte kommt nach Wiederherstellungen: Die Dateien im Volume haben nach dem Zurückspielen andere Besitzer, als die Anwendung erwartet. Das Anpassen mit `chown` im Hilfscontainer behebt es.

20.7 Datenbankverbindungsfehler

Die Fehlermeldungen von Datenbanken und ihre Ursachen hat Kapitel 14 in einer Tabelle gesammelt. In der Praxis hilft eine feste Prüfreihenfolge, die man von außen nach innen durchgeht.

Zuerst: Läuft die Datenbank und ist sie gesund (`docker compose ps`, `docker compose logs db`)? Dann: Sind Anwendung und Datenbank im selben Netz, und stimmt der Rechnername (`docker compose exec app getent hosts db`)? Dann: Antwortet die Datenbank auf dem Port, wenn man sie von einem zweiten Container anspricht (Test mit dem Client-Container aus Kapitel 14)? Schließlich: Stimmen Benutzer, Passwort und Datenbankname mit dem überein, was beim *ersten Start* eingerichtet wurde? Der letzte Punkt ist die häufigste Ursache bei `Access denied`. Wer das Passwort in der `.env` nachträglich ändert, ändert nicht das der Datenbank.

```
$ docker compose exec db sh -c 'mariadb -uroot -p"
    ↪ $MARIADB_ROOT_PASSWORD" -e "SELECT user, host FROM mysql.user
    ↪ "'
```

Dieser Befehl zeigt, welche Benutzer die Datenbank tatsächlich kennt. Fehlt der erwartete oder steht ein anderer Rechnername dahinter (`localhost` statt `%`), liegt der Fehler hier.

20.8 Der Docker-Daemon startet nicht

Startet der Daemon nicht, geht nichts mehr: Jeder `docker`-Befehl meldet, dass er den Daemon nicht erreicht. Die Diagnose beginnt beim Dienst.

```
$ sudo systemctl status docker --no-pager
$ sudo journalctl -u docker -b --no-pager | tail -n 30
```

Die Statusanzeige nennt, ob der Dienst fehlgeschlagen ist, und das Journal enthält die Fehlermeldung des Daemons. Die häufigste Ursache nach Änderungen ist ein Fehler in der Konfigurationsdatei. Man erkennt sie an Meldungen wie `unable to configure the Docker daemon with file /etc/docker/daemon.json`. Ob die Datei gültig ist, lässt sich prüfen, ohne den Daemon zu starten.

```
$ sudo dockerd --validate --config-file=/etc/docker/daemon.json
configuration OK
```

Bei einem Fehler nennt die Ausgabe die Stelle, meist ein fehlendes Komma oder eine fehlende Klammer. Zur Sicherheit legt man vor Änderungen eine Kopie der Datei an. Im Notfall verschiebt man die fehlerhafte Datei und startet den Daemon ohne sie.

```
$ sudo mv /etc/docker/daemon.json /etc/docker/daemon.json.defekt
$ sudo systemctl restart docker
```

Weitere Ursachen: Das Dateisystem ist voll, und der Daemon kann seine Datenbanken nicht schreiben (`df -h`). Rechte oder Besitzer von `/var/lib/docker` stimmen nicht. Ein anderes Programm blockiert den Socket oder Ports. Die Firewall-Werkzeuge widersprechen der Einstellung `iptables`, die Docker verwendet. Oder der Dienst ist maskiert (`Unit docker.service is masked`), dann hilft `sudo systemctl unmask docker.service docker.socket`. Nach einer Aktualisierung kann auch eine geänderte Hauptversion eine bisher akzeptierte Einstellung ablehnen. Das Journal ist dann die entscheidende Quelle.

20.9 Systematische Diagnosemethoden

Alle Fehlerbilder dieses Kapitels folgen demselben Ablauf, und der lässt sich als Methode festhalten. Er besteht aus sechs Schritten.

1. *Das Symptom genau beschreiben.* Was geht nicht? Seit wann? Was wurde zuletzt geändert? Eine präzise Beschreibung („der Container `app` beendet sich mit Wert 1 seit dem Update von gestern“) grenzt die Ursache stärker ein als jede Vermutung.
2. *Beweise sichern.* Bevor man etwas neu startet oder löscht, sichert man Protokolle und Zustand. Ein Neustart kann die entscheidende Meldung zerstören.
3. *Von außen nach innen prüfen.* Host (Platz, Speicher, Netzwerk), Daemon (läuft er, Journal), Image, Container (Zustand, Protokoll), Netzwerk und Volumes, zuletzt die Anwendung selbst.
4. *Eine Vermutung aufstellen und genau einen Test durchführen.* Der Test sollte die Vermutung bestätigen oder widerlegen. Man ändert nie mehrere Dinge gleichzeitig, sonst weiß man nicht, was gewirkt hat.
5. *Funktionierendes mit Nicht-Funktionierendem vergleichen.* Läuft derselbe Container auf einem anderen Server, in einem anderen Netz oder mit dem Vorjahres-Image, ist der Unterschied die Spur.
6. *Festhalten, was die Ursache war.* Eine kurze Notiz mit Symptom, Ursache und Lösung spart beim nächsten Mal die Hälfte der Zeit.

Für Schritt 2 hilft ein Skript, das alle wichtigen Angaben in eine Datei schreibt. Das folgende sammelt Systemzustand, Docker-Zustand, Speicherplatz, Netze, Journal und auf Wunsch Zustand und Protokoll eines Containers.

/usr/local/sbin/docker-diagnose.sh

```
#!/bin/bash
# Sammelt Diagnoseinformationen in einer Datei: docker-diagnose.sh [
    ↪ Containername]
ZIEL=/tmp/docker-diagnose-$(date +%Y-%m-%d_%H%M%S).txt
NAME=${1:-}
abschnitt() { echo; echo "===== $1 ====="; }
{
  abschnitt "Zeitpunkt und System"
  date; uname -a; uptime
```

```
  abschnitt "Docker-Versionen"
  docker version
  abschnitt "Docker-Dienst"
  systemctl status docker --no-pager | head -n 15
  abschnitt "Container"
  docker ps -a --format "table {{.Names}}\t{{.Image}}\t{{.Status}}"
  abschnitt "Speicherplatz"
  df -h / /var/lib/docker 2>/dev/null
  docker system df
  abschnitt "Netzwerke"
  docker network ls
  abschnitt "Letzte Meldungen des Daemons"
  journalctl -u docker --since "1 hour ago" --no-pager | tail -n 30
  if [ -n "$NAME" ]; then
    abschnitt "Container $NAME: Zustand"
    docker inspect --format 'Status={{.State.Status}} ExitCode={{.
    ↪ State.ExitCode}} OOM={{.State.OOMKilled}} Neustarts={{.
    ↪ RestartCount}} Fehler={{.State.Error}}' "$NAME"
    abschnitt "Container $NAME: letzte Protokollzeilen"
    docker logs --tail 50 "$NAME" 2>&1
  fi
} > "$ZIEL" 2>&1
echo "Diagnose gespeichert: $ZIEL"
```

Der Aufruf ist einfach, und die Datei lässt sich bei Bedarf einer zweiten Person schicken. Sie enthält keine Passwörter, aber Namen von Containern und Images. Prüfen Sie vor dem Weitergeben, was darin steht.

```
$ sudo docker-diagnose.sh projekt4-app-1
Diagnose gespeichert: /tmp/docker-diagnose-2026-11-11_183220.txt
```

Zur Orientierung fasst die Tabelle zusammen, mit welchem Befehl man welche Frage beantwortet.

Frage	Befehl
Was läuft, was ist beendet?	`docker ps -a`
Warum ist ein Container beendet?	`docker logs`, `docker inspect --format '{{.State.ExitCode}}'`
Wie ist ein Container konfiguriert?	`docker inspect`
Was hat sich im Container verändert?	`docker diff`
Was verbraucht Ressourcen?	`docker stats`, `docker top`
Wie voll ist die Platte?	`docker system df`, `df -h`
Was ist im Netz los?	`docker network inspect`, `docker port`, `ss -tlnp`
Was meldet der Daemon?	`journalctl -u docker`
Was ist gerade passiert?	`docker events --since 1h`
Ist eine Konfiguration gültig?	`docker compose config`, `nginx -t`, `dockerd --validate`

Praxisprojekt 6: Sicherer Docker-Server

In diesem Projekt sichern Sie den Übungsserver ab. Sie prüfen zuerst, wo er angreifbar ist, härten dann Host und Firewall, betreiben einen Webserver-Container mit minimalen Rechten und weisen am Ende mit einem Prüfskript nach, dass die Maßnahmen wirken. Das Projekt setzt Kapitel 18 in die Praxis um. Planen Sie etwa zweieinhalb Stunden ein, und arbeiten Sie bei den Schritten zu SSH und Firewall mit besonderer Sorgfalt: Ein Fehler dort kann Sie vom Server aussperren.

> **Zugang zur Konsole sicherstellen**
>
> Bevor Sie Änderungen an SSH oder der Firewall vornehmen, vergewissern Sie sich, dass Sie den Server notfalls auch ohne Netzwerk erreichen, also über die Konsole der virtuellen Maschine oder einen Bildschirm am Rechner. Behalten Sie bei jeder Änderung eine bestehende SSH-Sitzung offen und testen Sie die neue Einstellung in einer zweiten Sitzung, bevor Sie die erste schließen.

Ausgangslage und Ziel

Bereich	Sollzustand
Benutzerrechte	nur `admin` ist in der Gruppe `docker`, Anmeldung per SSH nur mit Schlüssel, kein Root-Login
Firewall	`ufw` blockiert eingehenden Verkehr, SSH nur aus dem Heimnetz; die Kette `DOCKER-USER` lässt Zugriffe auf Container nur aus dem Heimnetz zu und bleibt über einen Neustart erhalten
Container	Webserver ohne Root, mit schreibgeschütztem Dateisystem, ohne Capabilities, mit Limits
Prüfung	Skript `audit.sh` meldet für den gehärteten Container keine Warnungen

Schritt 1: Bestandsaufnahme

Das Skript `audit.sh` prüft laufende Container auf die Punkte aus Kapitel 18. Legen Sie es an und starten Sie es einmal gegen den Zustand, den Sie von den bisherigen Projekten haben.

```
$ mkdir -p /srv/docker/projekt-6/html && cd /srv/docker/projekt-6
```

/srv/docker/projekt-6/audit.sh

```
#!/bin/bash
# Sicherheitsprüfung für laufende Container: audit.sh [Namensteil]
BEFUNDE=0
FILTER=()
[ -n "$1" ] && FILTER=(--filter "name=$1")
befund()  { echo "   [WARNUNG] $1"; BEFUNDE=$((BEFUNDE + 1)); }
hinweis() { echo "   [HINWEIS] $1"; }
wert()    { docker inspect --format "$1" "$ID"; }

for ID in $(docker ps -q "${FILTER[@]}"); do
```

```
NAME=$(wert '{{.Name}}'); NAME=${NAME#/}
echo "Container $NAME"

[ "$(wert '{{.HostConfig.Privileged}}')" = "true" ] && befund "
   ↪ läuft privilegiert"

case "$(wert '{{.Config.User}}')" in
  ""|0|0:*|root|root:*) befund "läuft als Root (kein Benutzer
   ↪ gesetzt)" ;;
esac

[ "$(wert '{{.HostConfig.ReadonlyRootfs}}')" = "true" ] \
  || befund "Dateisystem ist beschreibbar"
wert '{{.HostConfig.CapDrop}}' | grep -q "ALL" \
  || befund "Capabilities sind nicht mit ALL entfernt"
wert '{{.HostConfig.SecurityOpt}}' | grep -q "no-new-privileges" \
  || befund "no-new-privileges fehlt"
[ "$(wert '{{.HostConfig.Memory}}')" = "0" ] \
  && befund "kein Speicherlimit gesetzt"
case "$(wert '{{.HostConfig.PidsLimit}}')" in
  "<nil>"|0|-1) befund "kein Limit für Prozesse gesetzt" ;;
esac
wert '{{range .Mounts}}{{.Source}} {{end}}' | grep -q "docker.sock"
   ↪  \
  && befund "bindet den Docker-Socket ein"

BILD=$(wert '{{.Config.Image}}')
if [[ "$BILD" == *@sha256:* ]]; then
  :
elif [[ "$BILD" =~ :([^/:]+)$ ]]; then
  [ "${BASH_REMATCH[1]}" = "latest" ] && befund "Image $BILD
   ↪ verwendet den Tag latest"
else
  befund "Image $BILD hat keinen Tag (gilt als latest)"
fi

case "$(wert '{{.HostConfig.RestartPolicy.Name}}')" in
  ""|no) hinweis "keine Neustartregel gesetzt" ;;
```

```
  esac

  OFFEN=$(wert '{{range $p, $c := .HostConfig.PortBindings}}{{range
    ↪ $c}}{{.HostIp}}:{{.HostPort}} {{end}}{{end}}')
  for PORT in $OFFEN; do
    case "$PORT" in
      :*|0.0.0.0:*) hinweis "Port ${PORT##*:} ist auf allen Adressen
    ↪ des Hosts erreichbar" ;;
    esac
  done
done

echo
echo "Befunde: $BEFUNDE"
[ "$BEFUNDE" -eq 0 ] && echo "Keine Warnungen."
exit $([ "$BEFUNDE" -eq 0 ] && echo 0 || echo 1)
```

```
$ chmod +x audit.sh && ./audit.sh
Container projekt5-proxy-1
   [WARNUNG] läuft als Root (kein Benutzer gesetzt)
   [WARNUNG] Dateisystem ist beschreibbar
   ...
Befunde: 24
```

Die Befunde sind nicht dramatisch. Die Container der früheren Projekte waren auf Lernen und Funktion ausgelegt, nicht auf Sicherheit. Das Skript liest mit `docker inspect` die Einstellungen jedes laufenden Containers aus und meldet Abweichungen vom Kapitel 18: privilegiert, Root, beschreibbares Dateisystem, nicht entfernte Capabilities, fehlende Optionen und Limits, eingebundener Docker-Socket, fehlender oder fester Tag. Hinweise (nicht Warnungen) betreffen Ports, die auf allen Adressen erreichbar sind, und fehlende Neustartregeln. Die genaue Zahl der Befunde hängt davon ab, welche Projekte laufen.

Schritt 2: Benutzerrechte und SSH

Prüfen Sie, wer Docker bedienen darf, und wer sich anmelden darf.

```
$ getent group docker
docker:x:998:admin
$ sudo sshd -T | grep -E "^(permitrootlogin|passwordauthentication|
    ↪ pubkeyauthentication) "
permitrootlogin without-password
passwordauthentication yes
pubkeyauthentication yes
```

Die Gruppe `docker` sollte nur Personen enthalten, die Root-Rechte haben dürfen (Kapitel 18). Bei der SSH-Konfiguration sind drei Dinge zu ändern: den Login als Root verbieten, die Anmeldung mit Passwort abschalten und nur den Benutzer `admin` zulassen. Das Abschalten der Passwortanmeldung setzt voraus, dass Sie einen SSH-Schlüssel hinterlegt haben. Erzeugen Sie ihn *auf Ihrem Arbeitsrechner* und übertragen Sie ihn.

```
$ ssh-keygen -t ed25519 -C "admin@arbeitsplatz"
$ ssh-copy-id admin@docker01
$ ssh admin@docker01 "echo Anmeldung mit Schlüssel funktioniert"
Anmeldung mit Schlüssel funktioniert
```

Erst wenn diese letzte Zeile ohne Passwortabfrage erscheint, ändern Sie die Konfiguration auf dem Server. Debian liest zusätzliche Einstellungen aus dem Verzeichnis `/etc/ssh/sshd\_config.d/`, das Sie nutzen, statt die Hauptdatei zu ändern.

/etc/ssh/sshd_config.d/10-haertung.conf

```
PermitRootLogin no
PasswordAuthentication no
PubkeyAuthentication yes
AllowUsers admin
```

```
$ sudo sshd -t && echo "Konfiguration ist gültig"
Konfiguration ist gültig
$ sudo systemctl reload ssh
```

Der Test `sshd -t` prüft die Datei, ohne etwas zu ändern. Das `reload` lädt die Konfiguration, ohne bestehende Sitzungen zu beenden. Öffnen Sie nun *in einem zweiten Fenster* eine neue SSH-Verbindung. Gelingt sie, ist alles in Ordnung. Misslingt sie, haben Sie noch die erste Sitzung, mit der Sie die Datei korrigieren können.

Schritt 3: Firewall

Die Firewall hat zwei Teile. `ufw` schützt den Host selbst. Die Kette `DOCKER-USER` schützt die Container, die Docker unter Umgehung von `ufw` nach außen führt (Kapitel 11).

ufw für den Host

```
$ sudo apt install ufw
$ sudo ufw default deny incoming
$ sudo ufw default allow outgoing
$ sudo ufw allow from 192.168.100.0/24 to any port 22 proto tcp
$ sudo ufw enable
Command may disrupt existing ssh connections. Proceed with operation
    ↪ (y|n)? y
$ sudo ufw status verbose
```

Die Regeln sperren alles Eingehende, außer SSH aus dem Heimnetz. Ausgehend ist alles erlaubt, damit der Server Updates und Images laden kann. Die Regel für SSH steht vor dem Einschalten, damit Ihre Sitzung nicht abreißt. Passen Sie das Netz `192.168.100.0/24` an Ihr Heimnetz an.

DOCKER-USER dauerhaft

Die Regeln in `DOCKER-USER` aus Kapitel 11 überstehen keinen Neustart. Damit sie dauerhaft gelten, legen Sie ein Skript und einen kleinen Dienst an, der es nach dem Start von Docker ausführt.

/usr/local/sbin/docker-user-regeln.sh

```
#!/bin/bash
# Setzt eigene Firewall-Regeln in der Kette DOCKER-USER:
# Zugriffe auf Container sind nur aus dem Heimnetz erlaubt.
SCHNITTSTELLE=enp0s3
HEIMNETZ=192.168.100.0/24

iptables -F DOCKER-USER
iptables -A DOCKER-USER -m conntrack --ctstate ESTABLISHED,RELATED -j
    ↪ ACCEPT
iptables -A DOCKER-USER -i "$SCHNITTSTELLE" ! -s "$HEIMNETZ" -j DROP
iptables -A DOCKER-USER -j RETURN
```

Das Skript leert zuerst die Kette, damit es sich beliebig oft ausführen lässt. Danach erlaubt es Pakete bestehender Verbindungen, verwirft alles, was über die Netzwerkkarte hereinkommt und nicht aus dem Heimnetz stammt, und endet mit einer Rückkehr-Regel, damit Docker seine eigenen Regeln danach anwenden kann. Den Namen der Netzwerkkarte kennen Sie aus Kapitel 4.

/etc/systemd/system/docker-user-firewall.service

```
[Unit]
Description=Eigene Firewall-Regeln in der Kette DOCKER-USER
After=docker.service
Requires=docker.service
PartOf=docker.service

[Service]
Type=oneshot
RemainAfterExit=yes
ExecStart=/usr/local/sbin/docker-user-regeln.sh

[Install]
```

```
WantedBy=multi-user.target
```

```
$ sudo chmod 750 /usr/local/sbin/docker-user-regeln.sh
$ sudo systemctl daemon-reload
$ sudo systemctl enable --now docker-user-firewall.service
$ sudo iptables -L DOCKER-USER -n --line-numbers
Chain DOCKER-USER (1 references)
num  target  prot opt source        destination
1    ACCEPT  0    --  0.0.0.0/0     0.0.0.0/0     ctstate RELATED,
     ↪ ESTABLISHED
2    DROP    0    --  !192.168.100.0/24 0.0.0.0/0
3    RETURN  0    --  0.0.0.0/0     0.0.0.0/0
```

Die Zeile `PartOf=docker.service` bewirkt, dass der Dienst bei jedem Neustart von Docker mit neu gestartet wird und die Regeln erneut setzt. Prüfen Sie den Schutz von einem zweiten Rechner *außerhalb* des Heimnetzes, wenn Sie die Möglichkeit haben. Ein Container-Port, der aus dem Heimnetz erreichbar ist, darf von dort nicht antworten.

Schritt 4: Der gehärtete Webserver

Jetzt folgt der Container, der alle Punkte des Kapitels 18 erfüllt. Als Image dient die Variante von nginx, die ohne Root-Rechte läuft. Sie lauscht auf Port 8080 und legt Prozesskennung und Zwischendateien unter `/tmp` ab.

html/index.html

```
<!DOCTYPE html>
<html lang="de">
<head><meta charset="utf-8"><title>Gehärteter Webserver</title></head
     ↪ >
<body><h1>Gehärteter Webserver</h1><p>Läuft ohne Root,
     ↪ schreibgeschützt und mit Limits.</p></body>
</html>
```

compose.yaml

```
name: projekt6

services:
  web:
    image: nginxinc/nginx-unprivileged:stable-alpine
    restart: unless-stopped
    ports:
      - "8081:8080"
    volumes:
      - ./html:/usr/share/nginx/html:ro
    read_only: true
    tmpfs:
      - /tmp
    cap_drop:
      - ALL
    security_opt:
      - no-new-privileges:true
    mem_limit: 128m
    cpus: 0.5
    pids_limit: 100
    healthcheck:
      test: ["CMD", "wget", "-q", "--spider", "http
    ↪ ://127.0.0.1:8080/"]
      interval: 15s
      timeout: 3s
      retries: 3
      start_period: 5s
```

Jede Einstellung hat ihren Abschnitt im Kapitel 18. Das Image läuft ohne Root. Das Wurzeldateisystem ist schreibgeschützt, und nur `/tmp` steht im Arbeitsspeicher zur Verfügung. Alle Capabilities sind entfernt, ohne dass etwas hinzugefügt wird, denn der Prozess belegt nur einen Port über 1024. `no-new-privileges` verhindert nachträglichen Rechtegewinn. Speicher, Prozessorzeit und Prozesszahl sind begrenzt. Die Webseite ist schreibgeschützt eingebunden.

```
$ chmod -R a+rX html
$ docker compose up -d
$ docker compose ps --format "table {{.Service}}\t{{.Status}}\t{{.
    ↪ Ports}}"
SERVICE   STATUS                    PORTS
web       Up 20 seconds (healthy)   0.0.0.0:8081->8080/tcp,
    ↪ [::]:8081->8080/tcp
$ curl -s http://localhost:8081 | grep h1
<body><h1>Gehärteter Webserver</h1><p>Läuft ohne Root,
    ↪ schreibgeschützt und mit Limits.</p></body>
```

Wie es nach der Härtung aussieht, lässt sich am Container selbst nachprüfen.

```
$ docker compose exec web id
uid=101(nginx) gid=101(nginx) groups=101(nginx)
$ docker compose exec web touch /usr/share/nginx/html/test
touch: /usr/share/nginx/html/test: Read-only file system
$ docker compose exec web sh -c "grep CapEff /proc/1/status"
CapEff:^^I0000000000000000
$ docker inspect --format '{{.HostConfig.Memory}}' projekt6-web-1
134217728
```

Der Prozess läuft als Benutzer `nginx`, die Webseite lässt sich nicht verändern, es sind keine Capabilities wirksam, und das Speicherlimit steht. Das Image und seine Benutzernummer können sich von Ausgabe zu Ausgabe unterscheiden. Entscheidend ist, dass `id` nicht `uid=0` zeigt.

Schritt 5: Image prüfen

Zum Abschluss prüfen Sie das verwendete Image auf bekannte Schwachstellen, mit den Vorsichtsmaßnahmen aus Kapitel 18: festgelegte Version des Werkzeugs und kein Docker-Socket.

```
$ TRIVY_VERSION=0.70.0
```

```
$ docker save nginxinc/nginx-unprivileged:stable-alpine -o /tmp/nginx-
    ↪ unpriv.tar
$ docker run --rm -v /tmp:/work aquasec/trivy:$TRIVY_VERSION \
    image --input /work/nginx-unpriv.tar --severity HIGH,CRITICAL
$ rm /tmp/nginx-unpriv.tar
```

Lesen Sie die Ausgabe wie in Kapitel 18 beschrieben. Prüfen Sie vorher, welche Fassung von Trivy aktuell empfohlen wird. Treffer, für die eine behobene Version genannt wird, lösen Sie durch ein Update des Images.

Schritt 6: Sicherheitsprüfung und Abnahme

Führen Sie die Prüfung für den gehärteten Container und für den Host durch. Das Skript fasst beides zusammen und braucht Root-Rechte, weil es Firewall und SSH-Einstellungen liest.

/srv/docker/projekt-6/check.sh

```
#!/bin/bash
# Funktionsprüfung für Projekt 6: Sicherer Docker-Server (mit sudo
    ↪ ausführen)
cd /srv/docker/projekt-6 || exit 1
FEHLER=0
ok()   { echo "[ OK ]     $1"; }
fail() { echo "[FEHLER]   $1"; FEHLER=1; }

./audit.sh projekt6 >/dev/null \
  && ok "audit.sh meldet für den Webserver keine Warnungen" || fail "
    ↪ audit.sh meldet Warnungen"
[ "$(docker compose exec -T web id -u)" != "0" ] \
  && ok "Webserver läuft nicht als Root" || fail "Webserver läuft als
    ↪  Root"
docker compose exec -T web touch /usr/share/nginx/html/test >/dev/
    ↪ null 2>&1 \
  && fail "Webseite ist beschreibbar" || ok "Webseite ist
    ↪ schreibgeschützt"
```

```
[ "$(docker compose exec -T web sh -c "grep CapEff /proc/1/status" |
    ↪ awk '{print $2}')" = "0000000000000000" ] \
  && ok "Keine Capabilities wirksam" || fail "Capabilities sind
    ↪ wirksam"
[ "$(curl -s -o /dev/null -w '%{http_code}' http://localhost:8081/)"
    ↪ = "200" ] \
  && ok "Webserver antwortet" || fail "Webserver antwortet nicht"

[ "$(getent group docker | cut -d: -f4)" = "admin" ] \
  && ok "Nur admin ist in der Gruppe docker" || fail "Gruppe docker
    ↪ enthält weitere Mitglieder"
sshd -T | grep -q "^permitrootlogin no" \
  && ok "SSH: Root-Login verboten" || fail "SSH: Root-Login ist
    ↪ erlaubt"
sshd -T | grep -q "^passwordauthentication no" \
  && ok "SSH: Passwortanmeldung abgeschaltet" || fail "SSH:
    ↪ Passwortanmeldung ist erlaubt"
ufw status | grep -q "^Status: active" \
  && ok "ufw ist aktiv" || fail "ufw ist nicht aktiv"
iptables -L DOCKER-USER -n | grep -q "DROP" \
  && ok "DOCKER-USER enthält die eigene Regel" || fail "DOCKER-USER
    ↪ enthält keine Regel"
systemctl is-enabled --quiet docker-user-firewall.service \
  && ok "Die Regeln überstehen einen Neustart" || fail "Der Dienst
    ↪ für die Regeln ist nicht aktiv"

echo
[ "$FEHLER" -eq 0 ] && echo "Alle Prüfungen bestanden." \
  || echo "Mindestens eine Prüfung ist fehlgeschlagen."
exit $FEHLER
```

```
$ chmod +x check.sh && sudo ./check.sh
[ OK ]     audit.sh meldet für den Webserver keine Warnungen
[ OK ]     Webserver läuft nicht als Root
[ OK ]     Webseite ist schreibgeschützt
[ OK ]     Keine Capabilities wirksam
[ OK ]     Webserver antwortet
```

```
[ OK ]     Nur admin ist in der Gruppe docker
[ OK ]     SSH: Root-Login verboten
[ OK ]     SSH: Passwortanmeldung abgeschaltet
[ OK ]     ufw ist aktiv
[ OK ]     DOCKER-USER enthält die eigene Regel
[ OK ]     Die Regeln überstehen einen Neustart

Alle Prüfungen bestanden.
```

Zur Abnahme gehört ein Neustart des Servers. Danach muss `iptables -L DOCKER-USER -n` die Regeln weiterhin zeigen, der Webserver muss laufen, und die SSH-Anmeldung muss mit Schlüssel funktionieren. Die Wirkung der Firewall bestätigt ein Zugriff auf `http://192.168.100.10:8081` von einem Rechner im Heimnetz (antwortet) und, wenn möglich, von einem Rechner außerhalb des Heimnetzes (antwortet nicht).

Wenn etwas nicht klappt

Symptom	Ursache und Abhilfe
Die SSH-Anmeldung mit Schlüssel scheitert nach der Härtung	Schlüssel nicht hinterlegt oder Rechte von `~/.ssh` falsch (`700` für das Verzeichnis, `600` für `authorized\_keys`). Über die Konsole die Datei `10-haertung.conf` entfernen und neu laden.
Container erreichen das Internet nicht mehr	Die Regel für bestehende Verbindungen fehlt oder steht nicht vor dem `DROP`. Reihenfolge mit `iptables -L DOCKER-USER -n --line-numbers` prüfen.
Der Container startet nicht, Meldung `Read-only file system`	Ein Verzeichnis braucht Schreibzugriff. Pfad aus dem Protokoll als `tmpfs` ergänzen.
`403` beim Aufruf der Seite	Dateien in `html` sind nicht lesbar. `chmod -R a+rX html` setzt die Rechte.
Der Dienst `docker-user-firewall` ist fehlgeschlagen	Name der Netzwerkkarte im Skript stimmt nicht, oder Docker läuft nicht. `journalctl -u docker-user-firewall` nennt die Meldung.
Das Image `nginx-unprivileged` lässt sich nicht laden	Den Tag auf Docker Hub prüfen (Kapitel 20) und einen vorhandenen verwenden.

Praxisprojekt 7: Vollständige Serverinfrastruktur

Dieses Abschlussprojekt führt alles zusammen. Sie bauen eine produktionsnahe Infrastruktur: ein Debian-System mit Docker und Compose, einen Reverse Proxy mit TLS, mehrere Anwendungscontainer, eine Datenbank mit Secrets und Volume, getrennte interne Netzwerke, ein Backup-Konzept, ein Sicherheitskonzept und eine technische Dokumentation. Das Projekt verwendet Bausteine aus allen früheren Kapiteln und Projekten. Planen Sie etwa vier Stunden ein, am besten verteilt auf zwei Termine.

Ausgangslage und Ziel

Ein kleiner Verein möchte zwei Anwendungen auf einem eigenen Server betreiben: seine Webseite und einen Besucherzähler mit Datenbank. Der Server soll nach Stand der Technik abgesichert sein, regelmäßig gesichert werden und so dokumentiert sein, dass ihn eine andere Person betreuen oder neu aufbauen kann. Der Sollzustand im Überblick:

Baustein	Umsetzung
Host	Debian 13 (Hostname `docker01`), aktuelle Updates, SSH nur mit Schlüssel, `ufw` und `DOCKER-USER` aus Projekt 6
Container-Plattform	Docker Engine und Compose-Plugin aus dem offiziellen Repository
Reverse Proxy	`proxy` (nginx ohne Root), Ports 80 und 443, TLS mit selbstsigniertem Zertifikat, Umleitung auf HTTPS
Anwendungen	`verein` (statische Seite, nginx ohne Root) und `zaehler` (eigene Python-Anwendung)
Datenbank	`db` (MariaDB 11.8), Zugangsdaten als Secrets, Daten im Volume `db-daten`
Netzwerke	`frontend` (Proxy nach außen), `apps` (intern: Proxy und Anwendungen), `backend` (intern: Anwendung und Datenbank)
Härtung der Container	schreibgeschützt, ohne Capabilities, `no-new-privileges`, Limits, Protokollrotation
Sicherung	täglich per Timer: Datenbank-Dump, Konfiguration, verschlüsselte Geheimnisse, Aufbewahrung 14 Tage
Dokumentation	Architekturplan, Kommunikationsmatrix, Betriebshandbuch und Wiederherstellungsverfahren

Der Architekturplan

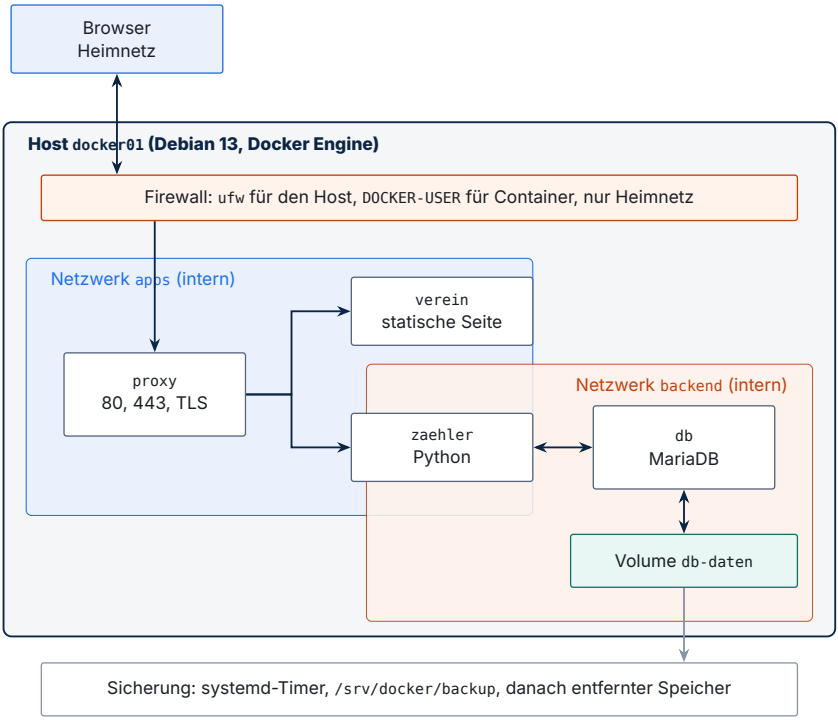


Abbildung P7: Architektur der Gesamtinfrastruktur

Die Abbildung zeigt die Gesamtanordnung, die Tabelle fasst die Kommunikation zwischen den Komponenten zusammen.

Von	Nach	Netz	Port	Zweck
Browser	proxy	Heimnetz	80, 443	Zugriff auf beide Anwendungen, HTTP wird umgeleitet
proxy	verein	apps	8080	Weiterleitung der Anfragen für `verein.docker01.lan`
proxy	zaehler	apps	8000	Weiterleitung der Anfragen für `zaehler.docker01.lan`
zaehler	db	backend	3306	Lesen und Schreiben des Zählers
Host (Skript)	db	Docker	kein Port	Datenbank-Dump per `docker compose exec`
Host (Skript)	Sicherungsziel	Heimnetz	22	Übertragung der Sicherungen per `rsync` über SSH

Alles, was in dieser Tabelle nicht steht, ist nicht erlaubt und technisch nicht möglich: Der Proxy kann die Datenbank nicht erreichen, weil er nicht im Netz `backend` steht. Die Anwendungen können nicht ins Internet, weil ihre Netze intern sind. Von außen ist nur der Proxy erreichbar.

Schritt 1: Host vorbereiten

Der Host entspricht dem Zustand nach den Projekten 1 und 6. Prüfen Sie ihn, bevor Sie beginnen.

```
$ hostnamectl --static
docker01
$ sudo apt update && sudo apt upgrade
$ docker compose version
$ sudo ufw status | head -n 1
Status: active
```

```
$ sudo iptables -L DOCKER-USER -n | grep -c DROP
1
$ sudo ss -tlnp | grep -E ":80 |:443 "
```

Die letzte Abfrage darf nichts zeigen. Stoppen Sie vorher Container früherer Projekte, die die Ports 80 und 443 belegen. Legen Sie die Verzeichnisse an.

```
$ mkdir -p /srv/docker/projekt-7/apps/zaehler /srv/docker/projekt-7/
    ↪ apps/verein/html \
          /srv/docker/projekt-7/nginx/conf.d /srv/docker/projekt-7/
    ↪ nginx/snippets \
          /srv/docker/backup/projekt-7
$ mkdir -m 700 /srv/docker/projekt-7/secrets
$ cd /srv/docker/projekt-7
$ cp /srv/docker/projekt-6/audit.sh .
```

Schritt 2: Secrets und Zertifikat

Die Zugangsdaten der Datenbank entstehen als Dateien, nicht als Variablen (Kapitel 18).

```
$ tr -dc A-Za-z0-9 </dev/urandom | head -c 24 > secrets/
    ↪ db_root_password
$ tr -dc A-Za-z0-9 </dev/urandom | head -c 24 > secrets/db_password
$ chmod 644 secrets/db_root_password secrets/db_password
```

Das Verzeichnis `secrets` hat die Rechte `700`, schützt die Dateien also vor anderen Benutzern des Hosts. Die Dateien selbst sind für den Benutzer im Container lesbar. Das Zertifikat erzeugt ein Skript wie in Projekt 5, diesmal für die beiden Namen dieser Anwendungen. Der Schlüssel gehört anschließend der Benutzernummer 101, dem Benutzer des Proxy-Containers, und ist nur für ihn lesbar.

/srv/docker/projekt-7/zertifikat.sh

```
#!/bin/bash
# Erzeugt ein selbstsigniertes Zertifikat für das Übungsnetz
set -e
cd /srv/docker/projekt-7
mkdir -p certs
openssl req -x509 -nodes -newkey rsa:2048 -days 365 \
  -keyout certs/server.key -out certs/server.crt \
  -subj "/CN=docker01.lan" \
  -addext "subjectAltName=DNS:docker01.lan,DNS:zaehler.docker01.lan,
    ↪ DNS:verein.docker01.lan" 2>/dev/null
sudo chown 101:101 certs/server.key
sudo chmod 600 certs/server.key
echo "Zertifikat erstellt: certs/server.crt"
```

```
$ chmod +x zertifikat.sh && ./zertifikat.sh
Zertifikat erstellt: certs/server.crt
$ ls -ln certs
-rw-r--r-- 1 1000 1000 1294 Nov 11 15:02 server.crt
-rw------- 1  101  101 1704 Nov 11 15:02 server.key
```

Der Schlüssel gehört nun Benutzer 101, mit Rechten nur für den Besitzer. Der Benutzer `admin` kann ihn ohne `sudo` nicht mehr lesen, der Proxy-Container schon. Das ist das Prinzip der geringsten Rechte auf Dateiebene.

Schritt 3: Die Anwendungen

Die Vereinsseite ist eine einfache Datei.

apps/verein/html/index.html

```
<!DOCTYPE html>
<html lang="de">
<head><meta charset="utf-8"><title>Verein</title></head>
<body><h1>Willkommen beim Verein</h1><p>Diese Seite liefert ein
    ↪ gehärteter nginx-Container.</p></body>
```

```
</html>
```

Der Besucherzähler ist die Anwendung aus Projekt 4, mit einer Änderung: Das Passwort liest sie aus der Datei, auf die die Umgebungsvariable `DB_PASSWORD_FILE` zeigt.

apps/zaehler/app.py

```
import html
import os
from http.server import BaseHTTPRequestHandler, HTTPServer

import pymysql


def passwort():
    datei = os.environ.get("DB_PASSWORD_FILE")
    if datei:
        with open(datei, encoding="utf-8") as f:
            return f.read().strip()
    return os.environ["DB_PASSWORD"]


def verbinden():
    return pymysql.connect(
        host=os.environ["DB_HOST"],
        user=os.environ["DB_USER"],
        password=passwort(),
        database=os.environ["DB_NAME"],
        autocommit=True,
    )


def zaehler_erhoehen():
    db = verbinden()
    try:
        cursor = db.cursor()
        cursor.execute(
```

```
            "CREATE TABLE IF NOT EXISTS besuche "
            "(id INT PRIMARY KEY, anzahl INT NOT NULL)"
        )
        cursor.execute(
            "INSERT INTO besuche (id, anzahl) VALUES (1, 1) "
            "ON DUPLICATE KEY UPDATE anzahl = anzahl + 1"
        )
        cursor.execute("SELECT anzahl FROM besuche WHERE id = 1")
        return cursor.fetchone()[0]
    finally:
        db.close()


class Handler(BaseHTTPRequestHandler):
    def antworten(self, status, inhalt, typ="text/html; charset=utf
    ↪ -8"):
        daten = inhalt.encode("utf-8")
        self.send_response(status)
        self.send_header("Content-Type", typ)
        self.send_header("Content-Length", str(len(daten)))
        self.end_headers()
        self.wfile.write(daten)

    def do_GET(self):
        if self.path == "/health":
            self.antworten(200, "ok", "text/plain; charset=utf-8")
        elif self.path == "/":
            try:
                anzahl = zaehler_erhoehen()
                text = f"Diese Seite wurde {anzahl} Mal aufgerufen."
                status = 200
            except Exception as fehler:
                text = "Datenbankfehler: " + html.escape(str(fehler))
                status = 500
            self.antworten(status, f"<h1>Besucherzähler</h1><p>{text
    ↪ }</p>")
        else:
```

```
            self.antworten(404, "Nicht gefunden", "text/plain;
    ↪ charset=utf-8")


if __name__ == "__main__":
    HTTPServer(("", 8000), Handler).serve_forever()
```

apps/zaehler/Dockerfile

```
FROM python:3-slim

RUN useradd --create-home --uid 10001 app \
    && pip install --no-cache-dir pymysql

WORKDIR /home/app
COPY --chown=app:app app.py .

USER app
EXPOSE 8000

HEALTHCHECK --interval=10s --timeout=3s --start-period=5s --retries=3
    ↪ \
  CMD python -c "import urllib.request; urllib.request.urlopen('http
    ↪ ://localhost:8000/health')"

CMD ["python", "app.py"]
```

Schritt 4: Der Reverse Proxy

Die Konfiguration verbindet Kapitel 16 und Projekt 5. Weil der Proxy ohne Root läuft, lauscht er im Container auf die Ports 8080 und 8443. Compose führt die Ports 80 und 443 des Hosts darauf.

nginx/snippets/proxy-header.conf

```
proxy_set_header Host $host;
proxy_set_header X-Real-IP $remote_addr;
```

```
proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
proxy_set_header X-Forwarded-Proto $scheme;
```

nginx/conf.d/default.conf

```
# Docker-DNS: Namen der Anwendungen bei jeder Anfrage neu auflösen
resolver 127.0.0.11 valid=10s;

# Begrenzung der Anfragen je Besucher
limit_req_zone $binary_remote_addr zone=pro_besucher:10m rate=10r/s;

# Auffangblöcke: unbekannte Namen und reine IP-Adressen
server {
    listen 8080 default_server;
    return 444;
}

server {
    listen 8443 ssl default_server;
    ssl_reject_handshake on;
}

# HTTP leitet auf HTTPS um
server {
    listen 8080;
    server_name zaehler.docker01.lan verein.docker01.lan;
    return 301 https://$host$request_uri;
}

# Anwendung Besucherzähler
server {
    listen 8443 ssl;
    server_name zaehler.docker01.lan;

    ssl_certificate     /etc/nginx/certs/server.crt;
    ssl_certificate_key /etc/nginx/certs/server.key;
    ssl_protocols       TLSv1.2 TLSv1.3;
```

```
    server_tokens off;
    client_max_body_size 1m;
    add_header X-Content-Type-Options "nosniff" always;
    add_header X-Frame-Options "SAMEORIGIN" always;

    location / {
        limit_req zone=pro_besucher burst=20 nodelay;
        set $ziel http://zaehler:8000;
        proxy_pass $ziel;
        include /etc/nginx/snippets/proxy-header.conf;
    }
}

# Anwendung Vereinsseite
server {
    listen 8443 ssl;
    server_name verein.docker01.lan;

    ssl_certificate     /etc/nginx/certs/server.crt;
    ssl_certificate_key /etc/nginx/certs/server.key;
    ssl_protocols       TLSv1.2 TLSv1.3;

    server_tokens off;
    client_max_body_size 1m;
    add_header X-Content-Type-Options "nosniff" always;
    add_header X-Frame-Options "SAMEORIGIN" always;

    location / {
        limit_req zone=pro_besucher burst=20 nodelay;
        set $ziel http://verein:8080;
        proxy_pass $ziel;
        include /etc/nginx/snippets/proxy-header.conf;
    }
}
```

Neu gegenüber Projekt 5 sind die Begrenzung der Anfragen je Besucher mit `limit_req`, die zusätzliche Sicherheitskopfzeile und die Ports für den Betrieb ohne Root.

Schritt 5: Die Compose-Datei

Die Dateien fasst die `compose.yaml` zusammen. Für die gemeinsamen Härtungseinstellungen der drei Webcontainer verwendet sie einen YAML-Anker (`&gehaertet`), den die Dienste mit `<<: *gehaertet` übernehmen.

compose.yaml

```
name: projekt7

x-gehaertet: &gehaertet
  restart: unless-stopped
  read_only: true
  tmpfs:
    - /tmp
  cap_drop:
    - ALL
  security_opt:
    - no-new-privileges:true
  pids_limit: 100
  logging:
    driver: json-file
    options:
      max-size: "10m"
      max-file: "3"

services:
  proxy:
    <<: *gehaertet
    image: nginxinc/nginx-unprivileged:stable-alpine
    mem_limit: 128m
    cpus: 0.5
    ports:
      - "80:8080"
      - "443:8443"
    volumes:
      - ./nginx/conf.d:/etc/nginx/conf.d:ro
      - ./nginx/snippets:/etc/nginx/snippets:ro
      - ./certs:/etc/nginx/certs:ro
```

```
    networks:
      - frontend
      - apps

  verein:
    <<: *gehaertet
    image: nginxinc/nginx-unprivileged:stable-alpine
    mem_limit: 64m
    cpus: 0.25
    volumes:
      - ./apps/verein/html:/usr/share/nginx/html:ro
    healthcheck:
      test: ["CMD", "wget", "-q", "--spider", "http
    ↪ ://127.0.0.1:8080/"]
      interval: 15s
      timeout: 3s
      retries: 3
      start_period: 5s
    networks:
      - apps

  zaehler:
    <<: *gehaertet
    build: ./apps/zaehler
    image: projekt7-zaehler:1.0
    mem_limit: 128m
    cpus: 0.5
    environment:
      DB_HOST: db
      DB_NAME: besucher
      DB_USER: zaehler
      DB_PASSWORD_FILE: /run/secrets/db_password
    secrets:
      - db_password
    depends_on:
      db:
        condition: service_healthy
    networks:
```

```
      - apps
      - backend

  db:
    image: mariadb:11.8
    restart: unless-stopped
    security_opt:
      - no-new-privileges:true
    mem_limit: 512m
    cpus: 1
    pids_limit: 200
    environment:
      MARIADB_ROOT_PASSWORD_FILE: /run/secrets/db_root_password
      MARIADB_PASSWORD_FILE: /run/secrets/db_password
      MARIADB_DATABASE: besucher
      MARIADB_USER: zaehler
    secrets:
      - db_root_password
      - db_password
    volumes:
      - db-daten:/var/lib/mysql
    healthcheck:
      test: ["CMD", "healthcheck.sh", "--connect", "--
    ↪ innodb_initialized"]
      interval: 10s
      timeout: 5s
      retries: 5
      start_period: 20s
    logging:
      driver: json-file
      options:
        max-size: "10m"
        max-file: "3"
    networks:
      - backend

secrets:
  db_root_password:
```

```
    file: ./secrets/db_root_password
  db_password:
    file: ./secrets/db_password

networks:
  frontend:
  apps:
    internal: true
  backend:
    internal: true

volumes:
  db-daten:
```

Zwei Einzelheiten verdienen Beachtung. Die Datenbank bekommt weniger Härtung als die übrigen Container: Sie läuft ohne `read_only` und mit den Standard-Capabilities, weil das Image beim ersten Start zunächst als Root das Datenverzeichnis einrichtet und dann die Rechte abgibt. Das ist ein bewusst dokumentiertes Restrisiko, dem die Isolation im Netz `backend`, die Secrets und die Limits entgegenwirken. Und der Dienst `proxy` hängt in `frontend`, damit die Portfreigabe funktioniert, und in `apps`, um die Anwendungen zu erreichen. Nie in `backend`.

Schritt 6: Starten und prüfen

```
$ docker compose config --quiet && echo "Datei ist gültig"
Datei ist gültig
$ docker compose up -d --build --wait
$ docker compose ps --format "table {{.Service}}\t{{.Status}}\t{{.
    ↪ Ports}}"
SERVICE   STATUS                    PORTS
db        Up 50 seconds (healthy)   3306/tcp
proxy     Up 30 seconds             0.0.0.0:80->8080/tcp,
    ↪ [::]:80->8080/tcp, 0.0.0.0:443->8443/tcp, [::]:443->8443/tcp
verein    Up 40 seconds (healthy)   8080/tcp
zaehler   Up 45 seconds (healthy)   8000/tcp
```

Tragen Sie die Namen in die `hosts`-Datei Ihres Arbeitsrechners ein und öffnen Sie beide Adressen im Browser.

/etc/hosts auf dem Arbeitsrechner

```
192.168.100.10   docker01.lan zaehler.docker01.lan verein.docker01.
    ↪ lan
```

```
$ curl -sk --resolve zaehler.docker01.lan:443:127.0.0.1 https://
    ↪ zaehler.docker01.lan/
<h1>Besucherzähler</h1><p>Diese Seite wurde 1 Mal aufgerufen.</p>
$ curl -sk --resolve verein.docker01.lan:443:127.0.0.1 https://verein.
    ↪ docker01.lan/ | grep h1
<body><h1>Willkommen beim Verein</h1>...
```

Schritt 7: Sicherung einrichten

Das Sicherungsskript legt drei Dateien ab: einen Dump der Datenbank, ein Archiv der Konfiguration ohne Geheimnisse und ein verschlüsseltes Archiv mit den Secrets und dem Zertifikat. Die Verschlüsselung braucht ein Passwort, das nur Root lesen kann. Es gehört zusätzlich an einen anderen Ort als den Server, wie in Kapitel 19 begründet.

```
$ sudo apt install gnupg
$ tr -dc A-Za-z0-9 </dev/urandom | head -c 32 | sudo tee /root/.
    ↪ sicherung-passphrase
$ sudo chmod 600 /root/.sicherung-passphrase
```

/srv/docker/projekt-7/sicherung.sh

```
#!/bin/bash
# Sicherung der Infrastruktur (Projekt 7): Datenbank, Konfiguration,
    ↪ Geheimnisse
set -eo pipefail
PROJEKT=${PROJEKT:-/srv/docker/projekt-7}
ZIEL=${ZIEL:-/srv/docker/backup/projekt-7}
```

```
PHRASE=${PHRASE:-/root/.sicherung-passphrase}
STAMP=$(date +%Y-%m-%d_%H%M%S)
mkdir -p "$ZIEL"
cd "$PROJEKT"

# 1. Datenbank als SQL-Dump
docker compose exec -T db sh -c \
  'mariadb-dump -uroot -p"$(cat /run/secrets/db_root_password)" --
    ↪ single-transaction --all-databases' \
  | gzip > "$ZIEL/db-$STAMP.sql.gz"

# 2. Konfiguration ohne Geheimnisse
tar czf "$ZIEL/konfiguration-$STAMP.tar.gz" compose.yaml nginx apps

# 3. Geheimnisse und Zertifikate, verschlüsselt
tar czf - secrets certs | gpg --batch --yes --pinentry-mode loopback
    ↪ \
  --passphrase-file "$PHRASE" --symmetric --cipher-algo AES256 \
  -o "$ZIEL/geheimnisse-$STAMP.tar.gz.gpg"

chmod 600 "$ZIEL"/*-"$STAMP".*

# 4. Sicherungen älter als 14 Tage löschen
find "$ZIEL" -type f \( -name "*.gz" -o -name "*.gpg" \) -mtime +14 -
    ↪ delete
echo "Sicherung: $STAMP"
```

/srv/docker/projekt-7/wiederherstellung.sh

```
#!/bin/bash
# Spielt einen Datenbank-Dump ein: wiederherstellung.sh DUMP.sql.gz
set -eo pipefail
cd "${PROJEKT:-/srv/docker/projekt-7}"
[ -f "$1" ] || { echo "Aufruf: $0 DUMP.sql.gz"; exit 1; }
gunzip -c "$1" | docker compose exec -T db sh -c 'mariadb -uroot -p"$
    ↪ (cat /run/secrets/db_root_password)"'
echo "Datenbank wiederhergestellt aus: $1"
```

```
$ chmod +x sicherung.sh wiederherstellung.sh
$ sudo ./sicherung.sh
Sicherung: 2026-11-11_150845
$ sudo ls -l /srv/docker/backup/projekt-7
-rw------- 1 root root  1830 Nov 11 15:08 db-2026-11-11_150845.sql.gz
-rw------- 1 root root  2100 Nov 11 15:08 geheimnisse-2026-11-11
    ↪ _150845.tar.gz.gpg
-rw------- 1 root root  3544 Nov 11 15:08 konfiguration-2026-11-11
    ↪ _150845.tar.gz
```

Damit die Sicherung täglich läuft, richten Sie einen Timer ein. Der Dienst führt das Skript aus, der Timer startet ihn jede Nacht, wie in Kapitel 19 beschrieben.

/etc/systemd/system/projekt7-sicherung.service

```
[Unit]
Description=Sicherung der Infrastruktur (Projekt 7)

[Service]
Type=oneshot
ExecStart=/srv/docker/projekt-7/sicherung.sh
```

/etc/systemd/system/projekt7-sicherung.timer

```
[Unit]
Description=Tägliche Sicherung der Infrastruktur (Projekt 7)

[Timer]
OnCalendar=*-*-* 02:30:00
Persistent=true
RandomizedDelaySec=300

[Install]
WantedBy=timers.target
```

```
$ sudo systemctl daemon-reload
$ sudo systemctl enable --now projekt7-sicherung.timer
```

```
$ systemctl list-timers projekt7-sicherung.timer
```

Die dritte Kopie gehört auf einen anderen Rechner. Dafür dient der Befehl aus Kapitel 19, den Sie an Ihre Umgebung anpassen: `rsync -a --delete /srv/docker/backup/ BENUTZER@ZIEL:/PFAD/`. Auch ihn lässt man per Timer laufen, zum Beispiel eine halbe Stunde nach der Sicherung.

Schritt 8: Funktionsprüfung

Das Prüfskript fasst alle Anforderungen zusammen und führt auch den Wiederherstellungstest durch.

/srv/docker/projekt-7/check.sh

```
#!/bin/bash
# Funktionsprüfung für Projekt 7 (mit sudo ausführen)
# Der Test legt eine Sicherung an und spielt die Datenbank daraus
    ↪ zurück.
cd /srv/docker/projekt-7 || exit 1
FEHLER=0
ok()   { echo "[ OK ]     $1"; }
fail() { echo "[FEHLER]   $1"; FEHLER=1; }
H=${HTTP_PORT:-80}; S=${HTTPS_PORT:-443}
holen() { curl -sk --resolve "$1:$S:127.0.0.1" "https://$1:$S/"; }
zahl()  { holen zaehler.docker01.lan | sed -n 's/.*wurde \([0-9]*\)
    ↪ Mal.*/\1/p'; }
sql() {
  docker compose exec -T db sh -c \
    'mariadb -uroot -p"$(cat /run/secrets/db_root_password)" -N -e "
    ↪ $1"' sh "$1" \
    2>/dev/null
}

[ "$(docker compose ps --status running --services | sort | tr '\n' '
    ↪  ')" = "db proxy verein zaehler " ] \
  && ok "Alle vier Dienste laufen" || fail "Nicht alle Dienste laufen
    ↪ "
```

```
holen verein.docker01.lan | grep -q "Willkommen beim Verein" \
  && ok "verein.docker01.lan liefert die Vereinsseite" || fail "
     ↪ Vereinsseite fehlt"
A=$(zahl); B=$(zahl)
[ -n "$A" ] && [ "$B" -eq $((A + 1)) ] 2>/dev/null \
  && ok "Zähler steigt ($A, dann $B)" || fail "Zähler steigt nicht"
[ "$(curl -s -o /dev/null -w '%{http_code}' --resolve zaehler.
     ↪ docker01.lan:$H:127.0.0.1 http://zaehler.docker01.lan:$H/)" =
     ↪  "301" ] \
  && ok "HTTP wird auf HTTPS umgeleitet" || fail "Umleitung fehlt"
curl -sk -o /dev/null https://127.0.0.1:$S/ \
  && fail "Reine IP-Adresse wird bedient" || ok "Reine IP-Adresse
     ↪ wird abgewiesen"

[ -z "$(docker compose port db 3306 2>/dev/null)" ] \
  && ok "Datenbank-Port ist nicht veröffentlicht" || fail "Datenbank-
     ↪ Port ist veröffentlicht"
docker inspect --format '{{json .NetworkSettings.Networks}}' projekt7-
     ↪ proxy-1 | grep -q backend \
  && fail "proxy hängt im Netz backend" || ok "proxy hängt nicht im
     ↪ Netz backend"
docker inspect --format '{{.Config.Env}}' projekt7-db-1 | grep -qF "$
     ↪ (cat secrets/db_password)" \
  && fail "Passwort steht in der Umgebung der Datenbank" || ok "
     ↪ Passwort steht nicht in der Umgebung"
for DIENST in proxy verein zaehler; do
  ./audit.sh projekt7-$DIENST >/dev/null \
    && ok "audit.sh: $DIENST ohne Warnungen" || fail "audit.sh:
     ↪ $DIENST hat Warnungen"
done

VOR=$(sql 'SELECT anzahl FROM besucher.besuche WHERE id = 1')
DUMP=$(./sicherung.sh >/dev/null && ls -t /srv/docker/backup/projekt
     ↪ -7/db-*.sql.gz | head -n 1)
[ -s "$DUMP" ] && zcat "$DUMP" | grep -q 'CREATE TABLE `besuche`' \
  && ok "Sicherung enthält die Tabelle besuche" || fail "Sicherung
     ↪ ist unvollständig"
sql 'DROP TABLE besucher.besuche' >/dev/null
```

```
./wiederherstellung.sh "$DUMP" >/dev/null 2>&1
NACH=$(sql 'SELECT anzahl FROM besucher.besuche WHERE id = 1')
[ -n "$VOR" ] && [ "$VOR" = "$NACH" ] \
  && ok "Wiederherstellung bringt den Zählerstand $NACH zurück" ||
     ↪ fail "Wiederherstellung unvollständig"

V=$(zahl)
docker compose down >/dev/null 2>&1
docker compose up -d --wait >/dev/null 2>&1
N=$(zahl)
[ -n "$V" ] && [ "$N" -eq $((V + 1)) ] 2>/dev/null \
  && ok "Daten überstehen den Austausch aller Container" || fail "
     ↪ Daten sind verloren"

echo
[ "$FEHLER" -eq 0 ] && echo "Alle Prüfungen bestanden." \
  || echo "Mindestens eine Prüfung ist fehlgeschlagen."
exit $FEHLER
```

```
$ chmod +x check.sh && sudo ./check.sh
[ OK ]     Alle vier Dienste laufen
[ OK ]     verein.docker01.lan liefert die Vereinsseite
[ OK ]     Zähler steigt (3, dann 4)
[ OK ]     HTTP wird auf HTTPS umgeleitet
[ OK ]     Reine IP-Adresse wird abgewiesen
[ OK ]     Datenbank-Port ist nicht veröffentlicht
[ OK ]     proxy hängt nicht im Netz backend
[ OK ]     Passwort steht nicht in der Umgebung
[ OK ]     audit.sh: proxy ohne Warnungen
[ OK ]     audit.sh: verein ohne Warnungen
[ OK ]     audit.sh: zaehler ohne Warnungen
[ OK ]     Sicherung enthält die Tabelle besuche
[ OK ]     Wiederherstellung bringt den Zählerstand 4 zurück
[ OK ]     Daten überstehen den Austausch aller Container

Alle Prüfungen bestanden.
```

Schritt 9: Technische Dokumentation

Eine Infrastruktur ist erst fertig, wenn sie dokumentiert ist. Legen Sie im Projektverzeichnis eine Datei `DOKUMENTATION.md` an, die ein fremder Administrator lesen kann, ohne Sie zu fragen. Sie enthält die folgenden Kapitel. Die Tabellen dieses Projekts liefern den Inhalt.

Abschnitt	Inhalt
Zweck und Umfang	wozu die Infrastruktur dient, wer sie betreut, wo die Dateien liegen
Architektur	Abbildung P7, Komponenten, Netzwerke, Namen und Adressen
Kommunikationsmatrix	Tabelle der erlaubten Verbindungen mit Port und Zweck
Daten und Speicher	Volumes, Verzeichnisse, wo Geheimnisse und Zertifikate liegen
Sicherheitskonzept	Maßnahmen je Schicht, Prüfskripte, bekannte Restrisiken
Backup-Konzept	was, wann, wohin, wie lange aufbewahrt, wie geprüft
Betrieb	Starten, Stoppen, Updates, Überwachung, wichtige Befehle
Wiederherstellung	Schritt-für-Schritt-Verfahren nach Kapitel 19
Änderungsprotokoll	Datum, Änderung, Name

Das Sicherheitskonzept dieser Infrastruktur lässt sich in einer Tabelle zusammenfassen, die in die Dokumentation gehört.

Schicht	Maßnahme	Prüfung
Netzwerk	`ufw`, `DOCKER-USER` nur Heimnetz, nur Ports 80 und 443	`ufw status`, `iptables -L DOCKER-USER -n`
Host	Updates, SSH nur mit Schlüssel, Gruppe `docker` nur für `admin`	`check.sh` aus Projekt 6
Docker-Daemon	kein Fernzugriff, kein Socket in Containern	`audit.sh`
Container	Non-Root, schreibgeschützt, Capabilities entfernt, `no-new-privileges`, Limits	`audit.sh`, `check.sh`
Netzwerke	Anwendungen und Datenbank in internen Netzen, Proxy nicht im Netz `backend`	`check.sh`
Geheimnisse	Secrets als Dateien, Sicherung verschlüsselt	`check.sh`, Rechte der Dateien
Images	feste Tags, regelmäßige Prüfung auf Schwachstellen, Updates mit Abstand	Prüfung mit Trivy nach Kapitel 18
Restrisiken	Datenbank mit Standardrechten, selbstsigniertes Zertifikat, keine zentrale Protokollierung	dokumentiert, bei der nächsten Überarbeitung zu verbessern

Das Backup-Konzept lässt sich genauso knapp fassen.

Frage	Antwort
Was wird gesichert?	Datenbank (Dump), Konfiguration, Secrets und Zertifikat (verschlüsselt)
Wann?	täglich um 02:30 Uhr per `systemd`-Timer
Wohin?	lokal unter `/srv/docker/backup/projekt-7`, zusätzlich per `rsync` auf einen zweiten Rechner
Wie lange?	14 Tage, ältere Dateien werden automatisch gelöscht
Wie geprüft?	`check.sh` spielt den Dump zurück, einmal im Jahr eine vollständige Wiederherstellung auf einem Testsystem
Verlustzeit (RPO)	höchstens 24 Stunden
Ausfallzeit (RTO)	nach dem ersten Test festzulegen, Richtwert: wenige Stunden

Abnahme

Das Projekt gilt als abgeschlossen, wenn die folgenden Punkte erfüllt sind. Das Prüfskript meldet „Alle Prüfungen bestanden“. Der Server startet nach einem Neustart ohne Eingriff, und beide Anwendungen sind danach erreichbar. Die Sicherung läuft einmal von Hand und einmal per Timer durch, und die Dateien liegen auch auf dem zweiten Rechner. Eine Wiederherstellung auf einem frisch aufgesetzten Testsystem nach der Anleitung in Ihrer Dokumentation gelingt. Die Dokumentation ist vollständig und enthält die Tabellen dieses Projekts. Und ein Schwachstellenscan der verwendeten Images (Kapitel 18) liegt vor, mit Entscheidung zu jedem schweren Treffer.

Wenn etwas nicht klappt

Symptom	Ursache und Abhilfe
Der Proxy startet nicht, Meldung zu Zertifikat oder Schlüssel	Der Schlüssel gehört nicht der Benutzernummer 101. `ls -ln certs` prüfen, `sudo chown 101:101 certs/server.key`.
`502 Bad Gateway` beim Zähler	Anwendung nicht gesund oder Datenbank nicht erreichbar. `docker compose logs zaehler db` lesen.
Die Datenbank bleibt `unhealthy`	Beim ersten Start dauert die Einrichtung. Wurden Secrets nach dem ersten Start geändert, passt das Passwort nicht mehr (Kapitel 14).
`audit.sh` meldet Warnungen	Die Meldung nennt die Einstellung. Meist fehlt eine Zeile aus dem Anker `gehaertet`.
Die Sicherung bricht ab	`/root/.sicherung-passphrase` fehlt oder `gpg` ist nicht installiert. Mit `sudo bash -x ./sicherung.sh` den Schritt finden.
Nach `down` ist der Zähler zurückgesetzt	Das Volume `db-daten` wurde gelöscht (`down -v`), oder die Datenbank hat ein anonymes Volume benutzt.
Der Zugriff von anderen Rechnern funktioniert nicht	Name in der `hosts`-Datei, Heimnetz in `DOCKER-USER`, Port in der Firewall. Schritt für Schritt wie in Kapitel 20.

Lernkontrolle zu Teil VII

Die Aufgaben prüfen die Kapitel 18 bis 20. Die praktischen Aufgaben führen Sie auf Ihrem Übungsserver durch. Die Lösungen stehen im Anhang.

Verständnisfragen

V1. Nennen Sie die fünf Schichten der Sicherheitsarchitektur aus Abbildung 18.1 und erklären Sie das Prinzip der Verteidigung in der Tiefe.

V2. Warum ist Root im Container trotz Namespaces ein Risiko? Welche beiden Wege gibt es, es zu verringern?

V3. Warum gilt der Docker-Socket als Root-Zugang, und warum schützt `:ro` bei der Einbindung nicht?

V4. Was sind Linux Capabilities, und was bewirken `--cap-drop ALL` und `no-new-privileges`?

V5. Welche Aufgabe haben Seccomp und AppArmor, und woran erkennen Sie, dass sie auf einem Container aktiv sind?

V6. Warum sind Secrets als Dateien besser als Umgebungsvariablen?

V7. Welche Lehren zieht man aus dem Vorfall mit Trivy im März 2026 für den Umgang mit Image-Tags und mit dem Docker-Socket?

V8. Was bedeuten RPO und RTO, und welche Rolle spielt die 3-2-1-Regel bei der Sicherung?

Multiple-Choice-Aufgaben

M1. Welche Option macht das Wurzeldateisystem eines Containers schreibgeschützt?
(a) `--readonly-root`
(b) `--read-only`
(c) `--protect`
(d) `--lock`

M2. Was zeigt der Rückgabewert 137 eines beendeten Containers an?
(a) Ein sauberes Beenden
(b) Ein nicht gefundener Befehl
(c) Einen Absturz durch einen Speicherzugriffsfehler
(d) Ein hartes Beenden mit SIGKILL, oft durch Speichermangel

M3. Welcher Befehl prüft eine `daemon.json`, ohne den Daemon zu starten?
(a) `dockerd --validate --config-file=/etc/docker/daemon.json`
(b) `docker config check`
(c) `docker daemon test`
(d) `systemctl verify docker`

M4. Wie bindet man ein Secret in Compose in einen Container ein?
(a) Als Umgebungsvariable mit dem Namen `SECRET`
(b) Als Teil des Image-Namens
(c) Als Datei unter `/run/secrets/NAME`
(d) Als Volume mit dem Namen `secrets`

M5. Was bewirkt die Option `--filter "until=720h"` bei `docker image prune -a`?
(a) Sie entfernt nur ungenutzte Images, die älter als 30 Tage sind.
(b) Sie begrenzt die Größe der Images.
(c) Sie entfernt alle Images außer den neuesten.
(d) Sie wartet 720 Stunden bis zum Löschen.

M6. Welche Reihenfolge der Schichten entspricht einer sinnvollen Fehlersuche?
(a) Anwendung, Container, Daemon, Host
(b) Host, Daemon, Image, Container, Netzwerk und Volumes, Anwendung
(c) Zufällig, je nach Vermutung

(d) Netzwerk, dann sofort Neustart des Servers

Praktische Aufgaben

P1. Starten Sie einen Alpine-Container mit `--read-only`, `--cap-drop ALL`, `--user 10001:10001` und einem `tmpfs` auf `/tmp`. Weisen Sie nach: Schreiben außerhalb von `/tmp` scheitert, in `/tmp` gelingt es, und `CapEff` ist null.

P2. Schreiben Sie mit `docker inspect` und Go-Vorlagen einen Einzeiler, der für jeden laufenden Container ausgibt, ob er als Root läuft. Prüfen Sie Ihre Container mit dem Skript `audit.sh` aus Projekt 6 und beheben Sie zwei Befunde.

P3. Richten Sie eine Sicherung eines Volumes oder einer Datenbank per `systemd`-Timer ein und prüfen Sie mit `systemctl list-timers`, dass der Timer aktiv ist. Spielen Sie die Sicherung in einem frischen Volume zurück.

Fehlersuche

F1. Ein Container beendet sich sofort mit `Exited (127)`. Was bedeutet der Wert, und wie gehen Sie vor?

F2. Nach Einrichtung der `DOCKER-USER`-Regel mit `! -s 192.168.100.0/24 -j DROP` kommen Container nicht mehr ins Internet, obwohl der Verkehr von außen wie gewünscht gefiltert wird. Was fehlt?

F3. Der Docker-Daemon startet nach dem Bearbeiten der `daemon.json` nicht mehr. Beschreiben Sie, wie Sie Ursache und Lösung finden.

Szenario aus dem Betrieb

Ein Administrator findet im Netz die Anleitung, einen Docker-Server werde „am schnellsten“ so repariert: `chmod 666 /var/run/docker.sock` gegen Meldungen mit `permission denied`. Beurteilen Sie den Rat und beschreiben Sie, was Sie stattdessen tun.

TEIL VIII

Docker unter Windows

KAPITEL 21

Docker unter Windows: Desktop, WSL 2 und Windows Server

Bisher lief jedes Beispiel dieses Buches auf Linux. Viele Menschen lernen Docker aber an einem Windows-Rechner, und viele Unternehmen betreiben Windows Server. Dieses Kapitel zeigt, wie Docker unter Windows funktioniert, welche Möglichkeiten es gibt und wo sie sich unterscheiden. Es trennt dabei sorgfältig vier Dinge, die in Anleitungen im Netz oft vermischt werden: Docker Desktop, Linux-Container mit WSL 2, Windows-Container und die Container-Runtime auf Windows Server. Wer diese Unterscheidung beherrscht, versteht auch Fehlermeldungen und Lizenzfragen.

Alle Aussagen dieses Kapitels beruhen auf den offiziellen Dokumentationen von Microsoft und Docker, Stand Ende 2026. Versionsnummern, Menübezeichnungen und Voraussetzungen ändern sich bei Docker Desktop und Windows häufiger als bei Docker Engine unter Linux. Prüfen Sie deshalb vor jeder Installation die jeweils aktuelle Seite des Herstellers. Die Stellen, an denen das besonders wichtig ist, sind im Text benannt.

Die Beispiele verwenden die *PowerShell*. Die Eingabeaufforderung steht verkürzt als `PS>`. Wie im ganzen Buch tippen Sie diese Zeichen nicht mit.

21.1 Grundlagen von Docker unter Windows

Kapitel 2 hat einen Satz hervorgehoben, der hier die Grundlage von allem ist: Alle Container eines Hosts teilen sich den Kernel des Hosts. Daraus folgt eine Einschränkung, die unter Linux kaum auffällt. Ein Linux-Container braucht einen Linux-Kernel. Ein Windows-Container braucht einen Windows-Kernel. Ein Windows-Rechner hat zunächst nur den zweiten.

Für Windows-Container ist das unproblematisch: Sie laufen direkt auf dem Kernel von Windows, so wie Linux-Container auf dem Kernel von Linux laufen. Für Linux-Container braucht Windows dagegen einen eigenen Linux-Kernel, und den liefert eine kleine virtuelle Maschine im Hintergrund. Das ist der Grund, warum es unter Windows mehr Varianten gibt als unter Linux.

Fachbegriff: Linux-Container und Windows-Container

Ein *Linux-Container* enthält Programme für Linux und läuft auf einem Linux-Kernel. Ein *Windows-Container* enthält Windows-Programme und läuft auf dem Windows-Kernel des Hosts. Beide lassen sich mit denselben Docker-Befehlen bedienen, aber ihre Images sind nicht austauschbar. Das Image `nginx:stable-alpine` ist ein Linux-Image, das Image `mcr.microsoft.com/windows/nanoserver:ltsc2022` ein Windows-Image.

Fachbegriff: Docker Desktop

Docker Desktop ist ein Programm für Windows, macOS und Linux mit grafischer Oberfläche. Es enthält die Docker Engine, den Client, Docker Compose und die dafür nötige virtuelle Umgebung. Unter Windows richtet es alles ein, was Linux-Container brauchen, und kann auf Wunsch auch Windows-Container ausführen. Docker Desktop ist ein Produkt für Arbeitsplätze und unterliegt eigenen Lizenzbedingungen.

Fachbegriff: WSL 2

Das Windows-Subsystem für Linux in der Version 2 (*Windows Subsystem for Linux*, WSL 2) lässt einen echten Linux-Kernel in einer schlanken virtuellen Maschine unter Windows laufen. Auf diesem Kernel laufen Linux-Distributionen wie Ubuntu, und Docker Desktop nutzt ihn für seine Linux-Container.

Fachbegriff: Container-Runtime

Eine Container-Runtime ist die Software, die Container tatsächlich anlegt und ausführt. Unter Windows Server unterstützt Microsoft derzeit Moby (Docker Community Edition), die Mirantis Container Runtime und containerd.

Aus diesen Bausteinen ergeben sich vier Wege, Container unter Windows zu betreiben.

Weg	Container	Host	Typischer Einsatz
Docker Desktop mit WSL 2	Linux	Windows 10/11	Lernen und Entwicklung am Arbeitsplatz
Docker Desktop mit Windows-Containern	Windows	Windows 10/11 Pro oder Enterprise	Entwicklung und Test von Windows-Anwendungen
Docker Engine in einer WSL-2-Distribution	Linux	Windows 10/11	Entwicklung ohne Docker Desktop
Container-Runtime auf Windows Server	Windows	Windows Server 2016 und neuer	Produktivbetrieb von Windows-Anwendungen

Die Abbildung zeigt die Schichten der drei wichtigsten Wege. Der vierte Weg, Docker Engine in einer WSL-Distribution, gleicht im Aufbau dem ersten, nur dass Sie den Daemon selbst in der Distribution installieren.

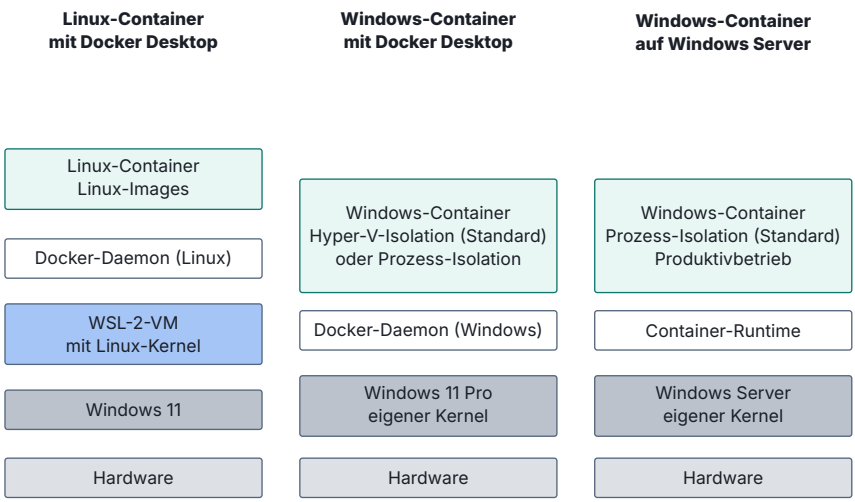


Abbildung 21.1: Docker unter Windows: Linux-Container, Windows-Container und Windows Server

Zwei Folgen dieses Aufbaus sollten Sie sich merken. Erstens teilen sich Linux-Container unter Docker Desktop den Kernel der WSL-2-Umgebung, nicht den von Windows. Zweitens sind Windows-Container an die Windows-Version gebunden, auf der sie laufen. Dazu mehr in Abschnitt 21.6.

21.2 Docker Desktop unter Windows 11

Docker Desktop ist der übliche Einstieg unter Windows. Es installiert Docker Engine, den Client `docker`, Compose und eine grafische Verwaltung, und es richtet die virtuelle Umgebung ein, in der die Linux-Container laufen.

21.2.1 Voraussetzungen

Die Voraussetzungen hängen vom Backend ab, also von der Technik, mit der Docker Desktop die virtuelle Umgebung betreibt. Docker Desktop kennt drei Backends: WSL 2, Hyper-V und, noch als Beta, Docker VMM. WSL 2 ist die Voreinstellung und genügt den meisten Anwendern. Nach der aktuellen Dokumentation von Docker gilt für das WSL-2-Backend unter Windows 11:

Anforderung	Wert
Windows	64-Bit-Windows 11 in den Editionen Enterprise, Pro oder Education, Version 23H2 (Build 22631) oder neuer
WSL	Version 2.1.5 oder neuer, für Enhanced Container Isolation 2.6 oder neuer
Arbeitsspeicher	8 GB
Prozessor	64-Bit mit Second Level Address Translation (SLAT)
Firmware	Hardware-Virtualisierung im BIOS/UEFI eingeschaltet
Dienste	Der Windows-Dienst `LanmanServer` (Server) muss aktiv und auf Automatisch gestellt sein

Windows Home

Die Anforderungsliste von Docker nennt für Windows 11 Pro, Enterprise und Education. Im selben Text steht aber auch, dass Home und Education nur Linux-Container erlauben. Ältere Anleitungen listeten Home ausdrücklich auf. Wenn Sie Windows Home einsetzen, lesen Sie vor der Installation die aktuelle Anforderungsseite. Für Windows-Container brauchen Sie in jedem Fall Pro oder Enterprise.

Docker unterstützt Docker Desktop nur für Windows-Versionen, die noch im Support von Microsoft sind. Und er unterstützt es *nicht* auf Windows Server. Das steht ausdrücklich in der Docker-Dokumentation: Auf Windows Server nutzt man eine Container-Runtime, wie Abschnitt 21.7 zeigt.

21.2.2 Zwei Installationsmodi

Docker Desktop kennt zwei Installationsmodi, die sich in den Rechten und in den Möglichkeiten unterscheiden. Die Wahl ist wichtig, weil man sie später nur durch Deinstallieren und Neuinstallieren ändern kann.

	Pro Benutzer (empfohlen)	Für alle Benutzer
Installationsort	`%LOCALAPPDATA%\Programs\DockerDesktop`	`C:\Program Files\Docker\Docker`
Administratorrechte zum Installieren und Aktualisieren	nicht nötig	nötig
Backend für Linux-Container	WSL 2 oder Docker VMM (Beta)	WSL 2, Hyper-V oder Docker VMM (Beta)
Windows-Container	nicht möglich	möglich
Privilegierter Systemdienst	keiner	`com.docker.service` läuft mit SYSTEM-Rechten

Der Modus pro Benutzer hat die kleinere Angriffsfläche, weil er keinen privilegierten Dienst installiert. Wer nur Linux-Container braucht, wählt ihn. Der Modus für alle Benutzer ist nötig für Windows-Container und für das Hyper-V-Backend.

21.2.3 Lizenzbedingungen

Docker Desktop ist nicht für jeden Zweck kostenlos. Nach der Dokumentation von Docker ist es kostenlos für kleine Unternehmen (weniger als 250 Beschäftigte *und* weniger als 10 Millionen US-Dollar Jahresumsatz), für den persönlichen Gebrauch, für Bildung und für nicht kommerzielle Open-Source-Projekte. In größeren Unternehmen erfordert die berufliche Nutzung ein kostenpflichtiges Abonnement, ebenso bei Behörden. Das gilt für Docker Desktop, nicht für Docker Engine, die weiter unter der Apache-Lizenz steht und frei nutzbar ist.

> **Lizenz vor dem Einsatz im Betrieb klären**

> Wer in einem Unternehmen mit Docker Desktop arbeitet, sollte die Lizenzfrage mit der IT-Abteilung klären, bevor er das Programm installiert. Das ist ein häufiger Stolperstein in Ausbildung und Beruf. Für den Lernbetrieb zu Hause und in der Ausbildung gilt die Ausnahme für Bildung und persönlichen Gebrauch. Die genauen Bedingungen stehen auf der Seite des Herstellers und ändern sich.

21.2.4 Wichtige Einstellungen

Docker Desktop startet nicht von selbst, wenn Windows hochfährt, sofern Sie das nicht einstellen. Unter den Einstellungen finden Sie drei Gruppen, die Anfänger kennen sollten. *General* legt unter anderem fest, ob Docker Desktop mit Windows startet und welches Backend es nutzt. *Resources* begrenzt Prozessoren, Arbeitsspeicher und Auslagerung. Bei WSL 2 steuert man diese Werte in der Datei `.wslconfig`, wie Abschnitt 21.3 erklärt. In der Rubrik für die WSL-Integration schaltet man ein, in welchen installierten Linux-Distributionen der Befehl `docker` verfügbar sein soll. Die Bezeichnungen der Menüpunkte ändern sich zwischen den Versionen, die Funktionen bleiben.

Aktualisierungen kommen als neue Fassungen von Docker Desktop. Im Modus pro Benutzer lassen sie sich ohne Administratorrechte einspielen. WSL 2 hat eigene Updates, die Sie mit `wsl --update` einspielen.

21.3 Windows Subsystem for Linux (WSL 2)

WSL 2 ist die Technik, die Linux-Container unter Windows ermöglicht, und gleichzeitig ein nützliches Werkzeug für sich. Es lässt eine Linux-Distribution wie Ubuntu neben Windows laufen, ohne dass man eine klassische virtuelle Maschine einrichten oder den Rechner mit zwei Betriebssystemen starten muss.

21.3.1 Aufbau

WSL 2 verwendet eine schlanke virtuelle Maschine mit einem von Microsoft gepflegten Linux-Kernel. Alle installierten Distributionen laufen als getrennte Umgebungen auf diesem einen gemeinsamen Kernel. Der Festplattenspeicher jeder

Distribution liegt in einer virtuellen Festplatte (einer `.vhdx`-Datei). Docker Desktop legt zusätzlich eine eigene interne Distribution an, die den Docker-Daemon betreibt. Mit `wsl --list --verbose` sehen Sie sie in der Liste.

21.3.2 Installation und Befehle

Seit Windows 10 Version 2004 und unter Windows 11 genügt ein Befehl. Öffnen Sie die PowerShell *als Administrator*.

```
PS> wsl --install
```

Der Befehl schaltet die nötigen Windows-Funktionen ein und installiert die Distribution Ubuntu. Danach muss der Rechner neu starten. Beim ersten Start der Distribution legen Sie einen Linux-Benutzer samt Passwort an. Die Tabelle nennt die Befehle, die Sie im Alltag brauchen.

Befehl	Wirkung
`wsl --install -d Debian`	installiert eine bestimmte Distribution (Liste mit `wsl --list --online`)
`wsl --list --verbose`	zeigt installierte Distributionen, ihren Zustand und die WSL-Version
`wsl --version`	zeigt die Version von WSL und des Linux-Kernels
`wsl --update`	aktualisiert WSL und den Kernel
`wsl --set-version Ubuntu 2`	stellt eine Distribution auf WSL 2 um
`wsl --set-default Ubuntu`	legt die Standarddistribution fest
`wsl --shutdown`	beendet die virtuelle Maschine und alle Distributionen
`wsl --unregister Ubuntu`	entfernt eine Distribution samt allen Daten

```
PS> wsl --list --verbose
  NAME              STATE           VERSION
* Ubuntu            Running         2
  docker-desktop    Running         2
```

Die Spalte `VERSION` muss bei Distributionen, die Docker nutzen soll, `2` lauten. Steht dort `1`, stellt `wsl --set-version` sie um. Ob die installierte WSL-Fassung neu genug für Docker Desktop ist, zeigt `wsl --version`. Erscheinen dort keine Versionsangaben, nutzen Sie die ältere, im Betriebssystem eingebaute Fassung. Sie muss mit `wsl --update` ersetzt werden.

wsl –unregister löscht ohne Rückfrage

Der Befehl `wsl --unregister` entfernt eine Distribution und ihre virtuelle Festplatte sofort und ohne Papierkorb. Alle Dateien darin sind danach verloren. Sichern Sie vorher alles Wichtige, etwa mit `wsl --export`.

21.3.3 Ressourcen begrenzen

Standardmäßig darf die WSL-2-Umgebung einen großen Teil des Arbeitsspeichers und alle Prozessoren verwenden. Auf einem Arbeitsplatzrechner kann das stören. Die Grenzen legt die Datei `%UserProfile%\.wslconfig` fest, die Sie selbst anlegen. Sie gilt für alle WSL-2-Distributionen.

%UserProfile%\.wslconfig

```
[wsl2]
memory=6GB
processors=4
swap=2GB
```

Die Angaben begrenzen den Arbeitsspeicher auf 6 GB, die Prozessoren auf vier und die Auslagerungsdatei auf 2 GB. Damit sie wirken, beenden Sie WSL mit `wsl --shutdown` und starten Docker Desktop neu. Passen Sie die Werte an Ihren Rechner an. Zu knappe Grenzen lassen Container und Builds langsam werden oder abstürzen, und der Rückgabewert 137 aus Kapitel 20 gilt auch hier.

21.3.4 Dateien und Zusammenarbeit

Windows und die Linux-Distribution sehen die Dateien des jeweils anderen. Aus Linux erreichen Sie die Windows-Laufwerke unter `/mnt/c`. Aus Windows erreichen Sie das Dateisystem einer Distribution im Explorer und in der PowerShell über einen Netzwerkpfad der Form `\\wsl.localhost\Ubuntu\home`. Für Docker

folgt daraus eine Regel, die in der Dokumentation von Docker ausdrücklich steht: Dateien, die in einen Linux-Container eingebunden werden, gehören für die beste Leistung in das *Linux*-Dateisystem der Distribution. Eine Bindung aus dem Windows-Dateisystem (`/mnt/c/...`) ist deutlich langsamer. Außerdem erhalten Linux-Container nur dann Benachrichtigungen über Dateiänderungen (inotify-Ereignisse), wenn die Dateien im Linux-Dateisystem liegen. Das betrifft Entwicklungsabläufe, die bei Änderungen automatisch neu laden.

21.3.5 Docker Engine direkt in WSL

Neben Docker Desktop gibt es einen zweiten Weg zu Linux-Containern: Sie installieren die Docker Engine wie in Kapitel 5 direkt in eine WSL-2-Distribution. Das Ergebnis ist ein gewöhnlicher Linux-Daemon ohne Docker Desktop. Er ist lizenzrechtlich unproblematisch, weil die Engine frei ist, und eignet sich gut für Menschen, die ohnehin im Terminal arbeiten. Zwei Hinweise: Die Dienstverwaltung setzt voraus, dass `systemd` in der Distribution aktiviert ist, was in der Datei `/etc/wsl.conf` im Abschnitt `[boot]` mit `systemd=true` geschieht. Und die Engine in der Distribution und die Integration von Docker Desktop sollten nicht zugleich in derselben Distribution laufen, weil sie sich stören. Dieses Buch verwendet im Folgenden Docker Desktop.

21.4 Docker mit Windows PowerShell

Der Docker-Client ist unter Windows dasselbe Programm `docker.exe` wie unter Linux, und die Befehle der Kapitel 6 bis 9 funktionieren unverändert. Unterschiede entstehen bei dem, was um den Befehl herum passiert: bei der Shell. Die PowerShell ist keine Bash, und wer Befehle aus Linux-Anleitungen einfach kopiert, stößt auf typische Fallen.

21.4.1 Windows PowerShell und PowerShell 7

Zwei verschiedene Programme tragen den Namen PowerShell. *Windows PowerShell* (Version 5.1) ist in Windows eingebaut und startet mit dem Befehl `powershell`. *PowerShell 7* (auch PowerShell Core genannt) ist ein getrenntes, plattformübergreifendes Produkt, das Sie zusätzlich installieren und mit `pwsh`

starten. Für Docker funktionieren beide. Die Unterschiede in einigen Verhaltensweisen sind aber wichtig und werden unten ausgewiesen. Für die Arbeit mit Containern empfiehlt Microsoft das Windows Terminal. Die PowerShell ISE eignet sich laut Microsoft nicht für interaktive Container-Sitzungen, die Container scheinen dort nicht mehr zu reagieren.

21.4.2 Typische Unterschiede zu Bash

Aufgabe	Bash (Linux)	PowerShell
Zeilenumbruch im Befehl	`\` am Zeilenende	Gravis (Backtick) `` ` `` am Zeilenende
Umgebungsvariable lesen	`$HOME`	`$env:USERPROFILE`
Umgebungsvariable setzen	`export NAME=wert`	`$env:NAME='wert'`
Aktuelles Verzeichnis	`$PWD`	`${PWD}`
Text filtern	`grep`	`Select-String`
Anfang einer Ausgabe	`head -n 5`	`Select-Object -First 5`
Befehle verketten	`befehl1 && befehl2`	in PowerShell 7 `&&`, in 5.1 `;`
Webanfrage	`curl`	`curl.exe` oder `Invoke-WebRequest`

Einige dieser Punkte verdienen eine Erklärung. In Anführungszeichen unterscheidet die PowerShell zwei Arten. Zeichenketten in einfachen Anführungszeichen (`'...'`) bleiben unverändert. In doppelten (`"..."`) ersetzt die PowerShell Variablen wie `$env:NAME` durch ihren Wert. Für Docker-Vorlagen mit geschweiften Klammern sind einfache Anführungszeichen die sichere Wahl.

```
PS> docker inspect --format '{{.State.Status}}' web
running
PS> docker run --rm -v "${PWD}:/daten:ro" alpine ls /daten
```

Der zweite Befehl bindet das aktuelle Verzeichnis ein. Dort braucht man doppelte Anführungszeichen, damit die PowerShell `${PWD}` ersetzt. Die geschweiften Klam-

mern um `PWD` sind nötig, weil der Doppelpunkt danach sonst als Teil des Variablennamens gelesen würde.

Der Befehl `curl` verhält sich in Windows PowerShell 5.1 anders als erwartet, weil dort `curl` ein Alias für das Cmdlet `Invoke-WebRequest` ist. Wer das Programm `curl.exe` meint, das in Windows 11 mitgeliefert wird, schreibt die Endung dazu.

```
PS> curl.exe -s http://localhost:8080
PS> (Invoke-WebRequest -Uri http://localhost:8080 -UseBasicParsing).
    ↪ StatusCode
200
```

21.4.3 Docker-Ausgaben als Objekte

Die PowerShell kann Ergebnisse in strukturierter Form weiterverarbeiten. `docker inspect` liefert JSON, das `ConvertFrom-Json` in Objekte umwandelt. Damit sind Abfragen möglich, die in Bash `jq` bräuchten.

```
PS> $info = docker inspect web | ConvertFrom-Json
PS> $info.State.Status
running
PS> $info.NetworkSettings.Ports
```

Auch eine Auswahl von Containern lässt sich mit `--format` als JSON ausgeben und weiterverarbeiten, zum Beispiel `docker ps --format '{{json .}}' | ConvertFrom-Json`.

21.4.4 Umleitung und Zeichenkodierung

Eine Falle betrifft die Umleitung der Ausgabe in eine Datei. In Windows PowerShell 5.1 schreibt der Operator > die Ausgabe in der Kodierung UTF-16, ein Format, mit dem viele Werkzeuge und Container nichts anfangen können. Wer etwa `docker compose config > pruefung.yaml` ausführt und die Datei danach wieder verwendet, erhält in 5.1 eine UTF-16-Datei, die Docker Compose nicht sauber lesen kann. PowerShell 7 schreibt dagegen UTF-8 ohne Byte-Order-Mark (BOM). Wenn Sie Dateien erzeugen, die später in Containern gelesen werden, etwa Konfigurationsdateien für nginx, sollten Sie die Kodierung deshalb bewusst wählen.

Eine zuverlässige Methode, die in beiden Versionen *ohne* BOM schreibt, ist die .NET-Funktion.

```
PS> function Write-Utf8NoBom([string]$Path, [string]$Text) {
>>   $enc = New-Object System.Text.UTF8Encoding($false)
>>   [System.IO.File]::WriteAllText((Join-Path $PWD $Path), $Text,
     ↪ $enc)
>> }
PS> Write-Utf8NoBom 'default.conf' "server { listen 80; }`n"
```

Die Funktion setzt den Pfad mit `Join-Path $PWD` zusammen, denn .NET kennt das aktuelle Verzeichnis der PowerShell nicht und würde relative Pfade sonst an einer anderen Stelle ablegen. Das Gegenstück ist `Set-Content -Encoding utf8`. Es schreibt in 5.1 UTF-8 *mit* BOM. Das BOM ist für eine HTML-Datei harmlos, bringt aber nginx und manche Skripte zum Absturz: Eine Konfigurationsdatei mit BOM lässt nginx die erste Zeile als unbekannte Anweisung ablehnen.

Dateien binär kopieren

Auch beim Weiterleiten binärer Daten durch die PowerShell-Pipeline können Bytes verfälscht werden. Wer etwa einen komprimierten Datenbank-Dump von einem Container holen will, erzeugt ihn besser im Container in eine Datei und kopiert diese mit `docker cp` heraus, statt die Ausgabe umzuleiten.

21.4.5 Skripte und Ausführungsrichtlinie

Unter Windows 11 ist das Ausführen von PowerShell-Skripten (`.ps1`-Dateien) in der Grundeinstellung gesperrt. Die *Ausführungsrichtlinie* (*Execution Policy*) bestimmt, was erlaubt ist. Für eine Übung genügt es, sie nur für das aktuelle Fenster zu lockern.

```
PS> Get-ExecutionPolicy -List
PS> Set-ExecutionPolicy -Scope Process -ExecutionPolicy RemoteSigned
```

Mit `-Scope Process` gilt die Änderung nur, bis Sie das Fenster schließen. Die Richtlinie ist eine Schutzmaßnahme gegen versehentliches Starten fremder

Skripte. Lockern Sie sie nie dauerhaft für den ganzen Rechner, um ein einzelnes Skript zu starten, und lesen Sie jedes Skript, bevor Sie es ausführen.

21.5 Linux-Container unter Windows

Mit Docker Desktop und WSL 2 funktionieren Linux-Container fast wie unter Linux. Alle Images aus den Kapiteln 7 bis 14 laufen, die Befehle stimmen, Compose-Dateien lassen sich übernehmen. Ein Blick auf den Kernel zeigt, was darunter passiert.

```
PS> docker run --rm alpine uname -a
Linux 3f2a9c1b7d44 6.6.x.y-microsoft-standard-WSL2 #1 SMP
    ↪ PREEMPT_DYNAMIC ... x86_64 Linux
PS> docker info --format '{{.OSType}} {{.Architecture}}'
linux x86_64
```

Der Container meldet einen Linux-Kernel, dessen Name `microsoft-standard-WSL2` verrät, dass er aus WSL 2 stammt. Die genaue Fassung ist bei Ihnen eine andere. Der Kernel stammt aus der virtuellen Umgebung und hat mit dem Kernel von Windows nichts zu tun.

21.5.1 Was anders ist als unter Linux

Einige Unterschiede betreffen den Alltag. Das Docker-Netzwerk der Linux-Container liegt in der virtuellen Maschine. Auf dem Windows-Host existiert deshalb keine Schnittstelle `docker0`, und man kann die Container nicht direkt über ihre Docker-Adresse erreichen, etwa mit `ping`. Der Zugriff von Windows auf einen Container geschieht über veröffentlichte Ports, und `localhost` funktioniert dafür. Umgekehrt erreicht ein Container Dienste auf dem Windows-Rechner über den besonderen Namen `host.docker.internal`.

```
PS> docker run -d --name web -p 8080:80 nginx:stable-alpine
PS> curl.exe -s -o NUL -w "%{http_code}`n" http://localhost:8080
200
PS> docker exec web wget -q -O- http://host.docker.internal:9000
```

Der letzte Befehl funktioniert nur, wenn auf dem Windows-Rechner tatsächlich ein Dienst auf Port 9000 lauscht. Er zeigt das Muster. Zwei weitere Eigenheiten betreffen Dateien und Rechte. Windows und Linux verwalten Zugriffsrechte unterschiedlich: Windows verwendet Zugriffssteuerungslisten, Linux die Benutzer- und Gruppennummern aus Kapitel 10. Bei Dateien, die aus dem Windows-Dateisystem in einen Linux-Container eingebunden werden, bildet Docker Desktop die Rechte auf eine Standardeinstellung ab. Verlassen Sie sich nicht darauf, dass `chmod` und `chown` auf solchen Dateien wie unter Linux wirken. Und Zeilenenden sind ein klassischer Fehler: Skripte, die in Windows mit Zeilenenden aus Wagenrücklauf und Zeilenvorschub (`CRLF`) gespeichert wurden, scheitern in Linux-Containern mit `exec format error` oder `no such file or directory`, wie Kapitel 20 beschrieben hat. Wer Git verwendet, legt in einer Datei `.gitattributes` fest, dass Skripte mit Linux-Zeilenenden ausgecheckt werden, zum Beispiel mit der Zeile `*.sh text eol=lf`.

21.5.2 Prozessorarchitektur

Auf einem Windows-Rechner mit x86-64-Prozessor laufen Linux-Images für `linux/amd64`. Die Unterstützung für Windows auf ARM-Prozessoren ist bei Docker Desktop derzeit als frühe Version (*Early Access*) gekennzeichnet. Auf solchen Rechnern müssen die Images für `linux/arm64` verfügbar sein, wie Kapitel 20 bei der Meldung `no matching manifest` beschrieben hat.

21.6 Windows-Container

Windows-Container sind das Gegenstück zu den Linux-Containern: Sie enthalten Windows-Programme, etwa Anwendungen für das .NET Framework oder den Webserver IIS, und laufen auf dem Windows-Kernel des Hosts. Für viele Anwendungen, die es nur für Windows gibt, sind sie der einzige Weg in die Container-Welt. Sie folgen denselben Grundideen wie alles in diesem Buch, haben aber eigene Regeln, vor allem bei Images, Isolation und Versionen.

21.6.1 Basisimages

Jedes Windows-Image baut auf einem Basisimage von Microsoft auf, das aus der Microsoft Container Registry (`mcr.microsoft.com`) stammt. Microsoft bietet vier Basisimages an. Die beiden gebräuchlichsten sind Windows Server Core und Nano Server.

Basisimage	Eigenschaft
`servercore`	große Teile der Windows-Server-Programmierschnittstellen, deshalb geeignet für Anwendungen, die das volle .NET Framework brauchen; mehrere Gigabyte groß
`nanoserver`	kleinste Variante mit deutlich kleinerer Programmierschnittstelle, ohne PowerShell, WMI und die Windows-Aktualisierungsdienste; geeignet für .NET-Anwendungen auf neuerer Basis; wenige hundert Megabyte
`windows`	größerer Umfang an Windows-Bibliotheken, für Anwendungen, denen bei Server Core etwas fehlt
`windows/server`	Windows-Server-Umfang mit Unterstützung für GPU-Beschleunigung

Windows-Images sind erheblich größer als Linux-Images. Ein Nano-Server-Image bringt es auf rund 300 MB, Server Core und die größeren Varianten auf mehrere Gigabyte. Wer mit Windows-Containern arbeitet, plant deshalb Plattenplatz und Downloadzeit anders als bei Alpine-Images. Zwei Besonderheiten bei den Tags: Es gibt kein `latest`. Das Tag `latest` wird für diese Repositories nicht mehr veröffentlicht, man muss immer eine bestimmte Fassung angeben. Und die Tags folgen den Windows-Versionen mit Langzeitunterstützung, zum Beispiel `ltsc2022` und `ltsc2025`.

```
PS> docker pull mcr.microsoft.com/windows/nanoserver:ltsc2022
PS> docker images --format "table {{.Repository}}\t{{.Tag}}\t{{.Size
    ↪ }}"
REPOSITORY                              TAG        SIZE
mcr.microsoft.com/windows/nanoserver    ltsc2022   296MB
```

Die Größe ist ein Beispiel aus der Microsoft-Dokumentation und weicht bei Ihnen ab. Microsoft aktualisiert die Basisimages nach seinen monatlichen Sicher-

heitsupdates, etwa fünf Arbeitstage nach dem Update-Termin. Ein Windows-Container hat keinen eigenen Aktualisierungsdienst. Zum Aktualisieren holt man die neue Fassung des Basisimages und baut das eigene Image neu, genau wie bei Linux-Images (Kapitel 7).

21.6.2 Isolationsmodi

Windows-Container können auf zwei Arten voneinander und vom Host getrennt sein. Das ist ein Unterschied, den es bei Linux-Containern in dieser Form nicht gibt.

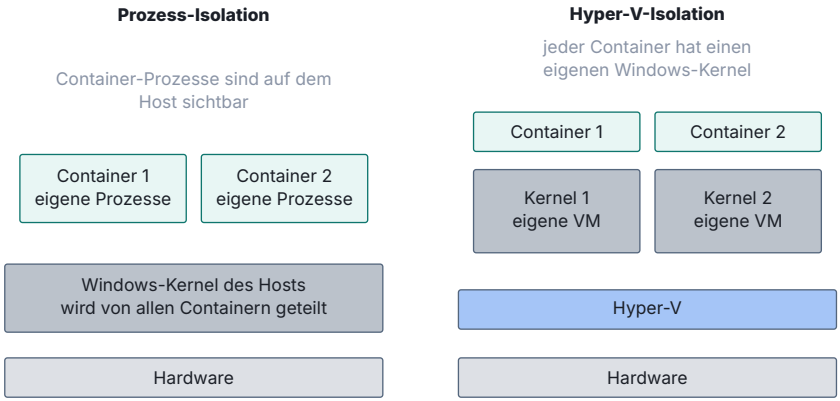


Abbildung 21.2: Prozess-Isolation und Hyper-V-Isolation bei Windows-Containern

Bei der *Prozess-Isolation* teilen sich alle Container den Kernel des Hosts, genau wie Linux-Container. Das ist schlank und schnell. Bei der *Hyper-V-Isolation* bekommt jeder Container eine eigene, für diesen Zweck optimierte kleine virtuelle Maschine mit einem eigenen Windows-Kernel. Das kostet etwas mehr Ressourcen, bietet aber eine deutlich stärkere Trennung und macht den Container unabhängig von der Windows-Version des Hosts. Das Docker-Image bleibt in beiden Fällen dasselbe. Die Wahl trifft man beim Start des Containers mit der Option `--isolation`.

```
PS> docker run --rm --isolation=process mcr.microsoft.com/windows/
    ↪ nanoserver:ltsc2022 cmd /c ver
```

```
PS> docker run --rm --isolation=hyperv  mcr.microsoft.com/windows/
    ↪ nanoserver:ltsc2022 cmd /c ver
```

Welcher Modus ohne Angabe gilt, hängt vom Host ab. Auf Windows 10 und Windows 11 verwenden Windows-Container nach der Dokumentation von Microsoft standardmäßig die Hyper-V-Isolation, damit Entwickler dieselbe Kernel-Version bekommen wie im Betrieb. Auf Windows Server gilt standardmäßig die Prozess-Isolation. Welchen Modus ein Container hat, zeigt `docker inspect`.

```
PS> docker inspect --format '{{.HostConfig.Isolation}}' meincontainer
hyperv
```

21.6.3 Versionskompatibilität

Hier liegt die wichtigste Falle bei Windows-Containern. Unter Windows sind Benutzer- und Kernelteil des Betriebssystems eng verzahnt und erscheinen gemeinsam. Ein Container mit Prozess-Isolation nutzt den Kernel des Hosts und muss deshalb zu dessen Version passen. Nach der Dokumentation von Microsoft gilt: Außer bei Windows Server 2022 zusammen mit Windows 11 lässt Windows Container nicht starten, wenn sich die *Build-Nummer* von Host und Image unterscheidet. Die Build-Nummer ändert sich mit jedem neuen Windows-Release, die Revisionsnummer mit jedem Update. Ein Windows-Server-2016-Container verlangt zusätzlich gleiche Revisionsnummern. Für Hosts und Images ab Windows Server, Version 1809, gilt diese Zusatzregel nicht.

Mit Hyper-V-Isolation entfällt die Einschränkung, weil der Container seinen eigenen passenden Kernel mitbringt. Für die Entscheidung dient die folgende Übersicht. Sie gibt wieder, was Microsoft für einen Windows-Server-2025-Host und für einen Windows-11-Host ausweist.

Basisimage	Server 2025 Prozess	Server 2025 Hyper-V	Windows 11 Prozess	Windows 11 Hyper-V
Windows Server 2025	ja	ja	ja, ab 24H2	ja, ab 24H2
Windows Server 2022	ja	ja	ja	ja
Windows Server 2019	nein	ja	nein	ja
Windows Server 2016	nein	ja	nein	ja

Auf Windows 10 ist die Prozess-Isolation für neuere Images nicht vorgesehen. Für andere Hosts gilt die Regel: Ist der Host neuer als das Image, hilft die Hyper-V-Isolation. Ein Image, das neuer ist als der Host, läuft nicht. Fehlt die Übereinstimmung, bricht der Start mit einer Meldung ab, die man sich merken sollte:

```
docker: Error response from daemon: container ... encountered an
    ↪ error during CreateContainer:
failure in a Windows system call: The operating system of the
    ↪ container does not match the
operating system of the host. (0xc0370101)
```

Die Fehlernummer `0xc0370101` ist das Erkennungszeichen. Es gibt drei Wege, das Problem zu lösen: das Image auf der zum Host passenden Basis neu bauen, den Container mit `--isolation=hyperv` starten, wenn der Host neuer ist, oder einen Host mit passender Windows-Version verwenden. Die Build-Nummern von Host und Container erfährt man so:

```
PS> (Get-ItemProperty 'HKLM:\SOFTWARE\Microsoft\Windows NT\
    ↪ CurrentVersion').CurrentBuild
26100
PS> docker run --rm mcr.microsoft.com/windows/nanoserver:ltsc2022 cmd
    ↪  /c ver
Microsoft Windows [Version 10.0.20348.xxxx]
```

Die Zahlen stehen für Beispiele. Der Host zeigt hier Build 26100, das ist die Basis von Windows Server 2025 und Windows 11 24H2. Das Image trägt Build 20348 für Windows Server 2022. Beides sind verschiedene Builds, und deshalb zeigt dieses Beispiel auch, warum bei Windows 11 die Ausnahme für Server-2022-Images so nützlich ist.

Aktualisierungen können Container brechen

Wer bei Windows Server mit Prozess-Isolation arbeitet und den Host auf eine neuere Windows-Version hebt, kann damit alle Container unbrauchbar machen, deren Image nicht zur neuen Version passt. Planen Sie Host-Updates und Image-Updates zusammen und testen Sie sie zuerst auf einem Testsystem. Microsoft empfiehlt, Host und Container regelmäßig auf den neuesten Stand zu bringen.

21.6.4 Lizenz und Einsatzgebiet

Die Lizenzbedingungen der Windows-Basisimages knüpfen die Nutzung an ein gültig lizenziertes Host-Betriebssystem. Wie viele Container laufen dürfen, hängt von der Edition und vom Isolationsmodus ab. Bei Windows Server Standard sind beliebig viele Container mit Prozess-Isolation und zwei mit Hyper-V-Isolation erlaubt, bei Datacenter in beiden Fällen beliebig viele. Auf Windows 10 und 11 gelten Windows-Container mit Prozess-Isolation als Entwicklungs- und Testwerkzeug. Für den Produktivbetrieb verlangt Microsoft Windows Server als Host. Beachten Sie auch die Ergänzende Lizenz, die beim Herunterladen der Images erscheint.

21.6.5 Dockerfiles für Windows

Die Syntax des Dockerfiles aus Kapitel 8 gilt unverändert. Es gibt einige Besonderheiten. Das Basisimage kommt von `mcr.microsoft.com`. Pfade enthalten Laufwerksbuchstaben und Rückwärtsstriche. Weil der Rückwärtsstrich im Dockerfile zugleich das Fluchtzeichen ist, setzt man mit einer Zeile `# escape=`` den Gravis als Fluchtzeichen. Und die Shell-Form von `RUN` nutzt unter Windows standardmäßig `cmd`. Wer PowerShell in Befehlen verwenden will, stellt das mit der Anweisung `SHELL` um. Ein einfaches Beispiel für eine statische Seite mit IIS sieht so aus:

Dockerfile für Windows

```
# escape=`
FROM mcr.microsoft.com/windows/servercore/iis:windowsservercore-
    ↪ ltsc2022
SHELL ["powershell", "-Command", "$ErrorActionPreference = 'Stop';"]
COPY site/ C:/inetpub/wwwroot/
```

```
EXPOSE 80
```

Die Anweisung `COPY` nimmt Schrägstriche auch bei Windows-Pfaden an. Das Beispiel ist für die Erklärung gedacht und kein Teil der Übungen.

21.6.6 Netzwerk und Speicher

Windows-Container haben einen eigenen Netzwerkstack mit einem virtuellen Switch. Beim ersten Start legt Docker ein Standardnetz mit dem Namen `nat` an, das mit dem Windows-Dienst WinNAT und einem internen Switch arbeitet. Container ohne weitere Angabe hängen an diesem Netz und bekommen dort eine Adresse. Der Standardbereich dafür ist nach der Dokumentation `172.16.0.0/16` und lässt sich über die Einstellung `fixed-cidr` in der Konfigurationsdatei des Daemons ändern. Die Portveröffentlichung mit `-p` funktioniert wie unter Linux.

Windows kennt fünf Netzwerktreiber: `nat`, `overlay`, `transparent`, `l2bridge` und `l2tunnel`. Der Treiber `transparent` entspricht grob einem externen Switch in Hyper-V. Alle Docker-Netzwerkbefehle aus Kapitel 11 gelten mit denselben Schreibweisen, mit Ausnahmen bei einigen Optionen. Ein Netzwerk im Modus `host` gibt es unter Windows *nicht*. Unter Windows Server 2019 und neuer bleiben selbst angelegte NAT-Netze nach einem Neustart nicht erhalten.

> **Windows-Container und Docker Desktop**
>
> Unter Docker Desktop sind Windows-Container im Netzwerk anders zu erreichen als Linux-Container. Für Linux-Container ist das Docker-Netz vom Windows-Host nicht erreichbar. Für Windows-Container ist eine Adressierung über die Container-Adresse dagegen möglich, und `ping` funktioniert.

Images und Schichten liegen bei Windows unter `C:\ProgramData\docker`, mit eigenem Speicherformat, das nicht mit OverlayFS aus Kapitel 2 arbeitet. Das Prinzip aus Kapitel 7 bleibt dasselbe: Schichten, Tags, Digest. Auch Volumes und Bind Mounts gibt es, mit Windows-Pfaden. Die Basisschichten der Microsoft-Images gehen aus lizenzrechtlichen Gründen in der Regel nicht in Ihre Registry. Sie werden von Microsoft bezogen.

21.7 Docker und Container auf Windows Server

Windows Server ist der Ort, an dem Windows-Container produktiv laufen. Hier gilt eine Regel, die nicht oft genug gesagt werden kann: *Docker Desktop ist keine Lösung für Windows Server.* Docker unterstützt Docker Desktop ausdrücklich nicht auf Serverversionen von Windows, also nicht auf Windows Server 2019, 2022 oder 2025. Für den Betrieb auf einem Server verwendet man stattdessen eine Container-Runtime ohne grafische Oberfläche, die als Windows-Dienst läuft.

21.7.1 Container-Runtimes für Windows Server

Nach der Dokumentation von Microsoft werden unter Windows drei Runtimes unterstützt.

Runtime	Eigenschaft
Moby / Docker Community Edition	freie Docker Engine der Open-Source-Gemeinschaft, die Variante dieses Buches
Mirantis Container Runtime	kommerziell unterstützte Fassung (früher Docker Enterprise); Microsoft hat die Unterstützung für die Windows-Container-Runtime an Mirantis übergeben
containerd	Runtime ohne Docker-Komfortschicht, wird vor allem von Kubernetes und anderen Werkzeugen verwendet

Für Lernzwecke und viele Betriebe genügt die Docker Community Edition. Wer Herstellersupport braucht, prüft die Mirantis-Variante. Azure Kubernetes Service (AKS) bietet ein vollständig verwaltetes Angebot, bei dem die Runtime im Dienst enthalten ist.

21.7.2 Installation mit dem Skript von Microsoft

Für Docker Community Edition auf Windows Server stellt Microsoft ein PowerShell-Skript bereit, das die nötigen Windows-Funktionen einschaltet und die Docker Engine installiert. Man lädt es herunter und führt es als Administrator aus. Lesen Sie es vorher, wie jedes Skript aus dem Netz. Was es tut, lässt sich dem Quelltext entnehmen:

Schritt	Aktion des Skripts
1	Prüft die Administratorrechte und schaltet die Windows-Funktion `Containers` ein. Mit dem Parameter `-HyperV` kommt die Hyper-V-Funktion hinzu.
2	Ist ein Neustart nötig, startet es den Rechner neu und richtet eine geplante Aufgabe ein, die das Skript bei der nächsten Anmeldung fortsetzt. Mit `-NoRestart` bricht es stattdessen ab.
3	Lädt die Docker-Programme als ZIP-Datei von `download.docker.com` und kopiert `docker.exe` und `dockerd.exe` nach `C:\Program Files\Docker`. Der Ordner kommt in die Umgebungsvariable `Path`.
4	Legt die Konfigurationsdatei `C:\ProgramData\docker\config\daemon.json` an, die den Daemon auf die lokale Named Pipe beschränkt.
5	Registriert den Windows-Dienst `docker` und startet ihn. Optional lädt es ein Basisimage (`-ContainerBaseImage`).

Das Skript installiert nur die Programme `docker.exe` und `dockerd.exe`. Docker Compose und Buildx gehören nicht dazu. Compose muss man gesondert einspielen, wie Abschnitt 21.8 zeigt. Weitere Parameter sind `-DockerVersion` für eine bestimmte Fassung, `-NATSubnet` für einen anderen Adressbereich des NAT-Netzes und `-TransparentNetwork` für ein Netz, das Adressen per DHCP erhält. Außerdem ist die Installation über die Windows-Admin-Center-Erweiterung *Containers* möglich, die Docker einrichtet und Images und Container grafisch verwaltet. Die Anleitung für die Installation der Binärdateien von Hand steht in der Docker-Dokumentation.

> **Ein Testsystem beschaffen**
>
> Für Tests stellt Microsoft Windows Server 2025 als Evaluierungsversion bereit. Sie können sie in einer virtuellen Maschine installieren und die Übung 5 aus Abschnitt 21.11 darauf durchführen.

21.7.3 Betrieb als Windows-Dienst

Der Docker-Daemon läuft unter Windows als Dienst mit dem Namen `docker`. Verwaltet wird er mit den PowerShell-Cmdlets für Dienste.

```
PS> Get-Service docker
Status   Name               DisplayName
------   ----               -----------
Running  docker             docker

PS> Restart-Service docker
```

Die Konfiguration steht in der Datei `C:\ProgramData\docker\config\daemon.json`, im selben JSON-Format wie unter Linux. Typische Einträge sind die Adresse des NAT-Netzes (`fixed-cidr`), Protokolloptionen und der Standardmodus der Isolation. Ein Standard für Hyper-V-Isolation sieht so aus:

C:\ProgramData\docker\config\daemon.json

```
{
  "exec-opts": ["isolation=hyperv"]
}
```

Nach jeder Änderung startet man den Dienst neu. Die Meldungen des Dienstes landen im Ereignisprotokoll von Windows, und die Daten von Docker liegen unter `C:\ProgramData\docker`. Ein Fehler in der JSON-Datei verhindert, wie unter Linux, den Start des Dienstes (Kapitel 20).

```
PS> Get-EventLog -LogName Application -Source docker -Newest 5
```

Lokal kommuniziert der Client über eine Named Pipe statt über einen Unix-Socket. Das Skript von Microsoft lässt den Daemon *nur* auf dieser Pipe lauschen. Wer Docker aus der Ferne steuern will, richtet TLS mit Zertifikaten ein oder verwendet einen SSH-Kontext vom Client aus, wie in Kapitel 3. Ein ungesicherter TCP-Zugang gehört auch unter Windows nicht ins Netz.

21.7.4 Produktionsbetrieb

Für den Betrieb auf Windows Server gelten zusätzlich zu den Regeln der Kapitel 18 bis 20 einige Punkte, die sich aus dem Windows-Modell ergeben. Host und Images müssen zueinander passen (Abschnitt 21.6.3). Der Host wird mit den Windows-Updates aktuell gehalten, die Images werden regelmäßig neu gebaut. Für Anwendungen, die nicht vertrauenswürdig sind oder verschiedene Mandanten trennen sollen, ist die Hyper-V-Isolation angeraten, denn sie sorgt dafür, dass Code in einem Container weder den Host noch andere Container beeinträchtigen kann. Windows-Container können mit gruppenverwalteten Dienstkonten (gMSA) bei Active Directory angemeldet werden, was für viele Unternehmensanwendungen nötig ist. Die Orchestrierung mit Docker Swarm oder Kubernetes ist möglich und geht über dieses Buch hinaus.

Ein Sicherheitsaspekt unterscheidet sich vom Linux-Verhalten aus Kapitel 11. Bei Veröffentlichung eines Ports legt der Host-Netzwerkdienst automatisch die zugehörigen Weiterleitungsregeln *und* die passenden Erlaubnisregeln der Windows-Firewall an. Ein veröffentlichter Port ist damit auch unter Windows Server in der Regel von außen erreichbar. Veröffentlichen Sie nur, was erreichbar sein soll.

21.8 Docker Compose unter Windows

Docker Compose funktioniert unter Windows nach denselben Regeln wie unter Linux. Die Kapitel 12 bis 14 gelten. Die Unterschiede betreffen die Installation und einige Besonderheiten der Umgebung.

21.8.1 Installation

Docker Desktop bringt Compose als Plugin mit, sodass `docker compose` sofort funktioniert. Auf Windows Server fehlt es, wenn man Docker mit dem Skript von Microsoft installiert hat. Die Docker-Dokumentation beschreibt dafür die eigenständige Variante (*Standalone*), die sie allerdings als Altlast (*Legacy*) bezeichnet und nur für Fälle empfiehlt, in denen weder Docker Desktop noch das Plugin infrage kommen. Auf Windows Server ist das der dokumentierte Weg. Die Schreibweise lautet `docker-compose` mit Bindestrich.

```
PS> [Net.ServicePointManager]::SecurityProtocol = [Net.
    ↪ SecurityProtocolType]::Tls12
PS> Start-BitsTransfer -Source "https://github.com/docker/compose/
    ↪ releases/download/v5.4.0/docker-compose-windows-x86_64.exe" `
>>     -Destination $Env:ProgramFiles\Docker\docker-compose.exe
PS> docker-compose.exe version
Docker Compose version v5.4.0
```

Die erste Zeile stellt sicher, dass für die Verbindung zu GitHub TLS 1.2 genutzt wird. Die Versionsnummer `v5.4.0` stammt aus der Dokumentation. Verwenden Sie die aktuelle Fassung von der Releaseseite des Projekts und ersetzen Sie die Nummer in der Adresse. Der Ordner `C:\Program Files\Docker` ist bereits im `Path`, sodass der Befehl danach überall funktioniert. Laden Sie die Datei wie jede Software nur aus der offiziellen Quelle. Es gibt zusätzlich die Möglichkeit, die Datei als Plugin des Docker-Clients abzulegen, damit `docker compose` mit Leerzeichen funktioniert. Diese Variante ist in der Dokumentation für Linux beschrieben, für Windows Server habe ich sie nicht geprüft.

21.8.2 Compose-Dateien für Windows

Das Dateiformat ist dasselbe, `compose.yaml` mit `services`, `networks` und `volumes`. Für Linux-Container unter Docker Desktop lassen sich die Dateien aus dem Buch unverändert verwenden. Bei Windows-Containern ändert sich der Inhalt: Das Image ist ein Windows-Image, Pfade sind Windows-Pfade, und die Netzwerke folgen den Windows-Treibern. Ein einfaches Beispiel mit IIS:

compose.yaml für Windows-Container

```
services:
  iis:
    image: mcr.microsoft.com/windows/servercore/iis:windowsservercore-
    ↪ ltsc2022
    ports:
      - "8080:80"
    isolation: process
```

Das Attribut `isolation` gehört zur Compose-Spezifikation und legt den Isolationsmodus des Dienstes fest. Ein Compose-Projekt läuft immer gegen *einen* Dae-

mon: Mit Docker Desktop entweder gegen den Linux- oder gegen den Windows-Daemon, je nachdem, was gerade aktiv ist. Ein Projekt kann Linux- und Windows-Container deshalb nicht mischen.

21.8.3 Besonderheiten der Windows-Umgebung

Die Platzhalter aus Kapitel 12 funktionieren mit PowerShell wie gewohnt. Umgebungsvariablen setzt man mit `$env:NAME = 'wert'` vor dem Aufruf, und eine Datei `.env` liest Compose wie unter Linux. Drei Fehlerquellen sollten Sie kennen. Erstens die Kodierung: Eine Datei mit UTF-16 oder mit BOM kann zu unlesbaren Konfigurationen führen, wie der Abschnitt über die PowerShell erklärt hat. Zweitens die Zeilenenden: Skripte und Konfigurationen, die in Linux-Containern gelesen werden, müssen mit Linux-Zeilenenden gespeichert sein. Drittens die Pfade: Relative Pfade mit `./` funktionieren, und bei Linux-Containern unter WSL 2 sind Verzeichnisse im Linux-Dateisystem deutlich schneller als solche unter `C:\`.

21.9 Docker Desktop und Sicherheit

Die Sicherheitsüberlegungen aus Kapitel 18 gelten auch unter Windows. Hinzu kommen Punkte, die sich aus dem Aufbau von Docker Desktop und aus Windows ergeben.

21.9.1 Rechte und Dienste

Im Installationsmodus für alle Benutzer läuft ein privilegierter Hilfsdienst mit dem Namen `com.docker.service` mit SYSTEM-Rechten. Er führt einige Aufgaben aus, die höhere Rechte brauchen, und lauscht auf einer Named Pipe, die nur Mitglieder der Gruppe `docker-users` nutzen dürfen. Daraus folgt eine Warnung, die Docker in der eigenen Dokumentation ausspricht: Die Mitgliedschaft in `docker-users` gewährt Zugriff auf den Docker-Daemon und ist damit gleichbedeutend mit Administratorrechten auf dem Host. Das ist dasselbe Prinzip wie bei der Gruppe `docker` unter Linux (Kapitel 18). Nehmen Sie nur Personen auf, die Windows-Container oder die Hyper-V-Verwaltung wirklich brauchen. Für Linux-Container mit dem WSL-2-Backend ist die Mitgliedschaft nicht nötig.

```
PS> net localgroup docker-users
PS> net localgroup docker-users MeinBenutzer /add
```

Bei aktivierten Windows-Containern gibt es einen zusätzlichen Punkt. Der Benutzer `ContainerAdministrator`, mit dem man innerhalb eines Windows-Containers verwaltet, ist dann zugleich ein lokaler Administrator auf dem Host. Wer Windows-Container beim Installieren aktiviert, ermöglicht damit den Mitgliedern von `docker-users`, sich zu Administratoren des Hosts hochzustufen. Wer keine Windows-Container braucht, schaltet sie bei der Installation mit der Option `--no-windows-containers` ab oder wählt den Modus pro Benutzer, in dem sie nicht möglich sind.

21.9.2 Der Daemon im Netz

Docker Desktop kann den Daemon unverschlüsselt über TCP auf dem lokalen Rechner anbieten, über eine Einstellung, deren Bezeichnung in den Einstellungen unter *General* steht. Die Voreinstellung ist aus. Lassen Sie sie ausgeschaltet. Wer sie einschaltet, öffnet jedem Programm auf dem Rechner den Zugang zu Docker und damit zu den Rechten aus dem vorigen Abschnitt, ganz ohne Anmeldung.

21.9.3 Das gemeinsame Kernel-Modell von WSL 2

Bei WSL 2 teilen sich alle Distributionen und die Docker-Umgebung einen Kernel und eine virtuelle Maschine. Das ist praktisch, aber es ist ein Isolationsmodell, das nicht dem eines eigenen Hypervisors je Container entspricht. Für Umgebungen mit strengeren Anforderungen nennt die Docker-Dokumentation zwei Wege. Docker Desktop lässt sich im Hyper-V-Modus statt mit WSL 2 betreiben, was das gemeinsame Kernel-Modell vermeidet. Und im *Business*-Abonnement gibt es die Funktion *Enhanced Container Isolation* (ECI), die alle Container in einem Linux-User-Namespace laufen lässt, sodass Root im Container einem unprivilegierten Benutzer in der virtuellen Maschine entspricht. ECI setzt WSL ab Version 2.6 voraus, weil es einen Kernel ab Version 6.3 braucht.

21.9.4 Weitere Maßnahmen

Bereich	Maßnahme
Installationsmodus	Pro Benutzer wählen, wenn keine Windows-Container nötig sind
Aktualität	Docker Desktop und WSL regelmäßig aktualisieren (`wsl --update`)
Installationsdatei	Prüfsumme der heruntergeladenen Datei mit `Get-FileHash` gegen die Angabe in den Release-Hinweisen prüfen
Anmeldedaten	Registry-Anmeldungen nur über den Windows-Anmeldeinformationsspeicher, nie im Klartext in Dateien
Firewall	Beim ersten Start fragt Windows, in welchen Netzen Docker kommunizieren darf. Nur private Netze erlauben.
Ports	Dienste für die Entwicklung mit `-p 127.0.0.1:...` an die lokale Adresse binden
Images und Lieferkette	Regeln aus Kapitel 18: feste Tags oder Digests, keine Fremdimages ohne Prüfung
Datenträger	Auf Notebooks die Festplatte verschlüsseln (BitLocker), denn Images und Volumes liegen lesbar auf der Platte
Lizenz	Lizenzbedingungen von Docker Desktop einhalten

21.10 Vergleich: Docker unter Linux und Windows

Die Tabelle stellt die Möglichkeiten nebeneinander. Sie zeigt auch, wofür man welche einsetzt.

Merkmal	Linux-Server	Docker Desktop (Linux-Container)	Windows Server (Windows-Container)
Kernel	Linux des Hosts	Linux in der WSL-2-VM	Windows des Hosts
Images	Linux-Images	Linux-Images	Windows-Images von Microsoft
Image-Größe	klein, ab wenigen MB	klein, ab wenigen MB	groß, ab etwa 300 MB bis mehrere GB
Typischer Einsatz	Produktion und Entwicklung	Entwicklung am Arbeitsplatz	Produktion von Windows-Anwendungen
Produktiv geeignet	ja	nein, ein Entwicklungswerkzeug	ja
Installation	Paket aus dem Docker-Repository	Installer	Skript, Admin Center oder Binärdateien
Daemon läuft als	systemd-Dienst	Dienst der Desktop-Anwendung	Windows-Dienst
Schnittstelle des Clients	Unix-Socket	Named Pipe	Named Pipe
Isolation	Namespaces und cgroups	wie Linux, in einer VM	Prozess- oder Hyper-V-Isolation
Netzwerk	Bridge, host, macvlan und andere	Bridge in der VM, kein `docker0` auf dem Host	`nat`, `transparent`, `overlay`, `l2bridge`; kein `host`
Firewall	iptables oder nftables, `DOCKER-USER`	Windows-Firewall am Host	Windows-Firewall, Regeln entstehen automatisch
Rechte im Dateisystem	Benutzer- und Gruppennummern	Abbildung zwischen Windows und Linux	Zugriffslisten (ACL)
Lizenz	Engine frei	Docker Desktop mit Bedingungen	Windows-Lizenz für Host und Images
Compose	Plugin	Plugin enthalten	Standalone `docker-compose`

Die Wahl folgt aus der Anwendung. Läuft die Anwendung unter Linux, ist ein Linux-Server die erste Wahl für den Betrieb, und am Arbeitsplatz genügt Docker Desktop mit WSL 2 oder eine Engine in WSL. Gibt es die Anwendung nur für Windows, etwa mit .NET Framework oder IIS, braucht man Windows-Container auf Windows Server. Für Tests von Windows-Anwendungen am Arbeitsplatz dient Docker Desktop mit Windows-Containern. Für Produktion bleibt es Windows Server.

21.11 Praxisprojekte

Die folgenden fünf Laborübungen setzen den Stoff dieses Kapitels in die Praxis um. Sie bauen aufeinander auf. Die ersten drei laufen auf einem Windows-11-Rechner mit Docker Desktop und Linux-Containern, die vierte untersucht Windows-Container auf demselben Rechner, die fünfte baut ein Labor mit Windows Server auf.

Übung	Thema	Umgebung
1	Docker Desktop auf Windows 11 installieren	Windows 11 Pro, Enterprise oder Education
2	nginx unter Windows mit Linux-Containern	Windows 11 mit Docker Desktop
3	Docker Compose mit PowerShell	Windows 11 mit Docker Desktop
4	Windows-Container untersuchen	Windows 11 Pro oder Enterprise, Docker Desktop für alle Benutzer
5	Windows-Server-Labor mit Container-Runtime	virtuelle Maschine mit Windows Server 2022 oder 2025

Alle Befehle sind für die PowerShell geschrieben. Wo Administratorrechte nötig sind, steht es dabei: Dann öffnen Sie das Windows Terminal oder die PowerShell mit einem Rechtsklick und dem Eintrag *Als Administrator ausführen*. Die Dateien der Übungen legen Sie mit dem Windows-Editor (Notepad) an und speichern sie als *UTF-8*. In Windows 11 ist das die Voreinstellung. Wählen Sie nicht „UTF-8 mit Signatur“, denn das BOM bringt nginx zum Absturz (Abschnitt über die PowerShell).

Hinweis zur Prüfung der Übungen

Die Befehle dieser Übungen habe ich anhand der offiziellen Dokumentation von Microsoft und Docker zusammengestellt und geprüft. Ausgeführt habe ich sie nicht, denn mir stand kein Windows-System zur Verfügung. Die gezeigten Ausgaben sind Beispiele aus der Dokumentation oder typische Ausgaben und weichen bei Ihnen ab. Prüfen Sie vor dem Einsatz im Unterricht, ob Menübezeichnungen, Versionsnummern und Tags noch stimmen.

21.11.1 Übung 1: Docker Desktop auf Windows 11 installieren

Ziel: Docker Desktop mit dem WSL-2-Backend im Modus pro Benutzer installieren und die Installation prüfen.

Voraussetzungen

Voraussetzung	Prüfung
Windows 11 Pro, Enterprise oder Education, Version 23H2 (Build 22631) oder neuer	Schritt 1
Hardware-Virtualisierung in der Firmware eingeschaltet	Schritt 2
Dienst `LanmanServer` aktiv und auf Automatisch	Schritt 3
mindestens 8 GB Arbeitsspeicher, 20 GB freier Plattenplatz	Task-Manager
Administratorrechte für die einmalige Einrichtung von WSL	Konto prüfen
Internetzugang	Browser

Schritt 1: Windows-Version prüfen

```
PS> Get-ComputerInfo -Property OsName, OsVersion, OsBuildNumber
OsName         : Microsoft Windows 11 Pro
OsVersion      : 10.0.26100
OsBuildNumber  : 26100
```

Die Build-Nummer muss mindestens 22631 betragen. Die Registry-Angabe `ProductName` wäre hier unzuverlässig, weil sie auch unter Windows 11 teilweise noch „Windows 10“ nennt.

Schritt 2: Virtualisierung prüfen

```
PS> Get-CimInstance Win32_Processor | Select-Object Name,
    ↪ VirtualizationFirmwareEnabled
Name                                  VirtualizationFirmwareEnabled
----                                  -----------------------------
12th Gen Intel(R) Core(TM) i7-1255U                            True
```

Steht dort `False`, schalten Sie im BIOS oder UEFI die Virtualisierung ein (bei Intel *VT-x*, bei AMD *SVM* oder *AMD-V*). Läuft Windows selbst in einer virtuellen Maschine, muss der Hypervisor die verschachtelte Virtualisierung freigeben.

Schritt 3: Den Dienst LanmanServer prüfen

```
PS> Get-Service LanmanServer | Select-Object Name, Status, StartType
Name          Status StartType
----          ------ ---------
LanmanServer Running Automatic
```

Ist der Dienst gestoppt oder deaktiviert, stellen Sie ihn in einer Administrator-PowerShell um: `Set-Service LanmanServer -StartupType Automatic` und `Start-Service LanmanServer`.

Schritt 4: WSL installieren und aktualisieren

Öffnen Sie die PowerShell als Administrator.

```
PS> wsl --install
```

Starten Sie den Rechner neu, wenn der Befehl dazu auffordert. Beim ersten Start der Distribution Ubuntu legen Sie einen Benutzernamen und ein Passwort an. Danach prüfen Sie die Version.

```
PS> wsl --update
PS> wsl --version
WSL-Version: 2.x.y.z
Kernelversion: 6.6.x.y-1
...
PS> wsl --list --verbose
  NAME      STATE           VERSION
* Ubuntu    Running         2
```

Die WSL-Version muss 2.1.5 oder neuer sein, und die Distribution muss unter `VERSION` die 2 zeigen.

Schritt 5: Installer laden und prüfen

```
PS> $installer = "$env:USERPROFILE\Downloads\Docker Desktop Installer.
    ↪ exe"
PS> Invoke-WebRequest -Uri 'https://desktop.docker.com/win/main/amd64
    ↪ /Docker%20Desktop%20Installer.exe' -OutFile $installer
PS> Get-FileHash $installer -Algorithm SHA256
```

Der Hash der Datei steht in den Release-Hinweisen von Docker Desktop. Vergleichen Sie beide Werte. Stimmen sie nicht überein, ist die Datei beschädigt oder verändert, und Sie installieren nicht.

Schritt 6: Im Modus pro Benutzer installieren

```
PS> Start-Process -FilePath $installer -Wait -ArgumentList 'install',
    ↪  '--user'
```

Der Parameter `--user` wählt den Modus pro Benutzer. Die Installation landet im Benutzerprofil und braucht keine Administratorrechte. Starten Sie danach Docker Desktop über das Startmenü. Beim ersten Start erscheint die Lizenzvereinbarung. Lesen Sie die Zusammenfassung (kostenlos für kleine Unternehmen, persönlichen Gebrauch, Bildung und nicht kommerzielle Open-Source-Projekte) und stimmen Sie zu, wenn Ihre Nutzung darunter fällt. Warten Sie, bis die Oberfläche anzeigt, dass die Engine läuft.

Schritt 7: Installation prüfen

```
PS> docker version
Client:
 Version:           29.x.y
 ...
Server: Docker Desktop 4.x.y (...)
 Engine:
  Version:          29.x.y
  OS/Arch:          linux/amd64

PS> docker info --format '{{.OSType}} {{.ServerVersion}}'
linux 29.x.y
PS> docker run --rm hello-world
Hello from Docker!
This message shows that your installation appears to be working
    ↪ correctly.
PS> docker compose version
Docker Compose version vX.Y.Z
```

Schritt 8 (optional): Ressourcen begrenzen

Legen Sie mit Notepad die Datei `%UserProfile%\.wslconfig` an, mit diesem Inhalt:

%UserProfile%\.wslconfig

```
[wsl2]
memory=6GB
processors=4
swap=2GB
```

```
PS> wsl --shutdown
PS> docker info --format 'Speicher: {{.MemTotal}} Byte, Prozessoren:
    ↪ {{.NCPU}}'
```

Nach dem `wsl --shutdown` startet Docker Desktop die Engine neu (starten Sie es bei Bedarf selbst). Der Wert für den Speicher liegt danach bei etwa 6 GB und die Zahl der Prozessoren bei 4. Passen Sie die Zahlen an Ihre Hardware an.

Erwartetes Ergebnis

Docker Desktop läuft. `docker version` zeigt Client und Server, `docker run --rm hello-world` gibt die Begrüßung aus, und `docker compose version` nennt eine Version. `wsl --list --verbose` zeigt die Distributionen mit `VERSION 2`.

Fehlerdiagnose

Symptom	Ursache und Abhilfe
Meldung, dass die Virtualisierung nicht aktiviert ist	Firmware-Einstellung (Schritt 2) prüfen, bei virtuellen Maschinen verschachtelte Virtualisierung freigeben.
Hinweis, WSL müsse aktualisiert werden, oder Docker Desktop startet nicht	`wsl --update` in einer Administrator-PowerShell, danach `wsl --shutdown` und Docker Desktop neu starten.
`wsl --version` zeigt keine Version	Es läuft die ältere, eingebaute WSL-Fassung. `wsl --update` installiert die aktuelle.
`The system cannot find the file specified` bei einem Docker-Befehl	Docker Desktop läuft nicht oder die Engine ist noch nicht bereit. Programm starten und warten.
`docker` wird nicht gefunden	Neues Terminal öffnen, damit der Suchpfad neu geladen wird.
Docker Desktop bleibt im Startzustand	Dienst `LanmanServer` prüfen (Schritt 3), Rechner neu starten.
Images lassen sich nicht laden	Netzwerk, VPN und Proxy prüfen (Einstellungen von Docker Desktop, Rubrik Proxies).

Sicherheitsinformationen

Die Installation pro Benutzer richtet keinen privilegierten Systemdienst ein und hat deshalb die kleinere Angriffsfläche. Laden Sie den Installer nur von der Seite von Docker und prüfen Sie den Hash. Fügen Sie Ihren Benutzer nicht ohne Not der Gruppe `docker-users` hinzu, weil das Administratorrechten gleichkommt. Lassen Sie die Einstellung für den unverschlüsselten TCP-Zugang zum Daemon ausgeschaltet. Halten Sie Docker Desktop und WSL aktuell. Und klären Sie die Lizenzfrage, bevor Sie Docker Desktop beruflich einsetzen.

21.11.2 Übung 2: nginx unter Windows mit Linux-Containern

Ziel: Einen nginx-Webserver in einem Linux-Container betreiben, dessen Webseite und Konfiguration auf Windows liegen, und das Verhalten mit dem Linux-Dateisystem von WSL vergleichen.

Voraussetzungen

Übung 1 ist abgeschlossen, Docker Desktop läuft im Modus für Linux-Container (`docker info --format '{{.OSType}}'` meldet `linux`). Der Port 8080 ist frei. Die Ordner dieser Übung legen Sie unter `C:\Labor\nginx` an.

Schritt 1: Ordner und Dateien

```
PS> New-Item -ItemType Directory -Force C:\Labor\nginx\html | Out-
    ↪ Null
PS> Set-Location C:\Labor\nginx
PS> notepad html\index.html
```

Notepad fragt, ob die Datei angelegt werden soll. Bestätigen Sie, fügen Sie den folgenden Inhalt ein und speichern Sie als UTF-8.

html\index.html

```
<!DOCTYPE html>
<html lang="de">
<head><meta charset="utf-8"><title>nginx unter Windows</title></head>
<body><h1>Hallo aus nginx unter Windows</h1></body>
</html>
```

Danach legen Sie mit `notepad default.conf` die Konfiguration an, ebenfalls als UTF-8.

default.conf

```
server {
    listen 80;
    server_name _;
    charset utf-8;

    root /usr/share/nginx/html;
    index index.html;

    location / {
        try_files $uri $uri/ =404;
```

```
    }
}
```

Schritt 2: Container starten

```
PS> docker run -d --name nginx-lab -p 127.0.0.1:8080:80 `
>>     -v "${PWD}\html:/usr/share/nginx/html:ro" `
>>     -v "${PWD}\default.conf:/etc/nginx/conf.d/default.conf:ro" `
>>     nginx:stable-alpine
```

Der Gravis am Zeilenende setzt den Befehl in der nächsten Zeile fort. Er muss das *letzte* Zeichen der Zeile sein, ein Leerzeichen dahinter bricht den Befehl. Die beiden Einbindungen sind schreibgeschützt (`:ro`), und der Port ist an die lokale Adresse gebunden, sodass nur dieser Rechner den Server erreicht.

Schritt 3: Testen

```
PS> curl.exe -s http://localhost:8080
<!DOCTYPE html>
<html lang="de">
...
PS> curl.exe -sI http://localhost:8080 | Select-String "HTTP", "
    ↪ Server", "Content-Type"
HTTP/1.1 200 OK
Server: nginx/1.x.y
Content-Type: text/html; charset=utf-8
```

Öffnen Sie dieselbe Adresse zusätzlich im Browser. Dort steht die Überschrift mit Umlauten.

Schritt 4: Änderungen, Protokoll und Konfigurationsprüfung

Ändern Sie in Notepad den Text der `html\index.html`, speichern Sie und laden Sie die Seite im Browser neu. Die Änderung ist sofort sichtbar, ohne dass der Container neu gestartet wird. Danach prüfen Sie die Konfiguration und die Protokolle.

```
PS> docker exec nginx-lab nginx -t
nginx: the configuration file /etc/nginx/nginx.conf syntax is ok
nginx: configuration file /etc/nginx/nginx.conf test is successful
PS> docker logs --tail 3 nginx-lab
172.17.0.1 - - [11/Nov/2026:15:20:01 +0000] "GET / HTTP/1.1" 200 138
    ↪ "-" "curl/8.x" "-"
PS> docker exec nginx-lab uname -r
6.6.x.y-microsoft-standard-WSL2
```

Der Kernel im Container trägt den Namen von WSL 2, wie im Abschnitt über Linux-Container erklärt. Das Adresspräfix in den Protokollzeilen ist ein Beispiel und kann bei Docker Desktop anders aussehen.

Schritt 5: Variante im Linux-Dateisystem von WSL

Öffnen Sie das Terminal der Distribution Ubuntu (Startmenü oder `wsl` in der PowerShell). Voraussetzung ist, dass in den Einstellungen von Docker Desktop die WSL-Integration für diese Distribution eingeschaltet ist. Dann arbeiten Sie dort, im Linux-Dateisystem:

```
$ mkdir -p ~/labor2/html
$ echo '<h1>Aus dem Linux-Dateisystem</h1>' > ~/labor2/html/index.
    ↪ html
$ docker run -d --name nginx-wsl -p 127.0.0.1:8081:80 -v ~/labor2/
    ↪ html:/usr/share/nginx/html:ro nginx:stable-alpine
```

Zurück in der PowerShell prüfen Sie den zweiten Server mit `curl.exe -s http://localhost:8081`. Beide Container liefern nginx. Der Unterschied liegt im Ort der Dateien: Beim ersten liegen sie im Windows-Dateisystem, beim zweiten im Linux-Dateisystem der Distribution. Letzteres ist für größere Projekte deutlich schneller und liefert Dateiänderungen als Ereignisse an den Container, wie die Dokumentation von Docker ausführt.

Aufräumen

```
PS> docker rm -f nginx-lab nginx-wsl
```

Erwartetes Ergebnis

`curl.exe` liefert in beiden Fällen die jeweilige Seite mit dem Statuscode 200. Die Änderung der Datei unter Windows ist ohne Neustart sichtbar. `nginx -t` meldet eine gültige Konfiguration.

Fehlerdiagnose

Symptom	Ursache und Abhilfe
`port is already allocated`	Der Port ist belegt. `Get-NetTCPConnection -LocalPort 8080` zeigt den Besitzer (Spalte `OwningProcess`), `Get-Process -Id NUMMER` nennt das Programm. Anderen Port wählen.
`invalid reference format`	Ein Gravis steht nicht am Zeilenende, oder dahinter folgt ein Leerzeichen. Befehl in einer Zeile eingeben.
nginx beendet sich sofort, `docker logs` nennt `unknown directive`	Die `default.conf` hat ein BOM. Datei neu als *UTF-8* (ohne Signatur) speichern.
Die Standardseite von nginx erscheint	Der Pfad der Einbindung stimmt nicht. `docker inspect nginx-lab` zeigt unter `Mounts` die Quelle. Aus dem Ordner `C:\Labor\nginx` starten.
`404 Not Found`	Die Datei `index.html` liegt nicht im Ordner `html`, oder der Name ist falsch geschrieben.
Der Browser erreicht den Server nicht	`docker ps` zeigt, ob der Container läuft und welche Ports er veröffentlicht. VPN und Firewall prüfen.
`mount denied` oder Pfad nicht freigegeben	Der Pfad liegt auf einem Laufwerk, das Docker nicht kennt, oder enthält Zeichen, die Probleme machen. Unter `C:\` bleiben.

Sicherheitsinformationen

Die Bindung an `127.0.0.1` verhindert, dass andere Rechner im Netz auf den Entwicklungsserver zugreifen. Windows fragt beim ersten Start gegebenenfalls, ob

Docker in privaten oder öffentlichen Netzen kommunizieren darf. Erlauben Sie nur private Netze. Die Einbindungen sind schreibgeschützt, und in die Webseite gehören keine Zugangsdaten. Der nginx-Container läuft mit dem Standard-Image als Root. Für den Betrieb gelten die Regeln aus Kapitel 18.

21.11.3 Übung 3: Docker Compose mit PowerShell

Ziel: Zwei Dienste mit Compose unter Windows betreiben und dabei die PowerShell-Eigenheiten kennenlernen: Variablen, Filter, Zeichenkodierung.

Voraussetzungen

Übung 1 ist abgeschlossen, Docker Desktop läuft mit Linux-Containern, `docker compose version` meldet eine Version. Der Port 8082 ist frei.

Schritt 1: Projektordner und Dateien

```
PS> New-Item -ItemType Directory -Force C:\Labor\compose\site, C:\
    ↪ Labor\compose\nginx | Out-Null
PS> Set-Location C:\Labor\compose
```

Legen Sie die folgenden Dateien mit Notepad an und speichern Sie sie als UTF-8.

site\index.html

```
<!DOCTYPE html>
<html lang="de">
<head><meta charset="utf-8"><title>Compose unter Windows</title></
    ↪ head>
<body><h1>Antwort der Anwendung</h1></body>
</html>
```

nginx\default.conf

```
server {
    listen 80;

    location / {
```

```
        proxy_pass http://app:8000;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
    }
}
```

.env

```
HTTP_PORT=8082
```

compose.yaml

```
name: labor3

services:
  web:
    image: nginx:stable-alpine
    restart: unless-stopped
    ports:
      - "127.0.0.1:${HTTP_PORT:-8080}:80"
    volumes:
      - ./nginx/default.conf:/etc/nginx/conf.d/default.conf:ro
    depends_on:
      app:
        condition: service_healthy

  app:
    image: python:3-slim
    restart: unless-stopped
    working_dir: /srv/site
    command: ["python", "-m", "http.server", "8000"]
    volumes:
      - ./site:/srv/site:ro
    healthcheck:
      test: ["CMD", "python", "-c", "import urllib.request; urllib.
    ↪ request.urlopen('http://localhost:8000/')"]
      interval: 10s
      timeout: 3s
```

```
      retries: 3
      start_period: 5s
```

Dass `.env` als Datei ohne Namensteil beginnt, ist bei Notepad ein Stolperstein: Wählen Sie beim Speichern als Dateityp „Alle Dateien", sonst hängt Notepad `.txt` an. Ob die Dateinamen stimmen, zeigt `Get-ChildItem -Force`.

Schritt 2: Prüfen und starten

```
PS> docker compose config --quiet; if ($?) { "Datei ist gültig" }
Datei ist gültig
PS> docker compose up -d --wait
[+] Running 3/3
 ✔ Network labor3_default  Created
 ✔ Container labor3-app-1  Healthy
 ✔ Container labor3-web-1  Started
PS> docker compose ps --format "table {{.Service}}\t{{.Status}}\t{{.
    ↪ Ports}}"
SERVICE   STATUS                    PORTS
app       Up 15 seconds (healthy)   8000/tcp
web       Up 14 seconds             127.0.0.1:8082->80/tcp
```

Die Prüfung mit `if ($?)` ersetzt das `&&` der Bash, das Windows PowerShell 5.1 nicht kennt. Die Zahl hinter `Running` im Beispiel hängt von der Compose-Version ab.

Schritt 3: Testen und Protokolle filtern

```
PS> curl.exe -s http://localhost:8082
<!DOCTYPE html>
...
PS> docker compose logs --tail 5
PS> docker compose logs | Select-String "GET"
```

Das Protokoll des Python-Servers enthält die Zeile mit der Anfrage, und `Select-String` zeigt nur die passenden Zeilen. Das Ergebnis entspricht dem `grep` unter Linux.

Schritt 4: Variablen aus der PowerShell

Eine Umgebungsvariable der Shell überstimmt die Datei `.env`. Das lässt sich mit der PowerShell-Schreibweise prüfen.

```
PS> $env:HTTP_PORT = '8083'
PS> docker compose up -d
PS> docker compose ps --format "table {{.Service}}\t{{.Ports}}"
SERVICE   PORTS
app       8000/tcp
web       127.0.0.1:8083->80/tcp
PS> Remove-Item Env:\HTTP_PORT
PS> docker compose up -d
```

Compose hat den Dienst `web` mit dem neuen Port neu erzeugt. Nach dem Entfernen der Variablen gilt wieder der Wert aus `.env`.

Schritt 5: Befehl im Dienst und Ausgabe weiterverarbeiten

```
PS> docker compose exec app python --version
Python 3.x.y
PS> docker compose ps -q | ForEach-Object { docker inspect --format
    ↪ '{{.Name}}: {{.State.Status}}' $_ }
/labor3-app-1: running
/labor3-web-1: running
```

Aufräumen

```
PS> docker compose down
PS> Set-Location C:\
```

Erwartetes Ergebnis

Beide Dienste laufen, der Webserver reicht die Anfrage an die Anwendung weiter, und `curl.exe` zeigt die Seite. Die Umgebungsvariable `$env:HTTP_PORT` ändert den Port beim nächsten `up`. Nach `down` sind Container und Netzwerk entfernt.

Fehlerdiagnose

Symptom	Ursache und Abhilfe
`docker compose` nicht erkannt oder Verbindung zum Daemon scheitert	Docker Desktop läuft nicht, oder das Terminal wurde vor der Installation geöffnet. Docker Desktop starten, Terminal neu öffnen.
Meldung über Zeichen, die nicht gelesen werden können, oder `yaml`:-Fehler mit Zeilennummer	Einrückung mit Tabulatoren oder falsche Kodierung der Datei. Mit Leerzeichen einrücken, als UTF-8 speichern.
Die Variable `HTTP_PORT` wird nicht erkannt	Die Datei heißt `.env.txt` oder liegt im falschen Ordner. `Get-ChildItem -Force` zeigt es.
`app` bleibt `unhealthy`	`docker compose logs app` zeigt den Fehler, meist ein falscher Pfad in `volumes` oder ein Fehler im Pfad der `site`.
`502 Bad Gateway`	nginx erreicht die Anwendung nicht. `docker compose ps` prüfen, `docker compose restart web` versuchen.
Port belegt	Wie in Übung 2: `Get-NetTCPConnection -LocalPort 8082`.
`docker compose config > datei.yaml` erzeugt eine unlesbare Datei	In Windows PowerShell 5.1 schreibt `>` UTF-16. Mit PowerShell 7 oder dem Cmdlet `Out-File -Encoding ascii` arbeiten.

Sicherheitsinformationen

Die `.env` enthält in dieser Übung keine Geheimnisse, enthält sie später welche, gehört sie nicht in ein Repository (Kapitel 12). Die Ports sind an `127.0.0.1` gebunden, und die Quellordner sind schreibgeschützt eingebunden. Der Dienst `app` läuft mit dem Image `python:3-slim` als Root. Für mehr als eine Übung wäre ein Image mit eigenem Benutzer nötig, wie in Kapitel 8 gezeigt.

21.11.4 Übung 4: Windows-Container untersuchen

Ziel: Windows-Container auf Windows 11 starten, die beiden Isolationsmodi vergleichen und einen Versionskonflikt gezielt auslösen und beheben.

Voraussetzungen

Voraussetzung	Hinweis
Windows 11 Pro oder Enterprise	Home und Education erlauben keine Windows-Container
Docker Desktop im Modus *für alle Benutzer*	Der Modus pro Benutzer aus Übung 1 unterstützt keine Windows-Container
Windows-Funktionen *Hyper-V* und *Containers*	Schritt 1
Administratorrechte	für Installation und Wechsel
etwa 10 GB freier Plattenplatz	Windows-Images sind groß

Schritt 0: Installationsmodus wechseln

Wenn Sie Übung 1 im Modus pro Benutzer durchgeführt haben, müssen Sie Docker Desktop deinstallieren und im Modus für alle Benutzer neu installieren. Ein Wechsel ohne Neuinstallation ist nicht möglich. Sichern Sie vorher, was Sie behalten wollen, denn Images und Container gehen bei der Deinstallation verloren. Deinstallieren Sie über *Einstellungen* und *Apps*. Dann installieren Sie in einer Administrator-PowerShell neu.

```
PS> Start-Process -FilePath $installer -Wait -ArgumentList 'install'
```

Die Variable `$installer` stammt aus Übung 1. Ohne den Parameter `--user` installiert das Programm für alle Benutzer und fragt nach Administratorrechten. Arbeitet Ihr Alltagskonto nicht mit demselben Konto wie der Administrator, fügen Sie es der Gruppe `docker-users` hinzu. Beachten Sie die Warnung am Ende dieser Übung.

```
PS> net localgroup docker-users MeinBenutzer /add
```

Melden Sie sich danach neu an und starten Sie Docker Desktop.

Schritt 1: Windows-Funktionen prüfen

```
PS> 'Microsoft-Hyper-V-All', 'Containers' | ForEach-Object { Get-
    ↪ WindowsOptionalFeature -Online -FeatureName $_ | Select-
    ↪ Object FeatureName, State }
FeatureName            State
-----------            -----
Microsoft-Hyper-V-All Enabled
Containers            Enabled
```

Steht dort `Disabled`, schalten Sie die Funktionen in einer Administrator-PowerShell ein und starten den Rechner neu.

```
PS> Enable-WindowsOptionalFeature -Online -FeatureName Containers,
    ↪ Microsoft-Hyper-V-All -All
```

Schritt 2: Auf Windows-Container umschalten

```
PS> & $Env:ProgramFiles\Docker\Docker\DockerCli.exe -SwitchDaemon
PS> docker info --format '{{.OSType}} {{.Isolation}}'
windows hyperv
```

Der Befehl schaltet den Client zwischen dem Linux- und dem Windows-Daemon um. Dasselbe leistet der Eintrag *Switch to Windows containers* im Menü des Docker-Symbols im Infobereich der Taskleiste. Die Antwort zeigt den Typ `windows` und den Standard-Isolationsmodus `hyperv`, wie er für Windows 11 üblich ist.

Schritt 3: Basisimage laden

```
PS> docker pull mcr.microsoft.com/windows/nanoserver:ltsc2022
PS> docker images --format "table {{.Repository}}\t{{.Tag}}\t{{.Size
    ↪ }}"
REPOSITORY                              TAG        SIZE
mcr.microsoft.com/windows/nanoserver    ltsc2022   296MB
```

Lesen Sie die Lizenzbedingungen, auf die Microsoft beim Herunterladen hinweist. Erscheint die Meldung `No matching manifest for linux/amd64`, steht Docker noch im Linux-Modus: Schritt 2 wiederholen.

Schritt 4: Host- und Container-Version vergleichen

```
PS> $hostBuild = (Get-ItemProperty 'HKLM:\SOFTWARE\Microsoft\Windows
    ↪ NT\CurrentVersion').CurrentBuild
PS> "Build des Hosts: $hostBuild"
Build des Hosts: 26100
PS> docker run --rm mcr.microsoft.com/windows/nanoserver:ltsc2022 cmd
    ↪  /c ver
Microsoft Windows [Version 10.0.20348.xxxx]
```

Host und Image haben unterschiedliche Build-Nummern. Der Container läuft trotzdem, weil Windows 11 für Windows-Server-2022-Images die Ausnahme der Versionsmatrix nutzt (Abschnitt 21.6.3).

Schritt 5: Die Isolationsmodi vergleichen

```
PS> docker run -d --name prozess --isolation=process mcr.microsoft.
    ↪ com/windows/nanoserver:ltsc2022 ping -t localhost
PS> docker run -d --name hyperv  --isolation=hyperv  mcr.microsoft.
    ↪ com/windows/nanoserver:ltsc2022 ping -t localhost
PS> docker inspect --format '{{.Name}}: {{.HostConfig.Isolation}}'
    ↪ prozess hyperv
/prozess: process
/hyperv: hyperv
```

```
PS> Get-Process -Name ping -ErrorAction SilentlyContinue | Select-
    ↪ Object Id, ProcessName
  Id ProcessName
  -- -----------
5120 PING
```

Beide Container führen dasselbe Programm aus, aber auf dem Host erscheint nur *ein* `ping`-Prozess: der des Containers mit Prozess-Isolation. Er teilt sich den Kernel mit Windows und ist deshalb im Task-Manager sichtbar. Der Container mit Hyper-V-Isolation läuft in seiner eigenen kleinen virtuellen Maschine, sein Prozess ist vom Host aus nicht sichtbar. Die Prozessnummer in der Ausgabe ist ein Beispiel.

Schritt 6: Im Container umsehen

```
PS> docker run -it --rm mcr.microsoft.com/windows/nanoserver:ltsc2022
    ↪  cmd
```

```
C:\> hostname
C:\> ipconfig
C:\> dir C:\
C:\> set
C:\> exit
```

Der Rechnername ist die Kennung des Containers. `ipconfig` zeigt eine Adresse aus dem NAT-Netz. Der Nano-Server-Container kennt die klassische Eingabeaufforderung, aber keine PowerShell, wie die Basisimages erklärt haben.

Schritt 7: Ein eigenes Image aus einem Container

```
PS> docker run --name notiz mcr.microsoft.com/windows/nanoserver:
    ↪ ltsc2022 cmd /c "echo Hallo > C:\notiz.txt"
PS> docker commit notiz mein-windows-image:1.0
PS> docker run --rm mein-windows-image:1.0 cmd /c type C:\notiz.txt
```

```
Hallo
```

Der Befehl `docker commit` macht aus einem Container ein neues Image. Er dient hier der Anschauung, denn es zeigt, dass die Schichten dem Prinzip aus Kapitel 7 folgen. Für den Alltag baut man Images mit einem Dockerfile, weil sich das nachvollziehen und wiederholen lässt.

Schritt 8: Den Versionskonflikt auslösen

```
PS> docker pull mcr.microsoft.com/windows/nanoserver:ltsc2019
PS> docker run --rm --isolation=process mcr.microsoft.com/windows/
    ↪ nanoserver:ltsc2019 cmd /c ver
docker: Error response from daemon: container ... encountered an
    ↪ error during CreateContainer:
failure in a Windows system call: The operating system of the
    ↪ container does not match the
operating system of the host. (0xc0370101) ...
PS> docker run --rm --isolation=hyperv mcr.microsoft.com/windows/
    ↪ nanoserver:ltsc2019 cmd /c ver
Microsoft Windows [Version 10.0.17763.xxxx]
```

Ein Windows-Server-2019-Image lässt sich auf Windows 11 nicht mit Prozess-Isolation starten und liefert die Fehlernummer `0xc0370101`. Mit Hyper-V-Isolation läuft dasselbe Image, weil der Container seinen eigenen passenden Kernel mitbringt. Das ist die Regel aus der Versionsmatrix in Abschnitt 21.6.3.

Schritt 9: Netzwerk ansehen

```
PS> docker network ls
PS> docker network inspect nat --format '{{range .IPAM.Config}}{{.
    ↪ Subnet}}{{end}}'
PS> docker inspect --format '{{.NetworkSettings.Networks.nat.
    ↪ IPAddress}}' prozess
```

Die erste Abfrage zeigt das Standardnetz `nat`, die zweite seinen Adressbereich, die dritte die Adresse des Containers. Bei Windows-Containern unter Docker Desk-

top ist diese Adresse vom Host aus erreichbar, anders als bei Linux-Containern. Versuchen Sie `ping` mit der gefundenen Adresse.

Aufräumen und Zurückschalten

```
PS> docker rm -f prozess hyperv notiz
PS> docker rmi mein-windows-image:1.0
PS> & $Env:ProgramFiles\Docker\Docker\DockerCli.exe -SwitchDaemon
PS> docker info --format '{{.OSType}}'
linux
```

Erwartetes Ergebnis

Die Windows-Container starten, `docker inspect` zeigt die beiden Isolationsmodi, und auf dem Host erscheint nur der Prozess des Containers mit Prozess-Isolation. Der Start des Server-2019-Images mit Prozess-Isolation scheitert mit der Fehlernummer `0xc0370101`, mit Hyper-V-Isolation gelingt er.

Fehlerdiagnose

Symptom	Ursache und Abhilfe
Der Eintrag zum Umschalten fehlt, `DockerCli.exe` fehlt	Docker Desktop ist pro Benutzer installiert oder die Edition unterstützt keine Windows-Container. Im Modus für alle Benutzer neu installieren.
`No matching manifest for linux/amd64`	Der Daemon steht im Linux-Modus. `DockerCli.exe -SwitchDaemon` ausführen.
`0xc0370101` beim Start	Host- und Image-Version passen nicht zu dem gewählten Modus. Mit `--isolation=hyperv` starten oder ein passendes Image wählen.
Meldung, der privilegierte Hilfsdienst laufe nicht	`com.docker.service` starten (Dienste-Verwaltung) oder die Sicherheitsabfrage von Windows bestätigen.
Fehler bei `--isolation=hyperv`, der Hyper-V erwähnt	Die Funktion Hyper-V ist nicht eingeschaltet (Schritt 1) oder der Rechner nicht neu gestartet. In einer virtuellen Maschine fehlt die verschachtelte Virtualisierung.
Container beenden sich sofort	Der Hauptprozess endet. `docker logs NAME` lesen. Zum Test den Befehl `ping -t localhost` verwenden, der nicht endet.
Interaktive Sitzung reagiert nicht	Die PowerShell ISE ist dafür ungeeignet. Windows Terminal verwenden.

Sicherheitsinformationen

Diese Übung berührt die heikelsten Punkte von Docker Desktop. Mit Windows-Containern ist der Benutzer `ContainerAdministrator` im Container zugleich lokaler Administrator auf dem Host, und die Mitglieder von `docker-users` können sich damit zu Administratoren hochstufen. Entfernen Sie Ihren Benutzer nach der Übung aus der Gruppe, wenn Sie die Windows-Container nicht weiter brauchen (`net localgroup docker-users MeinBenutzer /delete`). Windows-Container mit Prozess-Isolation teilen sich den Kernel mit dem Host und sind für nicht vertrauenswürdigen Code ungeeignet. Dafür gibt es die Hyper-V-Isolation. Auf Windows 11 sind Windows-Container ein Entwicklungs- und Testwerkzeug, für den Betrieb verlangt Microsoft Windows Server. Schalten Sie die Windows-Container-Integration bei der Installation mit `--no-windows-containers` ab,

wenn Sie sie nicht nutzen. Wählen Sie Images nur aus der Microsoft Container Registry und halten Sie Host und Images aktuell.

21.11.5 Übung 5: Windows-Server-Labor mit Container-Runtime

Ziel: Auf Windows Server die Docker Community Edition als Container-Runtime einrichten, Windows-Container betreiben und die Besonderheiten von Dienst, Netzwerk, Isolation und Compose kennenlernen. *Docker Desktop* kommt in dieser Übung *nicht* zum Einsatz.

Voraussetzungen

Voraussetzung	Hinweis
Virtuelle Maschine oder Server mit Windows Server 2022 oder 2025	Evaluierungsversion von Microsoft genügt für Tests
2 virtuelle Prozessoren, 4 GB Arbeitsspeicher, 60 GB Platte	Windows-Images sind groß
Internetzugang	Download von Skript, Docker und Images
Administratorrechte	PowerShell als Administrator
Optional: verschachtelte Virtualisierung	nur für Schritt 9 (Hyper-V-Isolation)

Schritt 1: System prüfen

```
PS> Get-ComputerInfo -Property OsName, OsVersion, OsBuildNumber
OsName          : Microsoft Windows Server 2022 Datacenter
OsVersion       : 10.0.20348
OsBuildNumber   : 20348
```

Notieren Sie die Build-Nummer. Sie entscheidet, welche Images mit Prozess-Isolation laufen. Die Übung verwendet Images mit dem Tag `ltsc2022`, die auf Windows Server 2022 und 2025 mit Prozess-Isolation laufen (Versionsmatrix in Abschnitt 21.6.3).

Schritt 2: Das Installationsskript laden und lesen

```
PS> Invoke-WebRequest -UseBasicParsing "https://raw.githubusercontent.
    ↪ com/microsoft/Windows-Containers/Main/helpful_tools/Install-
    ↪ DockerCE/install-docker-ce.ps1" -OutFile install-docker-ce.
    ↪ ps1
PS> Get-Content .\install-docker-ce.ps1 -TotalCount 30
```

Lesen Sie das Skript, bevor Sie es ausführen. Die Tabelle in Abschnitt 21.7 fasst zusammen, was es tut. Es stammt aus dem Repository von Microsoft.

Schritt 3: Installieren

```
PS> .\install-docker-ce.ps1
```

Das Skript schaltet die Funktion *Containers* ein und startet den Rechner neu. Nach der nächsten Anmeldung setzt eine geplante Aufgabe das Skript fort, es öffnet ein PowerShell-Fenster, lädt Docker herunter, registriert den Dienst und zeigt zum Schluss, dass es fertig ist und sich mit dem Daemon verbinden konnte. Lassen Sie das Fenster offen, bis es sich abschließt. Möchten Sie den Neustart selbst steuern, geben Sie `-NoRestart` an.

Schritt 4: Installation prüfen

```
PS> Get-Service docker
Status   Name               DisplayName
------   ----               -----------
Running  docker             docker
PS> docker version
PS> docker info --format '{{.OSType}} {{.Isolation}}'
windows process
PS> docker network ls
NETWORK ID     NAME      DRIVER    SCOPE
...            nat       nat       local
...            none      null      local
PS> Get-Content C:\ProgramData\docker\config\daemon.json
```

```
{
    "hosts":  [
                  "npipe://"
              ]
}
```

Der Standardmodus der Isolation ist auf einem Server `process`. Der Daemon lauscht nur auf der lokalen Named Pipe. Das Standardnetz heißt `nat`. Die Ausgabe der Konfigurationsdatei kann in der Formatierung abweichen.

Schritt 5: Ein erster Container

```
PS> docker pull mcr.microsoft.com/windows/nanoserver:ltsc2022
PS> docker run --rm mcr.microsoft.com/windows/nanoserver:ltsc2022 cmd
    ↪ /c echo Hallo vom Server
Hallo vom Server
```

Schritt 6: IIS im Container

```
PS> docker run -d --name iis -p 8080:80 mcr.microsoft.com/windows/
    ↪ servercore/iis:windowsservercore-ltsc2022
PS> docker ps --format "table {{.Names}}\t{{.Status}}\t{{.Ports}}"
NAMES   STATUS         PORTS
iis     Up 20 seconds  0.0.0.0:8080->80/tcp
PS> (Invoke-WebRequest -Uri http://localhost:8080 -UseBasicParsing).
    ↪ StatusCode
200
PS> docker inspect --format '{{.HostConfig.Isolation}}' iis
process
```

Das erste Laden dieses Images dauert wegen der Größe einige Minuten. Die Seite des IIS-Standardservers antwortet mit dem Statuscode 200. Funktioniert `localhost` auf Ihrem System nicht, verwenden Sie die Adresse des Containers: `$ip = docker inspect --format '{{.NetworkSettings.Networks.nat.IPAddress}}' iis` und danach `Invoke-WebRequest "http://$ip" -UseBasicParsing`. Von einem anderen

Rechner im Netz erreichen Sie den Server unter der Adresse des Servers und dem Port 8080.

Schritt 7: Eine eigene Seite einbinden

Legen Sie mit Notepad die Datei `C:\Labor5\site\index.html` mit dem Inhalt `<h1>Eigene Seite auf Windows Server</h1>` an und starten Sie einen zweiten Container, der den Ordner als Webverzeichnis einbindet.

```
PS> docker run -d --name iis-eigen -p 8081:80 -v C:\Labor5\site:C:\
    ↪ inetpub\wwwroot mcr.microsoft.com/windows/servercore/iis:
    ↪ windowsservercore-ltsc2022
PS> (Invoke-WebRequest -Uri http://localhost:8081 -UseBasicParsing).
    ↪ Content
```

Bei Windows-Containern bindet man Verzeichnisse mit Windows-Pfaden auf beiden Seiten ein. Die Ausgabe zeigt Ihre eigene Seite.

Schritt 8: Dienst, Protokolle und Ressourcen

```
PS> docker stats --no-stream
PS> Restart-Service docker
PS> docker ps
PS> Get-EventLog -LogName Application -Source docker -Newest 5
```

Nach `Restart-Service docker` laufen die Container ohne Neustartregel nicht mehr. Windows-Container unterstützen Neustartregeln wie `--restart unless-stopped`, die Sie bei Bedarf setzen.

Schritt 9 (optional): Hyper-V-Isolation

Hyper-V-Isolation braucht die Funktion Hyper-V im Server und, wenn der Server selbst eine virtuelle Maschine ist, verschachtelte Virtualisierung. Auf dem Hyper-V-Host geben Sie sie bei ausgeschalteter virtueller Maschine frei:

```
PS> Set-VMProcessor -VMName 'WS-Docker' -
    ↪ ExposeVirtualizationExtensions $true
```

Im Server installieren Sie danach die Rolle und starten neu:

```
PS> Install-WindowsFeature -Name Hyper-V -IncludeManagementTools -
    ↪ Restart
PS> docker run --rm --isolation=hyperv mcr.microsoft.com/windows/
    ↪ nanoserver:ltsc2022 cmd /c ver
```

Mit Hyper-V-Isolation laufen auch Images älterer Windows-Versionen, die mit Prozess-Isolation scheitern würden. Ein Beispiel ist `nanoserver:ltsc2019` auf einem Server 2022 (Abschnitt 21.6.3).

Schritt 10: Docker Compose installieren und nutzen

```
PS> [Net.ServicePointManager]::SecurityProtocol = [Net.
    ↪ SecurityProtocolType]::Tls12
PS> Start-BitsTransfer -Source "https://github.com/docker/compose/
    ↪ releases/download/v5.4.0/docker-compose-windows-x86_64.exe" -
    ↪ Destination $Env:ProgramFiles\Docker\docker-compose.exe
PS> docker-compose.exe version
```

Verwenden Sie die aktuelle Versionsnummer von der Releaseseite. Legen Sie mit Notepad im Ordner `C:\Labor5` die Datei `compose.yaml` an:

C:\Labor5\compose.yaml

```
services:
  iis:
    image: mcr.microsoft.com/windows/servercore/iis:windowsservercore-
    ↪ ltsc2022
    ports:
      - "8082:80"
    isolation: process
```

```
PS> Set-Location C:\Labor5
PS> docker-compose.exe up -d
PS> docker-compose.exe ps
PS> docker-compose.exe down
```

Auf Windows Server heißt der Befehl `docker-compose` mit Bindestrich.

Aufräumen

```
PS> docker rm -f iis iis-eigen
```

Erwartetes Ergebnis

Der Dienst `docker` läuft, der Daemon lauscht nur auf der Named Pipe, und der Standardmodus der Isolation ist `process`. Der Nano-Server-Container gibt den Text aus, der IIS-Container antwortet auf Port 8080 mit dem Statuscode 200, und die eigene Seite erscheint auf Port 8081. Compose startet und beendet das Projekt.

Fehlerdiagnose

Symptom	Ursache und Abhilfe
`You must run this script as administrator`	PowerShell ohne Administratorrechte gestartet. Als Administrator öffnen.
Das Skript scheint nach dem Neustart nichts zu tun	Es setzt sich bei der nächsten Anmeldung fort. Mit dem Konto anmelden, das das Skript gestartet hat, und das Fenster abwarten.
Docker-Befehl meldet, der Daemon sei nicht erreichbar	`Get-Service docker` prüfen, `Start-Service docker` ausführen, Ereignisprotokoll lesen (`Get-EventLog -LogName Application -Source docker`).
Start eines Containers scheitert mit `0xc0370101`	Image und Host passen nicht zusammen. Passendes Image wählen oder `--isolation=hyperv` verwenden (Schritt 9).
Fehler bei Hyper-V-Isolation	Die Rolle Hyper-V fehlt oder die verschachtelte Virtualisierung ist nicht freigegeben.
Port 8080 ist von einem anderen Rechner nicht erreichbar	Netzwerkregeln der virtuellen Umgebung oder der Cloud (zum Beispiel Netzwerksicherheitsgruppen) prüfen.
Das Laden der Images dauert sehr lange oder bricht ab	Windows-Images sind mehrere Gigabyte groß. Plattenplatz (`Get-PSDrive C`) und Netzwerk prüfen.
`docker-compose` nicht gefunden	Die Datei liegt nicht in einem Ordner des Suchpfads. Der Ordner `C:\Program Files\Docker` gehört dazu. Neues Fenster öffnen.

Sicherheitsinformationen

Spielen Sie die Windows-Updates auf dem Host ein, und halten Sie Images und Host in der Version zusammen. Das Skript von Microsoft konfiguriert den Daemon so, dass er nur auf der lokalen Named Pipe lauscht. Öffnen Sie ihn nie unverschlüsselt über TCP. Wer Docker aus der Ferne steuern will, nutzt TLS. Bei der Veröffentlichung von Ports legt Windows automatisch die zugehörigen Regeln der Windows-Firewall an, ein veröffentlichter Port ist also von außen erreichbar. Nicht vertrauenswürdigen Code und Mandanten trennt man mit Hyper-V-Isolation. Wo es möglich ist, laufen Anwendungen mit einem Konto ohne Ad-

ministratorrechte im Container. Die Windows-Server-Evaluierung ist zeitlich begrenzt und für den Produktivbetrieb ungeeignet. Windows-Images brauchen einen gültig lizenzierten Host, bei Windows Server Standard sind beliebig viele Container mit Prozess-Isolation, aber nur zwei mit Hyper-V-Isolation erlaubt.

21.12 Lernkontrolle und Prüfungsvorbereitung

Die Aufgaben prüfen den Stoff dieses Kapitels. Die praktischen Aufgaben führen Sie auf einem Windows-11-Rechner mit Docker Desktop aus, die Aufgaben zu Windows Server auf einem Testsystem. Die Lösungen stehen im Anhang.

Verständnisfragen

V1. Warum braucht ein Linux-Container unter Windows eine virtuelle Maschine, ein Windows-Container aber nicht?

V2. Worin unterscheiden sich die Installation von Docker Desktop pro Benutzer und für alle Benutzer? Was ist nur im zweiten Modus möglich?

V3. Welche Rolle spielt WSL 2 für Docker Desktop, und was legt die Datei `.wslconfig` fest?

V4. Warum ist die Mitgliedschaft in der Gruppe `docker-users` sicherheitsrelevant?

V5. Erklären Sie Prozess-Isolation und Hyper-V-Isolation. Nennen Sie je einen Vorteil.

V6. Was bedeutet die Fehlernummer `0xc0370101`, und welche Wege zur Lösung gibt es?

V7. Warum ist Docker Desktop keine Lösung für Windows Server, und was verwendet man stattdessen?

V8. Was installiert das Skript von Microsoft auf Windows Server, und was fehlt danach noch für Compose?

V9. Nennen Sie drei Stolperfallen der PowerShell im Umgang mit Docker.

V10. Wo sollten Projektdateien liegen, die in einen Linux-Container unter WSL 2 eingebunden werden, und warum?

Multiple-Choice-Aufgaben

M1. Welcher Installationsmodus von Docker Desktop erlaubt Windows-Container?
(a) Pro Benutzer
(b) Für alle Benutzer
(c) Beide
(d) Keiner

M2. Welcher Isolationsmodus gilt für Windows-Container auf Windows 11 in der Grundeinstellung?
(a) Hyper-V-Isolation
(b) Prozess-Isolation
(c) Keine Isolation
(d) Nur auf Servern gibt es Isolation

M3. Wie schaltet man Docker Desktop zwischen Linux- und Windows-Daemon um?
(a) `docker context switch windows`
(b) `wsl --switch-daemon`
(c) `DockerCli.exe -SwitchDaemon`
(d) `docker swarm leave`

M4. Welche Option startet einen Windows-Container mit eigenem Kernel?
(a) `--privileged`
(b) `--isolation=process`
(c) `--kernel=eigen`
(d) `--isolation=hyperv`

M5. Welcher Befehl zeigt installierte WSL-Distributionen mit ihrer Version?
(a) `wsl --list --verbose`
(b) `wsl --show`
(c) `wsl --kernel`
(d) `wsl --status --all`

M6. Welche Container-Runtimes unterstützt Microsoft unter Windows Server?
(a) Docker Desktop, Podman und LXC
(b) Nur Docker Desktop
(c) Moby, Mirantis Container Runtime und containerd

(d) Nur Hyper-V

M7. Welcher Netzwerkmodus fehlt bei Windows-Containern?
(a) `nat`
(b) `host`
(c) `transparent`
(d) `overlay`

M8. In welcher Kodierung schreibt der Operator `>` in Windows PowerShell 5.1?
(a) UTF-8 ohne BOM
(b) ASCII
(c) Latin-1
(d) UTF-16

Praktische Aufgaben

P1. Prüfen Sie mit PowerShell-Befehlen, ob Ihr Rechner die Voraussetzungen für Docker Desktop mit WSL 2 erfüllt (Windows-Build, Virtualisierung, Dienst `LanmanServer`, WSL-Version). Notieren Sie die Ergebnisse.

P2. Starten Sie einen nginx-Container so, dass er nur auf `127.0.0.1:8090` erreichbar ist und eine Seite aus dem Linux-Dateisystem Ihrer WSL-Distribution ausliefert. Weisen Sie mit `docker port` und `Get-NetTCPConnection` nach, auf welcher Adresse der Port lauscht.

P3. Begrenzen Sie WSL mit einer `.wslconfig` auf 4 GB Arbeitsspeicher und 2 Prozessoren und belegen Sie die Wirkung mit `docker info`.

P4. Starten Sie einen Windows-Container mit Prozess-Isolation und einen mit Hyper-V-Isolation und zeigen Sie mit `Get-Process` den Unterschied.

Fehlersuche

F1. Docker Desktop startet nicht und meldet, die Virtualisierung sei nicht aktiviert. Welche Schritte unternehmen Sie?

F2. `docker pull mcr.microsoft.com/windows/nanoserver:ltsc2022` meldet `No matching manifest for linux/amd64`. Was ist die Ursache, und wie beheben Sie sie?

F3. Ein Windows-Container bricht beim Start mit der Nummer `0xc0370101` ab. Wie gehen Sie vor?

F4. Ein nginx-Container beendet sich sofort, nachdem die Konfigurationsdatei auf dem Windows-Rechner angelegt wurde. Das Protokoll meldet `unknown directive`. Woran liegt das?

Szenario aus dem Betrieb

S1. Ein Team will eine produktive Anwendung, die das .NET Framework braucht, mit Docker Desktop auf einem Windows-Server-2022-System betreiben. Beurteilen Sie den Plan und beschreiben Sie eine tragfähige Lösung.

S2. Ein Entwickler berichtet, dass Builds und der automatische Neustart seiner Webanwendung in Linux-Containern langsam sind oder gar nicht funktionieren. Sein Projekt liegt unter `C:\Users`. Nennen Sie Ursache und Abhilfe.

Merkpunkte für die Prüfung

Für Prüfungen und Gespräche über Docker unter Windows lassen sich die wichtigsten Aussagen auf wenige Sätze verdichten.

Thema	Merksatz
Kernel	Container teilen den Kernel des Hosts. Linux-Container brauchen Linux, Windows-Container brauchen Windows.
Docker Desktop	Entwicklungswerkzeug für Arbeitsplätze mit WSL 2, Hyper-V oder Docker VMM, mit eigenen Lizenzbedingungen, nicht für Windows Server.
Installationsmodi	Pro Benutzer: ohne Administratorrechte, nur Linux-Container. Für alle Benutzer: Hilfsdienst, Hyper-V und Windows-Container möglich.
WSL 2	Echter Linux-Kernel in einer schlanken VM, gemeinsam für alle Distributionen. Grenzen in `.wslconfig`.
Dateien	Für Linux-Container Dateien im Linux-Dateisystem von WSL ablegen, nicht unter `/mnt/c`.
Windows-Container	Image muss zur Windows-Version passen, es sei denn, man nutzt Hyper-V-Isolation. Fehler `0xc0370101`.
Isolation	Prozess: gemeinsamer Kernel, schlank. Hyper-V: eigener Kernel je Container, stärker getrennt, versionsunabhängig.
Windows Server	Runtime statt Docker Desktop: Moby, Mirantis oder containerd, installiert zum Beispiel mit dem Skript von Microsoft.
Sicherheit	`docker-users` entspricht Administratorrechten. Daemon nie unverschlüsselt über TCP. Published Ports öffnen die Windows-Firewall.
PowerShell	Anführungszeichen, Gravis zur Fortsetzung, `curl.exe`, `>` schreibt in 5.1 UTF-16, BOM bei Konfigurationsdateien meiden.

21.13 Zusammenfassung

Docker unter Windows bedeutet, zwei verschiedene Welten sauber zu trennen. In der einen laufen *Linux-Container*: Docker Desktop startet dafür mit WSL 2 eine schlanke virtuelle Maschine mit Linux-Kernel, in der die Engine arbeitet. Das ist der übliche Weg für Lernen und Entwicklung am Arbeitsplatz. Er verhält sich fast wie Docker unter Linux, mit wenigen Eigenheiten bei Netzwerk, Dateisystem und Rechten. In der anderen Welt laufen *Windows-Container*, die den Kernel von Windows nutzen. Sie brauchen Windows-Images von Microsoft, die zur Version des Hosts passen müssen, und lassen sich mit Prozess-Isolation oder Hyper-V-Isolation betreiben.

Für den Produktivbetrieb von Windows-Containern ist Windows Server der Host. Dort läuft eine Container-Runtime als Windows-Dienst: Moby, die Mirantis Container Runtime oder containerd. Docker Desktop ist für Server ausdrücklich nicht vorgesehen. Die Docker Community Edition lässt sich mit einem Skript von Microsoft einrichten. Compose gehört bei dieser Installation nicht dazu und kommt als eigenständiges Programm hinzu.

Die Sicherheitsfragen folgen aus den Besonderheiten der Plattform. Die Gruppe `docker-users` und der Benutzer `ContainerAdministrator` verschaffen Administratorrechte auf dem Host. Der Daemon gehört nicht unverschlüsselt ins Netz, und veröffentlichte Ports öffnen unter Windows die Firewall. Die PowerShell bringt eigene Fallstricke bei Anführungszeichen, Kodierung und Zeichen am Zeilenende mit. Wer die Unterscheidung zwischen Linux- und Windows-Containern, Docker Desktop und Server-Runtime sicher beherrscht, kann Anleitungen richtig einordnen und Fehlermeldungen schnell zuordnen.

Ihre Situation	Passende Lösung
Lernen am Windows-Rechner, Linux-Anwendungen	Docker Desktop mit WSL 2 (oder Engine in WSL)
Entwickeln und Testen von Windows-Anwendungen	Docker Desktop (für alle Benutzer) mit Windows-Containern
Produktiver Betrieb von Linux-Anwendungen	Linux-Server mit Docker Engine
Produktiver Betrieb von Windows-Anwendungen	Windows Server mit Container-Runtime
Stärkere Trennung unter Windows-Containern	Hyper-V-Isolation

Lösungen zu den Lernkontrollen

Lösungen zu Teil I

Verständnisfragen

V1. Ohne Virtualisierung belegt jede Anwendung einen eigenen Server, der meist nur zu einem kleinen Teil ausgelastet ist. Mit Virtualisierung teilen sich mehrere abgeschottete Systeme die Ressourcen eines Servers, sodass die vorhandene Hardware besser genutzt wird.

V2. Ein Typ-1-Hypervisor läuft direkt auf der Hardware (zum Beispiel VMware ESXi oder KVM), ein Typ-2-Hypervisor als Programm auf einem normalen Betriebssystem (zum Beispiel VirtualBox).

V3. Ein Container startet nur einen Prozess und fährt kein Betriebssystem mit eigenem Kernel hoch. Der Kernel des Hosts läuft bereits.

V4. Ein Image ist die schreibgeschützte Vorlage, ein Container die ausgeführte oder gestoppte Instanz, die aus dem Image entsteht und eine eigene beschreibbare Schicht besitzt.

V5. Namespaces begrenzen, was ein Prozess sehen kann (Prozesse, Netzwerk, Dateisystem und anderes). Control Groups begrenzen, wie viel er verbrauchen darf (Prozessorzeit, Speicher, Prozessanzahl, Plattenzugriffe).

V6. Die Daten gehen mit dem Container verloren. Was erhalten bleiben soll, gehört in ein Volume.

V7. `dockerd` nimmt Aufträge entgegen und verwaltet Images, Netzwerke und Volumes. `containerd` verwaltet den Lebenszyklus der Container. `runc` legt den Container im Kernel an und startet den Prozess.

V8. `docker.io/library/nginx:latest`

Multiple-Choice-Aufgaben

M1	M2	M3	M4	M5	M6
(b)	(c)	(a)	(d)	(b)	(c)

Praktische Aufgaben

P1. Befehle: `sudo unshare --uts bash`, danach `hostname labor` und `hostname`, dann `exit` und erneut `hostname`. In der neuen Shell erscheint `labor`, auf dem Host bleibt der ursprüngliche Name stehen, weil die Änderung nur den uts-Namespace der Shell betraf.

P2. Auf dem Host ergibt die Zeile einige Dutzend bis Hunderte Zeilen, im neuen pid-Namespace nur wenige (die Kopfzeile, die Shell und `ps` selbst, also etwa drei). Der Namespace blendet alle Prozesse außerhalb aus, die Shell trägt dort die Nummer 1.

P3. Die Datei `neu.txt` erscheint in `merged` und in `upper`, nicht in `lower`. Zum Aufräumen: `sudo umount` auf `merged`, danach das Arbeitsverzeichnis mit `sudo rm -r` löschen.

Fehlersuche

F1. Es fehlt der Schalter `--mount-proc`. Das Programm `ps` liest seine Daten aus `/proc`, und ohne den Schalter zeigt `/proc` weiterhin die Prozesse des Hosts, obwohl die Shell selbst im neuen Namespace läuft.

F2. Es fehlt die Option `workdir`. OverlayFS braucht zusätzlich zum `upperdir` ein leeres Arbeitsverzeichnis, das auf demselben Dateisystem liegen muss wie `upperdir`.

Szenario aus dem Betrieb

Eine mögliche Lösung: Der neue Server bekommt einen Typ-1-Hypervisor, zum Beispiel KVM oder Hyper-V. Anwendung C läuft in einer eigenen Windows-VM, weil sie ein Windows-Betriebssystem braucht und Container den Linux-Kernel des Hosts teilen. Die Anwendungen A und B laufen als Container in einer Linux-VM, in der Docker installiert ist. Sie sind kleine, gleichartige Dienste, die sich ohne eigenes Betriebssystem schnell und sparsam betreiben lassen. Die Linux-VM sorgt zusätzlich für eine klare Trennung vom Windows-System. Wer weniger Aufwand wünscht, kann A und B auch jeweils in eine eigene Linux-VM legen. Der Preis dafür sind mehr Arbeitsspeicher und mehr Pflege von Betriebssystemen.

Lösungen zu Teil II

Verständnisfragen

V1. Nach einem Update läuft das System weiter mit dem alten Kernel, bis es neu startet. Alle Container nutzen den Kernel des Hosts, deshalb gilt für sie erst nach dem Neustart der aktuelle Stand mit allen Sicherheitskorrekturen.

V2. Mit dem Schlüssel prüft `apt`, ob ein Paket tatsächlich von Docker stammt und unterwegs nicht verändert wurde. In der Datei `docker.sources` verweist `Signed-By` darauf.

V3. `docker-ce` (Daemon), `docker-ce-cli` (Client), `containerd.io` (`containerd` und `runc`), `docker-buildx-plugin` (erweitertes Bauen von Images) und `docker-compose-plugin` (Compose).

V4. `docker run` erzeugt aus einem Image einen neuen Container und startet ihn. `docker exec` führt einen Befehl in einem bereits laufenden Container aus.

V5. Die Gruppe gewährt Zugriff auf den Docker-Socket, und der Daemon läuft als Root. Wer Docker bedienen darf, kann einen Container starten, der das Dateisystem des Hosts einbindet, und hat damit praktisch Root-Rechte.

V6. Docker schickt dem Hauptprozess zuerst SIGTERM mit der Bitte aufzuhören. Reagiert er nicht innerhalb von zehn Sekunden (Standardwert), beendet Docker ihn mit SIGKILL.

V7. Der Port 8080 des Hosts wird mit dem Port 80 im Container verbunden. Links steht der Host-Port, rechts der Container-Port.

V8. Docker trägt beim Veröffentlichen eines Ports eigene Regeln in die Firewall des Kernels ein, die vor den Regeln von `ufw` oder `firewalld` greifen. Die Sperre in `ufw` wird dadurch umgangen.

Multiple-Choice-Aufgaben

M1	M2	M3	M4	M5	M6
(b)	(a)	(c)	(d)	(b)	(c)

Praktische Aufgaben

P1. `docker run --name lernen alpine cat /etc/os-release`. Danach zeigt `docker ps -a` den Status `Exited (0)`. Entfernen mit `docker rm lernen`.

P2. `docker run -d --name web2 -p 8081:80 nginx`, danach `curl -s http://localhost:8081`, `docker logs --tail 5 web2` und `docker rm -f web2`.

P3. In der `daemon.json` stehen `"max-size": "5m"` und `"max-file": "2"` unter `log-opts`. Danach `sudo systemctl restart docker`. `docker info --format '{{.LoggingDriver}}'` gibt `json-file` aus.

Fehlersuche

F1. Der Host-Port 8080 wird bereits von einem anderen Programm oder Container verwendet. Abhilfe: einen anderen Host-Port wählen (zum Beispiel `-p 8081:80`) oder den Verursacher mit `sudo ss -tlnp | grep 8080` suchen und beenden.

F2. Zuerst die Container entfernen, die das Image verwenden (`docker ps -a`, dann `docker rm`), danach `docker rmi alpine` wiederholen.

F3. `sudo systemctl status docker` zeigt den Zustand, `journalctl -u docker` die Fehlermeldung des Daemons. Meist liegt ein Tippfehler im JSON der `daemon.json` vor, etwa ein fehlendes Komma oder eine fehlende Klammer.

Szenario aus dem Betrieb

Die Meldung bedeutet, dass der Benutzer keinen Zugriff auf den Socket hat. Die Gruppenmitgliedschaft wirkt erst bei einer neuen Anmeldung. Die Kollegin soll sich abmelden und neu anmelden (oder in der Shell `newgrp docker` ausführen) und mit `id` prüfen, ob `docker` in der Liste steht. Sicherheitstechnisch ist die Mitgliedschaft nicht harmlos, denn sie entspricht Root-Rechten. Sie sollte nur Personen gegeben werden, die auch Root-Zugriff auf den Server bekommen dürften. Für andere Fälle kommen der rootless-Modus oder eine kontrollierte Bereitstellung per `sudo` infrage, mit denen sich Kapitel 18 befasst.

Lösungen zu Teil III

Verständnisfragen

V1. Ein Layer ist eine schreibgeschützte Schicht aus Dateisystemänderungen. Container aus demselben Image teilen sich alle Schichten des Images, jeder bekommt nur eine eigene dünne beschreibbare Schicht.

V2. Ein Tag ist ein veränderbarer Zeiger auf ein Image und kann jederzeit auf ein anderes zeigen. Ein Digest ist eine Prüfsumme des Inhalts und bezeichnet genau eine Fassung. `latest` ist nur ein Tag und garantiert keine bestimmte Fassung, nach einem Neustart oder Update weiß man nicht mehr, was läuft.

V3. Er nennt den Build-Kontext, also das Verzeichnis (hier das aktuelle), dessen Inhalt Docker für den Bau erhält und aus dem `COPY` Dateien nimmt.

V4. Jeder Schritt baut auf der Schicht des vorigen auf. Ändert sich ein Schritt, sind alle folgenden Schichten ungültig und werden neu erzeugt. Die Schritte davor kommen weiter aus dem Cache.

V5. `ENV` setzt eine Umgebungsvariable, die im fertigen Image und im Container erhalten bleibt. `ARG` gilt nur während des Baus.

V6. In der Exec-Form ist das Programm selbst der Hauptprozess und bekommt Signale wie SIGTERM direkt. In der Shell-Form wird die Shell zum Hauptprozess, und das Beenden dauert oft die vollen zehn Sekunden.

V7. `EXPOSE 80` vermerkt im Image, dass die Anwendung auf Port 80 lauscht, als Dokumentation. Es veröffentlicht den Port nicht, dafür braucht man `-p` beim Start.

V8. Das fertige Image wird deutlich kleiner, weil Compiler und Werkzeuge der Bauphase fehlen, und die Angriffsfläche sinkt. Außerdem laden Downloads schneller.

Multiple-Choice-Aufgaben

M1	M2	M3	M4	M5	M6
(b)	(c)	(d)	(a)	(d)	(b)

Praktische Aufgaben

P1. Dockerfile mit `FROM nginx:stable-alpine` und `COPY index.html /usr/share/nginx/html/index.html`. Bau: `docker build -t meine-seite:2.0 .`. Start: `docker run -d --name seite2 -p 8083:80 meine-seite:2.0`. Prüfung: `curl -s http://localhost:8083`. Aufräumen: `docker rm -f seite2` und `docker rmi meine-seite:2.0`.

P2. Beim zweiten Bau steht bei den Schritten `CACHED`. Nach der Änderung der `index.html` wird der Schritt `COPY` neu ausgeführt, die Schritte davor (`FROM`) bleiben im Cache.

P3. Aufruf: `docker run -d --name klein --memory 64m --cpus 0.25 nginx`. In der cgroup des Containers (Verzeichnis `/sys/fs/cgroup/system.slice/docker-<ID>.scope`) steht in `memory.max` der Wert `67108864` und in `cpu.max` der Wert `25000 100000`.

Fehlersuche

F1. Der Build-Kontext fehlt, meist ein Punkt für das aktuelle Verzeichnis am Ende des Befehls.

F2. Die Datei liegt nicht im Build-Kontext (etwa weil der Befehl aus einem anderen Verzeichnis gestartet wurde oder die Datei in einem Unterverzeichnis

liegt), oder sie wird durch die `.dockerignore` ausgeschlossen. Auch Tippfehler im Namen kommen vor.

F3. Das Verzeichnis gehört Root, der Benutzer `app` darf dort nicht schreiben. Abhilfe: beim Bau Verzeichnis oder Dateien dem Benutzer übergeben (`COPY --chown=app:app` oder `RUN mkdir /data && chown app:app /data` vor `USER app`).

Szenario aus dem Betrieb

Zuerst einen festen Tag statt `latest` verwenden, damit der Betrieb nachvollziehbar bleibt. Dann die Bauzeit verkürzen: Das Dockerfile so ordnen, dass Pakete und Abhängigkeiten oben stehen und der eigene Code zuletzt kopiert wird, damit der Cache greift, und eine `.dockerignore` anlegen. Danach die Größe senken: kleineres Basisimage wählen (`-slim` oder `-alpine`), Aufräumen im selben `RUN`-Schritt, nicht benötigte Werkzeuge weglassen und, wenn kompiliert wird, einen mehrstufigen Build verwenden. Abschließend den Prozess als normalen Benutzer laufen lassen und das fertige Image mit `docker build --check` und den Schwachstellenwerkzeugen aus Kapitel 18 prüfen.

Lösungen zu Teil IV

Verständnisfragen

V1. Die beschreibbare Schicht gehört zum Container und wird mit ihm gelöscht. Dauerhafte Daten gehören in ein Volume oder einen Bind Mount.

V2. Ein Volume wird von Docker verwaltet (Ort unter `/var/lib/docker/volumes`) und hat einen Namen, ein Bind Mount bindet ein frei gewähltes Verzeichnis des Hosts ein. Ein leeres Volume übernimmt beim ersten Einbinden den Inhalt des Image-Verzeichnisses, ein Bind Mount verdeckt ihn. Auch die Sicherung unterscheidet sich: Volumes sichert man mit Docker-Befehlen, Bind Mounts mit gewöhnlichen Dateiwerkzeugen.

V3. Docker kopiert die vorhandenen Dateien beim ersten Einbinden in das Volume. Bei einem Bind Mount verdeckt das eingebundene Verzeichnis den Inhalt des Images, die Dateien des Images sind nicht sichtbar.

V4. `:ro` bindet das Verzeichnis schreibgeschützt ein (read-only). Der Container darf lesen, aber nicht verändern.

V5. Die Dateien gehören der Benutzernummer, mit der der Prozess im Container läuft. Läuft er als Root (Nummer 0), gehören sie auf dem Host Root, und `admin` darf sie nicht ändern.

V6. Das Standardnetz hat keinen Namensdienst. In benutzerdefinierten Netzen betreibt Docker einen eigenen DNS-Server, der Containernamen und Aliasse auflöst.

V7. Docker betreibt einen eingebauten DNS-Server, der im Container unter `127.0.0.11` erreichbar ist. Er löst Container- und Aliasnamen im Netz auf und leitet andere Anfragen an die Namensserver des Hosts weiter.

V8. Docker trägt eigene Firewall-Regeln ein, die vor denen von `ufw` greifen. Gegenmaßnahmen: den Port nur auf `127.0.0.1` veröffentlichen, den Port gar nicht veröffentlichen und nur im Docker-Netz anbieten, oder eigene Regeln in der Kette `DOCKER-USER` setzen.

Multiple-Choice-Aufgaben

M1	M2	M3	M4	M5	M6
(b)	(d)	(a)	(c)	(b)	(d)

Praktische Aufgaben

P1. `docker volume create uebung`, dann `docker run --rm -v uebung:/daten alpine sh -c "echo test > /daten/a.txt"` und `docker run --rm -v uebung:/daten alpine cat /daten/a.txt`. Sicherung: `docker run --rm -v uebung:/daten:ro -v "$PWD":/backup alpine tar czf /backup/uebung.tar.gz -C /daten .`. Wiederherstellung: neues Volume anlegen und `tar xzf` in einem Hilfscontainer ausführen. Aufräumen mit `docker volume rm`.

P2. `docker network create uebung-netz`, `docker run -d --name web --network uebung-netz nginx`, `docker run --rm --network uebung-netz alpine wget -qO- http://web`. Im Standardnetz meldet `docker run --rm alpine ping -c 1 web` den Fehler `bad address`.

P3. `docker run -d --name lokal -p 127.0.0.1:8090:80 nginx`. Die Ausgabe von `sudo ss -tlnp | grep 8090` zeigt `127.0.0.1:8090` als lauschende Adresse.

Fehlersuche

F1. Docker hat beim Start ein neues, leeres Volume namens `dten` angelegt. `docker volume ls` zeigt beide Volumes. Die Daten liegen unverändert im Volume `daten`. Beheben: den Container mit dem richtigen Namen neu starten und das versehentlich angelegte Volume mit `docker volume rm dten` entfernen.

F2. Ein Name ohne führenden Schrägstrich gilt als Volume. `html` wurde als neues, leeres Volume angelegt, in das Docker die Standarddateien des Images kopiert hat. Für ein Verzeichnis des Hosts braucht man einen absoluten Pfad, zum Beispiel `-v "$PWD/html":/usr/share/nginx/html`.

F3. Der Container `web` läuft nicht oder heißt anders. Er hängt in einem anderen Netz als der aufrufende Container. Der aufrufende Container wurde ohne `--network` im Standardnetz gestartet. Auch ein Tippfehler im Namen kommt infrage. Prüfen mit `docker ps`, `docker network inspect` und `docker inspect`.

Szenario aus dem Betrieb

Docker trägt beim Veröffentlichen des Ports eigene Regeln in die Firewall des Kernels ein, die vor denen von `ufw` gelten. Deshalb hilft die Sperre in `ufw` nicht. Dauerhaft lösen Sie das Problem, indem Sie den Port gar nicht veröffentlichen. Datenbank und Anwendung kommen in dasselbe benutzerdefinierte Docker-Netz, und die Anwendung spricht die Datenbank über deren Namen an. Sinnvoll ist ein Netz mit `--internal`, in dem nur die Datenbank steht. Muss der Port doch auf dem Host erreichbar sein, veröffentlichen Sie ihn nur auf `127.0.0.1` oder beschränken ihn mit einer Regel in der Kette `DOCKER-USER` auf bestimmte Absender.

Lösungen zu Teil V

Verständnisfragen

V1. Man beschreibt den gewünschten Endzustand, nicht die einzelnen Schritte. Nach einer Änderung vergleicht Compose Ist- und Sollzustand und ersetzt nur die Dienste, deren Definition sich geändert hat.

V2. Die heutige Compose-Spezifikation braucht das Feld nicht mehr. Es wird ignoriert und führt zu einer Warnung.

V3. `stop` hält Container nur an, `down` entfernt Container und Netzwerke. Volumes und Images bleiben bei `down` erhalten. Die Option `-v` löscht zusätzlich die Volumes des Projekts, also auch die Daten.

V4. Die einfache Form wartet nur, bis der Container gestartet ist. Eine Datenbank braucht danach noch Zeit, bis sie Verbindungen annimmt. `service_healthy` wartet, bis der Healthcheck des Dienstes erfolgreich ist.

V5. Das Image wertet die Variablen nur bei leerem Datenverzeichnis aus. Bei einer bestehenden Datenbank ändern sie nichts. Ein Passwort ändert man deshalb per SQL (`ALTER USER`).

V6. `mariadb`: `/var/lib/mysql`. `postgres:17`: `/var/lib/postgresql/data`. `postgres:18`: `/var/lib/postgresql`.

V7. Eine kopierte Datei kann mitten in einer Änderung entstehen und unbrauchbar sein. Besser ist ein Dump mit `mariadb-dump` oder `pg_dump`, oder ein Stopp des Containers vor dem Kopieren.

V8. Der Dienstname als Rechnername und der Standardport (3306 bei MariaDB, 5432 bei PostgreSQL). Ein Port muss nicht veröffentlicht werden, die Container im selben Netz erreichen sich direkt.

Multiple-Choice-Aufgaben

M1	M2	M3	M4	M5	M6
(c)	(a)	(b)	(d)	(b)	(b)

Praktische Aufgaben

P1. Eine mögliche `compose.yaml`:

compose.yaml

```
services:
  web:
    image: nginx:stable-alpine
    ports:
      - "8090:80"
    volumes:
      - ./html:/usr/share/nginx/html:ro
```

```
$ docker compose up -d
$ curl -s http://localhost:8090
$ docker compose logs web
$ docker compose down
```

P2. Die Datei wird um die Datenbank erweitert, und `web` wartet auf sie:

compose.yaml (Ergänzung)

```
services:
  web:
    depends_on:
      db:
        condition: service_healthy
  db:
    image: mariadb:11.8
    environment:
      MARIADB_ROOT_PASSWORD: ${DB_ROOT_PASSWORD:?setzen}
    volumes:
      - db-daten:/var/lib/mysql
    healthcheck:
      test: ["CMD", "healthcheck.sh", "--connect", "--
    ↪ innodb_initialized"]
      interval: 10s
      timeout: 5s
```

```
      retries: 5
      start_period: 20s

volumes:
  db-daten:
```

Das Passwort steht in der `.env` neben der Datei. Die beiden Dienste gehören in dieselbe `compose.yaml`.

P3. Sicherung, Löschen und Wiederherstellung:

```
$ docker compose exec -T db sh -c \
    'mariadb-dump -uroot -p"$MARIADB_ROOT_PASSWORD" --single-
    ↪ transaction --databases shop' \
    | gzip > shop.sql.gz
$ docker compose exec db sh -c 'mariadb -uroot -p"
    ↪ $MARIADB_ROOT_PASSWORD" -e "DROP TABLE shop.gaeste"'
$ gunzip -c shop.sql.gz | docker compose exec -T db sh -c \
    'mariadb -uroot -p"$MARIADB_ROOT_PASSWORD"'
```

Die Zeilenzahl vor dem Löschen und nach der Wiederherstellung muss übereinstimmen.

Fehlersuche

F1. Typische Ursachen: ein Tabulator statt Leerzeichen, eine falsche Einrückung, ein fehlendes Leerzeichen nach dem Doppelpunkt, nicht in Anführungszeichen gesetzte Werte mit Sonderzeichen. Die Zeilennummer der Meldung nennt die Stelle. Mit `docker compose config` lässt sich die Datei prüfen.

F2. Die Anwendung startet, bevor die Datenbank Verbindungen annimmt. Abhilfe: Healthcheck für die Datenbank und `depends_on` mit `condition: service_healthy` bei der Anwendung. Zusätzlich sollte die Anwendung Verbindungsfehler abfangen und erneut versuchen.

F3. Das Passwort wird nur beim ersten Start mit leerem Datenverzeichnis gesetzt. Die Datenbank kennt weiter das alte. Lösung: das Passwort per SQL

(`ALTER USER`) ändern oder, falls keine wichtigen Daten vorhanden sind, das Volume löschen und neu initialisieren lassen.

Szenario aus dem Betrieb

Das 18er-Image erwartet das Volume auf `/var/lib/postgresql`, das alte Volume ist aber auf `/var/lib/postgresql/data` eingebunden. Je nach Host startet der Container nicht oder initialisiert eine neue, leere Datenbank. Außerdem lassen sich Datenverzeichnisse einer älteren Hauptversion nicht einfach vom neuen Server öffnen, es braucht `pg_upgrade` oder einen Dump mit anschließendem Einspielen. Sicher vorbereiten: zuerst einen vollständigen Dump mit `pg_dump` oder `pg_dumpall` anlegen und seine Wiederherstellbarkeit testen. Dann die neue Version in einem Testprojekt mit leerem Volume starten, den Dump einspielen und die Anwendung dagegen prüfen. Erst danach den Produktivstack umstellen, mit dem Volume-Pfad aus der Dokumentation der neuen Version.

Lösungen zu Teil VI

Verständnisfragen

V1. Ein einziger Eingang (nur Port 80 und 443 sind offen), zentrale Verschlüsselung mit TLS und Verteilung nach Namen. Weitere Vorteile: Die Anwendungen brauchen keine öffentlichen Ports, neue Anwendungen kommen ohne Änderung für Besucher hinzu, und der Proxy kann Zugriffe protokollieren und Anfragen begrenzen.

V2. Ein Forward Proxy steht auf der Seite der Benutzer und fragt stellvertretend für sie im Internet an. Ein Reverse Proxy steht auf der Seite der Server und leitet Anfragen an die Anwendungen im Hintergrund weiter.

V3. `Host` nennt den Namen, den der Besucher eingegeben hat, `X-Forwarded-For` die Kette der Adressen der Anfrage. Ohne sie sieht die Anwendung nur die Adresse und den internen Namen des Proxys, nicht den tatsächlichen Besucher.

V4. Ohne Schrägstrich am Ende reicht nginx die Adresse unverändert weiter (`/api/info` bleibt `/api/info`). Mit Schrägstrich ersetzt es den Teil `/api/` (`/api/info` wird zu `/info`).

V5. nginx löst den Namen beim Start einmal auf und merkt sich die Adresse. Bekommt der Container nach einem Austausch eine neue, ruft nginx die alte auf. Abhilfe: `resolver 127.0.0.11 valid=10s;` und eine Variable in `proxy_pass`, damit nginx den Namen bei jeder Anfrage neu auflöst.

V6. Das Zertifikat enthält den Namen und den öffentlichen Schlüssel und ist von einer Zertifizierungsstelle unterschrieben. Der private Schlüssel bleibt geheim und beweist die Echtheit. Let's Encrypt stellt Zertifikate nur für öffentliche Namen aus, deren Kontrolle es überprüfen kann. `docker01.lan` ist ein interner Name.

V7. Portainer hat Zugriff auf den Docker-Socket und damit Root-Rechte über den Server. Ist der Port nur an `127.0.0.1` gebunden, erreicht niemand die Oberfläche aus dem Netz. Der Zugang erfolgt über einen SSH-Tunnel (`ssh -L 9443:127.0.0.1:9443 admin@docker01`) und dann `https://localhost:9443`.

V8. `:ro` macht nur die Socket-Datei schreibgeschützt. Über den Socket laufen weiterhin alle Befehle der Docker-Schnittstelle, auch schreibende wie das Starten eines Containers mit eingebundenem Hauptverzeichnis.

Multiple-Choice-Aufgaben

M1	M2	M3	M4	M5	M6
(b)	(c)	(d)	(a)	(b)	(b)

Praktische Aufgaben

P1. Das Dockerfile enthält diese Zeilen. Gebaut wird mit `docker build -t verein-web:1.0 .`. Der Healthcheck steht anschließend in der Compose-Datei.

Dockerfile

```
FROM nginx:stable-alpine
COPY nginx/default.conf /etc/nginx/conf.d/default.conf
COPY html/ /usr/share/nginx/html/
```

Healthcheck

```
healthcheck:
  test: ["CMD", "wget", "-q", "--spider", "http://127.0.0.1/"]
  interval: 15s
  timeout: 3s
  retries: 3
  start_period: 5s
```

Danach `docker compose exec web nginx -t` und `docker compose exec web nginx -s reload`. Den Zustand prüft `docker compose ps`.

P2. Im Skript `zertifikat.sh` kommt `DNS:rot.docker01.lan` in die `subjectAltName`-Liste, das Zertifikat wird neu erzeugt. In der `default.conf` entstehen ein weiterer `server`-Block für `rot.docker01.lan` mit `set $ziel http://app-rot:80;` und der Name `rot.docker01.lan` in der Umleitung. In der Compose-Datei kommt ein Dienst `app-rot` im Netz `intern` mit eigener Seite hinzu. Danach `docker compose up -d`, `docker compose exec proxy nginx -s reload` und `curl -sk --resolve rot.docker01.lan:443:127.0.0.1 https://rot.docker01.lan/`.

P3. Compose-Datei wie in Kapitel 17 mit `"127.0.0.1:9443:9443"`. `sudo ss -tlnp | grep 9443` zeigt `127.0.0.1:9443` als Adresse. Auf dem Arbeitsrechner `ssh -L 9443:127.0.0.1:9443 admin@docker01`, im Browser `https://localhost:9443`. Das Konto muss innerhalb weniger Minuten angelegt werden, sonst hilft `docker compose restart portainer`.

Fehlersuche

F1. Die Dateien sind für den nginx-Benutzer nicht lesbar, oder es gibt keine `index.html` und kein erlaubtes Verzeichnislisting. Prüfen mit `ls -l` und `docker exec NAME ls -l /usr/share/nginx/html`. Abhilfe: `chmod -R a+rX` auf das Verzeichnis, gegebenenfalls eine `index.html` anlegen.

F2. nginx kann den Namen `app` beim Start nicht auflösen, weil der Container fehlt, noch nicht gestartet ist oder in einem anderen Netz hängt. Abhilfe: den Container im selben Netz starten und für den Proxy `resolver 127.0.0.11` mit einer Variablen in `proxy_pass` verwenden, damit er auch ohne die Anwendung startet.

F3. Moderne Browser prüfen den Namen in der Liste `subjectAltName`, nicht im Feld `CN`. Der Name `blau.docker01.lan` muss dort stehen. Zertifikat mit `-addext "subjectAltName=DNS:…"` neu erzeugen.

Szenario aus dem Betrieb

Portainer hat Root-Rechte über den Server. Ein direkt erreichbarer Anmeldeschirm lädt zu Passwortversuchen und zu Angriffen auf Lücken ein, und ein erfolgreicher Angriff übernimmt den ganzen Server. Sicherere Wege: Zugang nur über einen SSH-Tunnel oder ein VPN, sodass der Port nie öffentlich ist, oder Portainer hinter einem Reverse Proxy mit gültigem TLS-Zertifikat, Beschränkung auf bekannte Adressen und starken, personenbezogenen Konten. In beiden Fällen bleibt Portainer aktuell.

Lösungen zu Teil VII

Verständnisfragen

V1. Netzwerk, Host, Docker-Daemon, Container, Anwendung und Daten. Verteidigung in der Tiefe heißt, sich nie auf eine einzelne Maßnahme zu verlassen: Jede Schicht fängt Fehler der äußeren auf.

V2. Gelingt ein Ausbruch, ist der Angreifer auf dem Host ebenfalls Root, weil Container und Host einen Kernel teilen. Verringern lässt sich das, indem der Prozess im Container nicht als Root läuft (`USER`), oder indem Daemon und Container im Rootless-Modus oder mit `userns-remap` laufen, sodass Root im Container einer unprivilegierten Nummer des Hosts entspricht.

V3. Wer den Socket bedienen kann, kann einen Container mit eingebundenem Hauptverzeichnis des Hosts starten und damit alles lesen und ändern. `:ro` macht nur die Socket-Datei schreibgeschützt, über den Socket laufen weiterhin alle Befehle der Docker-Schnittstelle.

V4. Capabilities zerlegen die Rechte von Root in einzelne Sonderrechte. `--cap-drop ALL` entfernt alle, sodass nur gezielt hinzugefügte gelten. `no-new-privileges` verhindert, dass ein Prozess nachträglich zusätzliche Rechte erlangt.

V5. Seccomp filtert die Systemaufrufe, die ein Prozess verwenden darf. AppArmor begrenzt Zugriffe auf Dateien und Funktionen über Profile. Aktiv sind sie, wenn `docker info --format '{{.SecurityOptions}}'` die Einträge `seccomp` und `apparmor` zeigt und `docker inspect` beim Container das Profil `docker-default` nennt.

V6. Umgebungsvariablen stehen in `docker inspect`, können in Protokolle geraten und werden an Kindprozesse vererbt. Eine Datei unter `/run/secrets` steht weder in der Compose-Datei noch in der Umgebung des Containers.

V7. Tags sind veränderlich, auch ein scheinbar fester Tag kann auf neue Inhalte zeigen. Wichtige Images legt man deshalb mit dem Digest fest und aktualisiert bewusst und mit Abstand. Ein Werkzeug mit eingebundenem Docker-Socket hat Root-Rechte, auch ein Sicherheitswerkzeug. Man bindet den Socket nicht ein und übergibt dem Scanner das Image als Archiv.

V8. RPO (Recovery Point Objective) ist der größte tolerierte Datenverlust, gemessen in Zeit. RTO (Recovery Time Objective) ist die längste tolerierte Ausfallzeit. Die 3-2-1-Regel verlangt drei Kopien auf zwei Arten von Datenträgern, davon eine an einem anderen Ort.

Multiple-Choice-Aufgaben

M1	M2	M3	M4	M5	M6
(b)	(d)	(a)	(c)	(a)	(b)

Praktische Aufgaben

P1. Ein möglicher Aufruf:

```
$ docker run --rm --read-only --cap-drop ALL --user 10001:10001 --
    ↪ tmpfs /tmp alpine sh -c \
    "touch /x; touch /tmp/y && echo tmp-ok; grep CapEff /proc/self/
    ↪ status"
touch: /x: Read-only file system
tmp-ok
CapEff:^^I0000000000000000
```

P2. Ein Einzeiler und die Prüfung mit dem Skript:

```
$ docker ps -q | xargs docker inspect --format '{{.Name}}: Benutzer
    ↪ ="{{.Config.User}}"'
$ ./audit.sh
```

Ein leerer Wert hinter `Benutzer=` bedeutet Root. Typische Befunde, die sich schnell beheben lassen, sind fehlende Limits (`mem_limit`, `pids_limit`), fehlendes `no-new-privileges` und ein beschreibbares Dateisystem (`read_only: true` mit `tmpfs`).

P3. Skript, Dienst und Timer wie in Kapitel 19, danach `sudo systemctl enable --now NAME.timer` und `systemctl list-timers NAME.timer`. Rückspielen eines Volume-Archivs: neues Volume anlegen und mit einem Hilfscontainer `tar xzf` ausführen, wie in Kapitel 10.

Fehlersuche

F1. 127 bedeutet, dass der Befehl im Container nicht gefunden wurde: Tippfehler, falscher Pfad oder ein Programm, das im Image fehlt. Vorgehen: `docker logs`, dann das Image mit `--entrypoint sh` starten und mit `ls` und `which` nachsehen. Auch Windows-Zeilenenden in einem Skript führen zu dieser Meldung.

F2. Die Regel, die Pakete bestehender Verbindungen durchlässt, fehlt oder steht nicht vor dem `DROP`. Ohne sie verwirft die Regel die Antworten auf Verbindungen, die Container nach außen aufgebaut haben. Abhilfe: Eine Regel mit `-m conntrack --ctstate ESTABLISHED,RELATED -j ACCEPT` an den Anfang der Kette `DOCKER-USER` setzen, wie in Kapitel 11.

F3. `sudo systemctl status docker` und `sudo journalctl -u docker -b` lesen, dann die Datei mit `sudo dockerd --validate` und der Option `--config-file` auf `/etc/docker/daemon.json` prüfen. Meist fehlt ein Komma oder eine Klammer. Im Notfall die Datei beiseite verschieben und den Dienst neu starten.

Szenario aus dem Betrieb

Der Rat ist gefährlich. Mit den Rechten `666` darf jeder Benutzer des Systems den Docker-Socket bedienen und hat damit Root-Rechte. Stattdessen prüfen Sie, ob der betroffene Benutzer in der Gruppe `docker` ist (`id`), nehmen ihn gegebenenfalls bewusst auf (`sudo usermod -aG docker NAME`) und lassen ihn sich neu anmelden. Ist die Mitgliedschaft nicht gewünscht, bleibt es bei `sudo`. Für Dienste, die den Socket brauchen, kommt ein Socket-Proxy oder der Rootless-Modus in Frage.

Lösungen zu Teil VIII

Verständnisfragen

V1. Container teilen den Kernel des Hosts. Ein Windows-Rechner hat nur den Windows-Kernel. Windows-Container nutzen ihn direkt, für Linux-Container braucht es einen Linux-Kernel in einer virtuellen Maschine (bei Docker Desktop mit WSL 2).

V2. Pro Benutzer: installiert ohne Administratorrechte in das Benutzerprofil, ohne privilegierten Dienst, Backends WSL 2 oder Docker VMM, keine Windows-Container. Für alle Benutzer: braucht Administratorrechte, installiert den Hilfsdienst `com.docker.service`, erlaubt zusätzlich das Hyper-V-Backend und Windows-Container.

V3. WSL 2 liefert den Linux-Kernel in einer schlanken virtuellen Maschine, in der die Docker Engine von Docker Desktop läuft. Die Datei `.wslconfig` begrenzt Arbeitsspeicher, Prozessoren und Auslagerung der WSL-2-Umgebung.

V4. Die Mitgliedschaft gewährt Zugriff auf den Docker-Daemon über die geschützte Named Pipe. Das entspricht Administratorrechten auf dem Host. Mit aktivierten Windows-Containern können sich Mitglieder zu Administratoren hochstufen.

V5. Bei der Prozess-Isolation teilen sich die Container den Kernel des Hosts (schlank, schnell). Bei der Hyper-V-Isolation hat jeder Container eine eigene kleine virtuelle Maschine mit eigenem Windows-Kernel (stärkere Trennung, unabhängig von der Host-Version).

V6. Der Kernel des Containers passt nicht zum Host, das Image und der Host haben eine unverträgliche Build-Nummer. Lösungen: Image auf passender Basis neu bauen, bei neuerem Host mit `--isolation=hyperv` starten oder einen Host mit passender Windows-Version verwenden.

V7. Docker unterstützt Docker Desktop ausdrücklich nicht auf Windows-Server-Versionen. Auf Windows Server verwendet man eine Container-Runtime (Moby/Docker Community Edition, Mirantis Container Runtime oder containerd), die als Windows-Dienst läuft.

V8. Das Skript schaltet die Windows-Funktion `Containers` ein, lädt `docker.exe` und `dockerd.exe`, legt die `daemon.json` an und registriert den Dienst `docker`. Compose (und Buildx) fehlen. Compose wird gesondert als `docker-compose.exe` installiert.

V9. Zum Beispiel: Anführungszeichen (einfache für Vorlagen, doppelte für Variablen), der Gravis am Zeilenende als letztes Zeichen, `curl` als Alias für `Invoke-WebRequest`, > schreibt in Version 5.1 UTF-16, && gibt es nur in PowerShell 7, ein BOM in Konfigurationsdateien.

V10. Im Linux-Dateisystem der WSL-Distribution. Das ist deutlich schneller als Bindungen aus dem Windows-Dateisystem, und Linux-Container erhalten nur dort Dateiänderungs-Ereignisse (inotify).

Multiple-Choice-Aufgaben

M1	M2	M3	M4	M5	M6	M7	M8
(b)	(a)	(c)	(d)	(a)	(c)	(b)	(d)

Praktische Aufgaben

P1. Zum Beispiel: `Get-ComputerInfo -Property OsName, OsVersion, OsBuildNumber`, `Get-CimInstance Win32_Processor | Select-Object VirtualizationFirmwareEnabled`, `Get-Service LanmanServer` und `wsl --version`. Anforderung: Build ab 22631, Virtualisierung `True`, Dienst laufend und automatisch, WSL ab 2.1.5.

P2. In der WSL-Distribution: Ordner mit `index.html` im Home-Verzeichnis anlegen und `docker run -d --name n90 -p 127.0.0.1:8090:80 -v`

`~/seite:/usr/share/nginx/html:ro nginx:stable-alpine` ausführen. `docker port n90` zeigt `127.0.0.1:8090`, `Get-NetTCPConnection -LocalPort 8090` zeigt die lokale Adresse `127.0.0.1`.

P3. Datei `.wslconfig` mit `[wsl2]`, `memory=4GB` und `processors=2` anlegen, `wsl --shutdown`, Docker Desktop neu starten, `docker info --format '{{.MemTotal}} {{.NCPU}}'` zeigt etwa 4 GB und 2.

P4. Nach dem Umschalten auf den Windows-Daemon: je ein Container mit `--isolation=process` und `--isolation=hyperv` und dem Befehl `ping -t localhost`. `Get-Process -Name ping` zeigt nur den Prozess des Containers mit Prozess-Isolation.

Fehlersuche

F1. Hardware-Virtualisierung in BIOS/UEFI einschalten (Intel VT-x, AMD SVM). Mit `Get-CimInstance Win32_Processor` prüfen. In einer virtuellen Maschine die verschachtelte Virtualisierung freigeben. Danach WSL aktualisieren (`wsl --update`) und Docker Desktop neu starten.

F2. Der Daemon steht im Linux-Modus. Mit `& $Env:ProgramFiles\Docker\Docker\DockerCli.exe -SwitchDaemon` auf Windows-Container umschalten. Das geht nur im Installationsmodus für alle Benutzer und bei Windows Pro oder Enterprise.

F3. Hostversion und Image mit `ver` und der Registry (`CurrentBuild`) vergleichen. Dann ein zur Host-Version passendes Image wählen oder mit `--isolation=hyperv` starten, wenn der Host neuer ist (Voraussetzung: Hyper-V).

F4. Die Datei wurde mit BOM gespeichert, nginx liest das BOM als Teil des ersten Befehls. Die Datei neu als UTF-8 ohne Signatur speichern (Notepad: „UTF-8“) oder mit der Funktion `Write-Utf8NoBom` schreiben.

Szenario aus dem Betrieb

S1. Der Plan ist nicht tragfähig: Docker Desktop wird auf Windows Server nicht unterstützt und ist ein Arbeitsplatzwerkzeug mit eigenen Lizenzbedingungen. Tragfähig ist Windows Server als Host mit einer Container-Runtime

(zum Beispiel Docker Community Edition per Microsoft-Skript oder Mirantis Container Runtime), einem Windows-Basisimage mit passendem Tag (`servercore:ltsc2022` für das .NET Framework), abgestimmter Host- und Image-Version, regelmäßigen Updates, Hyper-V-Isolation für kritische Trennung und einer geklärten Windows-Lizenz.

S2. Die Projektdateien liegen im Windows-Dateisystem. Bindungen von dort sind langsam, und Linux-Container erhalten keine Dateiänderungs-Ereignisse. Abhilfe: Das Projekt in das Linux-Dateisystem der WSL-Distribution verlegen (Home-Verzeichnis) und dort mit `docker run -v` oder Compose arbeiten. Zusätzlich die Ressourcen von WSL mit `.wslconfig` passend begrenzen.

Befehlsreferenz

Diese Referenz fasst die Befehle zusammen, die im Buch vorkommen, geordnet nach Aufgaben. Jeder Eintrag hat denselben Aufbau: Auf die Syntax folgen eine Erklärung, die wichtigsten Parameter, ein Beispiel mit der Ausgabe, und wo es nötig ist, ein Warnhinweis. In der Syntax stehen Großbuchstaben für Angaben, die Sie einsetzen, und eckige Klammern für Optionales.

Die Ausgaben in den Beispielen sind Beispiele und weichen bei Ihnen ab: Kennungen, Zeitangaben, Größen und Versionsnummern sind andere, und Zeilen mit `...` stehen für Gekürztes. Die Befehle gelten für Docker Engine 29 auf Debian 13 und Ubuntu Server. Die vollständige Beschreibung jedes Befehls liefert `docker BEFEHL --help` und die Dokumentation von Docker. Die letzte Gruppe enthält die Windows- und PowerShell-Befehle zu Kapitel 21, mit `PS>` als Eingabeaufforderung.

Docker-Grundbefehle

docker version
`docker version [OPTIONEN]`

ERKLÄRUNG

Zeigt die Versionen von Client und Server mit `containerd` und `runc`. Fehlt der Server-Block, erreicht der Client den Daemon nicht.

PARAMETER

Parameter	Wirkung
`-f, --format TEXT`	Gibt nur die angeforderten Felder aus (Vorlage in der Sprache Go).

BEISPIEL

```
$ docker version --format '{{.Server.Version}}'
29.x.y
```

docker info
`docker info [OPTIONEN]`

ERKLÄRUNG

Zeigt den Zustand des Daemons: Anzahl der Container und Images, Kernel, Speichertreiber, cgroup-Version, Sicherheitsoptionen und das Datenverzeichnis.

PARAMETER

Parameter	Wirkung
`-f, --format TEXT`	Gibt nur einzelne Felder aus.

BEISPIEL

```
$ docker info --format '{{.Driver}} {{.CgroupVersion}}'
overlayfs 2
```

docker –help

```
docker [BEFEHL [UNTERBEFEHL]] --help
```

ERKLÄRUNG

Zeigt Syntax und Optionen jedes Befehls. Die Hilfe stimmt immer mit der installierten Version überein und ist damit verlässlicher als Anleitungen im Netz.

PARAMETER

Parameter	Wirkung
`--help`	Zeigt die Hilfe zum Befehl.

BEISPIEL

```
$ docker run --help | head -n 3
Usage:  docker run [OPTIONS] IMAGE [COMMAND] [ARG...]

Create and run a new container from an image
```

docker login

```
docker login [OPTIONEN] [SERVER]
```

ERKLÄRUNG

Meldet am Docker-Daemon bei einer Registry an. Ohne Serverangabe gilt Docker Hub.

PARAMETER

Parameter	Wirkung
`-u, --username NAME`	Benutzername.
`--password-stdin`	Liest das Passwort oder den Zugriffstoken von der Standardeingabe.
`-p, --password TEXT`	Passwort auf der Kommandozeile. Nicht verwenden.

BEISPIEL

```
$ echo "$ZUGRIFFSTOKEN" | docker login -u benutzername --password-
    ↪ stdin
Login Succeeded
```

Warnhinweis

Die Zugangsdaten landen in der Datei `~/.docker/config.json`. Die Option `-p` zeigt das Passwort in der Prozessliste und im Verlauf der Shell. Verwenden Sie Zugriffstoken statt Passwörter und `--password-stdin`.

docker logout

```
docker logout [SERVER]
```

ERKLÄRUNG

Entfernt die gespeicherten Zugangsdaten für eine Registry.

PARAMETER

Parameter	Wirkung
SERVER	Adresse der Registry, ohne Angabe Docker Hub.

BEISPIEL

```
$ docker logout
Removing login credentials for https://index.docker.io/v1/
```

docker context

```
docker context ls | create NAME --docker "host=..." | use NAME | rm NAME
```

ERKLÄRUNG

Verwaltet Verbindungen zu verschiedenen Docker-Daemons. Mit einem SSH-Kontext steuert der lokale Client einen entfernten Server.

PARAMETER

Parameter	Wirkung
`--docker "host=ssh://BENUTZER@RECHNER"`	Legt die Adresse des Daemons fest.
`--description TEXT`	Beschreibung des Kontexts.

BEISPIEL

```
$ docker context create server1 --docker "host=ssh://admin@docker01"
server1
Successfully created context "server1"
$ docker context use server1
server1
Current context is now "server1"
$ docker context ls --format "table {{.Name}}\t{{.Current}}"
NAME      CURRENT
default   false
server1   true
```

Warnhinweis

Ein TCP-Kontext ohne TLS macht den Daemon für jeden im Netz zugänglich. Verwenden Sie SSH.

Image-Befehle

docker pull

```
docker pull [OPTIONEN] NAME[:TAG|@DIGEST]
```

ERKLÄRUNG

Lädt ein Image aus einer Registry. Ohne Tag gilt `latest`. Bei einem vorhandenen Tag holt der Befehl die aktuelle Fassung.

PARAMETER

Parameter	Wirkung
`-q, --quiet`	Unterdrückt die Fortschrittsanzeige.
`-a, --all-tags`	Lädt alle Tags des Images.
`--platform PLATTFORM`	Wählt eine Architektur, zum Beispiel `linux/amd64`.

BEISPIEL

```
$ docker pull alpine:3
3: Pulling from library/alpine
...: Pull complete
Digest: sha256:...
Status: Downloaded newer image for alpine:3
docker.io/library/alpine:3
```

Warnhinweis

Ein Tag ist veränderlich. Für wichtige Images verwenden Sie den Digest (`NAME@sha256:...`).

docker images

```
docker images [OPTIONEN] [NAME[:TAG]]
```

ERKLÄRUNG

Listet die lokalen Images. Seit Docker 29 erscheint standardmäßig die Spaltenansicht mit IMAGE, ID, DISK USAGE, CONTENT SIZE und EXTRA. Die frühere Ansicht lässt sich mit `--format` nachbilden. Der gleiche Befehl heißt `docker image ls`.

PARAMETER

Parameter	Wirkung
`-a, --all`	Zeigt auch Zwischenschichten.
`-q, --quiet`	Gibt nur die Image-Kennungen aus.
`--digests`	Zeigt die Prüfsummen.
`--no-trunc`	Zeigt vollständige Kennungen.
`-f, --filter FILTER`	Filtert, zum Beispiel `dangling=true` oder `reference=nginx*`.
`--format TEXT`	Wählt Spalten über eine Vorlage.

BEISPIEL

```
$ docker images --format "table {{.Repository}}\t{{.Tag}}\t{{.ID}}\t
    ↪ {{.Size}}"
REPOSITORY   TAG             IMAGE ID       SIZE
alpine       3               a8560b36e8b8   13.6MB
nginx        stable-alpine   d4918ca78576   225MB
```

docker build

```
docker build [OPTIONEN] KONTEXT
```

ERKLÄRUNG

Baut ein Image aus einem Dockerfile. Der Kontext ist das Verzeichnis, dessen Inhalt Docker für den Bau erhält, meist ..

PARAMETER

Parameter	Wirkung
`-t, --tag NAME[:TAG]`	Name und Tag des Ergebnisses.
`-f, --file DATEI`	Pfad zu einem Dockerfile mit anderem Namen oder Ort.
`--build-arg NAME=WERT`	Setzt ein `ARG` des Dockerfiles.
`--no-cache`	Ignoriert den Zwischenspeicher.
`--pull`	Holt das Basisimage frisch.
`--target STUFE`	Baut nur bis zu einer Stufe eines mehrstufigen Builds.
`--check`	Prüft das Dockerfile, ohne zu bauen.

BEISPIEL

```
$ docker build -t meine-seite:1.0 .
[+] Building 1.9s (7/7) FINISHED
 => [2/2] COPY index.html /usr/share/nginx/html/index.html
 => => naming to docker.io/library/meine-seite:1.0
```

Warnhinweis

Alles im Kontext wird an den Daemon übergeben. Legen Sie eine `.dockerignore` an und schließen Sie Zugangsdaten aus.

docker tag

```
docker tag QUELLE[:TAG] ZIEL[:TAG]
```

ERKLÄRUNG

Gibt einem vorhandenen Image einen zusätzlichen Namen, ohne es zu kopieren.

PARAMETER

Parameter	Wirkung
`QUELLE`	Vorhandenes Image.
`ZIEL`	Neuer Name, gegebenenfalls mit Registry-Adresse und Namensraum.

BEISPIEL

```
$ docker tag meine-seite:1.0 localhost:5000/meine-seite:1.0
$ docker images localhost:5000/meine-seite --format "{{.Repository
    ↪ }}:{{.Tag}}"
localhost:5000/meine-seite:1.0
```

docker push

```
docker push [OPTIONEN] NAME[:TAG]
```

ERKLÄRUNG

Lädt ein Image in eine Registry. Dabei gehen nur die Schichten über das Netz, die die Registry noch nicht kennt.

PARAMETER

Parameter	Wirkung
`-a, --all-tags`	Lädt alle lokalen Tags des Images hoch.
`-q, --quiet`	Unterdrückt die Fortschrittsanzeige.

BEISPIEL

```
$ docker push localhost:5000/meine-seite:1.0
The push refers to repository [localhost:5000/meine-seite]
...: Pushed
1.0: digest: sha256:... size: ...
```

Warnhinweis

Prüfen Sie vor dem Hochladen, dass das Image keine Geheimnisse enthält. Auch gelöschte Dateien bleiben in früheren Schichten lesbar.

docker rmi

```
docker rmi [OPTIONEN] IMAGE [IMAGE ...]
```

ERKLÄRUNG

Entfernt Images beziehungsweise ihre Namen (Tags). Der Befehl heißt auch `docker image rm`. Ein Image, das ein Container verwendet, lässt sich nicht löschen.

PARAMETER

Parameter	Wirkung
`-f, --force`	Erzwingt das Löschen.
`--no-prune`	Löscht ungenannte Elternschichten nicht mit.

BEISPIEL

```
$ docker rmi meine-seite:1.0
Untagged: meine-seite:1.0
Deleted: sha256:3f9c6d2a1b7e...
```

Warnhinweis

`-f` lässt Container ohne Image zurück. Entfernen Sie besser zuerst die Container.

docker history

```
docker history [OPTIONEN] IMAGE
```

ERKLÄRUNG

Zeigt die Schichten eines Images mit dem Befehl, der sie erzeugt hat, und ihrer Größe. Damit sieht man, woher die Größe stammt.

PARAMETER

Parameter	Wirkung
`--no-trunc`	Zeigt die Befehle vollständig.
`-q, --quiet`	Gibt nur Kennungen aus.
`--format TEXT`	Wählt Spalten über eine Vorlage.

BEISPIEL

```
$ docker history --format "{{.Size}}\t{{.CreatedBy}}" nginx:stable-
    ↪ alpine | head -n 3
0B      CMD ["nginx" "-g" "daemon off;"
0B      STOPSIGNAL SIGQUIT
0B      EXPOSE map[80/tcp:{}]
```

docker image inspect

```
docker image inspect [OPTIONEN] IMAGE
```

ERKLÄRUNG

Gibt alle Angaben zu einem Image als JSON aus: Schichten, Konfiguration, Architektur, Digest.

PARAMETER

Parameter	Wirkung
`-f, --format TEXT`	Gibt einzelne Felder aus.

BEISPIEL

```
$ docker image inspect nginx:stable-alpine --format '{{.Architecture
    ↪ }} {{.Os}}'
amd64 linux
```

docker save

```
docker save [OPTIONEN] IMAGE [IMAGE ...]
```

ERKLÄRUNG

Schreibt Images als Archiv (tar). Damit lassen sich Images ohne Registry weitergeben oder an Prüfwerkzeuge übergeben.

PARAMETER

Parameter	Wirkung
`-o, --output DATEI`	Zieldatei statt Standardausgabe.

BEISPIEL

```
$ docker save nginx:stable-alpine -o /tmp/nginx.tar
$ ls -lh /tmp/nginx.tar
-rw------- 1 admin admin 59M Nov 11 15:10 /tmp/nginx.tar
```

docker load

```
docker load [OPTIONEN]
```

ERKLÄRUNG

Liest Images aus einem Archiv, das `docker save` erzeugt hat.

PARAMETER

Parameter	Wirkung
`-i, --input DATEI`	Quelldatei statt Standardeingabe.
`-q, --quiet`	Unterdrückt die Fortschrittsanzeige.

BEISPIEL

```
$ docker load -i /tmp/nginx.tar
Loaded image: nginx:stable-alpine
```

> **Warnhinweis**
> Laden Sie nur Archive aus vertrauenswürdiger Quelle. Ein Archiv ist ein Image und kann beliebigen Code enthalten.

Container-Befehle

docker run

```
docker run [OPTIONEN] IMAGE [BEFEHL] [ARGUMENTE]
```

ERKLÄRUNG

Erzeugt aus einem Image einen Container und startet ihn. Fehlt das Image lokal, lädt Docker es zuerst. Die Tabelle nennt die häufigsten Optionen, weitere stehen im Abschnitt über sicherheitsrelevante Optionen.

PARAMETER

Parameter	Wirkung
`-d, --detach`	Startet im Hintergrund.
`-i, -t`	Hält die Eingabe offen und weist ein Terminal zu, meist als `-it`.
`--name NAME`	Vergibt einen festen Namen.
`-p HOST:CONTAINER`	Veröffentlicht einen Port, auch `127.0.0.1:HOST:CONTAINER`.
`-e NAME=WERT, --env-file DATEI`	Setzt Umgebungsvariablen.
`-v QUELLE:ZIEL[:ro]`	Bindet Volume oder Verzeichnis ein.
`--network NETZ`	Verbindet den Container mit einem Netzwerk.
`--rm`	Löscht den Container nach dem Ende.
`--restart REGEL`	Neustartregel: `no`, `on-failure[:n]`, `always`, `unless-stopped`.
`-w, --workdir PFAD`	Arbeitsverzeichnis im Container.
`--entrypoint BEFEHL`	Ersetzt den Einstiegspunkt des Images.

BEISPIEL

```
$ docker run -d --name web -p 8080:80 --restart unless-stopped nginx:
    ↪ stable-alpine
c41e8d0b6f2a8b1c0d7e5a9f3b2c4d6e8f0a1b3c5d7e9f1a2b4c6d8e0f2a4b6c
```

Warnhinweis

Ein veröffentlichter Port umgeht `ufw`. Veröffentlichen Sie nur, was von außen erreichbar sein soll.

docker create

```
docker create [OPTIONEN] IMAGE [BEFEHL] [ARGUMENTE]
```

ERKLÄRUNG

Legt einen Container an, ohne ihn zu starten. Er steht danach im Zustand `created`. Die Optionen entsprechen denen von `docker run`.

PARAMETER

Parameter	Wirkung
`--name NAME`	Vergibt einen festen Namen.
`-p, -v, -e`	Wie bei `docker run`.

BEISPIEL

```
$ docker create --name vorbereitet -p 8082:80 nginx:stable-alpine
8a1b5c7d9e0f2a3b4c5d6e7f8091a2b3c4d5e6f708192a3b4c5d6e7f8091a2b3
$ docker start vorbereitet
vorbereitet
```

docker start

```
docker start [OPTIONEN] CONTAINER [CONTAINER ...]
```

ERKLÄRUNG

Startet gestoppte oder angelegte Container. Die Daten der beschreibbaren Schicht bleiben erhalten.

PARAMETER

Parameter	Wirkung
`-a, --attach`	Verbindet die Ausgabe mit dem Terminal.
`-i, --interactive`	Hält die Eingabe offen.

BEISPIEL

```
$ docker start web
web
```

docker stop

```
docker stop [OPTIONEN] CONTAINER [CONTAINER ...]
```

ERKLÄRUNG

Beendet Container geordnet. Der Hauptprozess bekommt zuerst SIGTERM, nach der Wartezeit SIGKILL.

PARAMETER

Parameter	Wirkung
`-t, --time SEKUNDEN`	Wartezeit bis SIGKILL, Standard 10.
`-s, --signal SIGNAL`	Anderes Signal als SIGTERM.

BEISPIEL

```
$ docker stop -t 30 web
web
```

docker restart

```
docker restart [OPTIONEN] CONTAINER [CONTAINER ...]
```

ERKLÄRUNG

Stoppt und startet Container. Die Containerkennung und die beschreibbare Schicht bleiben bestehen.

PARAMETER

Parameter	Wirkung
`-t, --time SEKUNDEN`	Wartezeit beim Stoppen.

BEISPIEL

```
$ docker restart web
web
```

docker kill

```
docker kill [OPTIONEN] CONTAINER [CONTAINER ...]
```

ERKLÄRUNG

Sendet ein Signal an den Hauptprozess, standardmäßig SIGKILL, also ein sofortiges Ende ohne Aufräumen.

PARAMETER

Parameter	Wirkung
`-s, --signal SIGNAL`	Zu sendendes Signal.

BEISPIEL

```
$ docker kill web
web
```

> **Warnhinweis**
>
> Das Programm kann keine Dateien schließen und keine Datenbank sauber beenden. Nur anwenden, wenn `docker stop` scheitert.

docker pause und docker unpause

```
docker pause CONTAINER | docker unpause CONTAINER
```

ERKLÄRUNG

Friert alle Prozesse eines Containers ein und gibt sie wieder frei. Der Zustand heißt `paused`.

PARAMETER

Parameter	Wirkung
`CONTAINER`	Name oder Kennung.

BEISPIEL

```
$ docker pause web
web
$ docker unpause web
web
```

docker rm

```
docker rm [OPTIONEN] CONTAINER [CONTAINER ...]
```

ERKLÄRUNG

Entfernt Container samt beschreibbarer Schicht. Ein laufender Container muss vorher gestoppt werden.

PARAMETER

Parameter	Wirkung
`-f, --force`	Beendet laufende Container und entfernt sie.
`-v, --volumes`	Entfernt auch die anonymen Volumes des Containers.

BEISPIEL

```
$ docker rm -f web
web
```

Warnhinweis

Daten in der beschreibbaren Schicht sind danach verloren. Mit `-v` verschwinden auch anonyme Volumes, etwa die einer Datenbank.

docker ps

```
docker ps [OPTIONEN]
```

ERKLÄRUNG

Listet Container. Ohne Optionen erscheinen nur die laufenden. Die Spalten sind sehr breit, mit `--format` wählt man die wichtigen.

PARAMETER

Parameter	Wirkung
`-a, --all`	Zeigt auch beendete Container.
`-q, --quiet`	Gibt nur Kennungen aus.
`-n ANZAHL, -l`	Zeigt die zuletzt erzeugten Container.
`-s, --size`	Zeigt zusätzlich die Größe.
`-f, --filter FILTER`	Filtert, zum Beispiel `status=exited`, `name=web`, `health=unhealthy`.
`--format TEXT`	Wählt Spalten über eine Vorlage.

BEISPIEL

```
$ docker ps -a --format "table {{.Names}}\t{{.Image}}\t{{.Status}}"
NAMES    IMAGE                 STATUS
web      nginx:stable-alpine   Up 3 minutes
test     alpine                Exited (0) 6 minutes ago
```

docker logs

```
docker logs [OPTIONEN] CONTAINER
```

ERKLÄRUNG

Zeigt, was der Hauptprozess auf Standardausgabe und Standardfehlerausgabe geschrieben hat.

PARAMETER

Parameter	Wirkung
`-f, --follow`	Folgt neuen Zeilen bis `Strg+C`.
`--tail ANZAHL, -n`	Zeigt nur die letzten Zeilen.
`-t, --timestamps`	Setzt Zeitstempel vor jede Zeile.
`--since ZEIT, --until ZEIT`	Beschränkt auf einen Zeitraum, zum Beispiel `10m` oder `2026-11-11T10:00:00`.

BEISPIEL

```
$ docker logs --tail 2 -t web
2026-11-11T15:20:01.123456789Z 172.17.0.1 - - [11/Nov/2026:15:20:01
    ↪ +0000] "GET / HTTP/1.1" 200 615 "-" "curl/8.x" "-"
2026-11-11T15:20:09.654321987Z 172.17.0.1 - - [11/Nov/2026:15:20:09
    ↪ +0000] "GET /x HTTP/1.1" 404 153 "-" "curl/8.x" "-"
```

Warnhinweis

`docker logs` gibt Standardausgabe und Fehlerausgabe getrennt wieder. Wer mit `grep` sucht, hängt `2>&1` an.

docker exec

```
docker exec [OPTIONEN] CONTAINER BEFEHL [ARGUMENTE]
```

ERKLÄRUNG

Führt einen Befehl in einem laufenden Container aus. Mit `-it` und einer Shell arbeitet man darin wie in einem kleinen System. Beendet man die Shell, läuft der Container weiter.

PARAMETER

Parameter	Wirkung
`-i, -t`	Hält die Eingabe offen und weist ein Terminal zu.
`-u, --user NAME`	Führt den Befehl als anderer Benutzer aus.
`-w, --workdir PFAD`	Arbeitsverzeichnis des Befehls.
`-e NAME=WERT`	Setzt eine Umgebungsvariable.
`-d, --detach`	Führt den Befehl im Hintergrund aus.

BEISPIEL

```
$ docker exec web nginx -v
nginx version: nginx/1.x.y
$ docker exec -it web sh
/ # exit
```

Warnhinweis

Änderungen in einem Container per `exec` sind flüchtig und gehören in ein Image oder Volume.

docker inspect

```
docker inspect [OPTIONEN] OBJEKT [OBJEKT ...]
```

ERKLÄRUNG

Gibt alle Angaben zu einem Container, Image, Netzwerk oder Volume als JSON aus. Mit einer Vorlage greift man einzelne Werte heraus.

PARAMETER

Parameter	Wirkung
`-f, --format TEXT`	Vorlage, zum Beispiel `{{.State.Status}}`.
`-s, --size`	Zeigt bei Containern die Größe.
`--type TYP`	Beschränkt auf `container`, `image`, `network` oder `volume`.

BEISPIEL

```
$ docker inspect --format '{{.State.Status}} {{.RestartCount}}' web
running 0
```

docker cp

```
docker cp CONTAINER:QUELLE ZIEL | docker cp QUELLE CONTAINER:ZIEL
```

ERKLÄRUNG

Kopiert Dateien zwischen Container und Host, auch bei gestoppten Containern.

PARAMETER

Parameter	Wirkung
`-a, --archive`	Behält Besitzer und Gruppe.
`-L, --follow-link`	Folgt symbolischen Verweisen in der Quelle.

BEISPIEL

```
$ docker cp web:/etc/nginx/nginx.conf ./nginx.conf
Successfully copied 2.05kB to /home/admin/nginx.conf
```

Warnhinweis

Kopieren Sie keine Dateien aus Containern fremder Herkunft an Orte, an denen sie ausgeführt werden könnten.

docker top

```
docker top CONTAINER [PS-OPTIONEN]
```

ERKLÄRUNG

Listet die Prozesse eines Containers aus Sicht des Hosts. Die Prozessnummern sind die des Hosts, nicht die im Container.

PARAMETER

Parameter	Wirkung
PS-OPTIONEN	Optionen von ps, zum Beispiel aux.

BEISPIEL

```
$ docker top web
UID    PID    PPID   C   STIME   TTY   TIME       CMD
root   4721   4698   0   10:52   ?     00:00:00   nginx: master
    ↪ process nginx -g daemon off;
101    4770   4721   0   10:52   ?     00:00:00   nginx: worker
    ↪ process
```

docker stats

```
docker stats [OPTIONEN] [CONTAINER ...]
```

ERKLÄRUNG

Zeigt Prozessorlast, Arbeitsspeicher, Netzwerk, Plattenzugriffe und Prozesszahl laufender Container.

PARAMETER

Parameter	Wirkung
`--no-stream`	Gibt einmal aus und beendet sich.
`-a, --all`	Zeigt auch gestoppte Container.
`--format TEXT`	Wählt Spalten über eine Vorlage.

BEISPIEL

```
$ docker stats --no-stream --format "table {{.Name}}\t{{.CPUPerc}}\t
    ↪ {{.MemUsage}}"
NAME    CPU %    MEM USAGE / LIMIT
web     0.00%    4.2MiB / 128MiB
```

docker diff

```
docker diff CONTAINER
```

ERKLÄRUNG

Zeigt, welche Dateien sich in der beschreibbaren Schicht gegenüber dem Image geändert haben. `A` steht für hinzugefügt, `C` für geändert, `D` für gelöscht.

PARAMETER

Parameter	Wirkung
`CONTAINER`	Name oder Kennung.

BEISPIEL

```
$ docker exec web touch /tmp/test.txt
$ docker diff web
C /tmp
A /tmp/test.txt
```

docker attach

```
docker attach [OPTIONEN] CONTAINER
```

ERKLÄRUNG

Verbindet das Terminal mit Standardein- und Standardausgabe des Hauptprozesses eines laufenden Containers.

PARAMETER

Parameter	Wirkung
`--sig-proxy=false`	Leitet `Strg+C` nicht an den Prozess weiter, sodass man sich nur löst.
`--detach-keys TASTEN`	Tastenfolge zum Lösen, Standard `Strg+P Strg+Q`.
`--no-stdin`	Hängt nur die Ausgabe an.

BEISPIEL

```
$ docker attach --sig-proxy=false uhr
Wed Nov 11 15:20:01 UTC 2026
Wed Nov 11 15:20:03 UTC 2026
```

Warnhinweis

Ohne `--sig-proxy=false` beendet `Strg+C` den Hauptprozess und damit den Container. In den meisten Fällen ist `docker exec` die bessere Wahl.

docker rename

```
docker rename ALTER_NAME NEUER_NAME
```

ERKLÄRUNG

Benennt einen Container um, auch im laufenden Betrieb.

PARAMETER

Parameter	Wirkung
`ALTER_NAME, NEUER_NAME`	Namen des Containers.

BEISPIEL

```
$ docker rename test web-test
$ docker ps -a --format "{{.Names}}"
web-test
```

docker update

```
docker update [OPTIONEN] CONTAINER [CONTAINER ...]
```

ERKLÄRUNG

Ändert Limits und Neustartregel eines vorhandenen Containers, ohne ihn neu zu erzeugen.

PARAMETER

Parameter	Wirkung
`--memory GRÖSSE`, `--memory-swap GRÖSSE`	Speicherlimit und Obergrenze mit Auslagerung.
`--cpus ZAHL`	Prozessorleistung.
`--pids-limit ZAHL`	Höchstzahl der Prozesse.
`--restart REGEL`	Neustartregel.

BEISPIEL

```
$ docker update --memory 256m --memory-swap 256m web
web
```

docker port

```
docker port CONTAINER [PORT[/PROTOKOLL]]
```

ERKLÄRUNG

Zeigt, auf welche Adressen und Ports des Hosts die Ports eines Containers veröffentlicht sind.

PARAMETER

Parameter	Wirkung
PORT	Zeigt nur diesen Container-Port.

BEISPIEL

```
$ docker port web
80/tcp -> 0.0.0.0:8080
80/tcp -> [::]:8080
```

Netzwerkbefehle

docker network ls

```
docker network ls [OPTIONEN]
```

ERKLÄRUNG

Listet die Docker-Netzwerke. Immer vorhanden sind `bridge`, `host` und `none`.

PARAMETER

Parameter	Wirkung
`-f, --filter FILTER`	Filtert, zum Beispiel `driver=bridge`.
`--format TEXT`	Wählt Spalten.
`-q, --quiet`	Gibt nur Kennungen aus.

BEISPIEL

```
$ docker network ls --format "table {{.Name}}\t{{.Driver}}\t{{.Scope
    ↪ }}"
NAME      DRIVER    SCOPE
bridge    bridge    local
host      host      local
none      null      local
```

docker network create

```
docker network create [OPTIONEN] NAME
```

ERKLÄRUNG

Legt ein Netzwerk an. Benutzerdefinierte Bridge-Netze lösen Containernamen automatisch auf.

PARAMETER

Parameter	Wirkung
`-d, --driver TREIBER`	`bridge` (Standard), `macvlan`, `ipvlan` und andere.
`--subnet CIDR, --gateway ADRESSE`	Adressbereich und Gateway.
`--ip-range CIDR`	Bereich, aus dem Docker Adressen vergibt.
`--internal`	Netz ohne Zugang nach außen.
`-o, --opt SCHLÜSSEL=WERT`	Treiberoptionen, bei Macvlan `parent=SCHNITTSTELLE`.

BEISPIEL

```
$ docker network create --subnet 10.50.0.0/24 --internal backend
7f4c1d8e2a9b...
```

Warnhinweis

Ein Adressbereich, der sich mit dem Heimnetz überschneidet, führt zu schwer zu findenden Verbindungsproblemen.

docker network inspect

```
docker network inspect [OPTIONEN] NETZ [NETZ ...]
```

ERKLÄRUNG

Zeigt Adressbereich, Gateway und die angeschlossenen Container eines Netzwerks.

PARAMETER

Parameter	Wirkung
`-f, --format TEXT`	Gibt einzelne Felder aus.
`-v, --verbose`	Zeigt zusätzliche Angaben.

BEISPIEL

```
$ docker network inspect backend --format '{{range .IPAM.Config}}{{.
    ↪ Subnet}} {{.Gateway}}{{end}}'
10.50.0.0/24 10.50.0.1
```

docker network connect und disconnect

```
docker network connect [OPTIONEN] NETZ CONTAINER | docker network disconnect
[OPTIONEN] NETZ CONTAINER
```

ERKLÄRUNG

Hängt einen laufenden Container an ein weiteres Netz oder trennt ihn davon.

PARAMETER

Parameter	Wirkung
`--ip ADRESSE`	Feste Adresse im Netz.
`--alias NAME`	Zusätzlicher Name im Netz.
`-f, --force`	Erzwingt das Trennen (bei `disconnect`).

BEISPIEL

```
$ docker network connect --alias datenbank backend db1
$ docker network disconnect backend db1
```

docker network rm

```
docker network rm NETZ [NETZ ...]
```

ERKLÄRUNG

Entfernt ein Netzwerk. Es darf keinen angeschlossenen Container mehr haben.

PARAMETER

Parameter	Wirkung
`NETZ`	Name oder Kennung.

BEISPIEL

```
$ docker network rm backend
backend
```

Warnhinweis

Die Standardnetze `bridge`, `host` und `none` lassen sich nicht entfernen.

Volume-Befehle

docker volume ls

```
docker volume ls [OPTIONEN]
```

ERKLÄRUNG

Listet die Volumes. Anonyme Volumes erkennt man an einem Namen aus Zeichen und Ziffern.

PARAMETER

Parameter	Wirkung
`-q, --quiet`	Gibt nur Namen aus.
`-f, --filter FILTER`	Filtert, zum Beispiel `dangling=true` für ungenutzte Volumes.
`--format TEXT`	Wählt Spalten.

BEISPIEL

```
$ docker volume ls
DRIVER    VOLUME NAME
local     db-daten
local     notizen-daten
```

docker volume create

```
docker volume create [OPTIONEN] [NAME]
```

ERKLÄRUNG

Legt ein benanntes Volume an. Ohne Name vergibt Docker einen zufälligen.

PARAMETER

Parameter	Wirkung
`-d, --driver TREIBER`	Volume-Treiber, Standard `local`.
`--label SCHLÜSSEL=WERT`	Setzt eine Beschriftung.
`-o, --opt SCHLÜSSEL=WERT`	Treiberoptionen.

BEISPIEL

```
$ docker volume create daten
daten
```

docker volume inspect

```
docker volume inspect [OPTIONEN] VOLUME [VOLUME ...]
```

ERKLÄRUNG

Zeigt Treiber, Beschriftungen und den Speicherort eines Volumes auf dem Host.

PARAMETER

Parameter	Wirkung
`-f, --format TEXT`	Gibt einzelne Felder aus.

BEISPIEL

```
$ docker volume inspect daten --format '{{.Mountpoint}}'
/var/lib/docker/volumes/daten/_data
```

docker volume rm

```
docker volume rm [OPTIONEN] VOLUME [VOLUME ...]
```

ERKLÄRUNG

Löscht Volumes samt Inhalt. Ein Volume, das ein Container verwendet, lässt sich nicht löschen.

PARAMETER

Parameter	Wirkung
`-f, --force`	Unterdrückt Fehler bei nicht vorhandenen Volumes.

BEISPIEL

```
$ docker volume rm daten
daten
```

Warnhinweis

Die Daten sind danach unwiederbringlich verloren. Sichern Sie wichtige Volumes vorher.

Compose-Befehle

docker compose up

```
docker compose up [OPTIONEN] [DIENST ...]
```

ERKLÄRUNG

Baut gegebenenfalls Images, legt Netzwerke und Volumes an und startet die Container eines Projekts. Bei einer geänderten Datei ersetzt Compose nur die betroffenen Dienste.

PARAMETER

Parameter	Wirkung
`-d, --detach`	Startet im Hintergrund.
`--build`	Baut die Images vor dem Start neu.
`--wait`	Wartet, bis alle Dienste laufen oder gesund sind.
`--force-recreate`	Erzeugt alle Container neu.
`--pull POLITIK`	Wann Images geholt werden: `always`, `missing` oder `never`.
`--remove-orphans`	Entfernt Container von Diensten, die nicht mehr in der Datei stehen.

BEISPIEL

```
$ docker compose up -d --build --wait
[+] Running 3/3
 ✔ Network projekt4_default   Created
 ✔ Container projekt4-db-1    Healthy
 ✔ Container projekt4-app-1   Healthy
```

docker compose down

```
docker compose down [OPTIONEN]
```

ERKLÄRUNG

Stoppt die Container eines Projekts und entfernt sie samt Netzwerken. Volumes und Images bleiben in der Grundeinstellung erhalten.

PARAMETER

Parameter	Wirkung
`-v, --volumes`	Entfernt auch die im Projekt deklarierten Volumes.
`--rmi local`	Entfernt die für das Projekt gebauten Images.
`--remove-orphans`	Entfernt Container ohne zugehörigen Dienst.

BEISPIEL

```
$ docker compose down
[+] Running 3/3
 ✔ Container projekt4-app-1  Removed
 ✔ Container projekt4-db-1   Removed
 ✔ Network projekt4_default  Removed
```

Warnhinweis

Mit `-v` verschwinden die Daten der Datenbank. Nur nach einer Sicherung verwenden.

docker compose ps

```
docker compose ps [OPTIONEN] [DIENST ...]
```

ERKLÄRUNG

Listet die Container des Projekts mit Zustand und Ports.

PARAMETER

Parameter	Wirkung
`-a, --all`	Zeigt auch beendete Container.
`-q, --quiet`	Gibt nur Kennungen aus.
`--services`	Gibt nur die Namen der Dienste aus.
`--status ZUSTAND`	Filtert nach Zustand, zum Beispiel `running`.
`--format TEXT`	Wählt Spalten, auch `table` oder `json`.

BEISPIEL

```
$ docker compose ps --format "table {{.Service}}\t{{.Status}}\t{{.
    ↪ Ports}}"
SERVICE   STATUS                    PORTS
db        Up 2 minutes (healthy)    3306/tcp
web       Up 2 minutes              0.0.0.0:8080->80/tcp
```

docker compose logs

```
docker compose logs [OPTIONEN] [DIENST ...]
```

ERKLÄRUNG

Zeigt die Protokolle der Dienste eines Projekts, gekennzeichnet mit dem Dienstnamen.

PARAMETER

Parameter	Wirkung
`-f, --follow`	Folgt neuen Zeilen.
`--tail ANZAHL`	Zeigt nur die letzten Zeilen je Dienst.
`-t, --timestamps`	Setzt Zeitstempel.
`--since ZEIT`	Beschränkt auf einen Zeitraum.

BEISPIEL

```
$ docker compose logs --tail 1 web
web-1  | 172.18.0.1 - - [11/Nov/2026:12:41:07 +0000] "GET / HTTP/1.1"
     ↪ 200 25 "-" "curl/8.x" "-"
```

docker compose exec

```
docker compose exec [OPTIONEN] DIENST BEFEHL [ARGUMENTE]
```

ERKLÄRUNG

Führt einen Befehl in einem laufenden Container eines Dienstes aus.

PARAMETER

Parameter	Wirkung
`-T, --no-TTY`	Schaltet das Terminal ab. Nötig, wenn die Ein- oder Ausgabe umgeleitet wird.
`-u, --user NAME`	Führt den Befehl als anderer Benutzer aus.
`-e NAME=WERT`	Setzt eine Umgebungsvariable.

BEISPIEL

```
$ docker compose exec -T db sh -c 'mariadb -uroot -p"
    ↪ $MARIADB_ROOT_PASSWORD" -e "SELECT 1"'
1
1
```

Warnhinweis

Ohne `-T` vermischt sich umgeleitete Ausgabe mit Steuerzeichen.

docker compose run

```
docker compose run [OPTIONEN] DIENST [BEFEHL] [ARGUMENTE]
```

ERKLÄRUNG

Startet für einen Dienst einen einmaligen, neuen Container, zum Beispiel für Wartungsaufgaben.

PARAMETER

Parameter	Wirkung
`--rm`	Löscht den Container nach dem Ende.
`-e NAME=WERT`	Setzt eine Umgebungsvariable.
`--no-deps`	Startet die abhängigen Dienste nicht mit.

BEISPIEL

```
$ docker compose run --rm app python --version
Python 3.x.y
```

docker compose build

```
docker compose build [OPTIONEN] [DIENST ...]
```

ERKLÄRUNG

Baut die Images der Dienste, die in der Datei ein `build` enthalten.

PARAMETER

Parameter	Wirkung
`--pull`	Holt das Basisimage frisch.
`--no-cache`	Ignoriert den Zwischenspeicher.
`--build-arg NAME=WERT`	Setzt ein `ARG`.

BEISPIEL

```
$ docker compose build --pull app
[+] Building 4.3s (9/9) FINISHED
```

docker compose pull

```
docker compose pull [OPTIONEN] [DIENST ...]
```

ERKLÄRUNG

Lädt die Images der Dienste, die ein `image` angeben. Danach erzeugt `docker compose up -d` die Container neu, deren Image sich geändert hat.

PARAMETER

Parameter	Wirkung
`-q, --quiet`	Unterdrückt die Fortschrittsanzeige.
`--ignore-buildable`	Überspringt Dienste mit `build`.

BEISPIEL

```
$ docker compose pull -q
$ docker compose up -d
```

docker compose restart, stop und start

```
docker compose restart|stop|start [OPTIONEN] [DIENST ...]
```

ERKLÄRUNG

Startet Dienste neu, hält sie an oder startet sie wieder. Die Container bleiben erhalten, anders als bei `docker compose down`.

PARAMETER

Parameter	Wirkung
`-t, --timeout SEKUNDEN`	Wartezeit beim Stoppen.

BEISPIEL

```
$ docker compose restart web
[+] Restarting 1/1
 ✔ Container projekt4-web-1  Started
```

docker compose config

```
docker compose config [OPTIONEN]
```

ERKLÄRUNG

Liest die Compose-Dateien, setzt alle Variablen ein und gibt das Ergebnis aus. Ein guter erster Schritt, wenn sich eine Datei nicht wie erwartet verhält.

PARAMETER

Parameter	Wirkung
`-q, --quiet`	Gibt nichts aus, meldet nur Fehler.
`--services`	Gibt nur die Namen der Dienste aus.
`--no-interpolate`	Setzt Variablen nicht ein.

BEISPIEL

```
$ docker compose config --quiet && echo "Datei ist gültig"
Datei ist gültig
```

Warnhinweis

Die Ausgabe enthält eingesetzte Passwörter aus der `.env`. Geben Sie sie nicht weiter.

docker compose ls

```
docker compose ls [OPTIONEN]
```

ERKLÄRUNG

Listet alle Compose-Projekte des Servers mit Zustand und Dateipfad, unabhängig vom aktuellen Verzeichnis.

PARAMETER

Parameter	Wirkung
`-a, --all`	Zeigt auch gestoppte Projekte.

BEISPIEL

```
$ docker compose ls
NAME         STATUS       CONFIG FILES
projekt4     running(3)   /srv/docker/projekt-4/compose.yaml
projekt5     running(3)   /srv/docker/projekt-5/compose.yaml
```

docker compose port

```
docker compose port [OPTIONEN] DIENST PORT
```

ERKLÄRUNG

Zeigt, auf welchen Host-Port ein Port eines Dienstes veröffentlicht ist. Die Ausgabe ist leer, wenn der Port nicht veröffentlicht wird.

PARAMETER

Parameter	Wirkung
`--protocol PROTOKOLL`	`tcp` (Standard) oder `udp`.

BEISPIEL

```
$ docker compose port web 80
0.0.0.0:8080
```

Diagnosebefehle

docker events
`docker events [OPTIONEN]`

ERKLÄRUNG

Zeigt laufend Ereignisse des Daemons: Container starten, stoppen, sterben, Healthchecks wechseln.

PARAMETER

Parameter	Wirkung
`--since ZEIT, --until ZEIT`	Zeitraum, auch rückwirkend.
`-f, --filter FILTER`	Filtert, zum Beispiel `type=container` oder `event=die`.
`--format TEXT`	Wählt Felder.

BEISPIEL

```
$ docker events --since 1h --until 0s --filter event=die --format "{{.
    ↪ Actor.Attributes.name}}"
alt-test
```

docker system df

```
docker system df [OPTIONEN]
```

ERKLÄRUNG

Zeigt den Platz, den Images, Container, Volumes und Build-Zwischenspeicher belegen, und wie viel davon sich freigeben ließe.

PARAMETER

Parameter	Wirkung
`-v, --verbose`	Zeigt die einzelnen Objekte.

BEISPIEL

```
$ docker system df
TYPE            TOTAL   ACTIVE   SIZE      RECLAIMABLE
Images          9       4        1.9GB     1.1GB (57%)
Containers      6       4        52MB      0B (0%)
Local Volumes   5       4        380MB     0B (0%)
Build Cache     21      0        640MB     640MB
```

ss

```
ss [OPTIONEN]
```

ERKLÄRUNG

Zeigt Netzwerkverbindungen und lauschende Ports des Hosts. Damit findet man, wer einen Port belegt.

PARAMETER

Parameter	Wirkung
`-t`	Nur TCP.
`-l`	Nur lauschende Ports.
`-n`	Zahlen statt Namen.
`-p`	Zeigt das zugehörige Programm (braucht Root-Rechte).

BEISPIEL

```
$ sudo ss -tlnp | grep ":8080 "
LISTEN 0 4096 0.0.0.0:8080 0.0.0.0:* users:(("docker-proxy",pid=3141,
    ↪ fd=7))
```

journalctl -u docker

```
journalctl -u docker [OPTIONEN]
```

ERKLÄRUNG

Zeigt die Meldungen des Docker-Dienstes im Journal von `systemd`. Die erste Anlaufstelle, wenn der Daemon nicht startet.

PARAMETER

Parameter	Wirkung
`-b`	Nur seit dem letzten Systemstart.
`--since ZEIT`	Zeitraum, zum Beispiel `"10 minutes ago"`.
`-f`	Folgt neuen Meldungen.
`--no-pager`	Gibt ohne Seitenanzeige aus.

BEISPIEL

```
$ sudo journalctl -u docker -b --no-pager | tail -n 2
... dockerd[2217]: time="..." level=info msg="Daemon has completed
    ↪ initialization"
... dockerd[2217]: time="..." level=info msg="API listen on /run/
    ↪ docker.sock"
```

systemctl status docker

```
systemctl status docker [OPTIONEN]
```

ERKLÄRUNG

Zeigt, ob der Docker-Dienst läuft, seit wann und mit welchen letzten Meldungen.

PARAMETER

Parameter	Wirkung
`--no-pager`	Gibt ohne Seitenanzeige aus.

BEISPIEL

```
$ sudo systemctl status docker --no-pager | head -n 3●
 docker.service - Docker Application Container Engine
     Loaded: loaded (/usr/lib/systemd/system/docker.service; enabled;
     ↪  preset: enabled)
     Active: active (running) since Wed 2026-11-11 10:21:04 CET; 2h 5
     ↪ min ago
```

dockerd –validate

```
dockerd --validate --config-file=DATEI
```

ERKLÄRUNG

Prüft eine `daemon.json`, ohne den Daemon zu starten.

PARAMETER

Parameter	Wirkung
`--config-file DATEI`	Zu prüfende Konfigurationsdatei.

BEISPIEL

```
$ sudo dockerd --validate --config-file=/etc/docker/daemon.json
configuration OK
```

curl –unix-socket

```
curl --unix-socket /var/run/docker.sock http://localhost/PFAD
```

ERKLÄRUNG

Fragt die REST-Schnittstelle des Daemons direkt ab. Damit lässt sich prüfen, ob ein Problem bei Docker oder bei einem Werkzeug darüber liegt.

PARAMETER

Parameter	Wirkung
`--unix-socket DATEI`	Verbindet über einen Unix-Socket statt über das Netzwerk.

BEISPIEL

```
$ sudo curl -s --unix-socket /var/run/docker.sock http://localhost/
    ↪ _ping
OK
```

Warnhinweis

Wer den Socket bedienen kann, hat Root-Rechte. Verwenden Sie nur lesende Pfade wie `/version` oder `/_ping`.

Bereinigungsbefehle

docker container prune

```
docker container prune [OPTIONEN]
```

ERKLÄRUNG

Entfernt alle gestoppten Container.

PARAMETER

Parameter	Wirkung
`-f, --force`	Fragt nicht nach.
`--filter FILTER`	Beschränkt, zum Beispiel `until=24h`.

BEISPIEL

```
$ docker container prune -f
Deleted Containers:
5d3c1f2a4b6c8d0e...
Total reclaimed space: 0B
```

docker image prune

```
docker image prune [OPTIONEN]
```

ERKLÄRUNG

Entfernt Images ohne Tag. Mit `-a` auch alle Images, die kein Container verwendet.

PARAMETER

Parameter	Wirkung
`-a, --all`	Entfernt alle ungenutzten Images, nicht nur untaggte.
`-f, --force`	Fragt nicht nach.
`--filter FILTER`	Beschränkt, zum Beispiel `until=720h` für Images, die älter als 30 Tage sind.

BEISPIEL

```
$ docker image prune -a -f --filter "until=720h"
Total reclaimed space: 412MB
```

> **Warnhinweis**
> Ein ungenutztes Image kann der Rückfallpunkt für ein Update sein.

docker network prune

```
docker network prune [OPTIONEN]
```

ERKLÄRUNG

Entfernt alle Netzwerke, an denen kein Container hängt.

PARAMETER

Parameter	Wirkung
`-f, --force`	Fragt nicht nach.
`--filter FILTER`	Beschränkt, zum Beispiel `until=24h`.

BEISPIEL

```
$ docker network prune -f
Deleted Networks:
projekt3_default
```

docker volume prune

```
docker volume prune [OPTIONEN]
```

ERKLÄRUNG

Entfernt unbenutzte Volumes. Seit Docker 23 trifft das in der Grundeinstellung nur anonyme Volumes.

PARAMETER

Parameter	Wirkung
`-a, --all`	Entfernt auch unbenutzte benannte Volumes.
`-f, --force`	Fragt nicht nach.
`--filter FILTER`	Beschränkt, zum Beispiel `label=...`.

BEISPIEL

```
$ docker volume prune
WARNING! This will remove anonymous local volumes not used by at
    ↪ least one container.
Are you sure you want to continue? [y/N] n
```

Warnhinweis

Das Löschen ist endgültig. Auch ein anonymes Volume kann die Datenbank eines gelöschten Containers enthalten. Mit `-a` gehören benannte Volumes dazu. Prüfen Sie vorher mit `docker volume ls`.

docker builder prune

```
docker builder prune [OPTIONEN]
```

ERKLÄRUNG

Räumt den Zwischenspeicher der Builds auf.

PARAMETER

Parameter	Wirkung
`-a, --all`	Entfernt den gesamten Zwischenspeicher.
`-f, --force`	Fragt nicht nach.
`--filter FILTER`	Beschränkt, zum Beispiel `until=168h`.

BEISPIEL

```
$ docker builder prune -f --filter "until=168h"
Total:^^I640MB
```

docker system prune

```
docker system prune [OPTIONEN]
```

ERKLÄRUNG

Entfernt gestoppte Container, ungenutzte Netzwerke, Images ohne Tag und ungenutzten Build-Zwischenspeicher in einem Schritt.

PARAMETER

Parameter	Wirkung
`-a, --all`	Entfernt auch alle ungenutzten Images.
`--volumes`	Entfernt zusätzlich anonyme Volumes.
`-f, --force`	Fragt nicht nach.
`--filter FILTER`	Beschränkt, zum Beispiel `until=24h`.

BEISPIEL

```
$ docker system prune
WARNING! This will remove:
  - all stopped containers
  - all networks not used by at least one container
  - all dangling images
  - unused build cache
Are you sure you want to continue? [y/N] n
```

Warnhinweis

Der Befehl wirkt breit. Verwenden Sie ihn nicht in automatischen Abläufen ohne Filter, und vermeiden Sie `--volumes` auf Servern mit Datenbanken.

Sicherheitsrelevante Optionen

–user

```
docker run --user UID[:GID] IMAGE
```

ERKLÄRUNG

Führt den Hauptprozess als Benutzer mit dieser Nummer aus statt als Root. In Compose heißt die Einstellung `user:`, im Dockerfile `USER`.

PARAMETER

Parameter	Wirkung
`UID[:GID]`	Benutzer- und Gruppennummer, zum Beispiel `10001:10001`.

BEISPIEL

```
$ docker run --rm --user 10001:10001 alpine id
uid=10001 gid=10001 groups=10001
```

Warnhinweis

Dateien in eingebundenen Verzeichnissen müssen dieser Nummer gehören oder für sie freigegeben sein, sonst erscheint `Permission denied`.

–read-only und –tmpfs

```
docker run --read-only [--tmpfs PFAD[:OPTIONEN]] IMAGE
```

ERKLÄRUNG

Macht das Wurzeldateisystem des Containers schreibgeschützt. Verzeichnisse, die das Programm beschreiben muss, stellt man mit `--tmpfs` im Arbeitsspeicher bereit. In Compose heißen sie `read_only: true` und `tmpfs:`.

PARAMETER

Parameter	Wirkung
`--tmpfs PFAD`	Hängt ein Verzeichnis im Arbeitsspeicher ein, das mit dem Container verschwindet.
`size=GRÖSSE`	Begrenzt die Größe, zum Beispiel `--tmpfs /tmp:size=64m`.

BEISPIEL

```
$ docker run --rm --read-only --tmpfs /tmp alpine sh -c "touch /x;
    ↪ touch /tmp/y && echo ok"
touch: /x: Read-only file system
ok
```

Warnhinweis

Daten in einem `tmpfs` belegen Arbeitsspeicher und gehen beim Beenden verloren.

–cap-drop und –cap-add

```
docker run --cap-drop FÄHIGKEIT [--cap-add FÄHIGKEIT] IMAGE
```

ERKLÄRUNG

Entfernt Linux Capabilities oder fügt sie hinzu. `--cap-drop ALL` entfernt alle, danach gibt man gezielt zurück, was das Programm braucht. In Compose heißen sie `cap_drop:` und `cap_add:`.

PARAMETER

Parameter	Wirkung
`ALL`	Alle Capabilities.
`NET_BIND_SERVICE`	Ports unter 1024 belegen.
`CHOWN, SETUID, SETGID`	Besitzer ändern und Benutzer wechseln.

BEISPIEL

```
$ docker run --rm --cap-drop ALL alpine sh -c "grep CapEff /proc/self
    ↪ /status"
CapEff:^^I0000000000000000
```

Warnhinweis

Fügen Sie `SYS_ADMIN` und `NET_ADMIN` nur mit zwingendem Grund hinzu. Sie kommen Root-Rechten im Kernel sehr nahe.

–security-opt

```
docker run --security-opt OPTION IMAGE
```

ERKLÄRUNG

Steuert zusätzliche Schutzfunktionen. `no-new-privileges:true` verhindert, dass Prozesse nachträglich Rechte erlangen. `seccomp=` und `apparmor=` wählen Profile.

PARAMETER

Parameter	Wirkung
`no-new-privileges:true`	Kein Rechtegewinn durch Programme mit Sonderrechten.
`seccomp=DATEI`	Eigenes Seccomp-Profil.
`apparmor=PROFIL`	Anderes AppArmor-Profil.

BEISPIEL

```
$ docker run -d --name aa-test --security-opt no-new-privileges:true
    ↪ nginx:stable-alpine
$ docker inspect --format '{{.HostConfig.SecurityOpt}}' aa-test
[no-new-privileges:true]
```

Warnhinweis

`seccomp=unconfined` und `apparmor=unconfined` schalten den Schutz ab. Nur zum kurzen Test verwenden.

–memory, –cpus und –pids-limit

```
docker run --memory GRÖSSE --cpus ZAHL --pids-limit ZAHL IMAGE
```

ERKLÄRUNG

Begrenzt Arbeitsspeicher, Prozessorleistung und Prozesszahl eines Containers über cgroups. Ein Container, der sein Speicherlimit überschreitet, wird vom Kernel beendet (Rückgabewert 137). In Compose heißen sie `mem_limit:`, `cpus:` und `pids_limit:`.

PARAMETER

Parameter	Wirkung
`--memory, -m GRÖSSE`	Arbeitsspeicher, zum Beispiel `128m`.
`--memory-swap GRÖSSE`	Obergrenze mit Auslagerung.
`--cpus ZAHL`	Anteil an Prozessorkernen, `0.5` ist ein halber.
`--pids-limit ZAHL`	Höchstzahl der Prozesse.

BEISPIEL

```
$ docker run -d --name klein --memory 64m --cpus 0.25 --pids-limit
    ↪ 100 nginx:stable-alpine
$ docker inspect --format '{{.HostConfig.Memory}}' klein
67108864
```

-p 127.0.0.1:HOST:CONTAINER

```
docker run -p [ADRESSE:]HOST-PORT:CONTAINER-PORT IMAGE
```

ERKLÄRUNG

Veröffentlicht einen Port nur auf einer bestimmten Adresse des Hosts. Mit `127.0.0.1` ist der Dienst nur vom Server selbst erreichbar.

PARAMETER

Parameter	Wirkung
`127.0.0.1:8080:80`	Nur lokal erreichbar.
`8080:80`	Auf allen Adressen erreichbar, auch aus dem Netz.

BEISPIEL

```
$ docker run -d --name intern -p 127.0.0.1:8085:80 nginx:stable-
    ↪ alpine
$ docker port intern
80/tcp -> 127.0.0.1:8085
```

Warnhinweis

Ein veröffentlichter Port auf allen Adressen umgeht `ufw`. Schützen Sie ihn mit Regeln in `DOCKER-USER`.

–network none und –internal

```
docker run --network none IMAGE | docker network create --internal NETZ
```

ERKLÄRUNG

`--network none` gibt dem Container nur die Loopback-Schnittstelle. Ein `--internal`-Netz erlaubt die Verbindung der Container untereinander, aber keine nach außen.

PARAMETER

Parameter	Wirkung
`--network none`	Kein Netzwerk.
`--internal`	Netzwerk ohne Zugang zur Außenwelt (bei `docker network create`).

BEISPIEL

```
$ docker run --rm --network none alpine ip -br address
lo               UNKNOWN        127.0.0.1/8
```

–privileged

```
docker run --privileged IMAGE
```

ERKLÄRUNG

Gibt dem Container alle Capabilities und Zugriff auf alle Geräte des Hosts und schaltet Seccomp und AppArmor ab. In Compose heißt die Einstellung `privileged: true`.

PARAMETER

Parameter	Wirkung
`--privileged`	Hebt fast alle Schutzmaßnahmen auf.

BEISPIEL

```
$ docker inspect --format '{{.HostConfig.Privileged}}' web
false
```

Warnhinweis

Ein privilegierter Container ist praktisch ein Prozess mit Root-Rechten auf dem Host. Verwenden Sie die Option nicht. Das Beispiel zeigt nur, wie man ihre Abwesenheit prüft.

Windows- und PowerShell-Befehle

wsl –install

```
wsl --install [-d DISTRIBUTION] [--no-distribution]
```

ERKLÄRUNG

Schaltet die Windows-Funktionen für WSL ein und installiert standardmäßig die Distribution Ubuntu. Der Befehl braucht eine PowerShell mit Administratorrechten, danach ist meist ein Neustart nötig.

PARAMETER

Parameter	Wirkung
`-d, --distribution NAME`	Installiert eine bestimmte Distribution, die Liste liefert `wsl --list --online`.
`--no-distribution`	Installiert WSL ohne Distribution.
`--web-download`	Lädt die Distribution aus dem Netz, wenn der Store-Weg hängt.

BEISPIEL

```
PS> wsl --install -d Debian
```

Warnhinweis

Die Anleitung gilt für Windows 10 ab Version 2004 und für Windows 11. Läuft WSL schon, zeigt der Befehl nur die Hilfe an. Dann hilft `wsl --install -d NAME`.

wsl –list –verbose

```
wsl --list --verbose
```

ERKLÄRUNG

Zeigt die installierten Distributionen mit Zustand und WSL-Version. Docker Desktop erscheint hier mit einer eigenen Distribution.

PARAMETER

Parameter	Wirkung
`-v, --verbose`	Zeigt Zustand und Version.
`--online`	Listet die Distributionen, die sich installieren lassen.
`--running`	Zeigt nur laufende Distributionen.

BEISPIEL

```
PS> wsl --list --verbose
  NAME              STATE           VERSION
* Ubuntu            Running         2
  docker-desktop    Running         2
```

wsl –version und wsl –update

`wsl --version | wsl --update`

ERKLÄRUNG

`--version` zeigt die Version von WSL und des Linux-Kernels, `--update` aktualisiert beides. Erscheinen bei `--version` keine Angaben, läuft die ältere, im Betriebssystem eingebaute Fassung.

PARAMETER

Parameter	Wirkung
`--version`	Zeigt die Versionen.
`--update`	Aktualisiert WSL und den Kernel.
`--pre-release`	Installiert die Vorabversion (nur zum Testen).

BEISPIEL

```
PS> wsl --update
PS> wsl --version
WSL-Version: 2.x.y.z
Kernelversion: 6.6.x.y-1
```

Warnhinweis

Docker Desktop verlangt WSL ab Version 2.1.5, für Enhanced Container Isolation ab 2.6.

wsl –shutdown

```
wsl --shutdown
```

ERKLÄRUNG

Beendet die virtuelle Maschine von WSL 2 mit allen Distributionen. Nötig, damit Änderungen an der `.wslconfig` wirken.

PARAMETER

Parameter	Wirkung
`--shutdown`	Beendet alle Distributionen und die VM.
`-t, --terminate NAME`	Beendet nur eine Distribution.

BEISPIEL

```
PS> wsl --shutdown
```

> **Warnhinweis**
>
> Laufende Container und alle offenen Sitzungen in WSL enden sofort. Docker Desktop startet die Engine danach neu.

wsl –set-version

```
wsl --set-version DISTRIBUTION 1|2
```

ERKLÄRUNG

Stellt eine installierte Distribution auf WSL 1 oder WSL 2 um. Für Docker muss sie auf 2 stehen.

PARAMETER

Parameter	Wirkung
`DISTRIBUTION`	Name aus `wsl --list --verbose`.
`2`	Zielversion WSL 2.

BEISPIEL

```
PS> wsl --set-version Ubuntu 2
```

wsl –unregister

```
wsl --unregister DISTRIBUTION
```

ERKLÄRUNG

Entfernt eine Distribution samt ihrer virtuellen Festplatte.

PARAMETER

Parameter	Wirkung
`DISTRIBUTION`	Name aus `wsl --list --verbose`.

BEISPIEL

```
PS> wsl --export Ubuntu C:\Sicherung\ubuntu.tar
PS> wsl --unregister Ubuntu
```

Warnhinweis

Der Befehl löscht sofort und ohne Rückfrage alle Daten der Distribution. Sichern Sie vorher mit `wsl --export`.

DockerCli.exe -SwitchDaemon

```
& $Env:ProgramFiles\Docker\Docker\DockerCli.exe -SwitchDaemon
```

ERKLÄRUNG

Schaltet Docker Desktop zwischen dem Linux- und dem Windows-Daemon um. Dasselbe leistet der Eintrag im Menü des Docker-Symbols. Windows-Container gibt es nur bei der Installation für alle Benutzer und unter Windows Pro oder Enterprise.

PARAMETER

Parameter	Wirkung
`-SwitchDaemon`	Wechselt zwischen Linux- und Windows-Containern.

BEISPIEL

```
PS> & $Env:ProgramFiles\Docker\Docker\DockerCli.exe -SwitchDaemon
PS> docker info --format '{{.OSType}}'
windows
```

Warnhinweis

Nach dem Umschalten sehen Sie die Container und Images des anderen Daemons nicht mehr.

docker run –isolation

```
docker run --isolation=process|hyperv IMAGE
```

ERKLÄRUNG

Wählt bei Windows-Containern den Isolationsmodus. Bei `process` teilt sich der Container den Kernel mit dem Host, bei `hyperv` läuft er in einer eigenen kleinen virtuellen Maschine. Auf Windows 10 und 11 gilt standardmäßig `hyperv`, auf Windows Server `process`.

PARAMETER

Parameter	Wirkung
`--isolation=process`	Prozess-Isolation, Image und Host müssen zusammenpassen.
`--isolation=hyperv`	Hyper-V-Isolation, unabhängig von der Version des Hosts.

BEISPIEL

```
PS> docker run --rm --isolation=hyperv mcr.microsoft.com/windows/
    ↪ nanoserver:ltsc2022 cmd /c ver
Microsoft Windows [Version 10.0.20348.xxxx]
```

Warnhinweis

Passt die Version nicht, endet der Start mit der Fehlernummer `0xc0370101`. Dann hilft `hyperv`, wenn der Host neuer ist als das Image.

Get-ComputerInfo

```
Get-ComputerInfo [-Property NAME[,NAME ...]]
```

ERKLÄRUNG

Liefert Angaben zu Windows und Hardware. Mit `-Property` wählt man einzelne Felder, damit die Ausgabe kurz bleibt.

PARAMETER

Parameter	Wirkung
`-Property NAME`	Gewünschte Felder, zum Beispiel `OsName`, `OsVersion`, `OsBuildNumber`.

BEISPIEL

```
PS> Get-ComputerInfo -Property OsName, OsVersion, OsBuildNumber
OsName          : Microsoft Windows 11 Pro
OsVersion       : 10.0.26100
OsBuildNumber   : 26100
```

Get-WindowsOptionalFeature und Enable-WindowsOptionalFeature

```
Get-WindowsOptionalFeature -Online -FeatureName NAME |
Enable-WindowsOptionalFeature -Online -FeatureName NAME -All
```

ERKLÄRUNG

Zeigt den Zustand optionaler Windows-Funktionen und schaltet sie ein. Für Windows-Container sind `Containers` und `Microsoft-Hyper-V-All` wichtig. Beide Befehle brauchen Administratorrechte.

PARAMETER

Parameter	Wirkung
`-Online`	Wirkt auf das laufende System.
`-FeatureName NAME`	Name der Funktion.
`-All`	Schaltet abhängige Funktionen mit ein.
`-NoRestart`	Startet den Rechner nicht selbst neu.

BEISPIEL

```
PS> Get-WindowsOptionalFeature -Online -FeatureName Containers |
    ↪ Select-Object FeatureName, State
FeatureName State
----------- -----
Containers Enabled
```

Warnhinweis

Das Einschalten von Hyper-V verändert den Rechner erheblich und verlangt einen Neustart. Auf Windows Server schaltet `Install-WindowsFeature` die Rollen ein.

Get-Service und Restart-Service

```
Get-Service docker | Start-Service docker | Stop-Service docker |
Restart-Service docker
```

ERKLÄRUNG

Verwaltet den Docker-Dienst unter Windows Server. Der Dienst heißt `docker`. Änderungen an der `daemon.json` wirken erst nach einem Neustart des Dienstes.

PARAMETER

Parameter	Wirkung
`NAME`	Dienstname, hier `docker`.

BEISPIEL

```
PS> Get-Service docker
Status   Name               DisplayName
------   ----               -----------
Running  docker             docker
```

> **Warnhinweis**
> Ein Neustart des Dienstes beendet Container ohne Neustartregel.

Get-EventLog

```
Get-EventLog -LogName Application -Source docker -Newest ANZAHL
```

ERKLÄRUNG

Liest Meldungen des Docker-Dienstes aus dem Ereignisprotokoll von Windows. Das entspricht dem `journalctl -u docker` unter Linux. In PowerShell 7 heißt der Befehl `Get-WinEvent`.

PARAMETER

Parameter	Wirkung
`-LogName NAME`	Protokoll, hier `Application`.
`-Source NAME`	Quelle der Meldungen, hier `docker`.
`-Newest ANZAHL`	Zeigt die neuesten Einträge.

BEISPIEL

```
PS> Get-EventLog -LogName Application -Source docker -Newest 5
```

Get-NetTCPConnection

```
Get-NetTCPConnection -LocalPort PORT
```

ERKLÄRUNG

Zeigt, wer einen TCP-Port belegt. Mit `Get-Process -Id` findet man das zugehörige Programm. Das Gegenstück unter Linux ist `ss -tlnp`.

PARAMETER

Parameter	Wirkung
`-LocalPort PORT`	Lokaler Port.
`-State ZUSTAND`	Filtert nach Zustand, zum Beispiel `Listen`.

BEISPIEL

```
PS> Get-NetTCPConnection -LocalPort 8080 | Select-Object LocalAddress,
    ↪ State, OwningProcess
PS> Get-Process -Id 4120
```

Get-FileHash

`Get-FileHash DATEI [-Algorithm ALGORITHMUS]`

ERKLÄRUNG

Berechnet die Prüfsumme einer Datei. Damit vergleicht man heruntergeladene Installationsdateien mit der Angabe des Herstellers.

PARAMETER

Parameter	Wirkung
`-Algorithm NAME`	Zum Beispiel `SHA256` (Standard), `SHA512`.
`-Path DATEI`	Datei, deren Prüfsumme berechnet wird.

BEISPIEL

```
PS> Get-FileHash "$env:USERPROFILE\Downloads\Docker Desktop Installer.
    ↪ exe" -Algorithm SHA256
```

> **Warnhinweis**
>
> Stimmt die Prüfsumme nicht mit der Herstellerangabe überein, die Datei nicht ausführen.

Invoke-WebRequest und curl.exe

```
Invoke-WebRequest -Uri URL [-OutFile DATEI] [-UseBasicParsing] | curl.exe
[OPTIONEN] URL
```

ERKLÄRUNG

Lädt eine Webseite oder Datei. In Windows PowerShell 5.1 ist `curl` ein Alias für `Invoke-WebRequest`. Das echte Programm ruft man mit `curl.exe` auf.

PARAMETER

Parameter	Wirkung
`-Uri URL`	Adresse.
`-OutFile DATEI`	Speichert die Antwort in einer Datei.
`-UseBasicParsing`	Verzichtet auf den Internet Explorer (in 5.1 empfohlen).
`curl.exe -s, -I`	Stiller Modus und nur Kopfzeilen, wie unter Linux.

BEISPIEL

```
PS> (Invoke-WebRequest -Uri http://localhost:8080 -UseBasicParsing).
    ↪ StatusCode
200
PS> curl.exe -sI http://localhost:8080 | Select-String "HTTP"
HTTP/1.1 200 OK
```

Select-String

```
Select-String -Pattern MUSTER [-Path DATEI] | BEFEHL | Select-String MUSTER
```

ERKLÄRUNG

Filtert Text nach einem Muster. Es ist das PowerShell-Gegenstück zu `grep`.

PARAMETER

Parameter	Wirkung
`-Pattern MUSTER`	Suchmuster (regulärer Ausdruck).
`-SimpleMatch`	Sucht den Text wörtlich.
`-CaseSensitive`	Beachtet Groß- und Kleinschreibung.

BEISPIEL

```
PS> docker compose logs | Select-String "GET"
web-1  | 172.18.0.1 - - [11/Nov/2026:12:41:07 +0000] "GET / HTTP/1.1"
     ↪  200 25
```

ConvertFrom-Json

```
BEFEHL | ConvertFrom-Json
```

ERKLÄRUNG

Wandelt JSON, zum Beispiel die Ausgabe von `docker inspect`, in PowerShell-Objekte um. Einzelne Felder lassen sich danach direkt ansprechen.

PARAMETER

Parameter	Wirkung
`-Depth ZAHL`	Tiefe der Umwandlung bei verschachtelten Daten.

BEISPIEL

```
PS> $info = docker inspect web | ConvertFrom-Json
PS> $info.State.Status
running
```

Start-Process

```
Start-Process -FilePath DATEI -ArgumentList ARGUMENT[,ARGUMENT ...] -Wait
```

ERKLÄRUNG

Startet ein Programm mit Argumenten und wartet auf sein Ende. So installiert man Docker Desktop aus der PowerShell.

PARAMETER

Parameter	Wirkung
`-ArgumentList LISTE`	Argumente des Programms, durch Kommas getrennt.
`-Wait`	Wartet, bis das Programm endet.
`-Verb RunAs`	Startet mit Administratorrechten.

BEISPIEL

```
PS> Start-Process -FilePath $installer -Wait -ArgumentList 'install',
    ↪ '--user'
```

> **Warnhinweis**
>
> Ohne `--user` installiert Docker Desktop für alle Benutzer und verlangt Administratorrechte.

Set-ExecutionPolicy

```
Set-ExecutionPolicy -Scope Process -ExecutionPolicy RemoteSigned
```

ERKLÄRUNG

Legt fest, welche PowerShell-Skripte ausgeführt werden dürfen. Mit `-Scope Process` gilt die Änderung nur für das aktuelle Fenster.

PARAMETER

Parameter	Wirkung
`-Scope Process`	Nur für dieses Fenster.
`-ExecutionPolicy NAME`	Zum Beispiel `RemoteSigned` oder `Restricted`.

BEISPIEL

```
PS> Get-ExecutionPolicy -List
PS> Set-ExecutionPolicy -Scope Process -ExecutionPolicy RemoteSigned
```

Warnhinweis

Lockern Sie die Richtlinie nie dauerhaft für den ganzen Rechner, um ein einzelnes Skript zu starten. Lesen Sie jedes Skript, bevor Sie es ausführen.

net localgroup docker-users

```
net localgroup docker-users [BENUTZER /add | BENUTZER /delete]
```

ERKLÄRUNG

Zeigt die Mitglieder der Gruppe `docker-users` oder verändert sie. Die Gruppe gibt Zugriff auf den Docker-Daemon von Docker Desktop (Installation für alle Benutzer).

PARAMETER

Parameter	Wirkung
`BENUTZER /add`	Fügt einen Benutzer hinzu.
`BENUTZER /delete`	Entfernt einen Benutzer.

BEISPIEL

```
PS> net localgroup docker-users
PS> net localgroup docker-users MeinBenutzer /add
```

Warnhinweis

Die Mitgliedschaft entspricht Administratorrechten auf dem Host. Fügen Sie nur Personen hinzu, die sie brauchen.

Start-BitsTransfer

```
Start-BitsTransfer -Source URL -Destination ZIEL
```

ERKLÄRUNG

Lädt eine Datei im Hintergrund über den Windows-Übertragungsdienst. Die Docker-Dokumentation verwendet den Befehl, um `docker-compose.exe` auf Windows Server zu installieren.

PARAMETER

Parameter	Wirkung
`-Source URL`	Quelladresse.
`-Destination PFAD`	Zieldatei.

BEISPIEL

```
PS> [Net.ServicePointManager]::SecurityProtocol = [Net.
    ↪ SecurityProtocolType]::Tls12
PS> Start-BitsTransfer -Source "https://github.com/docker/compose/
    ↪ releases/download/v5.4.0/docker-compose-windows-x86_64.exe" -
    ↪ Destination $Env:ProgramFiles\Docker\docker-compose.exe
```

Warnhinweis

Laden Sie Programme nur aus der offiziellen Quelle. Die Versionsnummer in der Adresse ist ein Beispiel, verwenden Sie die aktuelle.

install-docker-ce.ps1

```
.\install-docker-ce.ps1 [-DockerVersion VERSION] [-HyperV] [-NoRestart]
```

ERKLÄRUNG

Skript von Microsoft, das auf Windows Server die Funktion Containers einschaltet und die Docker Community Edition als Dienst einrichtet. Es installiert `docker.exe` und `dockerd.exe`, aber kein Compose.

PARAMETER

Parameter	Wirkung
`-DockerVersion VERSION`	Bestimmte Docker-Version statt der neuesten.
`-HyperV`	Bereitet den Rechner für Hyper-V-Isolation vor.
`-NoRestart`	Beendet das Skript, wenn ein Neustart nötig ist.
`-NATSubnet CIDR`	Anderer Adressbereich für das NAT-Netz.
`-ContainerBaseImage URL`	Lädt ein Basisimage vor.

BEISPIEL

```
PS> Invoke-WebRequest -UseBasicParsing "https://raw.githubusercontent.
    ↪ com/microsoft/Windows-Containers/Main/helpful_tools/Install-
    ↪ DockerCE/install-docker-ce.ps1" -OutFile install-docker-ce.
    ↪ ps1
PS> .\install-docker-ce.ps1
```

Warnhinweis

Lesen Sie das Skript vor der Ausführung. Es läuft mit Administratorrechten und startet den Rechner gegebenenfalls neu.

docker-compose.exe

```
docker-compose.exe [OPTIONEN] BEFEHL
```

ERKLÄRUNG

Eigenständige Fassung von Compose, die man auf Windows Server installiert. Die Befehle entsprechen `docker compose`, mit Bindestrich geschrieben. Die Docker-Dokumentation nennt diese Variante eine Altlast und empfiehlt sie nur, wenn weder Docker Desktop noch das Plugin infrage kommen.

PARAMETER

Parameter	Wirkung
`up -d`	Startet das Projekt im Hintergrund.
`ps`	Listet die Container des Projekts.
`down`	Stoppt das Projekt und entfernt Container und Netze.
`version`	Zeigt die Version.

BEISPIEL

```
PS> docker-compose.exe version
Docker Compose version v5.4.0
PS> docker-compose.exe up -d
```

Prüfungsfragen

Dieser Fragenkatalog fasst den Stoff aller acht Teile des Buches zusammen. Jede Frage hat vier Antworten, von denen genau eine richtig ist. Die Fragen sind nach Teilen geordnet und eignen sich zur Wiederholung vor Prüfungen. Die Lösungen mit kurzen Begründungen stehen am Ende des Abschnitts. Decken Sie sie beim Üben zu und begründen Sie Ihre Wahl, bevor Sie nachsehen.

Teil I: Virtualisierung, Container und Docker

1. Welche Aufgabe hat ein Hypervisor?

(a) Er verschlüsselt den Datenverkehr zwischen Containern.
(b) Er ersetzt den Kernel des Hosts durch einen kleineren.
(c) Er baut Container-Images aus Dockerfiles.
(d) Er teilt die Hardware auf mehrere virtuelle Maschinen auf und verwaltet sie.

2. Was unterscheidet einen Container grundsätzlich von einer virtuellen Maschine?

(a) Alle Container teilen sich den Kernel des Hosts.
(b) Container brauchen keinen Arbeitsspeicher.
(c) Container laufen nur auf Linux-Servern mit Hypervisor.
(d) Jeder Container bringt einen eigenen Kernel mit.

3. Welche Kernel-Funktion sorgt dafür, dass ein Container nur seine eigenen Prozesse und Netzwerkschnittstellen sieht?

(a) Namespaces
(b) Cgroups
(c) Seccomp
(d) OverlayFS

4. Wofür werden cgroups bei Containern eingesetzt?

(a) Zum Signieren von Images.
(b) Zum Auflösen von Containernamen im Netzwerk.
(c) Zum Begrenzen und Messen von Ressourcen wie Speicher und Prozessorzeit.
(d) Zum Verschlüsseln der Image-Schichten.

5. Was ist ein Docker-Image?

(a) Eine virtuelle Festplatte für eine VM.
(b) Eine Konfigurationsdatei des Daemons.
(c) Ein laufender Container mit allen Daten.
(d) Eine schreibgeschützte, aus Schichten aufgebaute Vorlage für Container.

6. Welche Komponente nimmt die Befehle des Docker-Clients entgegen?

(a) Die Registry
(b) Der Befehl `docker selbst`
(c) Der Daemon `dockerd`
(d) Die Datei `compose.yaml`

Teil II: Installation und erste Schritte

7. Welche Folge hat die Mitgliedschaft in der Gruppe `docker unter` Linux?

(a) Sie hat keine Auswirkung auf die Sicherheit.
(b) Sie erlaubt nur das Lesen von Container-Protokollen.
(c) Sie ist praktisch gleichbedeutend mit Root-Rechten auf dem Host.
(d) Sie gibt Zugriff auf die Registry.

8. In welchem Format ist das Docker-Repository unter Debian 13 in diesem Buch eingerichtet?

(a) Als deb822-Datei mit der Endung .sources
(b) Als Snap-Paket
(c) Als einzelne Zeile in `/etc/fstab`
(d) Als Eintrag in der `daemon.json`

9. Mit welchem Befehl prüft man eine frische Installation am einfachsten?

(a) `docker build hello-world`
(b) `docker run --rm hello-world`
(c) `docker stop hello-world`
(d) `docker login hello-world`

10. Was bewirkt die Option -d bei `docker run`?

(a) Der Container läuft im Debug-Modus.
(b) Der Container wird nach dem Ende gelöscht.
(c) Der Container läuft im Hintergrund.
(d) Der Container bekommt ein Terminal.

11. Was zeigt `docker ps -a` zusätzlich zu `docker ps`?

(a) Auch Netzwerke
(b) Auch beendete Container
(c) Auch ungenutzte Images
(d) Auch Volumes

12. Welche Option löscht einen Container automatisch, wenn er endet?

(a) `-x`
(b) `--clean`
(c) `--rm`
(d) `--auto`

13. Warum ist ein veröffentlichter Port trotz aktiver ufw von außen erreichbar?

(a) Docker schreibt eigene Weiterleitungsregeln, die vor den Regeln von ufw greifen.
(b) Veröffentlichte Ports sind immer verschlüsselt.
(c) ufw erlaubt Docker grundsätzlich alles.
(d) Die Firewall gilt nur für IPv6.

Teil III: Images, Dockerfile und Containerverwaltung

14. Welche Anweisung im Dockerfile legt das Basisimage fest?

(a) CMD
(b) COPY
(c) RUN
(d) FROM

15. Welche Anweisung kopiert Dateien aus dem Build-Kontext in das Image?

(a) HEALTHCHECK
(b) EXPOSE
(c) FROM
(d) COPY

16. Wozu dient ein mehrstufiger Build (Multi-Stage-Build)?

(a) Das fertige Image bleibt klein, weil Build-Werkzeuge nicht enthalten sind.
(b) Das Image wird verschlüsselt.
(c) Der Build läuft auf mehreren Servern.
(d) Das Image startet schneller als jeder Container.

17. Wofür steht eine Datei .dockerignore?

(a) Sie schließt Dateien vom Build-Kontext aus.
(b) Sie verbietet das Starten von Containern.
(c) Sie löscht ungenutzte Images.
(d) Sie legt Neustartregeln fest.

18. Ein Container endet mit dem Rückgabewert 137. Was ist die wahrscheinlichste Ursache?

(a) Der Befehl im Container wurde nicht gefunden.
(b) Das Image ist für eine andere Architektur gebaut.
(c) Der Container wurde sauber beendet.
(d) Der Prozess wurde mit SIGKILL beendet, oft wegen Speichermangels.

19. Was bewirkt `--memory` 128m?

(a) Der Container läuft nur mit 128 Prozessen.
(b) Der Container wird nach 128 Minuten beendet.
(c) Der Container darf höchstens 128 MB Arbeitsspeicher verwenden.
(d) Der Container bekommt eine 128 MB große Festplatte.

20. Was bedeutet die Neustartregel unless-stopped?

(a) Der Container startet nur nach einem Fehler neu.
(b) Der Container startet immer neu, außer er wurde von Hand gestoppt.
(c) Der Container startet nie neu.
(d) Der Container startet nach jedem Befehl neu.

Teil IV: Speicher und Netzwerk

21. Womit überleben Daten das Löschen eines Containers?

(a) Mit der beschreibbaren Schicht
(b) Mit einem tmpfs
(c) Mit einem Volume
(d) Mit dem Tag latest

22. Wodurch unterscheidet sich ein Bind Mount von einem Volume?

(a) Ein Bind Mount bindet einen Pfad des Hosts direkt ein.
(b) Ein Bind Mount liegt immer im Arbeitsspeicher.
(c) Ein Bind Mount wird von Docker automatisch gesichert.
(d) Ein Bind Mount ist nur für Datenbanken erlaubt.

23. Welche Besonderheit haben benutzerdefinierte Bridge-Netzwerke gegenüber dem Standardnetz?

(a) Container lösen sich dort gegenseitig über ihren Namen auf.
(b) Sie sind immer verschlüsselt.
(c) Sie funktionieren nur unter Windows.
(d) Sie erlauben nur einen Container.

24. Was bewirkt `docker network create --internal`?

(a) Das Netzwerk hat keinen Zugang nach außen.
(b) Das Netzwerk ist nur für Windows-Container.
(c) Das Netzwerk wird nach einer Stunde gelöscht.
(d) Das Netzwerk verschlüsselt alle Pakete.

25. Was bewirkt -p 127.0.0.1:8080:80?

(a) Der Container bekommt die Adresse 127.0.0.1.
(b) Der Dienst ist nur vom Server selbst erreichbar.
(c) Der Dienst ist aus dem ganzen Internet erreichbar.
(d) Der Port 80 des Hosts wird gesperrt.

26. Wozu dient die Kette DOCKER-USER?

(a) Für das Speichern von Passwörtern.
(b) Für eigene Firewall-Regeln, die vor den Regeln von Docker geprüft werden.
(c) Für die Benutzerverwaltung von Docker.
(d) Für die Anmeldung an der Registry.

27. Warum gehört in DOCKER-USER vor eine Regel mit DROP eine Regel für ESTABLISHED,RELATED?

(a) Sonst funktioniert die DNS-Auflösung nicht.
(b) Sonst startet der Daemon nicht.
(c) Sonst wird der Port 80 gesperrt.
(d) Sonst werden auch die Antworten auf Verbindungen der Container nach außen verworfen.

Teil V: Compose und Datenbanken

28. Wie heißt die empfohlene Datei für ein Compose-Projekt?

(a) Composefile.txt
(b) `compose.yaml`
(c) `docker`.config
(d) dockerfile.yml

29. Welchen Status hat das Feld version in einer Compose-Datei?

(a) Es ist veraltet und wird nicht mehr gebraucht.
(b) Es ist Pflicht.
(c) Es legt den Projektnamen fest.
(d) Es bestimmt die Docker-Version.

30. Was bewirkt depends_on mit condition: service_healthy?

(a) Der Dienst startet erst, wenn der andere Dienst gesund gemeldet ist.
(b) Der Dienst wird nach einem Fehler neu gestartet.
(c) Der Dienst teilt sich das Netzwerk des anderen.
(d) Der Dienst läuft nur auf demselben Host.

31. Was löscht `docker compose down -v` zusätzlich?

(a) Alle Images des Servers
(b) Die im Projekt deklarierten Volumes
(c) Die Compose-Datei
(d) Die Docker-Konfiguration

32. Wann wirken die Variablen `MARIADB_DATABASE` und `MARIADB_USER`?

(a) Nur bei leerem Datenverzeichnis, also beim ersten Start
(b) Nur mit `docker exec`
(c) Bei jedem Start
(d) Nur bei Neustart des Servers

33. Wie sichert man eine laufende MariaDB-Datenbank konsistent?

(a) Durch Löschen des Volumes
(b) Durch `docker pause`
(c) Durch Kopieren der Container-Schicht
(d) Mit `mariadb-dump und der` Option `--single-transaction`

34. Wo erwartet das Image PostgreSQL 18 das Volume für die Daten?

(a) Unter `/etc/postgresql`
(b) Unter `/data`
(c) Unter `/var/lib/postgresql`
(d) Unter `/var/lib/postgresql/data`

Teil VI: Webserver, Reverse Proxy und Portainer

35. Welche Aufgabe hat ein Reverse Proxy?

(a) Er baut Images.
(b) Er ersetzt die Registry.
(c) Er nimmt Anfragen entgegen und leitet sie an interne Dienste weiter.
(d) Er verschlüsselt den Docker-Socket.

36. Wozu dient in der nginx-Konfiguration die Zeile resolver 127.0.0.11?

(a) Zum Aktivieren von TLS
(b) Zur Namensauflösung über den DNS-Dienst von Docker bei jeder Anfrage
(c) Zum Sperren von Adressen
(d) Zum Begrenzen der Anfragen

37. Was bestätigt ein TLS-Zertifikat?

(a) Dass ein öffentlicher Schlüssel zu einem Namen gehört
(b) Dass der Container als Nicht-Root läuft
(c) Dass keine Schwachstellen bekannt sind
(d) Dass der Server fehlerfrei läuft

38. Warum warnt der Browser bei einem selbstsignierten Zertifikat?

(a) Es enthält keinen Namen.
(b) Es wurde nicht von einer vertrauten Stelle signiert.
(c) Es funktioniert nur mit HTTP.
(d) Es ist zu kurz.

39. Wie wird Portainer im Buch abgesichert?

(a) Ohne Passwort betrieben
(b) Port 9443 für alle geöffnet
(c) Mit privilegiertem Container
(d) Port nur an 127.0.0.1 gebunden, Zugriff über einen SSH-Tunnel

40. Was bewirkt :ro bei der Einbindung des Docker-Sockets?

(a) Es macht den Socket sicher.
(b) Es verschlüsselt den Socket.
(c) Es verhindert das Starten von Containern.
(d) Es schützt nicht, die Docker-Schnittstelle bleibt voll nutzbar.

41. Was bewirkt server_tokens off in nginx?

(a) Der Server blockiert alle Besucher.
(b) TLS wird abgeschaltet.
(c) Die Versionsnummer wird nicht mehr angezeigt.
(d) Der Server beendet sich nach einer Anfrage.

Teil VII: Sicherheit, Betrieb und Fehlerdiagnose

42. Was bewirkt `--cap-drop` ALL?

(a) Alle Linux Capabilities werden entfernt.
(b) Alle Container werden gelöscht.
(c) Der Container läuft als Root.
(d) Der Container verliert sein Netzwerk.

43. Was verhindert die Option no-new-privileges?

(a) Dass Images geladen werden
(b) Dass der Port veröffentlicht wird
(c) Dass ein Prozess nachträglich zusätzliche Rechte erlangt
(d) Dass der Container startet

44. Warum ist ein im Container eingebundener Docker-Socket gefährlich?

(a) Er verlangsamt das Netzwerk.
(b) Er ist gleichbedeutend mit Root-Zugriff auf den Host.
(c) Er löscht die Logs.
(d) Er verhindert Updates.

45. Was ändert der Rootless-Modus?

(a) Container brauchen keine Images mehr.
(b) Daemon und Container laufen als normaler Benutzer ohne Root-Rechte.
(c) Alle Ports sind erlaubt.
(d) Der Daemon läuft ohne Netzwerk.

46. Wo liegt ein Compose-Secret im Container?

(a) In der `compose.yaml` des Containers
(b) Als Umgebungsvariable
(c) Als Datei unter `/run/secrets/NAME`
(d) Im Image

47. Was besagt die 3-2-1-Regel?

(a) Drei Backups pro Woche, zwei pro Tag, eines pro Stunde
(b) Drei Passwörter, zwei Schlüssel, ein Zertifikat
(c) Drei Kopien, zwei Arten von Datenträgern, eine davon an einem anderen Ort
(d) Drei Server, zwei Netze, ein Router

48. Was beschreibt der RPO?

(a) Die Zahl der Backups
(b) Den größten tolerierten Datenverlust, gemessen in Zeit
(c) Die längste tolerierte Ausfallzeit
(d) Die Größe eines Images

49. Welche Lehre zieht das Buch aus dem Vorfall mit Trivy im März 2026?

(a) Docker Hub darf nicht benutzt werden.
(b) Tags sind veränderlich, wichtige Images legt man mit Digest fest, und der Docker-Socket gehört nicht in Werkzeug-Container.
(c) Images dürfen nie aktualisiert werden.
(d) Scanner sind grundsätzlich unsicher.

50. Mit welchem Befehl prüft man eine `daemon.json`, ohne den Daemon zu starten?

(a) `docker daemon test`
(b) systemctl verify `docker`
(c) `docker config check`
(d) `dockerd --validate --config-file=/etc/docker/daemon.json`

51. Was braucht ein Container mit `--read-only` für Schreibzugriffe?

(a) Zugriff auf den Docker-Socket
(b) Das Tag latest
(c) Root-Rechte
(d) Ein tmpfs oder ein Volume für die betroffenen Verzeichnisse

Teil VIII: Docker unter Windows

52. Welche Aussage über Docker Desktop und Windows Server stimmt?

(a) Docker Desktop läuft nur auf Windows Server.
(b) Docker Desktop ist die empfohlene Lösung für Windows Server.
(c) Docker Desktop ersetzt Windows Server.
(d) Docker Desktop wird auf Windows Server nicht unterstützt, dort verwendet man eine Container-Runtime.

53. Was bedeutet die Fehlernummer `0xc0370101`?

(a) Das Betriebssystem des Containers passt nicht zu dem des Hosts.
(b) Der Port ist belegt.
(c) Das Passwort ist falsch.
(d) Die Registry ist nicht erreichbar.

54. Welche Isolationsart teilt sich den Kernel mit dem Host?

(a) Beide
(b) Die Prozess-Isolation
(c) Die Hyper-V-Isolation
(d) Keine von beiden

55. Wozu dient die Datei .wslconfig?

(a) Sie speichert Docker-Passwörter.
(b) Sie begrenzt Arbeitsspeicher, Prozessoren und Auslagerung der WSL-2-Umgebung.
(c) Sie installiert Linux-Distributionen.
(d) Sie legt Netzwerkregeln fest.

56. Was passiert, wenn eine nginx-Konfigurationsdatei mit BOM gespeichert wurde?

(a) nginx wechselt auf HTTPS.
(b) nginx meldet eine unbekannte Anweisung und startet nicht.
(c) nginx ignoriert die Datei still.
(d) nginx läuft schneller.

57. Welcher Netzwerkmodus fehlt bei Windows-Containern?

(a) host
(b) nat
(c) transparent
(d) overlay

58. Warum ist die Mitgliedschaft in `docker`-users sicherheitsrelevant?

(a) Sie erlaubt nur Lesezugriff.
(b) Sie entspricht Administratorrechten auf dem Host.
(c) Sie schaltet die Firewall ab.
(d) Sie hat keine Wirkung.

59. Wo gehören Projektdateien für Linux-Container unter WSL 2 am besten hin?

(a) In den Windows-Papierkorb
(b) In den Ordner Dokumente
(c) Auf einen USB-Stick
(d) In das Linux-Dateisystem der WSL-Distribution

Lösungen und Begründungen

1. (d) Ein Hypervisor stellt jeder virtuellen Maschine eigene virtuelle Hardware bereit und verteilt die echte Hardware darauf.

2. (a) Container sind isolierte Prozesse auf dem Kernel des Hosts, virtuelle Maschinen bringen einen eigenen Kernel mit.

3. (a) Namespaces trennen die Sicht auf Prozesse, Netzwerk, Dateisysteme und mehr.

4. (c) Control Groups begrenzen und messen die Ressourcen einer Prozessgruppe.

5. (d) Aus einem Image werden Container erzeugt, es selbst wird dabei nicht verändert.

6. (c) Der Client schickt seine Aufträge an den Daemon, der Images, Container, Netze und Volumes verwaltet.

7. (c) Wer den Docker-Socket bedienen kann, kann einen Container mit eingebundenem Hauptverzeichnis des Hosts starten.

8. (a) Die Quelle liegt als deb822-Datei unter `/etc/apt/sources.list.d`.

9. (b) Der Befehl lädt das Image, startet einen Container und zeigt eine Begrüßung.

10. (c) Mit -d (detach) kehrt die Shell sofort zurück.

11. (b) Ohne -a erscheinen nur laufende Container.

12. (c) Mit `--rm` entfernt Docker den Container beim Beenden.

13. (a) Docker umgeht ufw bei veröffentlichten Ports, Schutz bietet die Kette DOCKER-USER.

14. (d) FROM benennt das Image, auf dem das neue aufbaut.

15. (d) COPY übernimmt Dateien aus dem Verzeichnis, das beim Bauen übergeben wird.

16. (a) In der letzten Stufe kopiert man nur das Ergebnis aus der Build-Stufe.

17. (a) Damit gelangen etwa Zugangsdaten oder große Verzeichnisse nicht in den Build.

18. (d) 137 ist 128 plus Signal 9. Mit `docker inspect` prüft man OOMKilled.

19. (c) Die Grenze wird über cgroups durchgesetzt.

20. (b) Die Regel hält Dienste am Laufen, respektiert aber ein bewusstes Stoppen.

21. (c) Ein Volume wird getrennt vom Container verwaltet.

22. (a) Beim Bind Mount bestimmen Sie den Ort selbst, damit auch Rechte und Besitzer des Hosts.

23. (a) Docker betreibt dafür einen eingebauten DNS-Dienst.

24. (a) Container im internen Netz erreichen sich gegenseitig, aber nicht das Internet.

25. (b) Die Bindung an die Loopback-Adresse schützt vor Zugriffen aus dem Netz.

26. (b) Docker leert diese Kette nicht, eigene Regeln bleiben erhalten.

27. (d) Die Antworten stammen aus dem Internet und sehen aus wie unerwünschter Verkehr von außen.

28. (b) Compose liest `compose.yaml` und die Namen compose.yml, `docker-compose.yaml` und `docker-compose.yml`.

29. (a) Moderne Compose-Versionen ignorieren das Feld.

30. (a) Voraussetzung ist ein Healthcheck beim abhängigen Dienst.

31. (b) Damit verschwinden auch die Daten der Datenbank.

32. (a) Spätere Änderungen in der Compose-Datei verändern eine vorhandene Datenbank nicht.

33. (d) Der Dump liest die Daten konsistent, ohne den Betrieb zu sperren.

34. (c) Ab Version 18 liegen die Daten in einem Unterverzeichnis, das zur Hauptversion gehört.

35. (c) Nach außen ist nur der Proxy sichtbar, die Anwendungen bleiben in internen Netzen.

36. (b) Mit einer Variable in proxy_pass löst nginx den Namen nicht nur beim Start auf.

37. (a) Eine Zertifizierungsstelle signiert diese Zuordnung.

38. (b) Für ein internes Übungsnetz ist das vertretbar, für das Internet nicht.

39. (d) Portainer hat Zugriff auf den Docker-Socket, deshalb darf er nicht offen im Netz stehen.

40. (d) Über den Socket laufen alle Befehle, auch schreibende.

41. (c) Angreifer erfahren so weniger über die eingesetzte Version.

42. (a) Danach fügt man gezielt hinzu, was das Programm braucht.

43. (c) Auch Programme mit Sonderrechten gewinnen dann keine zusätzlichen Rechte.

44. (b) Wer den Socket bedient, kann beliebige Container mit beliebigen Rechten starten.

45. (b) Root im Container entspricht dann einer unprivilegierten Nummer auf dem Host.

46. (c) Die Datei steht weder in der Compose-Datei noch in `docker inspect`.

47. (c) Eine Sicherung auf derselben Platte schützt nicht vor deren Ausfall.

48. (b) Die Ausfallzeit beschreibt der RTO.

49. (b) Auch ein Sicherheitswerkzeug kann zum Einfallstor werden.

50. (d) Bei einem Fehler nennt die Ausgabe die Stelle.

51. (d) Alles andere bleibt schreibgeschützt.

52. (d) Unterstützte Runtimes sind Moby, Mirantis Container Runtime und containerd.

53. (a) Abhilfe sind ein passendes Image oder die Hyper-V-Isolation.

54. (b) Bei der Hyper-V-Isolation bekommt jeder Container einen eigenen Kernel.

55. (b) Änderungen wirken nach `wsl --shutdown`.

56. (b) Das BOM wird als Teil des ersten Befehls gelesen.

57. (a) Windows kennt nat, overlay, transparent, l2bridge und l2tunnel.

58. (b) Wie bei der Gruppe `docker unter` Linux gewährt sie Zugriff auf den Daemon.

59. (d) Dort sind Bindungen deutlich schneller und Dateiänderungen kommen als Ereignisse an.

Zusätzliche Praxisaufgaben

Die Aufgaben vertiefen den Stoff der Teile I bis VIII über die Praxisprojekte hinaus. Sie sind nach Teilen geordnet und brauchen nur den Übungsserver (Aufgaben zu Windows: einen Windows-Rechner). Zu jeder Aufgabe steht ein Lösungshinweis, der den Weg beschreibt, ohne die Arbeit abzunehmen. Versuchen Sie zuerst, die Aufgabe ohne den Hinweis zu lösen.

Aufgaben zu Teil I bis III

Aufgabe 1: Ein Image mit Healthcheck

Bauen Sie ein eigenes nginx-Image, das eine Seite ausliefert und einen Healthcheck mit `wget` enthält. Starten Sie den Container und prüfen Sie mit `docker ps`, wann er als `healthy` gilt. Stoppen Sie nginx im Container und beobachten Sie den Zustand.

> **Lösungshinweis**
>
> Im Dockerfile steht die Anweisung `HEALTHCHECK` mit `wget -q --spider http://127.0.0.1/`. Den Zustand zeigt `docker inspect --format '{{.State.Health.Status}}' NAME`.

Aufgabe 2: Mehrstufiger Build

Schreiben Sie ein kleines Go-Programm und bauen Sie es in zwei Stufen. Vergleichen Sie die Größe des Endimages mit der Größe eines Images, das die Build-Werkzeuge enthält.

> **Lösungshinweis**
>
> Erste Stufe `golang:1-alpine` zum Bauen, zweite Stufe `alpine:3` mit `COPY --from=build`. Größen zeigt `docker images`.

Aufgabe 3: Speichergrenze erzwingen

Starten Sie einen Container mit `--memory 64m` und lassen Sie ein Programm mehr Speicher belegen. Weisen Sie nach, dass der Container beendet wurde und warum.

> **Lösungshinweis**
>
> Rückgabewert 137, `docker inspect --format '{{.State.OOMKilled}}' NAME` meldet `true`.

Aufgabe 4: Fehlersuche Rückgabewert 127

Starten Sie einen Container mit einem Befehl, den es im Image nicht gibt. Erklären Sie den Rückgabewert und finden Sie die Ursache mit `--entrypoint sh`.

> **Lösungshinweis**
>
> Wert 127 heißt Befehl nicht gefunden. In der Shell prüft man mit `ls` und `which`.

Aufgaben zu Teil IV und V

Aufgabe 5: Volume sichern und zurückspielen

Legen Sie ein Volume mit Testdaten an, sichern Sie es mit einem Hilfscontainer als Archiv und stellen Sie es in einem neuen Volume wieder her. Vergleichen Sie die Dateien.

> **Lösungshinweis**
>
> Archiv mit `tar czf` in einem Container, der das Volume und ein Bind-Verzeichnis einbindet. Zurückspielen mit `tar xzf`.

Aufgabe 6: Namensauflösung im eigenen Netz

Legen Sie ein benutzerdefiniertes Netzwerk an und starten Sie zwei Container darin. Erreichen Sie den einen vom anderen über den Namen. Zeigen Sie, dass es im Standardnetz nicht funktioniert.

> **Lösungshinweis**
>
> Benutzerdefinierte Bridge-Netze haben eingebauten DNS. Test mit `docker exec NAME1 ping -c 1 NAME2` oder `wget`.

Aufgabe 7: Internes Netz

Bauen Sie ein Netz mit `--internal` und weisen Sie nach, dass ein Container darin das Internet nicht erreicht, aber den Nachbarn.

> **Lösungshinweis**
>
> `docker exec NAME wget -T 3 -q -O- http://example.com` schlägt fehl, der Nachbarname antwortet.

Aufgabe 8: Compose mit Healthcheck und Reihenfolge

Schreiben Sie eine `compose.yaml` mit Datenbank und Anwendung. Die Anwendung soll erst starten, wenn die Datenbank gesund ist.

> **Lösungshinweis**
>
> `healthcheck` bei der Datenbank, bei der Anwendung `depends_on` mit `condition: service_healthy`.

Aufgabe 9: PostgreSQL sichern und wiederherstellen

Starten Sie PostgreSQL 18 mit Volume, legen Sie eine Tabelle an, sichern Sie mit `pg_dump`, löschen Sie die Tabelle und stellen Sie sie wieder her.

> **Lösungshinweis**
>
> Dump mit `docker compose exec -T db pg_dump -U BENUTZER DATENBANK`, Rückspielen mit `psql`.

Aufgaben zu Teil VI und VII

Aufgabe 10: Reverse Proxy für zwei Namen

Betreiben Sie zwei Webcontainer hinter einem nginx-Proxy, der nach dem Namen verteilt. Prüfen Sie mit `curl --resolve` beide Namen und einen unbekannten Namen.

> **Lösungshinweis**
>
> Zwei `server`-Blöcke mit `server_name`, ein Auffangblock mit `return 444`. Test mit `curl --resolve NAME:443:127.0.0.1 https://NAME/`.

Aufgabe 11: Anfragen begrenzen

Aktivieren Sie `limit_req` im Proxy und lösen Sie mit einer Schleife viele schnelle Anfragen aus. Zählen Sie die Antworten 200 und 503.

> **Lösungshinweis**
>
> `limit_req_zone` im `http`-Teil, `limit_req zone=...` mit `burst`. Zählen mit `sort | uniq -c`.

Aufgabe 12: Capabilities untersuchen

Starten Sie einen Alpine-Container mit und ohne `--cap-drop NET_RAW` und führen Sie `ping` aus. Lesen Sie `CapEff` aus und erklären Sie die Werte.

> **Lösungshinweis**
>
> Ohne `NET_RAW` meldet `ping` einen Rechtefehler. `grep CapEff /proc/self/status` zeigt die wirksamen Capabilities als Hexadezimalzahl.

Aufgabe 13: Schreibgeschützter Container

Betreiben Sie nginx mit `read_only: true` und ermitteln Sie anhand der Fehlermeldungen, welche Verzeichnisse als `tmpfs` nötig sind.

> **Lösungshinweis**
>
> Die Meldung `Read-only file system` nennt den Pfad. Üblich sind `/tmp`, `/var/run` und `/var/cache/nginx`.

Aufgabe 14: Secrets als Dateien

Richten Sie MariaDB mit `MARIADB_ROOT_PASSWORD_FILE` und einem Secret ein. Zeigen Sie, dass das Passwort weder in `docker inspect` noch in den Umgebungsvariablen steht.

> **Lösungshinweis**
>
> `secrets:` in der Compose-Datei, Datei unter `/run/secrets`. Prüfung mit `docker inspect --format '{{.Config.Env}}' NAME`.

Aufgabe 15: Image ohne Socket prüfen

Prüfen Sie ein Image mit einem Schwachstellenscanner, ohne den Docker-Socket in den Scanner einzubinden.

> **Lösungshinweis**
>
> Image mit `docker save -o` als Archiv speichern und dem Scanner als Datei übergeben. Version des Scanners festlegen.

Aufgabe 16: Firewall dauerhaft machen

Setzen Sie in `DOCKER-USER` die Regeln für Heimnetz und bestehende Verbindungen und machen Sie sie mit einem `systemd`-Dienst reboot-fest. Prüfen Sie nach einem Neustart.

> **Lösungshinweis**
>
> Skript mit `iptables -F DOCKER-USER`, Regeln und `RETURN`. Dienst mit `After=docker.service` und `PartOf=docker.service`.

Aufgabe 17: Sicherung per Timer

Richten Sie eine tägliche Sicherung mit `systemd`-Timer ein und weisen Sie mit `systemctl list-timers` nach, dass er aktiv ist. Testen Sie die Wiederherstellung.

> **Lösungshinweis**
>
> Dienst mit `Type=oneshot`, Timer mit `OnCalendar` und `Persistent=true`. Wiederherstellung mit einem Dump in eine frische Datenbank.

Aufgaben zu Teil VIII

Aufgabe 18: WSL begrenzen und prüfen

Begrenzen Sie die WSL-2-Umgebung mit einer `.wslconfig` und belegen Sie die Wirkung mit `docker info`.

> **Lösungshinweis**
>
> Datei im Benutzerprofil, danach `wsl --shutdown`. Anzeige mit `docker info --format '{{.MemTotal}} {{.NCPU}}'`.

Aufgabe 19: Unterschied der Isolationsmodi

Starten Sie einen Windows-Container mit Prozess- und einen mit Hyper-V-Isolation und zeigen Sie mit `Get-Process` den Unterschied.

> **Lösungshinweis**
>
> Nur der Container mit Prozess-Isolation erscheint als Prozess auf dem Host. Voraussetzung: Docker Desktop für alle Benutzer.

Aufgabe 20: BOM-Experiment

Legen Sie eine nginx-Konfiguration einmal mit und einmal ohne BOM an und starten Sie jeweils einen Container. Erklären Sie den Unterschied.

> **Lösungshinweis**
>
> Mit BOM meldet nginx eine unbekannte Anweisung. Ohne BOM schreibt die Funktion `Write-Utf8NoBom` oder Notepad mit UTF-8.

Aufgabe 21: Windows-Server-Labor erweitern

Richten Sie auf einem Windows-Server-Testsystem Compose als `docker-compose.exe` ein und betreiben Sie zwei IIS-Dienste mit unterschiedlichen Ports.

> **Lösungshinweis**
>
> Standalone-Datei nach `C:\Program Files\Docker`. In der `compose.yaml` zwei Dienste mit `ports`.

Glossar

Das Glossar erklärt die Fachbegriffe des Buches in alphabetischer Reihenfolge, einschließlich der Begriffe zu Docker unter Windows aus Kapitel 21.

A

Angriffsfläche
Summe aller Stellen, an denen ein Angreifer ansetzen kann. Bei einem Docker-Server liegen sie auf mehreren Schichten, vom Netzwerk bis zur Anwendung.

AppArmor
Schutzsystem des Linux-Kernels, das über Profile begrenzt, auf welche Dateien und Funktionen ein Programm zugreifen darf. Docker wendet das Profil `docker-default` an.

Ausführungsrichtlinie (Execution Policy)
Einstellung der PowerShell, die festlegt, welche Skripte laufen dürfen.

B

Basisimage
Image, auf dem ein anderes Image aufbaut. Bei Windows-Containern stammt es von Microsoft (Server Core, Nano Server, Windows, Windows Server) und trägt immer ein bestimmtes Tag.

Bind Mount
Einbindung eines Pfades des Hosts in einen Container. Rechte und Besitzer des Hosts gelten.

Bridge-Netzwerk
Standardart eines Docker-Netzwerks, das Container über einen virtuellen Switch auf dem Host verbindet. In benutzerdefinierten Bridge-Netzen lösen sich Container über ihren Namen auf.

Build-Kontext
Verzeichnis, dessen Inhalt beim Bauen eines Images an den Daemon übergeben wird.

Build-Nummer
Teil der Windows-Versionsnummer, der sich mit jedem neuen Windows-Release ändert. Bei Prozess-Isolation müssen die Build-Nummern von Host und Image passen, mit der Ausnahme Windows Server 2022 auf Windows 11.

C

Capabilities
Zerlegung der Rechte von Root in einzelne Sonderrechte des Linux-Kernels. Docker gibt Containern 14 davon, `--cap-drop ALL` entfernt alle.

cgroups
Control Groups des Linux-Kernels, mit denen Ressourcen wie Speicher und Prozessorzeit einer Prozessgruppe begrenzt und gemessen werden.

Client
Das Programm `docker`, das Befehle an den Daemon schickt.

com.docker.service
Privilegierter Hilfsdienst von Docker Desktop unter Windows, der mit SYSTEM-Rechten läuft. Er gehört nur zur Installation für alle Benutzer.

Compose
Werkzeug, das mehrere Container in einer Datei (`compose.yaml`) beschreibt und gemeinsam startet, stoppt und verwaltet.

Container
Isolierter Prozess auf dem Kernel des Hosts, der aus einem Image entsteht und über Namespaces und cgroups getrennt und begrenzt wird.

Container-Runtime
Software, die Container tatsächlich anlegt und ausführt.

containerd
Container-Runtime, die unter Docker Engine arbeitet. Unter Windows Server unterstützt Microsoft sie neben Moby und der Mirantis Container Runtime. Bei frischen Installationen von Docker 29 hält sie auch die Images.

CVE
Eindeutige Kennung einer bekannten Sicherheitslücke nach dem Schema `CVE-Jahr-Nummer`.

D

Daemon
Hintergrunddienst `dockerd`, der Images, Container, Netzwerke und Volumes verwaltet.

Digest
Unveränderliche Prüfsumme (`sha256:...`) eines Images. Anders als ein Tag bezeichnet sie genau eine Fassung.

DNS (Docker)
Eingebauter Namensdienst in benutzerdefinierten Netzen, erreichbar unter der Adresse 127.0.0.11 im Container.

Docker Desktop
Programm mit grafischer Oberfläche, das Engine, Client, Compose und die virtuelle Umgebung enthält. Arbeitsplatzwerkzeug mit eigenen Lizenzbedingungen, auf Windows Server nicht unterstützt.

Docker Engine
Die freie Container-Plattform aus Daemon, Client und Zubehör.

Docker VMM
Hypervisor von Docker für Docker Desktop, derzeit als Beta. Er braucht die Windows-Funktion Hyper-V nicht.

Docker-Socket
Datei `/var/run/docker.sock`, über die der Client den Daemon bedient. Wer ihn bedienen kann, hat Root-Rechte auf dem Host.

DOCKER-USER

Kette der Firewall, in der eigene Regeln vor den Regeln von Docker geprüft werden.

docker-users
Lokale Windows-Gruppe, deren Mitglieder auf den Daemon von Docker Desktop zugreifen dürfen. Die Mitgliedschaft entspricht Administratorrechten auf dem Host.

Dockerfile
Textdatei mit den Anweisungen, aus denen ein Image gebaut wird.

E

Enhanced Container Isolation (ECI)
Funktion von Docker Desktop im Business-Abonnement, die Container in einem Linux-User-Namespace laufen lässt. Sie setzt WSL ab Version 2.6 voraus.

Exit-Code
Rückgabewert, mit dem sich ein Prozess beendet. Werte über 128 entstehen als 128 plus Signalnummer, zum Beispiel 137 für SIGKILL.

G

gMSA
Gruppenverwaltetes Dienstkonto, mit dem sich Windows-Container bei Active Directory anmelden können.

H

Healthcheck
Prüfbefehl, mit dem Docker feststellt, ob ein Dienst im Container gesund ist.

Host
Rechner, auf dem Container laufen und dessen Kernel sie nutzen.

Hyper-V-Isolation
Betriebsart für Windows-Container, bei der jeder Container in einer eigenen kleinen virtuellen Maschine mit eigenem Windows-Kernel läuft.

Hypervisor

Software, die die Hardware auf mehrere virtuelle Maschinen aufteilt.

I

Image
Schreibgeschützte, aus Schichten aufgebaute Vorlage, aus der Container entstehen.

iptables
Werkzeug zur Verwaltung der Firewall-Regeln im Linux-Kernel. Docker nutzt es standardmäßig als Backend.

Isolation
Trennung von Prozessen voneinander und vom Host.

L

Layer (Schicht)
Einzelne Ebene eines Images, die bei Änderungen wiederverwendet werden kann.

Least Privilege
Grundsatz der geringsten Rechte: Jede Komponente bekommt nur die Rechte, die sie braucht.

Linux-Container
Container mit Linux-Programmen, die einen Linux-Kernel brauchen. Unter Windows laufen sie in der virtuellen Umgebung von WSL 2.

live-restore
Einstellung in der `daemon.json`, bei der Container weiterlaufen, während der Daemon neu startet.

LTSC
Windows-Version mit Langzeitunterstützung. Die Tags der Basisimages heißen zum Beispiel `ltsc2022` und `ltsc2025`.

M

Microsoft Container Registry (MCR)
Registry unter `mcr.microsoft.com`, aus der die Windows-Basisimages stammen.

Mirantis Container Runtime
Kommerziell unterstützte Container-Runtime für Windows Server, früher Docker Enterprise.

Moby
Open-Source-Projekt hinter der Docker Community Edition.

Multi-Stage-Build
Mehrstufiger Build, bei dem das fertige Image nur das Ergebnis der Build-Stufe enthält.

N

Named Pipe
Verbindungsart unter Windows, über die der Docker-Client lokal mit dem Daemon spricht.

Namespace
Funktion des Linux-Kernels, die einem Prozess eine eigene Sicht auf Prozesse, Netzwerk, Dateisysteme und mehr gibt.

Nano Server
Kleinstes Windows-Basisimage mit kleiner Programmierschnittstelle, ohne PowerShell und WMI.

NAT (Netzwerk nat)
Standardnetz der Windows-Container, das mit dem Windows-Dienst WinNAT arbeitet.

nftables
Neuere Firewall-Technik des Linux-Kernels. Docker unterstützt sie nur experimentell, iptables bleibt der Standard.

Non-Root-Container
Container, dessen Hauptprozess ohne Root-Rechte läuft.

O

OOM-Kill
Beenden eines Prozesses durch den Kernel bei Speichermangel. Bei Containern mit Limit trifft es den Container.

OverlayFS
Union-Dateisystem, das die Schichten eines Images übereinanderlegt und eine beschreibbare Schicht obenauf setzt.

P

Port-Veröffentlichung
Weiterleitung eines Host-Ports an einen Container-Port mit `-p`.

PowerShell
Shell und Skriptsprache von Windows. Windows PowerShell 5.1 ist eingebaut, PowerShell 7 ist ein getrenntes Produkt.

Prozess-Isolation
Betriebsart für Windows-Container, bei der alle Container den Windows-Kernel des Hosts teilen.

Prune
Aufräumbefehle wie `docker image prune`, die ungenutzte Objekte entfernen.

R

Read-only-Container
Container mit schreibgeschütztem Wurzeldateisystem (`--read-only`), der nur in `tmpfs` und Volumes schreiben darf.

Registry
Dienst, der Images speichert und verteilt, etwa Docker Hub.

Restart-Policy
Regel, wann Docker einen Container neu startet, zum Beispiel `unless-stopped`.

Reverse Proxy
Server, der Anfragen entgegennimmt und an interne Dienste weiterleitet.

Rootless-Modus
Betrieb von Daemon und Containern als normaler Benutzer ohne Root-Rechte.

RPO
Recovery Point Objective, der größte tolerierte Datenverlust, gemessen in Zeit.

RTO
Recovery Time Objective, die längste tolerierte Ausfallzeit.

S

Seccomp
Filter des Linux-Kernels, der einem Prozess nur eine Auswahl von Systemaufrufen erlaubt.

Secret
Geheimnis wie Passwort oder Schlüssel, das als Datei unter `/run/secrets` in den Container gegeben wird.

Server Core
Windows-Basisimage mit großen Teilen der Programmierschnittstellen von Windows Server.

Socket-Proxy
Kleiner Container, der nur ausgewählte Anfragen an die Docker-Schnittstelle durchlässt.

Supply Chain (Lieferkette)
Alles, was in eine Anwendung hineinfließt, ohne dass man es selbst geschrieben hat: Basisimages, Bibliotheken, Werkzeuge, Registries.

systemd-Timer
Zeitgesteuerte Einheit von systemd, die einen Dienst regelmäßig startet.

T

Tag
Beschriftung eines Images, etwa `3` oder `stable-alpine`. Ein Tag ist veränderlich.

TLS
Verfahren zur Verschlüsselung der Verbindung, etwa bei HTTPS.

tmpfs
Dateisystem im Arbeitsspeicher, das mit dem Container verschwindet.

Trivy
Werkzeug zur Schwachstellenanalyse von Images. Im März 2026 wurde es kompromittiert, was im Buch als Beispiel für Supply-Chain-Risiken dient.

U

userns-remap
Daemon-Einstellung, die Benutzernummern der Container auf einen unprivilegierten Bereich des Hosts abbildet.

UTF-16 und BOM
Zeichenkodierung, in die Windows PowerShell 5.1 mit `>` schreibt, und die Markierung am Dateianfang (Byte-Order-Mark). Konfigurationsdateien für Container müssen UTF-8 ohne BOM sein.

V

Virtuelle Maschine
Nachgebildeter Rechner mit eigenem Kernel, der auf einem Hypervisor läuft.

Volume
Von Docker verwalteter Datenspeicher, der unabhängig vom Container bestehen bleibt.

W

Windows-Container
Container mit Windows-Programmen, die den Windows-Kernel des Hosts nutzen und Windows-Images von Microsoft brauchen.

WSL 2
Windows Subsystem for Linux in Version 2: ein echter Linux-Kernel in einer schlanken virtuellen Maschine, den Docker Desktop für Linux-Container nutzt.

.wslconfig
Datei im Benutzerprofil, die Arbeitsspeicher, Prozessoren und Auslagerung der WSL-2-Umgebung begrenzt.

Z

Zertifikat
Bestätigung, dass ein öffentlicher Schlüssel zu einem Namen gehört, ausgestellt von einer Zertifizierungsstelle oder selbst signiert.

SERVER & CLOUD · BAND 1

Docker und Container

Container sind aus modernen Serverlandschaften nicht mehr wegzudenken. Viele Anleitungen zeigen aber nur Befehle und erklären nicht, warum sie funktionieren. Dieses Buch beginnt bei der Frage, was ein Container überhaupt ist, und führt zu einer vollständigen, abgesicherten Serverinfrastruktur auf Debian 13.

Sie arbeiten in einer durchgehenden Laborumgebung und bauen Schritt für Schritt auf: Docker installieren, eigene Images erstellen, Daten und Netzwerke verwalten, Anwendungen mit Docker Compose zusammenstellen, Datenbanken und Webserver betreiben und einen Reverse Proxy mit TLS davorsetzen. Sieben Praxisprojekte, Lernkontrollen mit Lösungen und eine Befehlsreferenz begleiten Sie dabei, Prüfungsfragen und ein Glossar helfen beim Wiederholen. Ein eigener Teil zeigt Docker unter Windows mit Docker Desktop, WSL 2 und Windows Server.

Was Sie in diesem Buch lernen

- Docker-Architektur verstehen
- Container erstellen und verwalten
- Eigene Docker-Images bauen
- Docker Compose einsetzen
- Netzwerke und Volumes konfigurieren
- Webserver und Datenbanken betreiben
- Container absichern
- Serverinfrastrukturen aufbauen

Zielgruppe

Das Buch richtet sich an angehende Fachinformatiker für Systemintegration und an Administratoren, die Container einführen oder von Grund auf verstehen wollen. Vorkenntnisse in Docker sind nicht nötig.

Behandelte Umgebungen: Debian 13, Docker Engine 29, Windows 11 und Windows Server